

inTurno — Codice Sorgente Completo

Generato automaticamente dal repository. Stack: Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS, shadcn/ui, Supabase (Postgres + Auth + Realtime), PWA.

Indice dei file

- `public/sw.js`
- `src/app/(dipendente)/home/layout.tsx`
- `src/app/(dipendente)/home/loading.tsx`
- `src/app/(dipendente)/home/miei-turni/page.tsx`
- `src/app/(dipendente)/home/ods/loading.tsx`
- `src/app/(dipendente)/home/ods/page.tsx`
- `src/app/(dipendente)/home/page.tsx`
- `src/app/(dipendente)/layout.tsx`
- `src/app/(manager)/account-pendenti/loading.tsx`
- `src/app/(manager)/account-pendenti/page.tsx`
- `src/app/(manager)/approvazioni/loading.tsx`
- `src/app/(manager)/approvazioni/page.tsx`
- `src/app/(manager)/assenze/loading.tsx`
- `src/app/(manager)/assenze/page.tsx`
- `src/app/(manager)/bacheca/loading.tsx`
- `src/app/(manager)/bacheca/page.tsx`
- `src/app/(manager)/dashboard/loading.tsx`
- `src/app/(manager)/dashboard/page.tsx`
- `src/app/(manager)/dipendenti/loading.tsx`
- `src/app/(manager)/dipendenti/page.tsx`
- `src/app/(manager)/layout.tsx`
- `src/app/(manager)/ods/loading.tsx`
- `src/app/(manager)/ods/page.tsx`
- `src/app/(manager)/presenze/loading.tsx`
- `src/app/(manager)/presenze/page.tsx`
- `src/app/(manager)/report/loading.tsx`
- `src/app/(manager)/report/page.tsx`
- `src/app/(manager)/ristoranti/loading.tsx`
- `src/app/(manager)/ristoranti/page.tsx`
- `src/app/(manager)/turni/loading.tsx`
- `src/app/(manager)/turni/page.tsx`
- `src/app/actions/adminActions.ts`
- `src/app/actions/aiTurni.ts`
- `src/app/actions/ods.ts`
- `src/app/actions/telegram.ts`
- `src/app/actions/turni.ts`
- `src/app/api/activity/route.ts`
- `src/app/api/assenze/route.ts`
- `src/app/api/auth/callback/route.ts`
- `src/app/api/clock-in-fallback/route.ts`
- `src/app/api/consultant-messages/[id]/download/route.ts`
- `src/app/api/consultant-messages/[id]/read/route.ts`

- `src/app/api/consultant-messages/route.ts`
- `src/app/api/consultant-messages/upload/route.ts`
- `src/app/api/presenze/fallback-approve/route.ts`
- `src/app/api/presenze/route.ts`
- `src/app/api/push/send/route.ts`
- `src/app/api/push/subscribe/route.ts`
- `src/app/api/register/route.ts`
- `src/app/api/report/route.ts`
- `src/app/api/scan/route.ts`
- `src/app/api/telegram/setup/route.ts`
- `src/app/api/telegram/webhook/route.ts`
- `src/app/api/users/password/route.ts`
- `src/app/api/users/route.ts`
- `src/app/consulente/dashboard/page.tsx`
- `src/app/consulente/layout.tsx`
- `src/app/layout.tsx`
- `src/app/login/page.tsx`
- `src/app/page.tsx`
- `src/app/register/page.tsx`
- `src/components/consulente/ActivityTrackerMount.tsx`
- `src/components/consulente/ConsultantDashboard.tsx`
- `src/components/consulente/ConsultantInbox.tsx`
- `src/components/consulente/ConsultantReportClient.tsx`
- `src/components/dipendente/AbsenceRequestDialog.tsx`
- `src/components/dipendente/BottomNav.tsx`
- `src/components/dipendente/BulletinDrawer.tsx`
- `src/components/dipendente/EmployeeHomeClient.tsx`
- `src/components/dipendente/MieiTurniClient.tsx`
- `src/components/dipendente/OdsClient.tsx`
- `src/components/dipendente/QRScanner.tsx`
- `src/components/manager/AccountPendentiClient.tsx`
- `src/components/manager/AiScheduleDialog.tsx`
- `src/components/manager/AiScheduleDraftView.tsx`
- `src/components/manager/ApprovazioniClient.tsx`
- `src/components/manager/AssenzeClient.tsx`
- `src/components/manager/BachecaClient.tsx`
- `src/components/manager/CapoServizioTimbraturaSection.tsx`
- `src/components/manager/ConsulenteLavoroManager.tsx`
- `src/components/manager/DemoBanner.tsx`
- `src/components/manager/DipendentiClient.tsx`
- `src/components/manager/FallbackApprovalSection.tsx`
- `src/components/manager/LiveShiftCounter.tsx`
- `src/components/manager/ManagerSidebar.tsx`
- `src/components/manager/OdsManagerClient.tsx`
- `src/components/manager/PageSkeleton.tsx`
- `src/components/manager/PresenzeClient.tsx`
- `src/components/manager/PresenzePreviewClient.tsx`
- `src/components/manager/ReportClient.tsx`
- `src/components/manager/RestaurantQrCard.tsx`
- `src/components/manager/RestaurantsClient.tsx`
- `src/components/manager/TelegramLinkButton.tsx`
- `src/components/manager/TurniManagerClient.tsx`
- `src/components/manager/TurniTimeline.tsx`

- `src/components/shared/LoadingDots.tsx`
- `src/components/shared/LoginForm.tsx`
- `src/components/shared/OfflineSyncProvider.tsx`
- `src/components/shared/PushNotificationBanner.tsx`
- `src/components/shared/RegisterForm.tsx`
- `src/components/shared/ThemeProvider.tsx`
- `src/components/shared/ThemeToggle.tsx`
- `src/components/ui/badge.tsx`
- `src/components/ui/button.tsx`
- `src/components/ui/card.tsx`
- `src/components/ui/dialog.tsx`
- `src/components/ui/dropdown-menu.tsx`
- `src/components/ui/input.tsx`
- `src/components/ui/label.tsx`
- `src/components/ui/select.tsx`
- `src/components/ui/separator.tsx`
- `src/components/ui/skeleton.tsx`
- `src/components/ui/switch.tsx`
- `src/components/ui/tabs.tsx`
- `src/components/ui/textarea.tsx`
- `src/components/ui/time-input.tsx`
- `src/components/ui/toast.tsx`
- `src/contexts/AccountStatusContext.tsx`
- `src/hooks/useActivityTracker.ts`
- `src/hooks/useBadging.ts`
- `src/hooks/useGeofence.ts`
- `src/hooks/usePushNotifications.ts`
- `src/lib/attendanceTime.ts`
- `src/lib/autoCloseStaleShifts.ts`
- `src/lib/compressImage.ts`
- `src/lib/demoData.ts`
- `src/lib/email.ts`
- `src/lib/generateUnifiedPDF.ts`
- `src/lib/offlineSync.ts`
- `src/lib/supabase/admin.ts`
- `src/lib/supabase/client.ts`
- `src/lib/supabase/server.ts`
- `src/lib/telegram/ai.ts`
- `src/lib/telegram/aiHistory.ts`
- `src/lib/telegram/api.ts`
- `src/lib/telegram/auth.ts`
- `src/lib/telegram/commands/help.ts`
- `src/lib/telegram/commands/ods.ts`
- `src/lib/telegram/commands/presenze.ts`
- `src/lib/telegram/commands/turni.ts`
- `src/lib/telegram/context.ts`
- `src/lib/telegram/format.ts`
- `src/lib/telegram/router.ts`
- `src/lib/telegram/scope.ts`
- `src/lib/telegram/session.ts`
- `src/lib/telegram/types.ts`
- `src/lib/telegram/wizard.ts`
- `src/lib/turnColors.ts`

- `src/lib/turniScope.ts`
 - `src/lib/utils.ts`
 - `src/middleware.ts`
 - `src/types/index.ts`
 - `supabase/migrations/20260626_saas_multitenancy.sql`
 - `next.config.ts`
 - `package.json`
 - `vercel.json`
-


```

// — inTurno Service Worker —————
// Handles: push notifications, offline caching, App Badging

const STATIC_CACHE = 'inturno-static-v2'
const PAGES_CACHE = 'inturno-pages-v2'

// Assets to pre-cache on install (offline shell)
const PRECACHE = [
  '/manifest.json',
  '/icon-192.png',
  '/icon-512.png',
  '/logo-branding.png',
]

// — Install —————
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(STATIC_CACHE)
      .then(cache => cache.addAll(PRECACHE))
      .then(() => self.skipWaiting()),
  )
})

// — Activate: prune stale caches —————
self.addEventListener('activate', event => {
  const keep = new Set([STATIC_CACHE, PAGES_CACHE])
  event.waitUntil(
    caches.keys()
      .then(keys => Promise.all(keys.filter(k => !keep.has(k)).map(k => caches.delete(k))))
      .then(() => self.clients.claim()),
  )
})

// — Fetch —————
self.addEventListener('fetch', event => {
  const { request } = event
  const url = new URL(request.url)

  // Only handle same-origin GETs
  if (request.method !== 'GET' || url.origin !== self.location.origin) return

  // Skip API and Next.js data routes — always network
  if (url.pathname.startsWith('/api/') || url.pathname.startsWith('/_next/data/')) return

  // App Router client-side navigations & prefetches request RSC payloads
  // (accept: text/x-component / RSC header), NOT text/html. These carry
  // user-specific, time-sensitive data (the shift list, open timbratura),
  // so they must never be served cache-first — otherwise a newly added
  // split shift stays hidden until a hard reload. NetworkFirst, with the
  // cache used only as an offline fallback.
  const isRSC =
    request.headers.get('RSC') === '1' ||
    url.searchParams.has('_rsc') ||
    request.headers.get('accept')?.includes('text/x-component')
  if (isRSC) {
    event.respondWith(
      fetch(request)
        .then(res => {
          if (res.ok) {
            const copy = res.clone()
            caches.open(PAGES_CACHE).then(cache => cache.put(request, copy))
          }
          return res
        })
        .catch(() => caches.open(PAGES_CACHE).then(cache => cache.match(request))),
    )
    return
  }

  // CacheFirst for immutable hashed bundles
  if (url.pathname.startsWith('/_next/static/')) {
    event.respondWith(
      caches.open(STATIC_CACHE).then(cache =>

```

```

        cache.match(request).then(hit => hit || fetch(request).then(res => {
            if (res.ok) cache.put(request, res.clone())
            return res
        })),
    ),
)
return
}

// NetworkFirst for HTML pages – authenticated pages render time-sensitive,
// user-specific state (open shift / timbratura status), so they must always
// reflect fresh server data when online. StaleWhileRevalidate served a
// pre-check-in cached page for hours, making an open shift "disappear".
// Cache is used only as an offline fallback.
if (request.headers.get('accept')?.includes('text/html')) {
    event.respondWith(
        fetch(request)
            .then(res => {
                if (res.ok) {
                    const copy = res.clone()
                    caches.open(PAGES_CACHE).then(cache => cache.put(request, copy))
                }
                return res
            })
            .catch(() => caches.open(PAGES_CACHE).then(cache => cache.match(request))),
    )
    return
}

// CacheFirst for other public assets
event.respondWith(
    caches.open(STATIC_CACHE).then(cache =>
        cache.match(request).then(hit => hit || fetch(request).then(res => {
            if (res.ok) cache.put(request, res.clone())
            return res
        })),
    ),
)
})

// — Push notifications —————
self.addEventListener('push', event => {
    if (!event.data) return
    const data = event.data.json()
    event.waitUntil(
        self.registration.showNotification(data.title, {
            body: data.body,
            icon: '/icon-192.png',
            badge: '/icon-192.png',
            vibrate: [200, 100, 200],
            data: { url: data.url ?? '/' },
        }),
    )
})

self.addEventListener('notificationclick', event => {
    event.notification.close()
    const url = event.notification.data?.url ?? '/'
    event.waitUntil(
        clients.matchAll({ type: 'window', includeUncontrolled: true }).then(list => {
            for (const client of list) {
                if (client.url.includes(self.location.origin) && 'focus' in client) {
                    client.navigate(url)
                    return client.focus()
                }
            }
            return clients.openWindow(url)
        }),
    )
})

```

[src/app/\(dipendente\)/home/layout.tsx](#)

```
import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'

export default async function DipendenteLayout({ children }: { children: React.ReactNode }) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) redirect('/login')

  const { data: profile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', user.id)
    .single()

  // Solo i dipendenti accedono a questa area
  if (profile?.role !== 'dipendente') redirect('/dashboard')

  return <>{children}</>
}
```

[src/app/\(dipendente\)/home/loading.tsx](#)

```
import { Skeleton } from '@components/ui/skeleton'

export default function Loading() {
  return (
    <main className="min-h-screen bg-background flex flex-col">
      {/* Header */}
      <div className="flex items-center justify-between px-4 pt-4 pb-3">
        <div className="space-y-1.5">
          <Skeleton className="h-3 w-40" />
          <Skeleton className="h-4 w-32" />
          <Skeleton className="h-3 w-24" />
        </div>
        <div className="flex items-center gap-2">
          <Skeleton className="w-9 h-9 rounded-md" />
          <Skeleton className="w-9 h-9 rounded-md" />
          <Skeleton className="w-9 h-9 rounded-md" />
        </div>
      </div>

      {/* Center: clock + button + link */}
      <div className="flex-1 flex flex-col items-center justify-center gap-6 px-6">
        <Skeleton className="h-16 w-64 rounded-md" />
        <Skeleton className="w-full max-w-xs h-14 rounded-md" />
        <Skeleton className="h-4 w-36 rounded" />
      </div>
    </main>
  )
}
```


[src/app/\(dipendente\)/home/miei-turni/page.tsx](#)

```
import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'
import { MieiTorniClient } from '@components/dipendente/MieiTorniClient'

export default async function MieiTorniPage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) redirect('/login')

  // Query Scoping (RBAC) – Dipendente: visibilità PERSONALE, sola lettura
  const { data: turns } = await supabase
    .from('turns')
    .select('*')
    .eq('user_id', user.id)
    .order('date', { ascending: true })
    .order('start_time', { ascending: true })

  return (
    <MieiTorniClient
      initialTurns={(turns as unknown as import('@types').Turn[]) ?? []}
      userId={user.id}
    />
  )
}
```

[src/app/\(dipendente\)/home/ods/loading.tsx](#)

```
import { Skeleton } from '@components/ui/skeleton'

export default function Loading() {
  return (
    <main className="min-h-screen bg-background flex flex-col">
      { /* Header */ }
      <div className="flex items-center gap-3 px-4 pt-5 pb-3 border-b border-border">
        <Skeleton className="w-5 h-5 rounded shrink-0" />
        <div className="space-y-1">
          <Skeleton className="h-4 w-36" />
          <Skeleton className="h-3 w-20" />
        </div>
      </div>

      <div className="px-4 pt-4 space-y-6">
        { /* Filter pills */ }
        <div className="flex gap-1.5 flex-wrap">
          {[72, 88, 96, 80, 104].map((w, i) => (
            <Skeleton key={i} className="h-7 rounded-sm" style={{ width: w }} />
          ))}
        </div>

        { /* Task rows */ }
        <div className="space-y-1.5">
          {[...Array(6)].map((_, i) => (
            <Skeleton key={i} className="h-11 rounded-sm" />
          ))}
        </div>
      </div>
    </main>
  )
}
```

```
import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'
import { OdsClient } from '@components/dipendente/OdsClient'
import { formatInTimeZone, fromZonedTime } from 'date-fns-tz'

const TZ = 'Europe/Rome'

function getOdsCutoff(): string {
  const now = new Date()
  const romeHour = parseInt(formatInTimeZone(now, TZ, 'H'), 10)
  const refDate = romeHour < 4 ? new Date(now.getTime() - 86_400_000) : now
  const cutoffDate = formatInTimeZone(refDate, TZ, 'yyyy-MM-dd')
  return fromZonedTime(`${cutoffDate}T04:00:00`, TZ).toISOString()
}

export default async function OdsPage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) redirect('/login')

  const { data: profile } = await supabase
    .from('profiles')
    .select('department, restaurant_id')
    .eq('id', user.id)
    .single()

  const cutoff = getOdsCutoff()

  const [{ data: tasks }, { data: completions }] = await Promise.all([
    supabase
      .from('ods_tasks')
      .select('*', assignee:profiles!assigned_to(id, full_name)')
      .order('created_at', { ascending: false }),
    supabase
      .from('ods_completions')
      .select('task_id')
      .eq('user_id', user.id)
      .gte('completed_at', cutoff),
  ])

  return (
    <OdsClient
      tasks={{(tasks as unknown as import('@types').OdsTask[]) ?? []}
      completedTaskIds={{(completions ?? []).map(c => c.task_id)}
      userId={user.id}
      userDepartment={profile?.department ?? null}
    />
  )
}
```

`src/app/(dipendente)/home/page.tsx`

```
import { createClient } from '@lib/supabase/server'
import { EmployeeHomeClient } from '@components/dipendente/EmployeeHomeClient'
import { autoCloseStaleShifts } from '@lib/autoCloseStaleShifts'

export default async function EmployeeHomePage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('*', restaurant:restaurants(id, name, latitude, longitude)')
    .eq('id', user!.id)
    .single()

  // Close a shift the employee forgot to timbrare l'uscita on (open >16h) before
  // reading the open shift, so the UI doesn't show an endless running counter.
  await autoCloseStaleShifts(supabase, user!.id)

  // Open shift = any record with check_out IS NULL, regardless of when it started.
  // Date-filtering here caused cross-midnight shifts to disappear from the UI.
  const { data: openAttendance } = await supabase
    .from('attendances')
    .select('*')
    .eq('user_id', user!.id)
    .is('check_out', null)
    .order('check_in', { ascending: false })
    .limit(1)
    .maybeSingle()

  return (
    <EmployeeHomeClient
      profile={profile}
      openAttendance={openAttendance}
      userId={user!.id}
    />
  )
}
```

`src/app/(dipendente)/layout.tsx`

```
import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'
import { BottomNav } from '@components/dipendente/BottomNav'

export default async function DipendenteLayout({ children }: { children: React.ReactNode }) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) redirect('/login')

  const { data: profile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', user.id)
    .single()

  if (profile?.role !== 'dipendente') redirect('/dashboard')

  return (
    <div className="pb-14">
      {children}
      <BottomNav />
    </div>
  )
}
```

[src/app/\(manager\)/account-pendenti/loading.tsx](#)

```
import { PageSkeleton } from '@components/manager/PageSkeleton'
export default function Loading() { return <PageSkeleton /> }
```

[src/app/\(manager\)/account-pendenti/page.tsx](#)

```
import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'
import { getPendingAccounts } from '@app/actions/adminActions'
import { AccountPendentiClient } from '@components/manager/AccountPendentiClient'

export default async function AccountPendentiPage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, managed_restaurant_ids')
    .eq('id', user!.id)
    .single()

  if (profile?.role !== 'manager' || profile.managed_restaurant_ids !== null) {
    redirect('/dashboard')
  }

  const pending = await getPendingAccounts()

  return (
    <div className="p-6 lg:p-8 max-w-3xl">
      <div className="mb-6">
        <h1 className="text-xl font-semibold tracking-tight">Account Pendenti</h1>
        <p className="text-sm text-muted-foreground mt-1">
          Gestisci le richieste di accesso a inTurno.
        </p>
      </div>
      <AccountPendentiClient initialAccounts={pending} />
    </div>
  )
}
```

[src/app/\(manager\)/approvazioni/loading.tsx](#)

```
import { PageSkeleton } from '@components/manager/PageSkeleton'
export default function Loading() { return <PageSkeleton /> }
```

```

import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'
import { ApprovazioniClient } from '@components/manager/ApprovazioniClient'
import { FallbackApprovalSection, type PendingItem } from '@components/manager/FallbackApprovalSection'

export default async function ApprovazioniPage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, is_direttore')
    .eq('id', user!.id)
    .single()

  // Riservato a manager e direttori
  if (profile?.role === 'capo_servizio' && profile.is_direttore !== true) redirect('/dashboard')

  const isManager = profile?.role === 'manager'
  const isDirettore = profile?.role === 'capo_servizio' && profile.is_direttore === true
  const canSeeFallback = isManager || isDirettore

  let absencesQuery = supabase
    .from('absences')
    .select('*', profile:profiles!user_id(id, full_name), restaurant:restaurants(id, name))
    .eq('status', 'pending')
    .order('created_at', { ascending: false })

  if (profile?.role === 'capo_servizio' && profile.restaurant_id) {
    absencesQuery = absencesQuery.eq('restaurant_id', profile.restaurant_id)
  }

  // Pending fallback attendances – only for manager / direttore
  let pendingFallback: PendingItem[] = []
  if (canSeeFallback) {
    let pendingQuery = supabase
      .from('attendances')
      .select('id, user_id, check_in, check_out, fallback_photo_path, restaurant_id, profile:profiles(full_name), restaurant:restaurants(name)')
      .eq('needs_manager_approval', true)
      .order('check_in', { ascending: false })

    if (isDirettore && profile?.restaurant_id) {
      pendingQuery = pendingQuery.eq('restaurant_id', profile.restaurant_id)
    }

    const { data } = await pendingQuery
    pendingFallback = (data ?? []) as unknown as PendingItem[]
  }

  const { data: requests, error: requestsError } = await absencesQuery
  if (requestsError) console.error('[approvazioni] query error:', requestsError.message)

  return (
    <div className="p-6 lg:p-8">
      {canSeeFallback && (
        <FallbackApprovalSection initialPending={pendingFallback} />
      )}
      <ApprovazioniClient initialRequests={requests ?? []} />
    </div>
  )
}

```

`src/app/(manager)/assenze/loading.tsx`

```
import { PageSkeleton } from '@components/manager/PageSkeleton'
export default function Loading() { return <PageSkeleton /> }
```

```

import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'
import { AssenzeClient } from '@components/manager/AssenzeClient'
import { formatInTimeZone } from 'date-fns-tz'

const TZ = 'Europe/Rome'

export default async function AssenzePage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, is_direttore')
    .eq('id', user!.id)
    .single()

  // Riservato a manager e direttori
  if (profile?.role === 'capo_servizio' && profile.is_direttore !== true) redirect('/dashboard')

  const { data: restaurants } = await supabase
    .from('restaurants')
    .select('id, name')
    .order('name')

  // Managers can assign absences to capo_servizio too, not just dipendenti
  const { data: dipendenti } = await supabase
    .from('profiles')
    .select('id, full_name, restaurant_id')
    .in('role', ['dipendente', 'capo_servizio'])
    .order('full_name')

  const nowRome = formatInTimeZone(new Date(), TZ, 'yyyy-MM')
  const [year, month] = nowRome.split('-').map(Number)
  const startDate = new Date(Date.UTC(year, month - 1, 1)).toISOString().split('T')[0]
  const endDate = new Date(Date.UTC(year, month, 0)).toISOString().split('T')[0]

  let query = supabase
    .from('absences')
    .select('*', { profile: profiles!user_id(id, full_name), restaurant: restaurants(id, name) })
    .lte('start_date', endDate)
    .gte('end_date', startDate)
    .order('start_date', { ascending: false })

  if (profile?.role === 'capo_servizio' && profile.restaurant_id) {
    query = query.eq('restaurant_id', profile.restaurant_id)
  }

  const { data: absences, error: absencesError } = await query
  if (absencesError) console.error('[assenze] query error:', absencesError.message)

  return (
    <div className="p-6 lg:p-8">
      <AssenzeClient
        initialAbsences={absences ?? []}
        restaurants={restaurants ?? []}
        dipendenti={dipendenti ?? []}
        currentUserRole={profile?.role ?? 'capo_servizio'}
        currentRestaurantId={profile?.restaurant_id ?? null}
        isDirettore={profile?.is_direttore ?? false}
      />
    </div>
  )
}

```

[src/app/\(manager\)/bacheca/loading.tsx](#)

```
import { PageSkeleton } from '@components/manager/PageSkeleton'
export default function Loading() { return <PageSkeleton /> }
```

[src/app/\(manager\)/bacheca/page.tsx](#)

```
import { createClient } from '@lib/supabase/server'
import { BachecaClient } from '@components/manager/BachecaClient'

export default async function BachecaPage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, can_post_bulletin, is_direttore')
    .eq('id', user!.id)
    .single()

  const isDirettore = profile?.role === 'capo_servizio' && profile.is_direttore === true

  let dipendentiQuery = supabase
    .from('profiles')
    .select('id, full_name, role')
    .in('role', ['capo_servizio', 'dipendente'])
    .order('full_name')

  if (isDirettore && profile?.restaurant_id) {
    dipendentiQuery = dipendentiQuery.eq('restaurant_id', profile.restaurant_id)
  }

  const [{ data: restaurants }, { data: bulletins }, { data: dipendenti }] =
    await Promise.all([
      supabase.from('restaurants').select('id, name').order('name'),
      supabase.from('bulletins')
        .select('*', restaurant:restaurants(id, name), author:profiles!created_by(id, full_name))
        .order('created_at', { ascending: false })
        .limit(50),
      dipendentiQuery,
    ])

  const canPost = profile?.role === 'manager' ||
    (profile?.role === 'capo_servizio' && profile.can_post_bulletin)

  return (
    <div className="p-6 lg:p-8">
      <BachecaClient
        initialBulletins={bulletins ?? []}
        restaurants={restaurants ?? []}
        dipendenti={dipendenti ?? []}
        currentUserId={user!.id}
        currentUserRole={profile?.role ?? 'capo_servizio'}
        currentRestaurantId={profile?.restaurant_id ?? null}
        canPost={canPost ?? false}
        isDirettore={isDirettore}
      />
    </div>
  )
}
```

`src/app/(manager)/dashboard/loading.tsx`

```
import { PageSkeleton } from '@components/manager/PageSkeleton'
export default function Loading() { return <PageSkeleton /> }
```



```

import { createClient } from '@lib/supabase/server'
import { createAdminClient } from '@lib/supabase/admin'
import { autoCloseStaleShifts } from '@lib/autoCloseStaleShifts'
import { Users, Clock, CalendarX, CheckCircle, MessageSquare } from 'lucide-react'
import { formatInTimeZone, fromZonedTime } from 'date-fns-tz'
import { it } from 'date-fns/locale'
import Link from 'next/link'
import { CapoServizioTimbraturaSection } from '@components/manager/CapoServizioTimbraturaSection'
import { ConsulenteLavoroManager } from '@components/manager/ConsulenteLavoroManager'
import { FallbackApprovalSection, type PendingItem } from '@components/manager/FallbackApprovalSection'
import { PresenzePreviewClient, type PresenzaPreviewRow } from '@components/manager/PresenzePreviewClient'
import { RestaurantQrCard } from '@components/manager/RestaurantQrCard'
import { TelegramLinkButton } from '@components/manager/TelegramLinkButton'

const TZ = 'Europe/Rome'

export default async function DashboardPage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, is_direttore, restaurant:restaurants(name, latitude, longitude, qr_secret)')
    .eq('id', user!.id)
    .single()

  const isManager = profile?.role === 'manager'
  const isDirettore = profile?.role === 'capo_servizio' && (profile as { is_direttore?: boolean }).is_direttore
  const canSeeFallback = isManager || isDirettore

  // Chiudi le timbrature con uscita dimenticata prima di calcolare i KPI,
  // così "presenti oggi" e l'anteprima presenze non restano gonfiati.
  await autoCloseStaleShifts(createAdminClient())

  const todayRome = formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd')
  const todayStart = fromZonedTime(`${todayRome}T00:00:00`, TZ).toISOString()

  const isCapoServizio = profile?.role === 'capo_servizio'
  const restaurantFilter = isCapoServizio && profile?.restaurant_id
    ? profile.restaurant_id : null

  const baseProfiles = supabase.from('profiles').select('id', { count: 'exact' }).in('role', ['dipendente', 'capo_servizio'])
  const basePresenti = supabase.from('attendances').select('id', { count: 'exact' }).is('check_out', null).gte('check_in', todayStart)
  const baseAssenze = supabase.from('absences').select('id', { count: 'exact' }).eq('status', 'approved').lte('start_date', todayRome).gte('end_date', todayRome)
  const basePending = supabase.from('absences').select('id', { count: 'exact' }).eq('status', 'pending')
  let baseBulletins = supabase.from('bulletins').select('id', { count: 'exact', head: true })
  if (restaurantFilter) {
    baseBulletins = baseBulletins.or(`restaurant_id.eq.${restaurantFilter},restaurant_id.is.null`)
  }

  // Fetch own open attendance for capo_servizio timbratura section
  const [
    { count: totalDipendenti },
    { count: presenti },
    { count: assenzeOggi },
    { count: richiestePending },
    { count: bachecaCount },
    openAttendanceResult,
  ] = await Promise.all([
    restaurantFilter ? baseProfiles.eq('restaurant_id', restaurantFilter) : baseProfiles,
    restaurantFilter ? basePresenti.eq('restaurant_id', restaurantFilter) : basePresenti,
    restaurantFilter ? baseAssenze.eq('restaurant_id', restaurantFilter) : baseAssenze,
    restaurantFilter ? basePending.eq('restaurant_id', restaurantFilter) : basePending,
    baseBulletins,
    isCapoServizio
      ? supabase
        .from('attendances')
        .select('*')
        .eq('user_id', user!.id)

```

```

        .is('check_out', null)
        .order('check_in', { ascending: false })
        .limit(1)
        .maybeSingle()
    : Promise.resolve({ data: null })),
  ])

// Capo Servizio / Direttore: la 4ª card mostra i Messaggi in Bacheca al posto
// delle Richieste in Attesa (riservate ad Assenze/Approvazioni del manager)
const stats = isCapoServizio
  ? [
    { label: 'Dipendenti Totali', value: totalDipendenti ?? 0, icon: Users, color: 'text-blue-600', href: '/dipendenti' },
    { label: 'Presenti Oggi', value: presenti ?? 0, icon: CheckCircle, color: 'text-emerald-600', href: '/dashboard' },
    { label: 'Assenti Oggi', value: assenzeOggi ?? 0, icon: CalendarX, color: 'text-red-600', href: '/dashboard' },
    { label: 'Messaggi in Bacheca', value: bachecaCount ?? 0, icon: MessageSquare, color: 'text-violet-600', href: '/bacheca' },
  ]
  : [
    { label: 'Dipendenti Totali', value: totalDipendenti ?? 0, icon: Users, color: 'text-blue-600', href: '/dipendenti' },
    { label: 'Presenti Oggi', value: presenti ?? 0, icon: CheckCircle, color: 'text-emerald-600', href: '/presenze' },
    { label: 'Assenti Oggi', value: assenzeOggi ?? 0, icon: CalendarX, color: 'text-red-600', href: '/assenze' },
    { label: 'Richieste in Attesa', value: richiestePending ?? 0, icon: Clock, color: 'text-amber-600', href: '/assenze' },
  ]

return (
  <div className="p-6">
    <div className="mb-6">
      <h1 className="text-xl font-semibold tracking-tight">Dashboard</h1>
      <p className="text-muted-foreground text-sm mt-0.5 capitalize">
        {formatInTimeZone(new Date(), TZ, "EEEE d MMMM yyyy", { locale: it })}
      </p>
    </div>

    {/* KPI grid */}
    <div className="grid grid-cols-2 lg:grid-cols-4 gap-3">
      {stats.map(({ label, value, icon: Icon, color, href }) => (
        <Link
          key={label}
          href={href}
          className="bg-card border border-border rounded-md p-4 block
            transition-all duration-200 cursor-pointer
            hover:shadow-md hover:-translate-y-0.5
            hover:bg-zinc-50 dark:hover:bg-zinc-800/50
            focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2"
          >
          <p className="text-xs text-muted-foreground font-medium">{label}</p>
          <div className="flex items-end justify-between mt-2">
            <span className="text-3xl font-bold tracking-tight">{value}</span>
            <Icon className={`w-6 h-6 ${color} opacity-70`} />
          </div>
        </Link>
      ))}
    </div>

    {/* Timbratura – solo capo_servizio */}
    {isCapoServizio && (
      <CapoServizioTimbraturaSection
        initialOpenAttendance={openAttendanceResult.data ?? null}
        restaurantLat={((profile?.restaurant as { latitude?: number | null } | null)?.latitude ?? null)}
        restaurantLng={((profile?.restaurant as { longitude?: number | null } | null)?.longitude ?? null)}
      />
    )}

    {/* Timbrature di emergenza – manager e direttori */}
    {canSeeFallback && (
      <FallbackPendingSection
        isManager={isManager}

```

```

        restaurantId={isDirettore ? (profile?.restaurant_id ?? null) : null}
      />
    ))

    {/** QR Code timbratura – capo servizio e direttori */}
    {isCapoServizio && profile?.restaurant && (
      <RestaurantQrCard
        restaurantName={{(profile.restaurant as unknown as { name: string }).name}}
        qrSecret={{(profile.restaurant as unknown as { qr_secret: string }).qr_secret}}
      />
    )}

    {/** Preview presenze di oggi – capo servizio e direttori. Niente ore
      lavorate; cliccabile solo per il direttore (aggiunge/modifica). */}
    {isCapoServizio && restaurantFilter && (
      <PresenzePreviewSection restaurantId={restaurantFilter} isDirettore={isDirettore} />
    )}

    {/** Consulenti del Lavoro – solo manager */}
    {isManager && (
      <ConsulenteLavoroManagerSection managerId={user!.id} />
    )}

    {/** Bot Telegram – manager e capi servizio/direttori. Mai per
      dipendenti o consulenti (questa pagina non è accessibile a loro). */}
    <TelegramLinkButton />
  </div>
)
}

async function PresenzePreviewSection({
  restaurantId,
  isDirettore,
}: {
  restaurantId: string
  isDirettore: boolean
}) {
  const supabase = await createClient()
  const todayRome = formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd')
  const todayStart = fromZonedTime(`${todayRome}T00:00:00`, TZ).toISOString()

  const [{ data: presenze }, { data: dipendenti }] = await Promise.all([
    supabase
      .from('attendances')
      .select('id, user_id, check_in, check_out, profile:profiles(id, full_name)')
      .eq('restaurant_id', restaurantId)
      .gte('check_in', todayStart)
      .order('check_in', { ascending: false }),
    isDirettore
      ? supabase
          .from('profiles')
          .select('id, full_name')
          .eq('restaurant_id', restaurantId)
          .in('role', ['dipendente', 'capo_servizio'])
          .order('full_name')
      : Promise.resolve({ data: [] }),
  ])

  return (
    <PresenzePreviewClient
      initialRows={{(presenze as PresenzaPreviewRow[]) ?? []}}
      dipendenti={{dipendenti ?? []}}
      isDirettore={isDirettore}
    />
  )
}

async function FallbackPendingSection({
  isManager,
  restaurantId,
}: {
  isManager: boolean
  restaurantId: string | null
}) {

```

```

const supabase = await createClient()
let query = supabase
  .from('attendances')
  .select('id, user_id, check_in, check_out, fallback_photo_path, restaurant_id, profile:profiles(full_name),
restaurant:restaurants(name)')
  .eq('needs_manager_approval', true)
  .order('check_in', { ascending: false })

if (!isManager && restaurantId) {
  query = query.eq('restaurant_id', restaurantId)
}

const { data } = await query
if (!data?.length) return null

return (
  <div className="mt-6">
    <FallbackApprovalSection initialPending={data as unknown as PendingItem[]} />
  </div>
)
}

async function ConsulenteLavoroManagerSection({ managerId }: { managerId: string }) {
  const supabase = await createClient()
  const { data: restaurants } = await supabase
    .from('restaurants')
    .select('id, name')
    .order('name')

  return (
    <ConsulenteLavoroManager
      managerId={managerId}
      restaurants={restaurants ?? []}
    />
  )
}

```

[src/app/\(manager\)/dipendenti/loading.tsx](#)

```

import { PageSkeleton } from '@components/manager/PageSkeleton'
export default function Loading() { return <PageSkeleton /> }

```

```

import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'
import { DipendentiClient } from '@components/manager/DipendentiClient'

export default async function DipendentiPage({
  searchParams,
}): {
  searchParams: Promise<{ restaurant_id?: string }>
}) {
  const { restaurant_id: restaurantIdParam } = await searchParams

  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, is_direttore')
    .eq('id', user!.id)
    .single()

  // Riservato a manager e direttori – il capo servizio non gestisce dipendenti
  if (profile?.role === 'capo_servizio' && profile.is_direttore !== true) redirect('/dashboard')

  const { data: restaurants } = await supabase
    .from('restaurants')
    .select('id, name')
    .order('name')

  let query = supabase
    .from('profiles')
    .select('*', restaurant:restaurants(id, name))
    .order('full_name')

  if (profile?.role === 'capo_servizio' && profile.restaurant_id) {
    // Capo servizio is always locked to own restaurant – URL param ignored
    query = query.eq('restaurant_id', profile.restaurant_id)
  } else if (profile?.role === 'manager' && restaurantIdParam) {
    query = query.eq('restaurant_id', restaurantIdParam)
  }

  const { data: dipendenti } = await query

  return (
    <div className="p-6 lg:p-8">
      <DipendentiClient
        key={restaurantIdParam ?? 'all'}
        initialDipendenti={dipendenti ?? []}
        restaurants={restaurants ?? []}
        currentUserRole={profile?.role ?? 'capo_servizio'}
        currentIsDirettore={profile?.is_direttore ?? false}
        currentRestaurantId={profile?.restaurant_id ?? null}
        currentRestaurantFilter={restaurantIdParam ?? null}
      />
    </div>
  )
}

```

[src/app/\(manager\)/layout.tsx](#)

```
import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'
import { ManagerSidebar } from '@components/manager/ManagerSidebar'
import { DemoBanner } from '@components/manager/DemoBanner'
import { AccountStatusProvider } from '@contexts/AccountStatusContext'

export default async function ManagerLayout({ children }: { children: React.ReactNode }) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) redirect('/login')

  const { data: profile } = await supabase
    .from('profiles')
    .select('*', restaurant:restaurants(id, name))
    .eq('id', user.id)
    .single()

  if (!profile || profile.role === 'dipendente') redirect('/home')
  if (profile.role === 'consulente_lavoro') redirect('/consulente/dashboard')

  const isPending = profile.account_status === 'pending'

  return (
    <AccountStatusProvider isPending={isPending}>
      <div className="flex h-[100dvh] overflow-hidden bg-background">
        <ManagerSidebar profile={profile} />
        <main className="flex-1 h-full overflow-y-auto pt-14 lg:pt-0">
          {isPending && <DemoBanner />}
          {children}
        </main>
      </div>
    </AccountStatusProvider>
  )
}
```

[src/app/\(manager\)/ods/loading.tsx](#)

```
import { Skeleton } from '@components/ui/skeleton'

export default function Loading() {
  return (
    <div className="p-6 lg:p-8">
      <div className="flex items-center justify-between mb-6">
        <Skeleton className="h-7 w-48" />
        <Skeleton className="h-9 w-32 rounded-md" />
      </div>
      <div className="flex gap-2 mb-5 flex-wrap">
        {[96, 80, 88, 104, 80].map((w, i) => (
          <Skeleton key={i} className="h-7 rounded-sm" style={{ width: w }} />
        ))}
      </div>
      <div className="space-y-2">
        {[...Array(6)].map((_, i) => (
          <Skeleton key={i} className="h-16 rounded-sm" />
        ))}
      </div>
    </div>
  )
}
```



```

import { createClient } from '@lib/supabase/server'
import { OdsManagerClient } from '@components/manager/OdsManagerClient'
import { formatInTimeZone, fromZonedTime } from 'date-fns-tz'

const TZ = 'Europe/Rome'

function getOdsCutoff(): string {
  const now = new Date()
  const romeHour = parseInt(formatInTimeZone(now, TZ, 'H'), 10)
  const refDate = romeHour < 4 ? new Date(now.getTime() - 86_400_000) : now
  const cutoffDate = formatInTimeZone(refDate, TZ, 'yyyy-MM-dd')
  return fromZonedTime(`${cutoffDate}T04:00:00`, TZ).toISOString()
}

export default async function OdsPage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, department, restaurant_id, can_post_bulletin, is_direttore')
    .eq('id', user!.id)
    .single()

  const cutoff = getOdsCutoff()

  const isCapo = profile?.role === 'capo_servizio'
  const restaurantFilter = isCapo ? profile?.restaurant_id : null

  let tasksQuery = supabase
    .from('ods_tasks')
    .select('*', assignee:profiles!assigned_to(id, full_name)')
    .order('created_at', { ascending: false })

  // Direttore / capo_servizio: hard-scope to their restaurant
  if (restaurantFilter) {
    tasksQuery = tasksQuery.eq('restaurant_id', restaurantFilter)
  }

  let completionsQuery = supabase
    .from('ods_completions')
    .select('task_id, user_id, completed_at, profile:profiles!user_id(id, full_name)')
    .gte('completed_at', cutoff)

  // Dipendenti for assigned_to dropdown (scoped to restaurant)
  let staffQuery = supabase
    .from('profiles')
    .select('id, full_name, department')
    .in('role', ['dipendente', 'capo_servizio'])
    .order('full_name')

  if (restaurantFilter) {
    staffQuery = staffQuery.eq('restaurant_id', restaurantFilter)
  }

  const [
    { data: rawTasks },
    { data: rawCompletions },
    { data: staff },
    { data: restaurants },
  ] = await Promise.all([
    tasksQuery,
    completionsQuery,
    staffQuery,
    profile?.role === 'manager'
      ? supabase.from('restaurants').select('id, name').order('name')
      : Promise.resolve({ data: [] }),
  ])

  // Scope completions to visible task IDs – prevents capo_servizio from
  // receiving completion records that belong to other restaurants.
  const taskIdSet = new Set((rawTasks ?? []).map(t => (t as { id: string }).id))
  const completions = restaurantFilter

```

```

? (rawCompletions ?? []).filter(c => taskIdSet.has((c as { task_id: string }).task_id))
: rawCompletions

return (
  <div className="p-6 lg:p-8">
    <OdsManagerClient
      initialTasks= {(rawTasks as unknown as import('@types').OdsTask[]) ?? []}
      completions= {(completions ?? []) as unknown as (import('@types').OdsCompletion & { profile?: { id:
string; full_name: string } | null })[] }
      staff= {staff ?? []}
      restaurants= {restaurants ?? []}
      currentUserId= {user!.id}
      currentUserRole= {profile?.role ?? 'capo_servizio'}
      currentDepartment= {profile?.department ?? null}
      currentRestaurantId= {profile?.restaurant_id ?? null}
      currentIsDirettore= {profile?.is_direttore ?? false}
    />
  </div>
)
}

```

[src/app/\(manager\)/presenze/loading.tsx](#)

```

import { PageSkeleton } from '@components/manager/PageSkeleton'
export default function Loading() { return <PageSkeleton /> }

```

```

import { createClient } from '@lib/supabase/server'
import { createAdminClient } from '@lib/supabase/admin'
import { redirect } from 'next/navigation'
import { PresenzeClient, type AbsenceItem } from '@components/manager/PresenzeClient'
import { FallbackApprovalSection } from '@components/manager/FallbackApprovalSection'
import { autoCloseStaleShifts } from '@lib/autoCloseStaleShifts'
import { formatInTimeZone } from 'date-fns-tz'

const TZ = 'Europe/Rome'

export default async function PresenzePage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, is_direttore')
    .eq('id', user!.id)
    .single()

  // La tab "Presenze" è riservata al manager – capo servizio e direttori
  // hanno la preview presenze nella Dashboard.
  if (profile?.role === 'capo_servizio') redirect('/dashboard')

  // Chiudi i turni lasciati aperti (uscita dimenticata) prima di caricare la
  // lista, così il manager non vede mai una timbratura bloccata da ore/giorni.
  await autoCloseStaleShifts(createAdminClient())

  const { data: restaurants } = await supabase
    .from('restaurants')
    .select('id, name')
    .order('name')

  const nowRome = formatInTimeZone(new Date(), TZ, 'yyyy-MM')
  const [year, month] = nowRome.split('-').map(Number)
  const startDate = new Date(Date.UTC(year, month - 1, 1))
  const endDate = new Date(Date.UTC(year, month, 0, 23, 59, 59))

  // Date boundaries for absence range (YYYY-MM-DD strings, no timezone shift needed)
  const monthStart = `${nowRome}-01`
  const monthEnd = `${nowRome}-${String(new Date(Date.UTC(year, month, 0)).getDate()).padStart(2, '0')}`

  let presenzeQuery = supabase
    .from('attendances')
    .select('*', profile:profiles(id, full_name, role), restaurant:restaurants(id, name))
    .gte('check_in', startDate.toISOString())
    .lte('check_in', endDate.toISOString())
    .order('check_in', { ascending: false })

  // Approved absences that overlap with the current month
  let absencesQuery = supabase
    .from('absences')
    .select('id, user_id, restaurant_id, type, start_date, end_date, profile:profiles!user_id(id, full_name)')
    .eq('status', 'approved')
    .lte('start_date', monthEnd)
    .gte('end_date', monthStart)

  if (profile?.role === 'capo_servizio' && profile.restaurant_id) {
    presenzeQuery = presenzeQuery.eq('restaurant_id', profile.restaurant_id)
    absencesQuery = absencesQuery.eq('restaurant_id', profile.restaurant_id)
  }

  const isManager = profile?.role === 'manager'
  const isDirettore = profile?.role === 'capo_servizio' && profile.is_direttore === true
  const canSeeFallback = isManager || isDirettore

  // Fetch pending fallback attendances only for authorised roles
  let pendingFallback: unknown[] = []
  if (canSeeFallback) {
    let pendingQuery = supabase
      .from('attendances')
      .select('id, user_id, check_in, check_out, fallback_photo_path, restaurant_id, profile:profiles(full_name),
restaurant:restaurants(name)')

```

```

        .eq('needs_manager_approval', true)
        .order('check_in', { ascending: false })

    // Direttore is scoped to their own restaurant
    if (isDirettore && profile?.restaurant_id) {
        pendingQuery = pendingQuery.eq('restaurant_id', profile.restaurant_id)
    }

    const { data } = await pendingQuery
    pendingFallback = data ?? []
}

const [
    { data: presenze, error: presenzeError },
    { data: dipendenti },
    { data: absences },
] = await Promise.all([
    presenzeQuery,
    supabase
        .from('profiles')
        .select('id, full_name, role')
        .in('role', ['dipendente', 'capo_servizio'])
        .order('full_name'),
    absencesQuery,
])

if (presenzeError) console.error('[presenze] query error:', presenzeError.message)

return (
    <div className="p-6 lg:p-8">
        {canSeeFallback && (
            <FallbackApprovalSection
                // eslint-disable-next-line @typescript-eslint/no-explicit-any
                initialPending={pendingFallback as any}
            />
        )}
        <PresenzeClient
            initialPresenze={presenze ?? []}
            initialAbsences={{absences ?? []} as unknown as AbsenceItem[]}
            restaurants={restaurants ?? []}
            dipendenti={dipendenti ?? []}
            currentUserRole={profile?.role ?? 'capo_servizio'}
            currentRestaurantId={profile?.restaurant_id ?? null}
            isDirettore={profile?.is_direttore ?? false}
        />
    </div>
)
}

```

[src/app/\(manager\)/report/loading.tsx](#)

```

import { PageSkeleton } from '@components/manager/PageSkeleton'
export default function Loading() { return <PageSkeleton /> }

```

`src/app/(manager)/report/page.tsx`

```
import { createClient } from '@lib/supabase/server'
import { ReportClient } from '@components/manager/ReportClient'

export default async function ReportPage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, is_direttore')
    .eq('id', user!.id)
    .single()

  const { data: restaurants } = await supabase
    .from('restaurants')
    .select('id, name')
    .order('name')

  return (
    <div className="p-6 lg:p-8">
      <ReportClient
        restaurants={restaurants ?? []}
        currentUserRole={profile?.role ?? 'capo_servizio'}
        currentRestaurantId={profile?.restaurant_id ?? null}
        currentUserId={user!.id}
        isDirettore={profile?.is_direttore ?? false}
      />
    </div>
  )
}
```

`src/app/(manager)/ristoranti/loading.tsx`

```
import { PageSkeleton } from '@components/manager/PageSkeleton'
export default function Loading() { return <PageSkeleton /> }
```

`src/app/(manager)/ristoranti/page.tsx`

```
import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'
import { RestaurantsClient } from '@components/manager/RestaurantsClient'

export default async function RistorantiPage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', user!.id)
    .single()

  if (profile?.role !== 'manager') redirect('/dashboard')

  const { data: restaurants } = await supabase
    .from('restaurants')
    .select('*')
    .order('name')

  return (
    <div className="p-6 lg:p-8">
      <RestaurantsClient initialRestaurants={restaurants ?? []} />
    </div>
  )
}
```

`src/app/(manager)/turni/loading.tsx`

```
import { PageSkeleton } from '@components/manager/PageSkeleton'
export default function Loading() { return <PageSkeleton /> }
```

```

import { createClient } from '@lib/supabase/server'
import { TurniManagerClient } from '@components/manager/TurniManagerClient'
import { scopeTurnsQuery, scopeStaffQuery, type ScopeProfile } from '@lib/turniScope'

export default async function TurniPage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, department, restaurant_id, is_direttore')
    .eq('id', user!.id)
    .single()

  const scopeProfile: ScopeProfile = {
    role: profile?.role ?? 'dipendente',
    restaurant_id: profile?.restaurant_id ?? null,
    department: profile?.department ?? null,
    is_direttore: profile?.is_direttore ?? false,
  }

  // — Query Scoping (RBAC) — vedi src/lib/turniScope.ts —————
  let turnsQuery = supabase
    .from('turns')
    .select('*', profile:profiles!user_id(id, full_name), restaurant:restaurants(id, name))
    .order('date', { ascending: true })
    .order('start_time', { ascending: true })
  turnsQuery = scopeTurnsQuery(turnsQuery, scopeProfile, user!.id)

  // Dipendenti assegnabili al turno — stesso scoping dei turni stessi
  let staffQuery = supabase
    .from('profiles')
    .select('id, full_name, department, restaurant_id')
    .in('role', ['dipendente', 'capo_servizio'])
    .order('full_name')
  staffQuery = scopeStaffQuery(staffQuery, scopeProfile)

  // Turni standard (Pattern Master) — stesso scoping dei turni reali
  let standardQuery = supabase
    .from('standard_shifts')
    .select('*', profile:profiles!user_id(id, full_name))
    .order('day_of_week')
  standardQuery = scopeTurnsQuery(standardQuery, scopeProfile, user!.id)

  const [{ data: turns }, { data: staff }, { data: restaurants }, { data: standardShifts }] = await Promise.all([
    turnsQuery,
    staffQuery,
    profile?.role === 'manager'
      ? supabase.from('restaurants').select('id, name').order('name')
      : Promise.resolve({ data: [] }),
    standardQuery,
  ])

  return (
    <div className="p-6 lg:p-8">
      <TurniManagerClient
        initialTurns={{turns as unknown as import('@types').Turn[]}} ?? []
        initialStandardShifts={{standardShifts as unknown as import('@types').StandardShift[]}} ?? []
        staff={staff ?? []}
        restaurants={restaurants ?? []}
        currentUserId={user!.id}
        currentUserRole={profile?.role ?? 'capo_servizio'}
        currentDepartment={profile?.department ?? null}
        currentRestaurantId={profile?.restaurant_id ?? null}
        currentIsDirettore={profile?.is_direttore ?? false}
      />
    </div>
  )
}

```

`src/app/actions/adminActions.ts`

```

'use server'

import { createClient } from '@lib/supabase/server'
import { createAdminClient } from '@lib/supabase/admin'
import { deleteDemoData } from '@lib/demoData'
import { sendAccountActivatedEmail } from '@lib/email'
import { revalidatePath } from 'next/cache'

async function assertPlatformOwner() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) throw new Error('Non autenticato')

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, managed_restaurant_ids')
    .eq('id', user.id)
    .single()

  if (profile?.role !== 'manager' || profile.managed_restaurant_ids !== null) {
    throw new Error('Non autorizzato')
  }
  return { supabase, user }
}

export async function getPendingAccounts() {
  await assertPlatformOwner()
  const admin = createAdminClient()

  const { data, error } = await admin
    .from('profiles')
    .select('id, full_name, username, created_at')
    .eq('role', 'manager')
    .eq('account_status', 'pending')
    .order('created_at', { ascending: false })

  if (error) throw new Error(error.message)
  return data ?? []
}

export async function approveAccount(userId: string): Promise<void> {
  await assertPlatformOwner()
  const admin = createAdminClient()

  // Elimina dati demo
  await deleteDemoData(userId)

  // Attiva account
  const { error } = await admin
    .from('profiles')
    .update({
      account_status: 'active',
      managed_restaurant_ids: [], // vuoto = nessun ristorante, lo crea lui
    })
    .eq('id', userId)

  if (error) throw new Error(error.message)

  // Recupera email per notifica
  const { data: profile } = await admin
    .from('profiles')
    .select('full_name, username')
    .eq('id', userId)
    .single()

  if (profile?.username) {
    const appUrl = process.env.NEXT_PUBLIC_APP_URL ?? 'https://turni.vercel.app'
    sendAccountActivatedEmail({
      fullName: profile.full_name,
      email: profile.username,
      loginUrl: `${appUrl}/login`,
    }).catch(() => {})
  }
}

```

```
    revalidatePath('/account-pendenti')
  }

export async function rejectAccount(userId: string): Promise<void> {
  await assertPlatformOwner()
  const admin = createAdminClient()

  await deleteDemoData(userId)
  await admin.from('profiles').delete().eq('id', userId)
  await admin.auth.admin.deleteUser(userId)

  revalidatePath('/account-pendenti')
}
```

`src/app/actions/aiTurni.ts`

```

'use server'

import { createClient } from '@lib/supabase/server'
import { createClient as createAdminClient } from '@supabase/supabase-js'
import { revalidatePath } from 'next/cache'
import { startOfWeek, addDays, format, getDay, differenceInMinutes, parseISO } from 'date-fns'
import type {
  Department, Profile, Turn, ShiftSlot,
  AiScheduleDraft, AiScheduleDraftTurn,
  AiScheduleWarning, ExtraordinaryClosure, ExistingTurnsMode,
  SecondaryDepartment,
} from '@types'

// — Tipi interni —————

interface EmployeeWithProfile extends Profile {
  // ore settimanali già accumulate nella bozza (aggiornate durante la generazione)
  _weeklyMinutes: number
}

export interface GenerateParams {
  restaurantId: string
  weekStart: string // yyyy-MM-dd (lunedì)
  departmentScope: Department[] | null // null = tutti
  existingTurnsMode: ExistingTurnsMode
  extraordinaryClosures: ExtraordinaryClosure[]
  notes?: string // testo libero dal campo NL
}

interface AttendancePattern {
  userId: string
  isSplitShift: boolean // tipicamente fa lo spezzato
  avgSplitGap: number // minuti di pausa media nello spezzato
  morningStart: string // HH:mm — ora inizio media mattina
  morningEnd: string // HH:mm — ora fine media mattina
  eveningStart: string // HH:mm — ora inizio media sera (se spezzato)
  eveningEnd: string // HH:mm — ora fine media sera
}

interface DeadZone {
  department: Department
  startTime: string // HH:mm
  endTime: string // HH:mm
}

// — Helpers —————

function timeToMinutes(t: string): number {
  const [h, m] = t.split(':').map(Number)
  return h * 60 + m
}

function minutesToTime(m: number): string {
  const h = Math.floor(m / 60) % 24
  const min = m % 60
  return `${String(h).padStart(2, '0')}:${String(min).padStart(2, '0')}`
}

function avg(nums: number[]): number {
  if (!nums.length) return 0
  return Math.round(nums.reduce((a, b) => a + b, 0) / nums.length)
}

// — Autorizzazione —————

async function getCallerAndScope() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) throw new Error('Non autenticato')

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, department, is_direttore, account_status')

```

```

    .eq('id', user.id)
    .single()
    if (!profile) throw new Error('Profilo non trovato')
    if (!['manager', 'capo_servizio'].includes(profile.role)) throw new Error('Non autorizzato')
    if ((profile as { account_status?: string }).account_status === 'pending') {
        throw new Error('Account in attesa di approvazione. La demo è in sola lettura.')
    }
}

return { supabase, user, profile }
}

// — Analisi presenze storiche —————
// Legge le ultime 8 settimane di attendance per apprendere i pattern reali:
// - chi fa spezzato
// - orari tipici mattina/sera
// - fasce "morte" per reparto (ore di bassa presenza aggregata)

async function learnFromAttendance(
    supabase: Awaited<ReturnType<typeof createClient>>,
    restaurantId: string,
    weekStart: string,
): Promise<{ patterns: Map<string, AttendancePattern>; deadZones: DeadZone[] }> {
    const eightWeeksAgo = format(addDays(parseISO(weekStart), -56), 'yyyy-MM-dd')

    const { data: records } = await supabase
        .from('attendances')
        .select('user_id, check_in, check_out, restaurant_id')
        .eq('restaurant_id', restaurantId)
        .gte('check_in', eightWeeksAgo)
        .lt('check_in', weekStart)
        .not('check_out', 'is', null)

    const patterns = new Map<string, AttendancePattern>()
    if (!records?.length) return { patterns, deadZones: [] }

    // Raggruppa per (user_id, data)
    const byUserDay = new Map<string, typeof records>()
    for (const r of records) {
        const day = r.check_in.slice(0, 10)
        const key = `${r.user_id}|${day}`
        if (!byUserDay.has(key)) byUserDay.set(key, [])
        byUserDay.get(key)!.push(r)
    }

    // Per ogni utente, calcola se fa spezzato e gli orari medi
    const userStats = new Map<string, {
        splitCount: number; totalCount: number
        morningStarts: number[]; morningEnds: number[]
        eveningStarts: number[]; eveningEnds: number[]
        splitGaps: number[]
    }>()

    for (const [key, dayRecords] of byUserDay) {
        const userId = key.split('|')[0]
        if (!userStats.has(userId)) {
            userStats.set(userId, {
                splitCount: 0, totalCount: 0,
                morningStarts: [], morningEnds: [],
                eveningStarts: [], eveningEnds: [],
                splitGaps: [],
            })
        }
    }
    const stats = userStats.get(userId)!
    stats.totalCount++

    const sorted = dayRecords.sort((a, b) => a.check_in.localeCompare(b.check_in))
    const checkIn = sorted[0].check_in
    const checkOut = sorted[sorted.length - 1].check_out!

    const startMin = timeToMinutes(checkIn.slice(11, 16))
    const endMin = timeToMinutes(checkOut.slice(11, 16))

    if (sorted.length >= 2) {
        // Spezzato: due sessioni nella stessa giornata
    }
}

```



```

    stats.splitCount++
    const midEnd = timeToMinutes(sorted[0].check_out!.slice(11, 16))
    const midStart = timeToMinutes(sorted[1].check_in.slice(11, 16))
    stats.splitGaps.push(midStart - midEnd)
    stats.morningStarts.push(startMin)
    stats.morningEnds.push(midEnd)
    stats.eveningStarts.push(midStart)
    stats.eveningEnds.push(endMin)
  } else {
    stats.morningStarts.push(startMin)
    stats.morningEnds.push(endMin)
  }
}

for (const [userId, stats] of userStats) {
  const splitRatio = stats.splitCount / (stats.totalCount || 1)
  const isSplit = splitRatio >= 0.5 // spezzato nel ≥50% dei casi → pattern

  patterns.set(userId, {
    userId,
    isSplitShift: isSplit,
    avgSplitGap: avg(stats.splitGaps),
    morningStart: minutesToTime(avg(stats.morningStarts)),
    morningEnd: minutesToTime(avg(stats.morningEnds)),
    eveningStart: minutesToTime(avg(stats.eveningStarts.length ? stats.eveningStarts : stats.morningStarts)),
    eveningEnd: minutesToTime(avg(stats.eveningEnds.length ? stats.eveningEnds : stats.morningEnds)),
  })
}

// Calcola "dead zones" per reparto:
// le fasce orarie in cui la presenza aggregata scende sotto il 30% del picco
const SLOT_MIN = 30 // granularità 30 minuti
const slotCounts: Record<string, number[]> = {} // dept → array[48] (slots da 00:00 a 23:30)

const { data: profiles } = await supabase
  .from('profiles')
  .select('id, department')
  .eq('restaurant_id', restaurantId)

const userDept = new Map(profiles?.map(p => [p.id, p.department]) ?? [])

for (const r of records) {
  const dept = userDept.get(r.user_id)
  if (!dept || !r.check_out) continue
  if (!slotCounts[dept]) slotCounts[dept] = new Array(48).fill(0)

  const startMin = timeToMinutes(r.check_in.slice(11, 16))
  const endMin = timeToMinutes(r.check_out.slice(11, 16))

  for (let s = Math.floor(startMin / SLOT_MIN); s < Math.min(48, Math.ceil(endMin / SLOT_MIN)); s++) {
    slotCounts[dept][s]++
  }
}

const deadZones: DeadZone[] = []
for (const [dept, counts] of Object.entries(slotCounts)) {
  const peak = Math.max(...counts)
  if (peak < 3) continue // troppo pochi dati

  let inDead = false
  let deadStart = 0
  for (let s = 0; s < 48; s++) {
    const isDead = counts[s] < peak * 0.3
    if (isDead && !inDead) { inDead = true; deadStart = s }
    if (!isDead && inDead) {
      inDead = false
      const durationSlots = s - deadStart
      if (durationSlots >= 3) { // almeno 90 min di pausa significativa
        deadZones.push({
          department: dept as Department,
          startTime: minutesToTime(deadStart * SLOT_MIN),
          endTime: minutesToTime(s * SLOT_MIN),
        })
      }
    }
  }
}

```

```

    }
  }
}

return { patterns, deadZones }
}

// — Algoritmo principale —————

interface GenerationResult {
  draft: Omit<AiScheduleDraft, 'id' | 'created_at' | 'updated_at'>
  turns: Omit<AiScheduleDraftTurn, 'id' | 'draft_id' | 'created_at'>[]
  warnings: AiScheduleWarning[]
}

function buildSchedule(params: {
  weekDays: Date[]
  employees: EmployeeWithProfile[]
  slots: ShiftSlot[]
  absences: Array<{ user_id: string; start_date: string; end_date: string }>
  existingTurns: Turn[]
  existingTurnsMode: ExistingTurnsMode
  closingDays: number[]
  extraordinaryClosures: ExtraordinaryClosure[]
  departmentScope: Department[] | null
  patterns: Map<string, AttendancePattern>
  deadZones: DeadZone[]
}): GenerationResult['turns'] & { warnings: AiScheduleWarning[] } {
  const {
    weekDays, employees, slots, absences, existingTurns,
    existingTurnsMode, closingDays, extraordinaryClosures, departmentScope,
    patterns, deadZones,
  } = params

  const result: GenerationResult['turns'] = []
  const warnings: AiScheduleWarning[] = []

  // Indici rapidi
  const absenceSet = new Set(
    absences.map(a => {
      const start = a.start_date; const end = a.end_date
      const dates: string[] = []
      let cur = new Date(start + 'T00:00:00')
      const endD = new Date(end + 'T00:00:00')
      while (cur <= endD) {
        dates.push(format(cur, 'yyyy-MM-dd'))
        cur = addDays(cur, 1)
      }
      return dates.map(d => `${a.user_id}|${d}`)
    }).flat()
  )

  const existingKey = new Set(existingTurns.map(t => `${t.user_id}|${t.date}`))

  // Ore lavorate questa settimana per dipendente (aggiornate in tempo reale)
  const weeklyMinutes: Record<string, number> = {}
  for (const emp of employees) weeklyMinutes[emp.id] = 0

  // Dipendenti assegnati per giorno (evita doppi turni nello stesso slot)
  const assignedToday: Record<string, Set<string>> = {} // dateStr → Set<userId>

  // Dipendenti con ruolo di senior (can_substitute_capo_servizio)
  const isSenior = new Set(employees.filter(e => e.can_substitute_capo_servizio).map(e => e.id))

  // Reparti da generare
  const depts = departmentScope ?? (['Sala', 'Pizzeria', 'Bar', 'Cucina'] as Department[])

  for (const day of weekDays) {
    const dateStr = format(day, 'yyyy-MM-dd')
    const dow = getDay(day) // 0=Dom..6=Sab

    if (!assignedToday[dateStr]) assignedToday[dateStr] = new Set()

    // Giorno di chiusura ordinaria
    if (closingDays.includes(dow)) continue
  }
}

```

```

for (const dept of depts) {
  // Chiusura straordinaria per questo giorno/reparto
  const isClosed = extraordinaryClosures.some(
    c => c.date === dateStr && (!c.department || c.department === dept)
  )
  if (isClosed) continue

  // Fasce del reparto valide per questo giorno
  const daySlots = slots.filter(
    s => s.department === dept &&
    (s.days_of_week.length === 0 || s.days_of_week.includes(dow))
  )
  if (!daySlots.length) continue

  // Dipendenti primari del reparto disponibili oggi
  const primaryAvail = employees.filter(e =>
    e.department === dept &&
    !absenceSet.has(`${e.id}|${dateStr}`) &&
    (existingTurnsMode === 'replace' || !existingKey.has(`${e.id}|${dateStr}`)) &&
    !assignedToday[dateStr].has(e.id)
  )

  // Dead zone per questo reparto oggi
  const dz = deadZones.find(z => z.department === dept)

  for (const slot of daySlots) {
    // Evita slot nelle dead zone (< 2 richiesti = può restare con 1 persona)
    const slotStart = timeToMinutes(slot.start_time)
    // Per turni notturni (fine < inizio) aggiungiamo 24h così la durata è positiva
    let slotEnd = timeToMinutes(slot.end_time)
    if (slotEnd <= slotStart) slotEnd += 24 * 60
    const dzStart = dz ? timeToMinutes(dz.startTime) : 0
    const dzEnd = dz ? timeToMinutes(dz.endTime) : 0
    const inDeadZone = dz && slotStart >= dzStart && slotEnd <= dzEnd

    let needed = inDeadZone ? Math.min(1, slot.required_count) : slot.required_count
    const assignedThisSlot: string[] = []

    // — Step 1: dipendenti primari (min ore prima) —
    const candidates = [...primaryAvail]
      .filter(e => !assignedToday[dateStr].has(e.id))
      .sort((a, b) => (weeklyMinutes[a.id] ?? 0) - (weeklyMinutes[b.id] ?? 0))

    // Almeno 1 senior deve essere presente (se il reparto lo richiede)
    // Verifica se tra i candidati c'è già un senior
    const hasSeniorCandidate = candidates.some(c => isSenior.has(c.id))

    for (const emp of candidates) {
      if (needed <= 0) break
      // Rispetta ore target part-time
      const targetMin = emp.weekly_hours_target ? emp.weekly_hours_target * 60 : Infinity
      const slotMin = slotEnd - slotStart
      if (weeklyMinutes[emp.id] + slotMin > targetMin + 30) continue // +30 min tolleranza

      assignedThisSlot.push(emp.id)
      assignedToday[dateStr].add(emp.id)
      weeklyMinutes[emp.id] = (weeklyMinutes[emp.id] ?? 0) + slotMin

      const pattern = patterns.get(emp.id)
      if (pattern?.isSplitShift && !inDeadZone && slotStart < 720) {
        // Spezzato mattina
        result.push({
          user_id: emp.id, department: dept, date: dateStr,
          start_time: pattern.morningStart, end_time: pattern.morningEnd,
          is_rest_day: false, is_extraordinary: false,
          is_cross_dept: false, original_department: null,
          warning: null, status: 'pending',
        })
      }
      if (pattern?.eveningStart) {
        result.push({
          user_id: emp.id, department: dept, date: dateStr,
          start_time: pattern.eveningStart, end_time: pattern.eveningEnd,
          is_rest_day: false, is_extraordinary: false,

```

```

        is_cross_dept: false, original_department: null,
        warning: null, status: 'pending',
      })
    }
  } else {
    result.push({
      user_id: emp.id, department: dept, date: dateStr,
      start_time: slot.start_time, end_time: slot.end_time,
      is_rest_day: false, is_extraordinary: false,
      is_cross_dept: false, original_department: null,
      warning: null, status: 'pending',
    })
  }
  needed--
}

// — Step 2: jolly (secondary_departments) —
// Un dipendente è jolly per questo slot se ha il suo slot_id nei secondary_departments
if (needed > 0) {
  const jolly = employees.filter(e =>
    e.department !== dept &&
    (e.secondary_departments ?? []).some((sd: SecondaryDepartment) => sd.slot_id === slot.id) &&
    !absenceSet.has(`${e.id}|${dateStr}`) &&
    !assignedToday[dateStr].has(e.id) &&
    (existingTurnsMode === 'replace' || !existingKey.has(`${e.id}|${dateStr}`))
  ).sort((a, b) => {
    const pa = (a.secondary_departments ?? []).find((sd: SecondaryDepartment) => sd.slot_id ===
slot.id)?.priority ?? 99
    const pb = (b.secondary_departments ?? []).find((sd: SecondaryDepartment) => sd.slot_id ===
slot.id)?.priority ?? 99
    return pa - pb || (weeklyMinutes[a.id] ?? 0) - (weeklyMinutes[b.id] ?? 0)
  })

  for (const emp of jolly) {
    if (needed <= 0) break
    assignedThisSlot.push(emp.id)
    assignedToday[dateStr].add(emp.id)
    const slotMin = slotEnd - slotStart
    weeklyMinutes[emp.id] = (weeklyMinutes[emp.id] ?? 0) + slotMin

    result.push({
      user_id: emp.id, department: dept, date: dateStr,
      start_time: slot.start_time, end_time: slot.end_time,
      is_rest_day: false, is_extraordinary: false,
      is_cross_dept: true, original_department: emp.department,
      warning: null, status: 'pending',
    })
    needed--
  }
}

// — Step 3: straordinario (proponi al meno carico già assegnato) —
if (needed > 0) {
  const alreadyInSlot = result.filter(
    t => t.date === dateStr && t.department === dept && !t.is_rest_day
  )
  const extraCandidates = employees
    .filter(e => alreadyInSlot.some(t => t.user_id === e.id))
    .sort((a, b) => (weeklyMinutes[a.id] ?? 0) - (weeklyMinutes[b.id] ?? 0))

  if (extraCandidates.length > 0) {
    const emp = extraCandidates[0]
    result.push({
      user_id: emp.id, department: dept, date: dateStr,
      start_time: slot.start_time, end_time: slot.end_time,
      is_rest_day: false, is_extraordinary: true,
      is_cross_dept: false, original_department: null,
      warning: `Straordinario proposto: copertura mancante (${needed} pers.)`,
      status: 'pending',
    })
    needed--
  }
}
}

```

```

// — Verifica senior obbligatorio —————
// Se nessun assegnato è senior e il reparto ha personale che non può stare da solo
const anyoneAssigned = assignedThisSlot.length > 0
const seniorPresent = assignedThisSlot.some(id => isSenior.has(id))
if (anyoneAssigned && !seniorPresent && hasSeniorCandidate) {
  warnings.push({
    day: dateStr, department: dept, slot_name: slot.name,
    message: `Nessun senior presente — ${slot.name} ${dept}: i presenti potrebbero non poter stare da
    soli`,
  })
}

// — Gap residuo —————
if (needed > 0) {
  warnings.push({
    day: dateStr, department: dept, slot_name: slot.name,
    message: `Copertura insufficiente: mancano ${needed} ${needed === 1 ? 'persona' : 'persone'}`,
    missing_count: needed,
  })
}
}
}
}

// — Riposi —————
// Assegna giorni di riposo preferendo lun-ven (regola "no riposi weekend" = ultima spiaggia)
const WEEKDAY_PREFERENCE = [1, 2, 3, 4, 5, 6, 0] // Lun-Ven, poi Sab, poi Dom

for (const emp of employees) {
  if (emp.department && !depts.includes(emp.department)) continue

  const alreadyRestDays = result.filter(t => t.user_id === emp.id && t.is_rest_day).length
  const neededRest = emp.weekly_rest_days - alreadyRestDays
  if (neededRest <= 0) continue

  // Conta solo i giorni lavorativi (non chiusure)
  const workableDays = weekDays.filter(d => !closingDays.includes(getDay(d)))
  const workedDays = result.filter(
    t => t.user_id === emp.id && !t.is_rest_day
  ).map(t => t.date)

  const restCandidates = WEEKDAY_PREFERENCE
    .map(dow => workableDays.filter(d => getDay(d) === dow))
    .flat()
    .filter(d => {
      const ds = format(d, 'yyyy-MM-dd')
      return !absenceSet.has(`${emp.id}|${ds}`) &&
        !workedDays.includes(ds) &&
        !result.some(t => t.user_id === emp.id && t.date === ds && t.is_rest_day)
    })

  // Preferisce il giorno configurato nel profilo
  if (emp.preferred_rest_day !== null) {
    const preferred = restCandidates.filter(d => getDay(d) === emp.preferred_rest_day)
    const others = restCandidates.filter(d => getDay(d) !== emp.preferred_rest_day)
    restCandidates.splice(0, restCandidates.length, ...preferred, ...others)
  }

  const toAssign = restCandidates.slice(0, neededRest)
  for (const d of toAssign) {
    result.push({
      user_id: emp.id, department: emp.department, date: format(d, 'yyyy-MM-dd'),
      start_time: '00:00', end_time: '00:00',
      is_rest_day: true, is_extraordinary: false,
      is_cross_dept: false, original_department: null,
      warning: null, status: 'pending',
    })
  }
}

return Object.assign(result, { warnings })
}

// — Server Actions —————

```

```

export async function generateAiSchedule(params: GenerateParams): Promise<AiScheduleDraft & { turns:
AiScheduleDraftTurn[] }> {
  const { supabase, user, profile } = await getCallerAndScope()

  // RBAC: capo_servizio può generare solo per il proprio ristorante (e reparto se non direttore)
  if (profile.role === 'capo_servizio') {
    if (params.restaurantId !== profile.restaurant_id) throw new Error('Non autorizzato')
    if (!profile.is_direttore && params.departmentScope) {
      const allowed = params.departmentScope.filter(d => d === profile.department)
      if (!allowed.length) throw new Error('Non autorizzato: reparto non di competenza')
      params.departmentScope = allowed
    }
  }

  const weekStart = params.weekStart
  const weekDays = Array.from({ length: 7 }, (_, i) => addDays(parseISO(weekStart + 'T00:00:00'), i))
  const weekEnd = format(weekDays[6], 'yyyy-MM-dd')

  // Fetch parallelo di tutto il necessario
  const [
    restaurantRes, slotsRes, employeesRes, absencesRes, existingTurnsRes,
  ] = await Promise.all([
    supabase.from('restaurants').select('closing_days').eq('id', params.restaurantId).single(),
    supabase.from('shift_slots').select('*').eq('restaurant_id', params.restaurantId),
    supabase.from('profiles').select('*').eq('restaurant_id', params.restaurantId)
      .in('role', ['dipendente', 'capo_servizio'])
      .neq('role', 'consulente_lavoro'),
    supabase.from('absences').select('user_id, start_date, end_date')
      .eq('restaurant_id', params.restaurantId)
      .eq('status', 'approved')
      .lte('start_date', weekEnd)
      .gte('end_date', weekStart),
    params.existingTurnsMode === 'integrate'
      ? supabase.from('turns').select('*')
        .eq('restaurant_id', params.restaurantId)
        .gte('date', weekStart)
        .lte('date', weekEnd)
      : Promise.resolve({ data: [] })),
  ])

  const closingDays: number[] = restaurantRes.data?.closing_days ?? []
  const slots: ShiftSlot[] = (slotsRes.data ?? []) as ShiftSlot[]
  const employees: EmployeeWithProfile[] = ((employeesRes.data ?? []) as Profile[]).map(e => ({
    ...e,
    secondary_departments: (e.secondary_departments ?? []) as unknown as import('@types').SecondaryDepartment[],
    _weeklyMinutes: 0,
  })))
  const absences = absencesRes.data ?? []
  const existingTurns: Turn[] = (existingTurnsRes.data ?? []) as Turn[]

  // Apprendi dai dati storici di presenza
  const { patterns, deadZones } = await learnFromAttendance(supabase, params.restaurantId, weekStart)

  // Genera la bozza
  const generated = buildSchedule({
    weekDays, employees, slots, absences, existingTurns,
    existingTurnsMode: params.existingTurnsMode,
    closingDays,
    extraordinaryClosures: params.extraordinaryClosures,
    departmentScope: params.departmentScope,
    patterns, deadZones,
  })

  const warnings: AiScheduleWarning[] = generated.warnings ?? []

  // Salva bozza nel DB
  const { data: draft, error: draftError } = await supabase
    .from('ai_schedule_drafts')
    .insert({
      restaurant_id: params.restaurantId,
      week_start: weekStart,
      status: 'draft',
      department_scope: params.departmentScope,
    })

```

```

        generated_by:      user.id,
        generation_params: { existingTurnsMode: params.existingTurnsMode, notes: params.notes },
        extraordinary_closures: params.extraordinaryClosures,
        existing_turns_mode: params.existingTurnsMode,
        warnings,
    })
    .select()
    .single()

    if (draftError || !draft) throw new Error(draftError?.message ?? 'Errore creazione bozza')

    // Salva i turni della bozza
    const turnRows = generated.map(t => ({ ...t, draft_id: draft.id }))
    const { data: turns, error: turnsError } = await supabase
        .from('ai_schedule_draft_turns')
        .insert(turnRows)
        .select('*', profile:profiles!user_id(id, full_name)')

    if (turnsError) throw new Error(turnsError.message)

    return { ...draft, turns: turns as unknown as AiScheduleDraftTurn[] }
}

export async function updateDraftTurn(
    turnId: string,
    update: Partial<Pick<AiScheduleDraftTurn, 'start_time' | 'end_time' | 'is_rest_day' | 'is_extraordinary' | 'status'>>,
): Promise<void> {
    const { supabase } = await getCallerAndScope()
    const { error } = await supabase
        .from('ai_schedule_draft_turns')
        .update({ ...update, status: 'modified' })
        .eq('id', turnId)
    if (error) throw new Error(error.message)
}

export async function rejectDraftTurn(turnId: string): Promise<void> {
    const { supabase } = await getCallerAndScope()
    const { error } = await supabase
        .from('ai_schedule_draft_turns')
        .update({ status: 'rejected' })
        .eq('id', turnId)
    if (error) throw new Error(error.message)
}

export async function confirmAiDraft(draftId: string): Promise<{ created: number }> {
    const { supabase, user, profile } = await getCallerAndScope()

    // Recupera bozza + turni non rifiutati
    const { data: draft } = await supabase
        .from('ai_schedule_drafts')
        .select('*', turns:ai_schedule_draft_turns(*)')
        .eq('id', draftId)
        .single()

    if (!draft) throw new Error('Bozza non trovata')

    // RBAC
    if (profile.role === 'capo_servizio' && draft.restaurant_id !== profile.restaurant_id) {
        throw new Error('Non autorizzato')
    }

    // Se modalità 'replace': elimina i turni esistenti nella settimana/scope
    if (draft.existing_turns_mode === 'replace') {
        let delQuery = supabase
            .from('turns')
            .delete()
            .eq('restaurant_id', draft.restaurant_id)
            .gte('date', draft.week_start)
            .lte('date', format(addDays(parseISO(draft.week_start + 'T00:00:00'), 6), 'yyyy-MM-dd'))

        if (draft.department_scope?.length) {
            delQuery = delQuery.in('department', draft.department_scope)
        }
    }
}

```

```

    await delQuery
  }

  // Inserisci solo i turni non rifiutati
  const validTurns = (draft.turns as AiScheduleDraftTurn[]).filter(t => t.status !== 'rejected')

  if (!validTurns.length) {
    await supabase.from('ai_schedule_drafts').update({ status: 'confirmed' }).eq('id', draftId)
    return { created: 0 }
  }

  // Usa admin client per bypassare RLS sui jolly cross-dept
  const adminClient = createAdminClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!,
  )

  const rows = validTurns.map(t => ({
    user_id: t.user_id,
    restaurant_id: draft.restaurant_id,
    department: t.department,
    date: t.date,
    start_time: t.start_time,
    end_time: t.end_time,
    is_extraordinary: t.is_extraordinary,
    is_rest_day: t.is_rest_day,
    notes: t.is_cross_dept ? `Jolly da ${t.original_department}` : (t.warning ?? null),
    created_by: user.id,
  }))

  const { error } = await adminClient.from('turns').insert(rows)
  if (error) throw new Error(error.message)

  // Marca bozza confermata
  await supabase.from('ai_schedule_drafts').update({ status: 'confirmed' }).eq('id', draftId)

  // Push notification a tutti i dipendenti coinvolti
  const userIds = [...new Set(validTurns.map(t => t.user_id))]
  const { data: subs } = await supabase
    .from('push_subscriptions')
    .select('*')
    .in('user_id', userIds)

  if (subs?.length) {
    const weekLabel = format(parseISO(draft.week_start + 'T00:00:00'), "d MMM")
    const endLabel = format(addDays(parseISO(draft.week_start + 'T00:00:00'), 6), "d MMM yyyy")

    // Fire-and-forget tramite API push interna
    await fetch(`${process.env.NEXT_PUBLIC_APP_URL ?? ''}/api/push/send`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        user_ids: userIds,
        title: 'Nuovi turni pubblicati',
        body: `I turni della settimana ${weekLabel}-${endLabel} sono stati pubblicati.`,
        url: '/home/miei-turni',
      }),
    }).catch(() => { /* non bloccante */ })
  }

  revalidatePath('/turni')
  return { created: rows.length }
}

export async function discardAiDraft(draftId: string): Promise<void> {
  const { supabase, profile } = await getCallerAndScope()
  const { data: draft } = await supabase
    .from('ai_schedule_drafts')
    .select('restaurant_id')
    .eq('id', draftId)
    .single()
  if (!draft) throw new Error('Bozza non trovata')
  if (profile.role === 'capo_servizio' && draft.restaurant_id !== profile.restaurant_id) {
    throw new Error('Non autorizzato')
  }
}

```



```

    }
    await supabase.from('ai_schedule_drafts').update({ status: 'cancelled' }).eq('id', draftId)
  }

export async function checkExistingTurns(restaurantId: string, weekStart: string, departmentScope: Department[] |
null): Promise<number> {
  const { supabase } = await getCallerAndScope()
  const weekEnd = format(addDays(parseISO(weekStart + 'T00:00:00'), 6), 'yyyy-MM-dd')

  let q = supabase
    .from('turns')
    .select('id', { count: 'exact', head: true })
    .eq('restaurant_id', restaurantId)
    .gte('date', weekStart)
    .lte('date', weekEnd)

  if (departmentScope?.length) q = q.in('department', departmentScope)
  const { count } = await q
  return count ?? 0
}

```



```

'use server'

import { createClient } from '@lib/supabase/server'
import { createAdminClient } from '@lib/supabase/admin'
import webpush from 'web-push'
import type { OdsTask, OdsTaskType } from '@types'

webpush.setVapidDetails(
  process.env.VAPID_EMAIL!,
  process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY!,
  process.env.VAPID_PRIVATE_KEY!,
)

export interface CreateOdsInput {
  title: string
  department: string
  restaurant_id: string
  type: OdsTaskType
  recurrence_days: string[]
  assigned_to: string | null
}

export async function createOdsTask(input: CreateOdsInput): Promise<OdsTask> {
  // — 1. Auth —————
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) throw new Error('Non autenticato')

  const { data: caller } = await supabase
    .from('profiles')
    .select('role, restaurant_id, department, is_direttore, account_status')
    .eq('id', user.id)
    .single()

  if (!caller || ![ 'manager', 'capo_servizio' ].includes(caller.role)) {
    throw new Error('Non autorizzato')
  }

  if ((caller as { account_status?: string }).account_status === 'pending') {
    throw new Error('Account in attesa di approvazione. La demo è in sola lettura.')
  }

  // — 2. Capo servizio security: must own the restaurant + department —
  // A "Direttore" (capo_servizio with is_direttore) bypasses the department
  // restriction and operates across all departments of its own restaurant.
  if (caller.role === 'capo_servizio') {
    if (input.restaurant_id !== caller.restaurant_id) {
      throw new Error('Non autorizzato: ristorante non corrispondente')
    }
    if (!caller.is_direttore && caller.department && input.department !== caller.department) {
      throw new Error('Non autorizzato: reparto non corrispondente')
    }
  }

  const admin = createAdminClient()

  // — 3. Create task —————
  const { data: task, error: taskError } = await admin
    .from('ods_tasks')
    .insert({
      title: input.title,
      department: input.department,
      restaurant_id: input.restaurant_id,
      type: input.type,
      recurrence_days: input.recurrence_days,
      assigned_to: input.assigned_to,
      creator_id: user.id,
    })
    .select('*', assignee:profiles!assigned_to(id, full_name)')
    .single()

  if (taskError || !task) throw new Error('Errore nella creazione del compito')

  // — 4. Identify recipients (excluding the creator) —————

```

```

type Recipient = { id: string; role: string }
let recipients: Recipient[] = []

if (input.assigned_to) {
  const { data: assignee } = await admin
    .from('profiles')
    .select('id, role')
    .eq('id', input.assigned_to)
    .single()
  if (assignee) recipients = [assignee]
} else {
  const { data: staff } = await admin
    .from('profiles')
    .select('id, role')
    .eq('restaurant_id', input.restaurant_id)
    .eq('department', input.department)
    .in('role', ['dipendente', 'capo_servizio'])
    .neq('id', user.id)
  recipients = (staff ?? []) as Recipient[]
}

if (recipients.length === 0) return task as unknown as OdsTask

const isPersonal = !!input.assigned_to
const notifTitle = isPersonal ? 'Nuova mansione assegnata' : 'Nuova istruzione di Reparto'
const recipientIds = recipients.map(r => r.id)

// — 5+6. Parallel: bulk insert in-app notifications + fetch push subscriptions —
const [, { data: subs }] = await Promise.all([
  admin.from('notifications').insert(
    recipients.map(r => ({
      user_id: r.id,
      title: notifTitle,
      message: input.title,
      // dipendente → /home/ods, manager-area users → /ods
      link: r.role === 'dipendente' ? '/home/ods' : '/ods',
    })))
  ),
  admin
    .from('push_subscriptions')
    .select('endpoint, p256dh, auth_key, user_id')
    .in('user_id', recipientIds),
])

if (subs?.length) {
  await Promise.allSettled(
    subs.map(async (sub: { endpoint: string; p256dh: string; auth_key: string }) => {
      const recipient = recipients.find(r => r.id === (sub as { user_id?: string }).user_id)
      const url = recipient?.role === 'dipendente' ? '/home/ods' : '/ods'
      try {
        await webpush.sendNotification(
          { endpoint: sub.endpoint, keys: { p256dh: sub.p256dh, auth: sub.auth_key } },
          JSON.stringify({ title: notifTitle, body: input.title, url })
        )
      } catch (err: unknown) {
        const status = (err as { statusCode?: number }).statusCode
        if (status === 410 || status === 404) {
          await admin.from('push_subscriptions').delete().eq('endpoint', sub.endpoint)
        }
      }
    })
  )
}

return task as unknown as OdsTask
}

```

`src/app/actions/telegram.ts`

```

'use server'

import { randomInt } from 'crypto'
import { createClient } from '@lib/supabase/server'
import { getMe } from '@lib/telegram/api'
import type { Role } from '@types'

// Ruoli che NON possono mai collegare Telegram (vincolo di sicurezza,
// applicato anche qui in difesa rispetto al solo nascondere il bottone in UI).
const FORBIDDEN_ROLES: Role[] = ['dipendente', 'consulente_lavoro']

const PIN_TTL_MS = 5 * 60 * 1000

async function getCaller() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) throw new Error('Non autenticato')

  const { data: profile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', user.id)
    .single()
  if (!profile) throw new Error('Profilo non trovato')

  if (FORBIDDEN_ROLES.includes(profile.role as Role)) {
    throw new Error('Non autorizzato a collegare Telegram')
  }

  return { supabase, user }
}

export interface TelegramLinkInfo {
  linked: boolean
  username: string | null
  deepLink: string | null
  pin: string
}

export async function generateTelegramLink(): Promise<TelegramLinkInfo> {
  const { supabase, user } = await getCaller()

  const pin = randomInt(100000, 1000000).toString()
  const expiresAt = new Date(Date.now() + PIN_TTL_MS).toISOString()

  const { error } = await supabase.from('telegram_link_pins').insert({
    pin,
    user_id: user.id,
    expires_at: expiresAt,
  })
  if (error) throw new Error(error.message)

  const me = await getMe()
  const deepLink = `https://t.me/${me.username}?start=${pin}`

  return { linked: false, username: me.username ?? null, deepLink, pin }
}

export async function getTelegramLinkStatus(): Promise<{ linked: boolean; telegramUsername: string | null }> {
  const { supabase, user } = await getCaller()

  const { data } = await supabase
    .from('telegram_links')
    .select('telegram_username')
    .eq('user_id', user.id)
    .maybeSingle()

  return { linked: !!data, telegramUsername: data?.telegram_username ?? null }
}

export async function unlinkTelegram(): Promise<void> {
  const { supabase, user } = await getCaller()

```

```
const { error } = await supabase.from('telegram_links').delete().eq('user_id', user.id)
if (error) throw new Error(error.message)
}
```

`src/app/actions/turni.ts`


```

'use server'

import { createClient } from '@lib/supabase/server'
import { revalidatePath } from 'next/cache'
import { eachDayOfInterval, getDay, format } from 'date-fns'
import type { Department, Turn } from '@types'

export interface TurnInput {
  user_id: string
  restaurant_id: string
  department: Department | null
  date: string
  start_time: string
  end_time: string
  is_extraordinary: boolean
  is_rest_day?: boolean
  notes?: string | null
}

export interface BulkTurnInput {
  user_id: string
  restaurant_id: string
  department: Department | null
  start_date: string
  end_date: string
  days_of_week: number[] // 0=Dom .. 6=Sab (date-fns getDay)
  start_time: string
  end_time: string
  is_extraordinary: boolean
  is_rest_day?: boolean
  notes?: string | null
}

export interface StandardShiftInput {
  user_id: string
  restaurant_id: string
  department: Department | null
  day_of_week: number
  start_time: string
  end_time: string
}

interface CallerProfile {
  role: string
  restaurant_id: string | null
  department: Department | null
  is_direttore: boolean
}

async function getCaller() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) throw new Error('Non autenticato')

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, department, is_direttore, account_status')
    .eq('id', user.id)
    .single()
  if (!profile) throw new Error('Profilo non trovato')

  if (!['manager', 'capo_servizio'].includes(profile.role)) {
    throw new Error('Non autorizzato')
  }
  if ((profile as { account_status?: string }).account_status === 'pending') {
    throw new Error('Account in attesa di approvazione. La demo è in sola lettura.')
  }

  return { supabase, user, profile: profile as CallerProfile }
}

// — Query Scoping (RBAC) —————
// Manager → GLOBALE: nessun vincolo

```

```

// Direttore → LOCALE: solo restaurant_id proprio (tutti i reparti)
// Capo Serv. → DIPARTIMENTALE: restaurant_id + department propri
function assertWithinScope(profile: CallerProfile, target: { restaurant_id: string; department: Department | null }) {
  if (profile.role === 'manager') {
    return // visibilità e modifica GLOBALE
  }
  if (profile.role === 'capo_servizio') {
    if (target.restaurant_id !== profile.restaurant_id) {
      throw new Error('Non autorizzato: ristorante non di tua competenza')
    }
    if (profile.is_direttore) {
      return // Direttore: tutti i reparti del proprio ristorante
    }
    if (target.department !== profile.department) {
      throw new Error('Non autorizzato: reparto non di tua competenza')
    }
    return
  }
  throw new Error('Non autorizzato')
}

export async function createTurn(input: TurnInput): Promise<Turn> {
  const { supabase, user, profile } = await getCaller()

  assertWithinScope(profile, { restaurant_id: input.restaurant_id, department: input.department })

  // Verify the assignee actually belongs to the caller's scope
  const { data: assignee } = await supabase
    .from('profiles')
    .select('id, restaurant_id, department')
    .eq('id', input.user_id)
    .single()
  if (!assignee) throw new Error('Dipendente non trovato')
  assertWithinScope(profile, { restaurant_id: assignee.restaurant_id, department: assignee.department })

  const { data, error } = await supabase
    .from('turns')
    .insert({
      user_id: input.user_id,
      restaurant_id: input.restaurant_id,
      department: input.department,
      date: input.date,
      start_time: input.start_time,
      end_time: input.end_time,
      is_extraordinary: input.is_extraordinary,
      is_rest_day: input.is_rest_day ?? false,
      notes: input.notes ?? null,
      created_by: user.id,
    })
    .select('*', profile:profiles!user_id(id, full_name)')
    .single()

  if (error) throw new Error(error.message)

  revalidatePath('/turni')
  return data as unknown as Turn
}

export async function updateTurn(id: string, input: TurnInput): Promise<Turn> {
  const { supabase, profile } = await getCaller()

  assertWithinScope(profile, { restaurant_id: input.restaurant_id, department: input.department })

  const { data: assignee } = await supabase
    .from('profiles')
    .select('id, restaurant_id, department')
    .eq('id', input.user_id)
    .single()
  if (!assignee) throw new Error('Dipendente non trovato')
  assertWithinScope(profile, { restaurant_id: assignee.restaurant_id, department: assignee.department })

  // RLS scopes the row itself: a capo_servizio cannot touch a turn outside
  // their restaurant/department even if the payload above were spoofed.

```

```

const { data, error } = await supabase
  .from('turns')
  .update({
    user_id:      input.user_id,
    restaurant_id: input.restaurant_id,
    department:   input.department,
    date:         input.date,
    start_time:   input.start_time,
    end_time:     input.end_time,
    is_extraordinary: input.is_extraordinary,
    is_rest_day:   input.is_rest_day ?? false,
    notes:        input.notes ?? null,
  })
  .eq('id', id)
  .select('*', profile:profiles!user_id(id, full_name)')
  .single()

if (error) throw new Error(error.message)

revalidatePath('/turni')
return data as unknown as Turn
}

export async function deleteTurn(id: string): Promise<void> {
  const { supabase } = await getCaller()

  // RLS enforces the manager/direttore/capo_servizio scoping on DELETE
  const { error } = await supabase
    .from('turns')
    .delete()
    .eq('id', id)

  if (error) throw new Error(error.message)

  revalidatePath('/turni')
}

// — Inserimento Multiplo (Bulk Insert) —————
// Calcola tutte le date nel range [start_date, end_date] che cadono nei
// giorni della settimana selezionati e inserisce un turno per ciascuna
// in un'unica insert massiva.
export async function createTurnsBulk(input: BulkTurnInput): Promise<Turn[]> {
  const { supabase, user, profile } = await getCaller()

  assertWithinScope(profile, { restaurant_id: input.restaurant_id, department: input.department })

  const { data: assignee } = await supabase
    .from('profiles')
    .select('id, restaurant_id, department')
    .eq('id', input.user_id)
    .single()
  if (!assignee) throw new Error('Dipendente non trovato')
  assertWithinScope(profile, { restaurant_id: assignee.restaurant_id, department: assignee.department })

  if (!input.days_of_week.length) throw new Error('Seleziona almeno un giorno della settimana')

  const start = new Date(`${input.start_date}T00:00:00`)
  const end = new Date(`${input.end_date}T00:00:00`)
  if (end < start) throw new Error('La data di fine deve essere successiva alla data di inizio')

  const daysSet = new Set(input.days_of_week)
  const dates = eachDayOfInterval({ start, end })
    .filter(d => daysSet.has(getDay(d)))
    .map(d => format(d, 'yyyy-MM-dd'))

  if (!dates.length) throw new Error('Nessuna data corrisponde ai giorni selezionati nel periodo scelto')

  const rows = dates.map(date => ({
    user_id:      input.user_id,
    restaurant_id: input.restaurant_id,
    department:   input.department,
    date,
    start_time:   input.start_time,
    end_time:     input.end_time,
  }))

```

```

    is_extraordinary: input.is_extraordinary,
    is_rest_day:      input.is_rest_day ?? false,
    notes:            input.notes ?? null,
    created_by:        user.id,
  )))

const { data, error } = await supabase
  .from('turns')
  .insert(rows)
  .select('*', profile:profiles!user_id(id, full_name)')

if (error) throw new Error(error.message)

revalidatePath('/turni')
return data as unknown as Turn[]
}

// — Turni Standard (Pattern Master-Exception) —————

export async function listStandardShifts() {
  const { supabase, profile } = await getCaller()

  let query = supabase
    .from('standard_shifts')
    .select('*', profile:profiles!user_id(id, full_name)')
    .order('day_of_week')

  if (profile.role === 'capo_servizio') {
    query = query.eq('restaurant_id', profile.restaurant_id)
    if (!profile.is_direttore) {
      query = query.eq('department', profile.department)
    }
  }

  const { data, error } = await query
  if (error) throw new Error(error.message)
  return data ?? []
}

export async function upsertStandardShift(input: StandardShiftInput & { id?: string }) {
  const { supabase, user, profile } = await getCaller()

  assertWithinScope(profile, { restaurant_id: input.restaurant_id, department: input.department })

  const { data: assignee } = await supabase
    .from('profiles')
    .select('id, restaurant_id, department')
    .eq('id', input.user_id)
    .single()
  if (!assignee) throw new Error('Dipendente non trovato')
  assertWithinScope(profile, { restaurant_id: assignee.restaurant_id, department: assignee.department })

  const payload = {
    user_id:      input.user_id,
    restaurant_id: input.restaurant_id,
    department:   input.department,
    day_of_week:  input.day_of_week,
    start_time:   input.start_time,
    end_time:     input.end_time,
    created_by:   user.id,
  }

  const query = input.id
    ? supabase.from('standard_shifts').update(payload).eq('id', input.id)
    : supabase.from('standard_shifts').insert(payload)

  const { data, error } = await query
    .select('*', profile:profiles!user_id(id, full_name)')
    .single()

  if (error) throw new Error(error.message)

  revalidatePath('/turni')
  return data
}

```

```

}

export async function deleteStandardShift(id: string): Promise<void> {
  const { supabase } = await getCaller()

  const { error } = await supabase
    .from('standard_shifts')
    .delete()
    .eq('id', id)

  if (error) throw new Error(error.message)

  revalidatePath('/turni')
}

// — Popola da Turni Standard (Automazione) —————
// Per ogni standard_shift nello scope del chiamante, genera i turni
// reali per le date del periodo scelto che cadono nel relativo
// day_of_week, saltando le date per cui esiste già un turno per quel
// dipendente (anti-duplicazione).
export async function populateFromStandard(startDate: string, endDate: string): Promise<{ created: number;
skipped: number }> {
  const { supabase, user, profile } = await getCaller()

  const start = new Date(`${startDate}T00:00:00`)
  const end = new Date(`${endDate}T00:00:00`)
  if (end < start) throw new Error('La data di fine deve essere successiva alla data di inizio')

  let standardQuery = supabase.from('standard_shifts').select('*')
  if (profile.role === 'capo_servizio') {
    standardQuery = standardQuery.eq('restaurant_id', profile.restaurant_id)
    if (!profile.is_direttore) {
      standardQuery = standardQuery.eq('department', profile.department)
    }
  }
  const { data: standardShifts, error: standardError } = await standardQuery
  if (standardError) throw new Error(standardError.message)
  if (!standardShifts?.length) return { created: 0, skipped: 0 }

  // Turni già esistenti nel periodo, per evitare duplicati.
  // La chiave include l'ora di inizio: un turno spezzato (es. mattina +
  // sera nello stesso giorno) è composto da due turni fissi distinti che
  // devono materializzarsi entrambi. Deduplichiamo per utente+data+ora,
  // non per utente+data, altrimenti il secondo segmento verrebbe scartato.
  let existingQuery = supabase
    .from('turns')
    .select('user_id, date, start_time')
    .gte('date', startDate)
    .lte('date', endDate)
  if (profile.role === 'capo_servizio') {
    existingQuery = existingQuery.eq('restaurant_id', profile.restaurant_id)
    if (!profile.is_direttore) {
      existingQuery = existingQuery.eq('department', profile.department)
    }
  }
  const { data: existingTurns, error: existingError } = await existingQuery
  if (existingError) throw new Error(existingError.message)

  const existingKeys = new Set((existingTurns ?? []).map(t => `${t.user_id}|${t.date}|${t.start_time.slice(0, 5)}`))

  const dates = eachDayOfInterval({ start, end })

  const rows: Array<{
    user_id: string
    restaurant_id: string
    department: string | null
    date: string
    start_time: string
    end_time: string
    is_extraordinary: boolean
    created_by: string
  }> = []
  let skipped = 0

```

```

for (const shift of standardShifts) {
  for (const d of dates) {
    if (getDay(d) !== shift.day_of_week) continue
    const dateStr = format(d, 'yyyy-MM-dd')
    const key = `${shift.user_id}|${dateStr}|${shift.start_time.slice(0, 5)}`
    if (existingKeys.has(key)) {
      skipped++
      continue
    }
    existingKeys.add(key) // evita doppioni solo a parità di utente+data+ora inizio
    rows.push({
      user_id:      shift.user_id,
      restaurant_id: shift.restaurant_id,
      department:   shift.department,
      date:         dateStr,
      start_time:    shift.start_time,
      end_time:      shift.end_time,
      is_extraordinary: false,
      created_by:    user.id,
    })
  }
}

if (rows.length) {
  const { error } = await supabase.from('turns').insert(rows)
  if (error) throw new Error(error.message)
}

revalidatePath('/turni')
return { created: rows.length, skipped }
}

```

src/app/api/activity/route.ts

```

import { createClient } from '@lib/supabase/server'
import { NextResponse } from 'next/server'

// POST /api/activity
// Updates last_active_at for the authenticated user.
// The client throttles this to at most once every 3-5 minutes.
export async function POST() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const { error } = await supabase
    .from('profiles')
    .update({ last_active_at: new Date().toISOString() })
    .eq('id', user.id)

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })
  return NextResponse.json({ ok: true })
}

```

```
import { createClient } from '@lib/supabase/server'
import { revalidatePath } from 'next/cache'
import { NextResponse } from 'next/server'

// PATCH /api/assenze
// Body: { id: string, action: 'approve' | 'reject' }
// Approves or rejects a pending absence request and revalidates all affected pages.
export async function PATCH(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const { data: callerProfile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, is_direttore, account_status')
    .eq('id', user.id)
    .single()

  if ((callerProfile as { account_status?: string } | null)?.account_status === 'pending')
    return NextResponse.json({ error: 'Account in attesa di approvazione. La demo è in sola lettura.' }, { status: 403 })

  const isManager = callerProfile?.role === 'manager'
  const isDirettore = callerProfile?.role === 'capo_servizio' && callerProfile.is_direttore === true
  if (!isManager && !isDirettore) {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }

  const { id, action } = await request.json()
  if (!id || ![ 'approve', 'reject' ].includes(action)) {
    return NextResponse.json({ error: 'Parametri non validi' }, { status: 400 })
  }

  const newStatus = action === 'approve' ? 'approved' : 'rejected'

  // Direttore: can only act on own restaurant
  if (isDirettore) {
    const { data: absence } = await supabase
      .from('absences')
      .select('restaurant_id')
      .eq('id', id)
      .single()
    if (absence?.restaurant_id !== callerProfile?.restaurant_id) {
      return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
    }
  }

  const { error } = await supabase
    .from('absences')
    .update({ status: newStatus })
    .eq('id', id)

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })

  // Revalidate all pages that show pending-absence counts
  revalidatePath('/dashboard')
  revalidatePath('/approvazioni')
  revalidatePath('/assenze')

  return NextResponse.json({ success: true, status: newStatus })
}
```

`src/app/api/auth/callback/route.ts`

```
import { createClient } from '@lib/supabase/server'
import { NextResponse } from 'next/server'

export async function GET(request: Request) {
  const { searchParams, origin } = new URL(request.url)
  const code = searchParams.get('code')
  const next = searchParams.get('next') ?? '/'

  if (code) {
    const supabase = await createClient()
    await supabase.auth.exchangeCodeForSession(code)
  }

  return NextResponse.redirect(`${origin}${next}`)
}
```

`src/app/api/clock-in-fallback/route.ts`

```

import { createClient } from '@lib/supabase/server'
import { revalidatePath } from 'next/cache'
import { NextResponse } from 'next/server'

const BUCKET = 'clock_in_proofs'

// POST /api/clock-in-fallback
// Multipart FormData: photo (File) + type ('in' | 'out')
// Creates an attendance row flagged for manager approval with the proof photo.
export async function POST(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const { data: profile } = await supabase
    .from('profiles')
    .select('restaurant_id')
    .eq('id', user.id)
    .single()

  if (!profile?.restaurant_id) {
    return NextResponse.json({ error: 'Nessun ristorante associato al profilo' }, { status: 400 })
  }

  const formData = await request.formData()
  const photo = formData.get('photo') as File | null
  const type = formData.get('type') as string | null

  if (!photo) return NextResponse.json({ error: 'Foto mancante' }, { status: 400 })
  if (!['in', 'out'].includes(type ?? '')) {
    return NextResponse.json({ error: 'Tipo mancante (in/out)' }, { status: 400 })
  }
  if (photo.size > 10 * 1024 * 1024) {
    return NextResponse.json({ error: 'La foto non può superare 10 MB' }, { status: 413 })
  }

  // Upload photo – store under {userId}/{timestamp}.jpg
  const ext = photo.name.split('.').pop() ?? 'jpg'
  const storagePath = `${user.id}/${Date.now()}.${ext}`
  const arrayBuffer = await photo.arrayBuffer()

  const { error: uploadErr } = await supabase.storage
    .from(BUCKET)
    .upload(storagePath, arrayBuffer, {
      contentType: photo.type || 'image/jpeg',
      upsert: false,
    })

  if (uploadErr) {
    return NextResponse.json({ error: 'Errore upload foto: ' + uploadErr.message }, { status: 500 })
  }

  const nowUtc = new Date().toISOString()

  if (type === 'in') {
    // Guard: cannot open a second shift
    const { data: openShift } = await supabase
      .from('attendances')
      .select('id')
      .eq('user_id', user.id)
      .is('check_out', null)
      .maybeSingle()

    if (openShift) {
      // Clean up the photo we just uploaded
      await supabase.storage.from(BUCKET).remove([storagePath])
      return NextResponse.json({ error: 'Hai già un turno aperto' }, { status: 409 })
    }

    const { data: attendance, error } = await supabase
      .from('attendances')
      .insert({
        user_id: user.id,

```

```

        restaurant_id:      profile.restaurant_id,
        check_in:            nowUtc,
        fallback_photo_path: storagePath,
        needs_manager_approval: true,
    })
    .select()
    .single()

    if (error) {
        await supabase.storage.from(BUCKET).remove([storagePath])
        return NextResponse.json({ error: 'Errore registrazione' }, { status: 500 })
    }

    revalidatePath('/dashboard')
    revalidatePath('/presenze')
    revalidatePath('/approvazioni')
    return NextResponse.json({ attendance }, { status: 201 })
} else {
    // type === 'out'
    const { data: openAttendance } = await supabase
        .from('attendances')
        .select('*')
        .eq('user_id', user.id)
        .is('check_out', null)
        .order('check_in', { ascending: false })
        .limit(1)
        .maybeSingle()

    if (!openAttendance) {
        await supabase.storage.from(BUCKET).remove([storagePath])
        return NextResponse.json({ error: 'Nessun turno aperto trovato' }, { status: 404 })
    }

    const { data: attendance, error } = await supabase
        .from('attendances')
        .update({
            check_out:      nowUtc,
            fallback_photo_path: storagePath,
            needs_manager_approval: true,
        })
        .eq('id', openAttendance.id)
        .select()
        .single()

    if (error) {
        await supabase.storage.from(BUCKET).remove([storagePath])
        return NextResponse.json({ error: 'Errore registrazione uscita' }, { status: 500 })
    }

    return NextResponse.json({ attendance })
}
}

```

```
import { createClient } from '@lib/supabase/server'
import { NextResponse } from 'next/server'

// POST /api/consultant-messages/[id]/download
// Body: { path: string }
// Sets downloaded_at = now() on the message if the caller is the recipient,
// then returns a signed URL (valid 1h) for the attachment.
export async function POST(
  request: Request,
  { params }: { params: Promise<{ id: string }> },
) {
  const { id } = await params
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const { path, name } = await request.json()
  if (!path) return NextResponse.json({ error: 'Path mancante' }, { status: 400 })

  const { data: msg, error: fetchErr } = await supabase
    .from('consultant_messages')
    .select('manager_id, consultant_id, sent_by_manager')
    .eq('id', id)
    .single()

  if (fetchErr || !msg) return NextResponse.json({ error: 'Messaggio non trovato' }, { status: 404 })

  // Verify the caller is a party to this message
  if (user.id !== msg.manager_id && user.id !== msg.consultant_id) {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }

  // Only set downloaded_at if the caller is the recipient
  const isRecipient =
    (msg.sent_by_manager && user.id === msg.consultant_id) ||
    (!msg.sent_by_manager && user.id === msg.manager_id)

  if (isRecipient) {
    await supabase
      .from('consultant_messages')
      .update({ downloaded_at: new Date().toISOString() })
      .eq('id', id)
  }

  // Generate a 1-hour signed URL
  const filename = name ?? path.split('/').pop() ?? 'file'
  const { data: signedData, error: signErr } = await supabase.storage
    .from('consultant_files')
    .createSignedUrl(path, 3600, { download: filename })

  if (signErr || !signedData?.signedUrl) {
    return NextResponse.json({ error: 'Impossibile generare il link di download' }, { status: 500 })
  }

  return NextResponse.json({ signedUrl: signedData.signedUrl })
}
```

```
import { createClient } from '@lib/supabase/server'
import { NextResponse } from 'next/server'

// POST /api/consultant-messages/[id]/read
// Marks the message as read by the current user (sets read_at = now()).
// Only marks it if the caller is the recipient (not the sender).
export async function POST(
  _request: Request,
  { params }: { params: Promise<{ id: string }> },
) {
  const { id } = await params
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  // Fetch the message to verify ownership and current read_at
  const { data: msg, error: fetchErr } = await supabase
    .from('consultant_messages')
    .select('manager_id, consultant_id, sent_by_manager, read_at')
    .eq('id', id)
    .single()

  if (fetchErr || !msg) return NextResponse.json({ error: 'Messaggio non trovato' }, { status: 404 })

  // The recipient is: consultant if sent_by_manager=true, manager if sent_by_manager=false
  const isRecipient =
    (msg.sent_by_manager && user.id === msg.consultant_id) ||
    (!msg.sent_by_manager && user.id === msg.manager_id)

  if (!isRecipient) {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }
  if (msg.read_at) {
    // Already read – return current timestamp, no update needed
    return NextResponse.json({ read_at: msg.read_at })
  }

  const now = new Date().toISOString()
  const { error } = await supabase
    .from('consultant_messages')
    .update({ read_at: now })
    .eq('id', id)

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })
  return NextResponse.json({ read_at: now })
}
```

`src/app/api/consultant-messages/route.ts`

```

import { createClient } from '@lib/supabase/server'
import { revalidatePath } from 'next/cache'
import { NextResponse } from 'next/server'

// GET /api/consultant-messages?consultantId=<uuid>
// Returns all messages between the authenticated manager and the given consultant.
// Also used by the consultant themselves (consultantId = their own id).
export async function GET(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const { data: profile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', user.id)
    .single()
  if (!profile) return NextResponse.json({ error: 'Profilo non trovato' }, { status: 404 })

  const { searchParams } = new URL(request.url)
  const consultantId = searchParams.get('consultantId')
  if (!consultantId) return NextResponse.json({ error: 'consultantId mancante' }, { status: 400 })

  let query = supabase
    .from('consultant_messages')
    .select('*')
    .order('created_at', { ascending: false })

  if (profile.role === 'manager') {
    query = query.eq('manager_id', user.id).eq('consultant_id', consultantId)
  } else if (profile.role === 'consulente_lavoro') {
    // consultant can only see their own thread
    if (consultantId !== user.id) {
      return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
    }
    query = query.eq('consultant_id', user.id)
  } else {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }

  const { data, error } = await query
  if (error) return NextResponse.json({ error: error.message }, { status: 500 })
  return NextResponse.json(data ?? [])
}

// POST /api/consultant-messages
// Body: { consultantId, title, body, attachments?, sentByManager? }
export async function POST(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const { data: profile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', user.id)
    .single()
  if (!profile) return NextResponse.json({ error: 'Profilo non trovato' }, { status: 404 })

  if (profile.role !== 'manager' && profile.role !== 'consulente_lavoro') {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }

  const body = await request.json()
  const { consultantId, title, body: msgBody, attachments, reply_to_id } = body

  if (!consultantId || !title || !msgBody) {
    return NextResponse.json({ error: 'Campi obbligatori mancanti' }, { status: 400 })
  }

  let managerId: string
  let consultId: string
  let sentByManager: boolean

```

```

if (profile.role === 'manager') {
  managerId = user.id
  consultId = consultantId
  sentByManager = true
} else {
  // consultant replying – the consultantId param here is their own manager's id
  managerId = consultantId
  consultId = user.id
  sentByManager = false
}

const { data, error } = await supabase
  .from('consultant_messages')
  .insert({
    manager_id: managerId,
    consultant_id: consultId,
    title,
    body: msgBody,
    attachments: attachments ?? [],
    sent_by_manager: sentByManager,
    reply_to_id: reply_to_id ?? null,
  })
  .select()
  .single()

if (error) return NextResponse.json({ error: error.message }, { status: 500 })
return NextResponse.json(data, { status: 201 })
}

// DELETE /api/consultant-messages?id=<uuid>
// Manager-only. Order: 1) role check → 2) fetch attachments → 3) storage.remove → 4) db.delete
export async function DELETE(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  // RBAC hard lock – only managers may delete messages
  const { data: callerProfile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', user.id)
    .single()
  if (callerProfile?.role !== 'manager') {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }

  const { searchParams } = new URL(request.url)
  const id = searchParams.get('id')
  if (!id) return NextResponse.json({ error: 'ID mancante' }, { status: 400 })

  // Step 1: fetch the message (scoped to the caller) to retrieve attachment paths
  const { data: message } = await supabase
    .from('consultant_messages')
    .select('attachments')
    .eq('id', id)
    .or(`manager_id.eq.${user.id},consultant_id.eq.${user.id}`)
    .maybeSingle()

  // Step 2: remove physical files from storage before touching the DB row.
  // Errors are intentionally ignored – a file already missing in storage
  // must not prevent the message record from being deleted.
  if (message?.attachments?.length) {
    const paths = (message.attachments as Array<{ name: string; path: string }>)
      .map(a => a.path)
      .filter(Boolean)
    if (paths.length > 0) {
      await supabase.storage.from('consultant_files').remove(paths)
    }
  }

  // Step 3: delete the DB row (same authorization filter as the SELECT above)
  const { error } = await supabase
    .from('consultant_messages')

```



```

.delete()
.eq('id', id)
.or(`manager_id.eq.${user.id},consultant_id.eq.${user.id}`)

if (error) return NextResponse.json({ error: error.message }, { status: 500 })

revalidatePath('/dashboard')
revalidatePath('/consulente/dashboard')
return NextResponse.json({ success: true })
}

```

src/app/api/consultant-messages/upload/route.ts

```

import { createClient } from '@lib/supabase/server'
import { NextResponse } from 'next/server'

// POST /api/consultant-messages/upload
// Multipart form: file + consultantId
// Uploads to storage bucket and returns the object path.
export async function POST(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const formData = await request.formData()
  const file = formData.get('file') as File | null
  const consultantId = formData.get('consultantId') as string | null

  if (!file || !consultantId) {
    return NextResponse.json({ error: 'File o consultantId mancante' }, { status: 400 })
  }
  if (file.size > 10 * 1024 * 1024) {
    return NextResponse.json({ error: 'Il file supera il limite di 10 MB' }, { status: 413 })
  }

  const ext = file.name.split('.').pop() ?? 'bin'
  const path = `${consultantId}/${Date.now()}-${Math.random().toString(36).slice(2)}${ext}`

  const arrayBuffer = await file.arrayBuffer()
  const { error } = await supabase.storage
    .from('consultant_files')
    .upload(path, arrayBuffer, {
      contentType: file.type,
      upsert: false,
    })

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })
  return NextResponse.json({ path, name: file.name }, { status: 201 })
}

```

`src/app/api/presenze/fallback-approve/route.ts`

```

import { createClient } from '@lib/supabase/server'
import { createAdminClient } from '@lib/supabase/admin'
import { revalidatePath } from 'next/cache'
import { NextResponse } from 'next/server'

const BUCKET = 'clock_in_proofs'

// POST /api/presenze/fallback-approve
// Body: { attendanceId: string, action: 'approve' | 'reject' }
// Manager-only. On approve: clears the photo flag, deletes the file from storage.
// On reject: deletes the attendance row and the file from storage.
export async function POST(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const { data: callerProfile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, is_direttore, account_status')
    .eq('id', user.id)
    .single()

  if ((callerProfile as { account_status?: string } | null)?.account_status === 'pending')
    return NextResponse.json({ error: 'Account in attesa di approvazione. La demo è in sola lettura.' }, { status: 403 })

  const isManager = callerProfile?.role === 'manager'
  const isDirettore = callerProfile?.role === 'capo_servizio' && callerProfile.is_direttore === true
  if (!isManager && !isDirettore) {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }

  const { attendanceId, action } = await request.json()
  if (!attendanceId || !['approve', 'reject'].includes(action)) {
    return NextResponse.json({ error: 'Parametri non validi' }, { status: 400 })
  }

  const admin = createAdminClient()

  // Load the attendance to get the photo path and verify restaurant scope
  const { data: attendance } = await admin
    .from('attendances')
    .select('id, restaurant_id, fallback_photo_path, needs_manager_approval')
    .eq('id', attendanceId)
    .single()

  if (!attendance) {
    return NextResponse.json({ error: 'Timbratura non trovata' }, { status: 404 })
  }
  if (!attendance.needs_manager_approval) {
    return NextResponse.json({ error: 'Questa timbratura non richiede approvazione' }, { status: 400 })
  }

  // Direttore is scoped to their own restaurant
  if (isDirettore && attendance.restaurant_id !== callerProfile?.restaurant_id) {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }

  const photoPath = attendance.fallback_photo_path as string | null

  if (action === 'approve') {
    // Clear the approval flag and wipe the photo reference
    const { error } = await admin
      .from('attendances')
      .update({
        needs_manager_approval: false,
        fallback_photo_path: null,
      })
      .eq('id', attendanceId)

    if (error) return NextResponse.json({ error: error.message }, { status: 500 })

    // Delete the physical file from storage to free space

```

```

    if (photoPath) {
      await admin.storage.from(BUCKET).remove([photoPath])
    }

    revalidatePath('/dashboard')
    revalidatePath('/presenze')
    revalidatePath('/approvazioni')
    return NextResponse.json({ success: true, action: 'approved' })
  } else {
    // reject – delete the whole attendance row first, then the file
    const { error } = await admin
      .from('attendances')
      .delete()
      .eq('id', attendanceId)

    if (error) return NextResponse.json({ error: error.message }, { status: 500 })

    if (photoPath) {
      await admin.storage.from(BUCKET).remove([photoPath])
    }

    revalidatePath('/dashboard')
    revalidatePath('/presenze')
    revalidatePath('/approvazioni')
    return NextResponse.json({ success: true, action: 'rejected' })
  }
}

```

`src/app/api/presenze/route.ts`

```

import { NextRequest, NextResponse } from 'next/server'
import { createClient } from '@lib/supabase/server'
import { computeAttendanceIso } from '@lib/attendanceTime'

const TZ = 'Europe/Rome'

const DEMO_READONLY = NextResponse.json(
  { error: 'Account in attesa di approvazione. La demo è in sola lettura.' },
  { status: 403 }
)

export async function POST(request: NextRequest) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autorizzato' }, { status: 401 })

  const { data: callerProfile } = await supabase
    .from('profiles')
    .select('role, is_direttore, account_status')
    .eq('id', user.id)
    .single()

  if ((callerProfile as { account_status?: string } | null)?.account_status === 'pending') return DEMO_READONLY

  const isManager = callerProfile?.role === 'manager'
  const isDirettore = callerProfile?.role === 'capo_servizio' && callerProfile?.is_direttore === true

  if (!isManager && !isDirettore) {
    return NextResponse.json(
      { error: 'Non autorizzato. Solo i manager e i direttori possono modificare le presenze.' },
      { status: 403 }
    )
  }

  const body = await request.json()
  const { userId, date, checkIn, checkOut } = body as {
    userId: string
    date: string
    checkIn: string
    checkOut?: string
  }

  if (!userId || !date || !checkIn) {
    return NextResponse.json({ error: 'Campi obbligatori mancanti' }, { status: 400 })
  }

  const { data: empProfile } = await supabase
    .from('profiles')
    .select('restaurant_id')
    .eq('id', userId)
    .single()

  const { checkInIso, checkOutIso } = computeAttendanceIso(TZ, date, checkIn, checkOut)

  const { data, error } = await supabase
    .from('attendances')
    .insert({
      user_id: userId,
      restaurant_id: empProfile?.restaurant_id ?? null,
      check_in: checkInIso,
      check_out: checkOutIso,
    })
    .select('*', profile:profiles(id, full_name, role), restaurant:restaurants(id, name))
    .single()

  if (error) {
    return NextResponse.json({ error: error.message }, { status: 500 })
  }

  return NextResponse.json({ attendance: data })
}

// PATCH /api/presenze

```

```

// Body: { id, date, checkIn, checkOut? }
// Manager: globale. Direttore: solo presenze del proprio ristorante.
export async function PATCH(request: NextRequest) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autorizzato' }, { status: 401 })

  const { data: callerProfile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, is_direttore, account_status')
    .eq('id', user.id)
    .single()

  if ((callerProfile as { account_status?: string } | null)?.account_status === 'pending') return DEMO_READONLY

  const isManager = callerProfile?.role === 'manager'
  const isDirettore = callerProfile?.role === 'capo_servizio' && callerProfile?.is_direttore === true

  if (!isManager && !isDirettore) {
    return NextResponse.json(
      { error: 'Non autorizzato. Solo i manager e i direttori possono modificare le presenze.' },
      { status: 403 }
    )
  }

  const body = await request.json()
  const { id, date, checkIn, checkOut } = body as {
    id: string
    date: string
    checkIn: string
    checkOut?: string | null
  }

  if (!id || !date || !checkIn) {
    return NextResponse.json({ error: 'Campi obbligatori mancanti' }, { status: 400 })
  }

  const { checkInIso, checkOutIso } = computeAttendanceIso(TZ, date, checkIn, checkOut)

  let query = supabase
    .from('attendances')
    .update({ check_in: checkInIso, check_out: checkOutIso })
    .eq('id', id)

  // Direttore is hard-locked to its own restaurant's attendance records
  if (isDirettore && callerProfile?.restaurant_id) {
    query = query.eq('restaurant_id', callerProfile.restaurant_id)
  }

  const { data, error } = await query
    .select('*', profile:profiles(id, full_name, role), restaurant:restaurants(id, name))
    .single()

  if (error) {
    return NextResponse.json({ error: error.message }, { status: 500 })
  }

  return NextResponse.json({ attendance: data })
}

```

`src/app/api/push/send/route.ts`


```

import { createClient } from '@lib/supabase/server'
import { createAdminClient } from '@lib/supabase/admin'
import { NextResponse } from 'next/server'
import webpush from 'web-push'

export const dynamic = 'force-dynamic'

export async function POST(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const { data: profile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', user.id)
    .single()

  if (profile?.role !== 'manager') {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }

  const { bulletinId } = await request.json()
  if (!bulletinId) return NextResponse.json({ error: 'bulletinId mancante' }, { status: 400 })

  const admin = createAdminClient()

  const { data: bulletin } = await admin
    .from('bulletins')
    .select('title, body, target, target_roles, target_user_ids, restaurant_id')
    .eq('id', bulletinId)
    .single()

  if (!bulletin) return NextResponse.json({ error: 'Bulletin non trovato' }, { status: 404 })

  type SubRow = { id: string; endpoint: string; p256dh: string; auth_key: string }
  let subscriptions: SubRow[] = []

  if (bulletin.target === 'all') {
    const { data } = await admin
      .from('push_subscriptions')
      .select('id, endpoint, p256dh, auth_key')
    subscriptions = (data ?? []) as SubRow[]
  } else if (bulletin.target === 'users' && bulletin.target_user_ids?.length) {
    const { data } = await admin
      .from('push_subscriptions')
      .select('id, endpoint, p256dh, auth_key')
      .in('user_id', bulletin.target_user_ids)
    subscriptions = (data ?? []) as SubRow[]
  } else if (bulletin.target === 'role' && bulletin.target_roles?.length) {
    const { data: profiles } = await admin
      .from('profiles')
      .select('id')
      .in('role', bulletin.target_roles)
    if (profiles?.length) {
      const { data } = await admin
        .from('push_subscriptions')
        .select('id, endpoint, p256dh, auth_key')
        .in('user_id', profiles.map((p: { id: string }) => p.id))
      subscriptions = (data ?? []) as SubRow[]
    }
  } else if (bulletin.target === 'restaurant' && bulletin.restaurant_id) {
    const { data: profiles } = await admin
      .from('profiles')
      .select('id')
      .eq('restaurant_id', bulletin.restaurant_id)
    if (profiles?.length) {
      const { data } = await admin
        .from('push_subscriptions')
        .select('id, endpoint, p256dh, auth_key')
        .in('user_id', profiles.map((p: { id: string }) => p.id))
      subscriptions = (data ?? []) as SubRow[]
    }
  }
}

```

```

}

if (!subscriptions.length) return NextResponse.json({ sent: 0 })

webpush.setVapidDetails(
  process.env.VAPID_EMAIL!,
  process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY!,
  process.env.VAPID_PRIVATE_KEY!
)

const payload = JSON.stringify({
  title: bulletin.title,
  body: bulletin.body,
  url: '/',
})

let sent = 0

await Promise.allSettled(
  subscriptions.map(async (sub) => {
    try {
      await webpush.sendNotification(
        { endpoint: sub.endpoint, keys: { p256dh: sub.p256dh, auth: sub.auth_key } },
        payload
      )
      sent++
    } catch (err: unknown) {
      // Rimuovi solo subscription definitivamente scadute/non valide (410 Gone, 404 Not Found)
      const status = (err as { statusCode?: number }).statusCode
      if (status === 410 || status === 404) {
        await admin.from('push_subscriptions').delete().eq('id', sub.id)
      }
    }
  })
)

return NextResponse.json({ sent })
}

```

```
import { createClient } from '@lib/supabase/server'
import { NextResponse } from 'next/server'

export async function POST(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const subscription = await request.json()
  const { endpoint, keys } = subscription

  if (!endpoint || !keys?.p256dh || !keys?.auth) {
    return NextResponse.json({ error: 'Subscription non valida' }, { status: 400 })
  }

  const { error } = await supabase
    .from('push_subscriptions')
    .upsert({
      user_id: user.id,
      endpoint,
      p256dh: keys.p256dh,
      auth_key: keys.auth,
    }, { onConflict: 'user_id,endpoint' })

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })

  return NextResponse.json({ success: true })
}

export async function DELETE(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const { endpoint } = await request.json()
  await supabase.from('push_subscriptions').delete().eq('user_id', user.id).eq('endpoint', endpoint)

  return NextResponse.json({ success: true })
}
```

`src/app/api/register/route.ts`

```

import { createAdminClient } from '@lib/supabase/admin'
import { NextResponse } from 'next/server'
import { createDemoData } from '@lib/demoData'
import { sendNewRegistrationAlert } from '@lib/email'

const NAME_RE = /^[A-Za-zÀ-ÿ\s']-{2,60}$/
const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]+$/

export async function POST(request: Request) {
  const body = await request.json().catch(() => null)
  if (!body) return NextResponse.json({ error: 'Richiesta non valida' }, { status: 400 })

  const { full_name, email, password } = body as { full_name?: string; email?: string; password?: string }

  if (!full_name || !NAME_RE.test(full_name.trim())) {
    return NextResponse.json({ error: 'Nome non valido (solo lettere, 2-60 caratteri)' }, { status: 400 })
  }
  if (!email || !EMAIL_RE.test(email.trim().toLowerCase())) {
    return NextResponse.json({ error: 'Email non valida' }, { status: 400 })
  }
  if (!password || password.length < 8) {
    return NextResponse.json({ error: 'Password troppo corta (minimo 8 caratteri)' }, { status: 400 })
  }

  const admin = createAdminClient()

  // Verifica che l'email non sia già registrata
  const { data: existing } = await admin
    .from('profiles')
    .select('id')
    .eq('username', email.trim().toLowerCase())
    .maybeSingle()

  if (existing) {
    return NextResponse.json({ error: 'Email già registrata' }, { status: 409 })
  }

  // Crea l'auth user con email reale
  const { data: authData, error: authErr } = await admin.auth.admin.createUser({
    email: email.trim().toLowerCase(),
    password,
    email_confirm: true,
    user_metadata: { full_name: full_name.trim(), role: 'manager' },
  })

  if (authErr || !authData.user) {
    const msg = authErr?.message ?? ''
    if (msg.includes('already been registered') || msg.includes('already registered') || msg.includes('already exists')) {
      return NextResponse.json({ error: 'Email già registrata' }, { status: 409 })
    }
    return NextResponse.json({ error: 'Errore durante la registrazione: ' + msg }, { status: 500 })
  }

  const userId = authData.user.id

  // Crea il profilo manager in stato pending
  // Usa upsert perché Supabase potrebbe avere un trigger che crea la riga automaticamente
  const { error: profileErr } = await admin.from('profiles').upsert({
    id: userId,
    full_name: full_name.trim(),
    username: email.trim().toLowerCase(),
    role: 'manager',
    department: null,
    restaurant_id: null,
    account_status: 'pending',
    managed_restaurant_ids: null,
    can_post_bulletin: true,
    is_direttore: false,
    consultant_restaurant_ids: [],
    can_view_hours: true,
    weekly_rest_days: 1,
    primary_slot_ids: [],
  })

```

```

    secondary_departments: [],
  }, { onConflict: 'id' })

if (profileErr) {
  await admin.auth.admin.deleteUser(userId)
  return NextResponse.json({ error: 'Errore creazione profilo: ' + profileErr.message }, { status: 500 })
}

// Crea i dati demo e aggiorna managed_restaurant_ids per isolare il demo
try {
  const demoRestaurantIds = await createDemoData(userId)
  await admin.from('profiles').update({
    managed_restaurant_ids: demoRestaurantIds,
  }).eq('id', userId)
} catch (err) {
  console.error('[register] Errore creazione demo data:', err)
}

// Notifica via email + notifica in-app all'admin
const appUrl = process.env.NEXT_PUBLIC_APP_URL ?? 'https://turni.vercel.app'
sendNewRegistrationAlert({
  fullName: full_name.trim(),
  email: email.trim().toLowerCase(),
  approveUrl: `${appUrl}/account-pendenti`,
}).catch(() => {})

// Notifica in-app: cerca tutti i manager platform-owner
const { data: adminManagers } = await admin
  .from('profiles')
  .select('id')
  .eq('role', 'manager')
  .is('managed_restaurant_ids', null)
  .neq('id', userId)

if (adminManagers?.length) {
  await admin.from('notifications').insert(
    adminManagers.map(m => ({
      user_id: m.id,
      title: 'Nuova richiesta di accesso',
      message: `${full_name.trim()} (${email.trim().toLowerCase()}) ha richiesto l'accesso.`,
      link: '/account-pendenti',
    })))
  )
}

return NextResponse.json({ ok: true }, { status: 201 })
}

```

`src/app/api/report/route.ts`

```

import { createClient } from '@lib/supabase/server'
import { createAdminClient } from '@lib/supabase/admin'
import { autoCloseStaleShifts } from '@lib/autoCloseStaleShifts'
import { NextResponse } from 'next/server'
import ExcelJS from 'exceljs'
import { formatInTimeZone, fromZonedTime } from 'date-fns-tz'
import { differenceInMinutes, getDaysInMonth } from 'date-fns'
import { it } from 'date-fns/locale'
import { ABSENCE_CODES } from '@types'
import type { AbsenceType } from '@types'

const TZ = 'Europe/Rome'
const TARGET_HOURS_PER_DAY = 8.5

function minutesToHours(minutes: number): string {
  const h = Math.floor(minutes / 60)
  const m = minutes % 60
  return `${h}h ${String(m).padStart(2, '0')}m`
}

export async function POST(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })

  const { data: profile } = await supabase
    .from('profiles')
    .select('role, restaurant_id')
    .eq('id', user.id)
    .single()

  if (!profile || profile.role === 'dipendente') {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }

  const { month, restaurantIds, type } = await request.json()

  if (!month || !restaurantIds?.length || ![ 'presenze', 'ore' ].includes(type)) {
    return NextResponse.json({ error: 'Parametri non validi' }, { status: 400 })
  }

  // Capo servizio può accedere solo al proprio ristorante
  const allowedIds = profile.role === 'capo_servizio' && profile.restaurant_id
    ? restaurantIds.filter((id: string) => id === profile.restaurant_id)
    : restaurantIds

  const [year, monthNum] = month.split('-').map(Number)
  const daysInMonth = getDaysInMonth(new Date(year, monthNum - 1))
  const startDate = `${month}-01`
  const endDate = `${month}-${String(daysInMonth).padStart(2, '0')}`
  // Use Rome-midnight boundaries so after-midnight check-ins are included correctly
  const rangeStart = fromZonedTime(`${startDate}T00:00:00`, TZ).toISOString()
  const rangeEnd = fromZonedTime(`${endDate}T23:59:59`, TZ).toISOString()

  const admin = createAdminClient()

  // Close forgotten check-outs (shifts open >16h) so reported hours don't show
  // an open/zero session for an employee who simply forgot to timbrare l'uscita.
  await autoCloseStaleShifts(admin)

  const workbook = new ExcelJS.Workbook()
  workbook.creator = 'Turni App'
  workbook.created = new Date()

  const monthName = formatInTimeZone(new Date(year, monthNum - 1, 1), TZ, 'MMMM yyyy', { locale: it })

  for (const restaurantId of allowedIds) {
    const { data: restaurant } = await admin
      .from('restaurants')
      .select('id, name')
      .eq('id', restaurantId)
      .single()
  }

```



```

if (!restaurant) continue

// Dipendenti del ristorante
const { data: employees } = await admin
  .from('profiles')
  .select('id, full_name')
  .eq('restaurant_id', restaurantId)
  .in('role', ['dipendente', 'capo_servizio'])
  .order('full_name')

if (!employees?.length) continue

// Presenze del mese per il ristorante
const { data: attendances } = await admin
  .from('attendances')
  .select('*')
  .eq('restaurant_id', restaurantId)
  .gte('check_in', rangeStart)
  .lte('check_in', rangeEnd)

// Assenze del mese per il ristorante
const { data: absences } = await admin
  .from('absences')
  .select('*')
  .eq('restaurant_id', restaurantId)
  .eq('status', 'approved')
  .lte('start_date', endDate)
  .gte('end_date', startDate)

const sheet = workbook.addWorksheet(restaurant.name.substring(0, 31))

// Header row: dipendente + giorni + extra colonne
const extraCols = type === 'ore' ? ['Totale Ore', 'Differenza', 'Note'] : ['Note']
const headers = ['Dipendente', ...Array.from({ length: daysInMonth }, (_, i) => String(i + 1)), ...extraCols]
const headerRow = sheet.addRow(headers)
headerRow.font = { bold: true, size: 11 }
headerRow.fill = { type: 'pattern', pattern: 'solid', fgColor: { argb: 'FF0F172A' } }
headerRow.font = { bold: true, color: { argb: 'FFFFFF' } }
headerRow.alignment = { horizontal: 'center' }

// Freeze prima colonna e prima riga
sheet.views = [{ state: 'frozen', xSplit: 1, ySplit: 1 }]

// Larghezza colonne
sheet.getColumn(1).width = 22
for (let d = 2; d <= daysInMonth + 1; d++) {
  sheet.getColumn(d).width = 5
}
if (type === 'ore') {
  sheet.getColumn(daysInMonth + 2).width = 12
  sheet.getColumn(daysInMonth + 3).width = 12
  sheet.getColumn(daysInMonth + 4).width = 20
} else {
  sheet.getColumn(daysInMonth + 2).width = 25
}

const CELL_COLORS: Record<string, string> = {
  P: 'FFd4edda', // verde pastello
  PP: 'FF86efac', // verde intenso (doppio turno >12h)
  F: 'FFe8d5f5', // lilla
  M: 'FFd0e8f5', // azzurro
  R: 'FFf5d0d0', // rosso pastello
  AI: 'FFd8c5f0', // viola
}

for (const emp of employees) {
  const row: (string | number)[] = [emp.full_name]
  let totalMinutes = 0
  let workDays = 0

  for (let day = 1; day <= daysInMonth; day++) {
    const dateStr = `${month}-${String(day).padStart(2, '0')}`

    // Controlla assenza

```

```

const absence = absences?.find(a =>
  a.user_id === emp.id &&
  a.start_date <= dateStr &&
  a.end_date >= dateStr
)

if (absence) {
  const code = ABSENCE_CODES[absence.type as AbsenceType]
  if (type === 'presenze') {
    row.push(code)
  } else {
    row.push('')
  }
  continue
}

// Tutte le sessioni di questo dipendente in questo giorno (Rome time)
const daySessions = attendances?.filter(a => {
  if (a.user_id !== emp.id) return false
  const dayRome = formatInTimeZone(new Date(a.check_in), TZ, 'yyyy-MM-dd')
  return dayRome === dateStr
}) ?? []

if (daySessions.length > 0) {
  const hasOpen = daySessions.some(a => !a.check_out)
  const dayMinutes = daySessions.reduce((sum, a) => {
    if (!a.check_out) return sum
    return sum + differenceInMinutes(new Date(a.check_out), new Date(a.check_in))
  }, 0)

  if (type === 'presenze') {
    row.push(dayMinutes > 720 ? 'PP' : 'P')
  } else {
    if (hasOpen && dayMinutes === 0) {
      row.push('In corso')
    } else {
      totalMinutes += dayMinutes
      workDays++
      row.push(minutesToHours(dayMinutes))
    }
  }
} else {
  row.push('')
}
}

if (type === 'ore') {
  const targetMinutes = workDays * TARGET_HOURS_PER_DAY * 60
  const diffMinutes = totalMinutes - targetMinutes
  const diffSign = diffMinutes >= 0 ? '+' : '-'
  row.push(minutesToHours(totalMinutes))
  row.push(`${diffSign}${minutesToHours(Math.abs(diffMinutes))}`)
  row.push('')
} else {
  row.push('')
}

const dataRow = sheet.addRow(row)
dataRow.alignment = { horizontal: 'center' }
dataRow.getCell(1).alignment = { horizontal: 'left' }

// Colora celle
for (let colIdx = 2; colIdx <= daysInMonth + 1; colIdx++) {
  const cell = dataRow.getCell(colIdx)
  const val = String(cell.value ?? '')
  const color = CELL_COLORS[val]
  if (color) {
    cell.fill = { type: 'pattern', pattern: 'solid', fgColor: { argb: color } }
    cell.font = { color: { argb: 'FF1a1a1a' }, bold: true }
  }
}

// Colora totale e differenza per ore
if (type === 'ore') {

```

```

const diffCell = dataRow.getCell(daysInMonth + 3)
const diffVal = String(diffCell.value ?? '')
if (diffVal.startsWith('+')) {
  diffCell.font = { color: { argb: 'FF166534' }, bold: true }
} else if (diffVal.startsWith('-')) {
  diffCell.font = { color: { argb: 'FF991B1B' }, bold: true }
}
}
}

// Bordi
const totalRows = sheet.rowCount
const totalCols = headers.length
for (let r = 1; r <= totalRows; r++) {
  for (let c = 1; c <= totalCols; c++) {
    sheet.getCell(r, c).border = {
      top: { style: 'thin', color: { argb: 'FFe2e8f0' } },
      left: { style: 'thin', color: { argb: 'FFe2e8f0' } },
      bottom: { style: 'thin', color: { argb: 'FFe2e8f0' } },
      right: { style: 'thin', color: { argb: 'FFe2e8f0' } },
    }
  }
}

if (workbook.worksheets.length === 0) {
  return NextResponse.json({ error: 'Nessun dato trovato per il periodo selezionato' }, { status: 404 })
}

const buffer = await workbook.xlsx.writeBuffer()

return new NextResponse(buffer as unknown as BodyInit, {
  headers: {
    'Content-Type': 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
    'Content-Disposition': `attachment; filename="report-${type}-${month}.xlsx"`,
  },
})
}

```

`src/app/api/scan/route.ts`

```

import { createClient } from '@lib/supabase/server'
import { NextResponse } from 'next/server'
import { formatInTimeZone, fromZonedTime } from 'date-fns-tz'
import { autoCloseStaleShifts } from '@lib/autoCloseStaleShifts'

const TZ = 'Europe/Rome'
const RAGGIO_MAX = 100 // metres
// Mirror the client geofence: credit (capped) GPS uncertainty so a weak indoor
// fix doesn't reject an employee who is actually inside the restaurant.
const ACCURACY_BUFFER_MAX = 250 // metres

function haversine(lat1: number, lon1: number, lat2: number, lon2: number): number {
  const R = 6_371_000
  const φ1 = (lat1 * Math.PI) / 180
  const φ2 = (lat2 * Math.PI) / 180
  const Δφ = ((lat2 - lat1) * Math.PI) / 180
  const Δλ = ((lon2 - lon1) * Math.PI) / 180
  const a = Math.sin(Δφ / 2) ** 2 + Math.cos(φ1) * Math.cos(φ2) * Math.sin(Δλ / 2) ** 2
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a))
}

function todayRomeBounds(): { start: string; end: string } {
  const todayRome = formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd')
  const start = fromZonedTime(`${todayRome}T00:00:00`, TZ).toISOString()
  const end = fromZonedTime(`${todayRome}T23:59:59`, TZ).toISOString()
  return { start, end }
}

export async function POST(request: Request) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  if (!user) {
    return NextResponse.json({ error: 'Non autenticato' }, { status: 401 })
  }

  const { qr_secret, type, latitude, longitude, accuracy, frozenAt } = await request.json()

  if (!qr_secret || ![ 'in', 'out' ].includes(type)) {
    return NextResponse.json({ error: 'Parametri non validi' }, { status: 400 })
  }

  // In development: simula scansione con il ristorante del profilo
  let restaurant
  if (process.env.NODE_ENV === 'development' && qr_secret === '__SIMULATE__') {
    const { data: profile } = await supabase
      .from('profiles')
      .select('restaurant:restaurants(*)')
      .eq('id', user.id)
      .single()
    restaurant = (profile as unknown as { restaurant: typeof restaurant })?.restaurant

    if (!restaurant) {
      // Prendi il primo ristorante disponibile in dev
      const { data } = await supabase.from('restaurants').select('*').limit(1).single()
      restaurant = data
    }
  } else {
    // Valida il QR secret
    const { data } = await supabase
      .from('restaurants')
      .select('*')
      .eq('qr_secret', qr_secret)
      .single()
    restaurant = data
  }

  if (!restaurant) {
    return NextResponse.json({ error: 'QR Code non riconosciuto' }, { status: 404 })
  }

  // Geofencing server-side: confronta la posizione del device con le
  // coordinate del ristorante. La simulazione in dev bypassa il controllo.

```

```

const isDevSimulate = process.env.NODE_ENV === 'development' && qr_secret === '__SIMULATE__'
if (!isDevSimulate && restaurant.latitude != null && restaurant.longitude != null) {
  if (latitude == null || longitude == null) {
    return NextResponse.json({ error: 'Devi attivare il GPS per timbrare' }, { status: 400 })
  }
  const dist = haversine(latitude, longitude, restaurant.latitude, restaurant.longitude)
  const buffer = Math.min(typeof accuracy === 'number' && accuracy > 0 ? accuracy : 0, ACCURACY_BUFFER_MAX)
  if (dist > RAGGIO_MAX + buffer) {
    return NextResponse.json({ error: 'Sei troppo lontano dal ristorante per timbrare.' }, { status: 403 })
  }
}

// If the client sent a frozen timestamp (offline replay), honour it; otherwise use server clock.
const nowUtc = (typeof frozenAt === 'string' && frozenAt) ? frozenAt : new Date().toISOString()

if (type === 'in') {
  // Close any shift the employee forgot to timbrare l'uscita on (open >16h)
  // before the guard below, so a stale shift from yesterday doesn't block a
  // fresh check-in today.
  await autoCloseStaleShifts(supabase, user.id)

  // The open-shift guard must NOT be date-scoped: a shift started before
  // midnight Rome is still open if check_out is null. Only split-shift
  // detection (completed turni within today's Rome window) needs a date range.
  const { start: todayStart, end: todayEnd } = todayRomeBounds()

  const [{ data: openShift }, { data: completedToday }] = await Promise.all([
    supabase
      .from('attendances')
      .select('id')
      .eq('user_id', user.id)
      .is('check_out', null)
      .maybeSingle(), // any open shift, no date filter
    supabase
      .from('attendances')
      .select('id')
      .eq('user_id', user.id)
      .not('check_out', 'is', null)
      .gte('check_in', todayStart)
      .lte('check_in', todayEnd)
      .maybeSingle(),
  ])

  if (openShift) {
    return NextResponse.json({ error: 'Hai già un turno aperto' }, { status: 409 })
  }

  const isSplitShift = !completedToday

  const { data: attendance, error } = await supabase
    .from('attendances')
    .insert({
      user_id: user.id,
      restaurant_id: restaurant.id,
      check_in: nowUtc,
      is_split_shift: isSplitShift,
    })
    .select()
    .single()

  if (error) {
    return NextResponse.json({ error: 'Errore registrazione' }, { status: 500 })
  }

  return NextResponse.json({ attendance, splitShift: isSplitShift })
} else {
  // check-out: trova il turno aperto
  const { data: openAttendance, error: findError } = await supabase
    .from('attendances')
    .select('*')
    .eq('user_id', user.id)
    .is('check_out', null)
    .order('check_in', { ascending: false })
    .limit(1)

```

```

    .maybeSingle()

    if (findError || !openAttendance) {
      return NextResponse.json({ error: 'Nessun turno aperto trovato' }, { status: 404 })
    }

    const { data: attendance, error } = await supabase
      .from('attendances')
      .update({ check_out: nowUtc })
      .eq('id', openAttendance.id)
      .select()
      .single()

    if (error) {
      return NextResponse.json({ error: 'Errore registrazione uscita' }, { status: 500 })
    }

    return NextResponse.json({ attendance })
  }
}

```

src/app/api/telegram/setup/route.ts

```

import { NextRequest, NextResponse } from 'next/server'
import { createClient } from '@lib/supabase/server'
import { setWebhook, getMe } from '@lib/telegram/api'

// Endpoint una tantum per registrare il webhook del bot Telegram.
// Riservato ai manager: GET /api/telegram/setup
export async function GET(request: NextRequest) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Non autorizzato' }, { status: 401 })

  const { data: profile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', user.id)
    .single()
  if (profile?.role !== 'manager') {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }

  const secret = process.env.TELEGRAM_WEBHOOK_SECRET
  if (!secret) return NextResponse.json({ error: 'TELEGRAM_WEBHOOK_SECRET non configurato' }, { status: 500 })

  const webhookUrl = `${request.nextUrl.origin}/api/telegram/webhook`

  try {
    await setWebhook(webhookUrl, secret)
    const me = await getMe()
    return NextResponse.json({ ok: true, webhookUrl, bot: me })
  } catch (err) {
    const message = err instanceof Error ? err.message : String(err)
    return NextResponse.json({ ok: false, error: message, webhookUrl }, { status: 500 })
  }
}

```

```
import { NextRequest, NextResponse } from 'next/server'
import { handleUpdate } from '@lib/telegram/router'
import type { TgUpdate } from '@lib/telegram/types'

export async function POST(request: NextRequest) {
  const secret = request.headers.get('x-telegram-bot-api-secret-token')
  if (!process.env.TELEGRAM_WEBHOOK_SECRET || secret !== process.env.TELEGRAM_WEBHOOK_SECRET) {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 401 })
  }

  const update = (await request.json()) as TgUpdate

  try {
    await handleUpdate(update)
  } catch (err) {
    console.error('Errore webhook Telegram:', err)
  }

  // Risponde sempre 200: Telegram ritenta le consegne se non riceve OK.
  return NextResponse.json({ ok: true })
}
```



```

import { createClient } from '@lib/supabase/server'
import { createAdminClient } from '@lib/supabase/admin'
import { NextResponse } from 'next/server'

export async function PUT(request: Request) {
  // — Security Level 1: caller must be an authenticated manager —————
  const supabase = await createClient()
  const { data: { user: caller } } = await supabase.auth.getUser()

  if (!caller) {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 401 })
  }

  const { data: callerProfile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', caller.id)
    .single()

  if (callerProfile?.role !== 'manager') {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }

  // — Parse body —————
  const { id: targetId, password } = await request.json()

  if (!targetId || !password) {
    return NextResponse.json({ error: 'Campi obbligatori mancanti' }, { status: 400 })
  }

  if (typeof password !== 'string' || password.length < 6) {
    return NextResponse.json({ error: 'La password deve essere di almeno 6 caratteri' }, { status: 400 })
  }

  // — Security Level 2: target must NOT be a manager —————
  // Using admin client so RLS doesn't interfere with this lookup.
  const admin = createAdminClient()

  const { data: targetProfile } = await admin
    .from('profiles')
    .select('role, full_name')
    .eq('id', targetId)
    .single()

  if (!targetProfile) {
    return NextResponse.json({ error: 'Utente non trovato' }, { status: 404 })
  }

  if (targetProfile.role === 'manager') {
    return NextResponse.json(
      { error: 'Non puoi modificare la password di un altro Manager' },
      { status: 403 }
    )
  }

  // — Update password via Admin API (never exposed client-side) —————
  const { error } = await admin.auth.admin.updateUserById(targetId, { password })

  if (error) {
    return NextResponse.json({ error: error.message }, { status: 500 })
  }

  return NextResponse.json({ success: true })
}

```

`src/app/api/users/route.ts`

```

import { createClient } from '@lib/supabase/server'
import { createAdminClient } from '@lib/supabase/admin'
import { NextResponse } from 'next/server'

// Fake domain used to satisfy Supabase Auth email requirement for username-based accounts.
// Must never be a real domain - .local is a non-routable mDNS TLD.
const FAKE_DOMAIN = 'struttura.local'

// GET /api/users?role=consulente_lavoro
// Returns all profiles with the given role. Manager-only.
export async function GET(request: Request) {
  const caller = await getUserAdminContext()
  if (!caller || caller.role !== 'manager') {
    return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  }
  const { searchParams } = new URL(request.url)
  const role = searchParams.get('role')
  if (!role) return NextResponse.json({ error: 'role mancante' }, { status: 400 })

  // Scope consulti to the caller's managed restaurants.
  // managed_restaurant_ids = null → platform owner, no filter needed.
  // managed_restaurant_ids = [] → new/empty account, no shared restaurants yet → return empty.
  if (role === 'consulente_lavoro' && caller.managed_restaurant_ids !== null) {
    if (caller.managed_restaurant_ids.length === 0) return NextResponse.json([])
  }

  const admin = createAdminClient()
  let query = admin
    .from('profiles')
    .select('id, full_name, username, last_active_at, consultant_restaurant_ids, can_view_hours')
    .eq('role', role)
    .order('full_name')

  if (role === 'consulente_lavoro' && caller.managed_restaurant_ids !== null) {
    query = query.overlaps('consultant_restaurant_ids', caller.managed_restaurant_ids)
  }

  const { data, error } = await query

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })
  return NextResponse.json(data ?? [])
}

const USERNAME_RE = /^[a-z0-9._-]+$/

export function usernameToEmail(username: string): string {
  return `${username}@${FAKE_DOMAIN}`
}

interface Caller {
  id: string
  role: string
  restaurant_id: string | null
  is_direttore: boolean
  account_status: string
  managed_restaurant_ids: string[] | null
}

// Authorises a user-management request. Managers have full access; a
// "Direttore" (capo_servizio with is_direttore) is allowed but confined to
// its own restaurant (enforced by the callers below). Returns null otherwise.
async function getUserAdminContext(): Promise<Caller | null> {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return null
  const { data: profile } = await supabase
    .from('profiles')
    .select('role, restaurant_id, is_direttore, account_status, managed_restaurant_ids')
    .eq('id', user.id)
    .single()
  if (!profile) return null

  const isManager = profile.role === 'manager'

```

```

const isDirettore = profile.role === 'capo_servizio' && profile.is_direttore === true
if (!isManager && !isDirettore) return null

const p = profile as { account_status?: string; managed_restaurant_ids?: string[] | null }
return {
  id: user.id,
  role: profile.role,
  restaurant_id: profile.restaurant_id,
  is_direttore: profile.is_direttore === true,
  account_status: p.account_status ?? 'active',
  managed_restaurant_ids: p.managed_restaurant_ids ?? null,
}
}

const DEMO_READONLY = NextResponse.json(
  { error: 'Account in attesa di approvazione. La demo è in sola lettura.' },
  { status: 403 }
)

export async function POST(request: Request) {
  const caller = await getUserAdminContext()
  if (!caller) return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  if (caller.account_status === 'pending') return DEMO_READONLY

  const body = await request.json()
  const { username, password, full_name, role, can_post_bulletin, department } = body
  let { restaurant_id, is_direttore } = body

  const body2 = body
  const consultant_restaurant_ids: string[] = body2.consultant_restaurant_ids ?? []
  const can_view_hours: boolean = body2.can_view_hours ?? false

  if (!username || !password || !full_name || !role) {
    return NextResponse.json({ error: 'Campi obbligatori mancanti' }, { status: 400 })
  }
  if (!USERNAME_RE.test(username)) {
    return NextResponse.json(
      { error: 'Username: solo lettere minuscole, numeri, punti, trattini e underscore' },
      { status: 400 }
    )
  }
  const requiresDept = role !== 'manager' && role !== 'consulente_lavoro'
  if (requiresDept && !department) {
    return NextResponse.json({ error: 'Il reparto è obbligatorio' }, { status: 400 })
  }

  // Direttore (capo_servizio): cannot create managers or consulenti, confined to own restaurant.
  if (caller.role !== 'manager') {
    if (role === 'manager' || role === 'consulente_lavoro') {
      return NextResponse.json({ error: 'Non autorizzato a creare questo ruolo' }, { status: 403 })
    }
    restaurant_id = caller.restaurant_id
    is_direttore = false
  }
  // is_direttore only applies to capo_servizio
  const isDirettore = role === 'capo_servizio' ? is_direttore === true : false

  const email = usernameToEmail(username)
  const admin = createAdminClient()

  const { data: authData, error: authError } = await admin.auth.admin.createUser({
    email,
    password,
    email_confirm: true,
    user_metadata: { full_name, role },
  })

  if (authError) {
    // Make the "User already registered" error friendlier
    const msg = authError.message.includes('already registered')
      ? 'Username già in uso'
      : authError.message
    return NextResponse.json({ error: msg }, { status: 400 })
  }
}

```

```

const { data: profile, error: profileError } = await admin
  .from('profiles')
  .upsert({
    id: authData.user.id,
    full_name,
    username,
    role,
    department: role === 'consulente_lavoro' ? null : (department || null),
    restaurant_id: restaurant_id || null,
    can_post_bulletin: can_post_bulletin ?? false,
    is_direttore: isDirettore,
    consultant_restaurant_ids: role === 'consulente_lavoro' ? consultant_restaurant_ids : [],
    can_view_hours: role === 'consulente_lavoro' ? can_view_hours : false,
  })
  .select('*', restaurant:restaurants(id, name)')
  .single()

if (profileError) {
  return NextResponse.json({ error: profileError.message }, { status: 500 })
}

return NextResponse.json(profile, { status: 201 })
}

export async function PATCH(request: Request) {
  const caller = await getUserAdminContext()
  if (!caller) return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  if (caller.account_status === 'pending') return DEMO_READONLY

  const patchBody = await request.json()
  const { id, full_name, role, can_post_bulletin, department } = patchBody
  let { restaurant_id, is_direttore } = patchBody
  const consultant_restaurant_ids: string[] = patchBody.consultant_restaurant_ids ?? []
  const can_view_hours: boolean = patchBody.can_view_hours ?? false

  // — Campi AI scheduling —————
  const weekly_rest_days: number = patchBody.weekly_rest_days ?? 1
  const preferred_rest_day: number | null = patchBody.preferred_rest_day ?? null
  const primary_slot_ids: string[] = patchBody.primary_slot_ids ?? []
  const secondary_departments = patchBody.secondary_departments ?? []
  const weekly_hours_target: number | null = patchBody.weekly_hours_target ?? null
  const can_substitute_capo_servizio: boolean = patchBody.can_substitute_capo_servizio ?? false

  if (!id || !full_name || !role) {
    return NextResponse.json({ error: 'Campi obbligatori mancanti' }, { status: 400 })
  }

  const admin = createAdminClient()

  // Direttore: confined to its own restaurant, cannot manage managers or consulenti.
  if (caller.role !== 'manager') {
    const { data: target } = await admin
      .from('profiles').select('role, restaurant_id').eq('id', id).single()
    if (!target || target.restaurant_id !== caller.restaurant_id || target.role === 'manager' || target.role ===
'consulente_lavoro') {
      return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
    }
    if (role === 'manager' || role === 'consulente_lavoro') {
      return NextResponse.json({ error: 'Non autorizzato a promuovere a questo ruolo' }, { status: 403 })
    }
    restaurant_id = caller.restaurant_id
    is_direttore = false
  }
  const isDirettore = role === 'capo_servizio' ? is_direttore === true : false

  // secondary_departments è editabile solo dal manager (non dal direttore)
  const isCallerManager = caller.role === 'manager'

  const { data: profile, error } = await admin
    .from('profiles')
    .update({
      full_name,
      role,

```

```

    department: role === 'consulente_lavoro' ? null : (department || null),
    restaurant_id: restaurant_id || null,
    can_post_bulletin: can_post_bulletin ?? false,
    is_direttore: isDirettore,
    consultant_restaurant_ids: role === 'consulente_lavoro' ? consultant_restaurant_ids : [],
    can_view_hours: role === 'consulente_lavoro' ? can_view_hours : false,
    // AI scheduling
    weekly_rest_days,
    preferred_rest_day,
    primary_slot_ids,
    secondary_departments: isCallerManager ? secondary_departments : undefined,
    weekly_hours_target: weekly_hours_target || null,
    can_substitute_capo_servizio,
  })
  .eq('id', id)
  .select('*', restaurant:restaurants(id, name))
  .single()

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })

  return NextResponse.json(profile)
}

export async function DELETE(request: Request) {
  const caller = await getUserAdminContext()
  if (!caller) return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
  if (caller.account_status === 'pending') return DEMO_READONLY

  const { searchParams } = new URL(request.url)
  const id = searchParams.get('id')
  if (!id) return NextResponse.json({ error: 'ID mancante' }, { status: 400 })

  const admin = createAdminClient()

  // Direttore: can only delete users within its own restaurant, never managers.
  if (caller.role !== 'manager') {
    const { data: target } = await admin
      .from('profiles').select('role, restaurant_id').eq('id', id).single()
    if (!target || target.restaurant_id !== caller.restaurant_id || target.role === 'manager') {
      return NextResponse.json({ error: 'Non autorizzato' }, { status: 403 })
    }
  }

  const { error } = await admin.auth.admin.deleteUser(id)

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })

  return NextResponse.json({ success: true })
}

```

[src/app/consulente/dashboard/page.tsx](#)

```
import { createClient } from '@lib/supabase/server'
import { ConsultantDashboard } from '@components/consulente/ConsultantDashboard'

export default async function ConsulenteDashboardPage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  const { data: profile } = await supabase
    .from('profiles')
    .select('id, full_name, can_view_hours, consultant_restaurant_ids')
    .eq('id', user!.id)
    .single()

  const restaurantIds: string[] = (profile?.consultant_restaurant_ids as string[] | null) ?? []

  const { data: restaurants } = await supabase
    .from('restaurants')
    .select('id, name')
    .in('id', restaurantIds.length > 0 ? restaurantIds : ['00000000-0000-0000-0000-000000000000'])
    .order('name')

  return (
    <ConsultantDashboard
      userId={user!.id}
      fullName={profile?.full_name ?? ''}
      canViewHours={profile?.can_view_hours ?? false}
      restaurants={restaurants ?? []}
    />
  )
}
```

[src/app/consulente/layout.tsx](#)

```
import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'
import { ActivityTrackerMount } from '@components/consulente/ActivityTrackerMount'

export default async function ConsulenteLayout({ children }: { children: React.ReactNode }) {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) redirect('/login')

  const { data: profile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', user.id)
    .single()

  if (!profile || profile.role !== 'consulente_lavoro') redirect('/dashboard')

  return (
    <div className="min-h-[100dvh] bg-background text-foreground">
      <ActivityTrackerMount />
      {children}
    </div>
  )
}
```

```
import type { Metadata, Viewport } from 'next'
import { Geist } from 'next/font/google'
import { ThemeProvider } from '@components/shared/ThemeProvider'
import { OfflineSyncProvider } from '@components/shared/OfflineSyncProvider'
import './globals.css'

const geist = Geist({ subsets: ['latin'] })

export const metadata: Metadata = {
  title: 'inTurno - Gestione Turni, Presenze e ODS',
  description: 'Piattaforma aziendale per la gestione ottimizzata di turni, presenze e ordini di servizio',
  manifest: '/manifest.json',
  appleWebApp: {
    capable: true,
    statusBarStyle: 'black-translucent',
    title: 'inTurno',
    startupImage: '/icon-512.png',
  },
  icons: {
    apple: [
      { url: '/logo-branding.png', sizes: '180x180', type: 'image/png' },
    ],
  },
}

export const viewport: Viewport = {
  width: 'device-width',
  initialScale: 1,
  maximumScale: 1,
  userScalable: false,
  themeColor: '#18181b',
}

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="it" suppressHydrationWarning>
      <body className={` ${geist.className} antialiased overflow-x-hidden w-full max-w-full`} >
        <ThemeProvider attribute="class" defaultTheme="system" enableSystem disableTransitionOnChange>
          <OfflineSyncProvider>
            {children}
          </OfflineSyncProvider>
        </ThemeProvider>
      </body>
    </html>
  )
}
```


src/app/login/page.tsx

```
import Image from 'next/image'
import { LoginForm } from '@components/shared/LoginForm'

export default function LoginPage() {
  return (
    <main className="min-h-screen flex items-center justify-center bg-background p-4 transition-colors duration-300">
      <div className="w-full max-w-sm">

        {/* Logo + brand */}
        <div className="flex flex-col items-center mb-8 gap-4">
          <Image
            src="/logo-branding.png"
            alt="inTurno"
            width={88}
            height={88}
            className="rounded-[22px] shadow-[0_8px_24px_rgba(0,0,0,0.5)]"
            priority
          />
          <div className="text-center">
            <h1 className="text-2xl font-semibold tracking-tight text-foreground">inTurno</h1>
            <p className="text-muted-foreground mt-1 text-sm">Accedi al tuo account</p>
          </div>
        </div>

        {/* Glassmorphism card - dark: white/3; light: black/3 */}
        <div className="bg-black/[0.03] dark:bg-white/[0.03] backdrop-blur-md border border-black/10 dark:border-white/10 rounded-2xl p-6 shadow-[0_8px_32px_rgba(0,0,0,0.18)] dark:shadow-[0_8px_32px_rgba(0,0,0,0.5)]">
          <LoginForm />
        </div>

        <p className="text-center text-muted-foreground/50 text-xs mt-6">
          © {new Date().getFullYear()} inTurno
        </p>
      </div>
    </main>
  )
}
```

src/app/page.tsx

```
import { createClient } from '@lib/supabase/server'
import { redirect } from 'next/navigation'

export default async function HomePage() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()

  if (!user) redirect('/login')

  const { data: profile } = await supabase
    .from('profiles')
    .select('role')
    .eq('id', user.id)
    .single()

  if (!profile) redirect('/login')

  if (profile.role === 'dipendente') {
    redirect('/home')
  } else {
    redirect('/dashboard')
  }
}
```

src/app/register/page.tsx

```
import Image from 'next/image'
import { RegisterForm } from '@components/shared/RegisterForm'

export default function RegisterPage() {
  return (
    <main className="min-h-screen flex items-center justify-center bg-background p-4 transition-colors duration-300">
      <div className="w-full max-w-sm">

        <div className="flex flex-col items-center mb-8 gap-4">
          <Image
            src="/logo-branding.png"
            alt="inTurno"
            width={88}
            height={88}
            className="rounded-[22px] shadow-[0_8px_24px_rgba(0,0,0,0.5)]"
            priority
          />
          <div className="text-center">
            <h1 className="text-2xl font-semibold tracking-tight text-foreground">inTurno</h1>
            <p className="text-muted-foreground mt-1 text-sm">Crea il tuo account</p>
          </div>
        </div>

        <div className="bg-black/[0.03] dark:bg-white/[0.03] backdrop-blur-md border border-black/10 dark:border-white/10 rounded-2xl p-6 shadow-[0_8px_32px_rgba(0,0,0,0.18)] dark:shadow-[0_8px_32px_rgba(0,0,0,0.5)]">
          <RegisterForm />
        </div>

        <p className="text-center text-muted-foreground/50 text-xs mt-6">
          © {new Date().getFullYear()} inTurno
        </p>
      </div>
    </main>
  )
}
```

src/components/consulente/ActivityTrackerMount.tsx

```
'use client'
import { useActivityTracker } from '@hooks/useActivityTracker'

// Invisible client component that mounts the activity tracker in the consulente layout.
export function ActivityTrackerMount() {
  useActivityTracker()
  return null
}
```



```

'use client'
import { useState } from 'react'
import { FileSpreadsheet, MessageSquare, Logout } from 'lucide-react'
import { createClient } from '@lib/supabase/client'
import { useRouter } from 'next/navigation'
import { ThemeToggle } from '@components/shared/ThemeToggle'
import { ConsultantReportClient } from './ConsultantReportClient'
import { ConsultantInbox } from './ConsultantInbox'
import type { Restaurant } from '@types'

type Tab = 'report' | 'messaggi'

interface Props {
  userId: string
  fullName: string
  canViewHours: boolean
  restaurants: Pick<Restaurant, 'id' | 'name'>[]
}

export function ConsultantDashboard({ userId, fullName, canViewHours, restaurants }: Props) {
  const [tab, setTab] = useState<Tab>('report')
  const router = useRouter()

  async function handleLogout() {
    const supabase = createClient()
    await supabase.auth.signOut()
    router.push('/login')
  }

  return (
    <div className="min-h-[100dvh] flex flex-col">
      { /* Header */ }
      <header className="sticky top-0 z-30 flex items-center justify-between px-4 h-14 border-b border-border bg-background/95 backdrop-blur">
        <div>
          <span className="font-semibold text-sm">inTurno</span>
          <span className="ml-2 text-xs text-muted-foreground">· Consulente del Lavoro</span>
        </div>
        <div className="flex items-center gap-1">
          <ThemeToggle />
          <button
            onClick={handleLogout}
            className="flex items-center gap-1.5 text-xs text-muted-foreground hover:text-foreground px-2 py-1.5 rounded-md hover:bg-accent transition-colors"
          >
            <Logout className="w-3.5 h-3.5" />
            Esci
          </button>
        </div>
      </header>

      { /* Tab bar */ }
      <div className="flex border-b border-border bg-background">
        <TabBtn active={tab === 'report'} onClick={() => setTab('report')}>
          <FileSpreadsheet className="w-4 h-4" />
          Report
        </TabBtn>
        <TabBtn active={tab === 'messaggi'} onClick={() => setTab('messaggi')}>
          <MessageSquare className="w-4 h-4" />
          Messaggi
        </TabBtn>
      </div>

      { /* Content */ }
      <div className="flex-1 overflow-y-auto">
        {tab === 'report' && (
          <div className="p-4 lg:p-6">
            <div className="mb-4">
              <h1 className="text-lg font-semibold">Benvenuto, {fullName}</h1>
              <p className="text-sm text-muted-foreground mt-0.5">Vista in sola lettura · nessuna modifica consentita</p>
            </div>
            {restaurants.length === 0 ? (

```

```

        <p className="text-sm text-muted-foreground">Nessun ristorante assegnato al tuo account. Contatta il
manager.</p>
      ) : (
        <ConsultantReportClient
          restaurants={restaurants}
          canViewHours={canViewHours}
        />
      )}
    </div>
  )}
  {tab === 'messaggi' && (
    <div className="p-4 lg:p-6">
      <ConsultantInbox userId={userId} />
    </div>
  )}
</div>
</div>
)
}

function TabBtn({ active, onClick, children }: { active: boolean; onClick: () => void; children:
React.ReactNode }) {
  return (
    <button
      onClick={onClick}
      className={`flex items-center gap-2 px-5 py-3 text-sm font-medium border-b-2 transition-colors ${
        active
          ? 'border-primary text-foreground'
          : 'border-transparent text-muted-foreground hover:text-foreground'
      }`}
    >
      {children}
    </button>
  )
}

```



```

'use client'
import { useState, useEffect, useCallback } from 'react'
import { ChevronDown, ChevronUp, Paperclip, Send, CornerDownLeft, X } from 'lucide-react'
import { formatInTimeZone } from 'date-fns-tz'
import { it } from 'date-fns/locale'
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import { Button } from '@components/ui/button'
import type { ConsultantMessage } from '@types'
import { createClient } from '@lib/supabase/client'

const TZ = 'Europe/Rome'
const MAX_BYTES = 10 * 1024 * 1024

function fmtDate(iso: string | null) {
  if (!iso) return null
  return formatInTimeZone(new Date(iso), TZ, 'dd-MM-yyyy HH:mm', { locale: it })
}

interface Props {
  userId: string
}

export function ConsultantInbox({ userId }: Props) {
  const [messages, setMessages] = useState<ConsultantMessage[]>([])
  const [loading, setLoading] = useState(true)
  const [expandedId, setExpandedId] = useState<string | null>(null)
  const [managerId, setManagerId] = useState<string | null>(null)

  // Compose state
  const [showCompose, setShowCompose] = useState(false)
  const [msgTitle, setMsgTitle] = useState('')
  const [msgBody, setMsgBody] = useState('')
  const [files, setFiles] = useState<File[]>([])
  const [sending, setSending] = useState(false)
  const [sendError, setSendError] = useState<string | null>(null)
  const [replyToMsg, setReplyToMsg] = useState<ConsultantMessage | null>(null)

  const loadMessages = useCallback(async () => {
    const res = await fetch(`/api/consultant-messages?consultantId=${userId}`)
    if (!res.ok) return
    const data: ConsultantMessage[] = await res.json()
    setMessages(data)
    if (data.length > 0) setManagerId(data[0].manager_id)
    setLoading(false)
  }, [userId])

  useEffect(() => { loadMessages() }, [loadMessages])

  // Supabase Realtime – reflect manager inserts and deletes without a page refresh
  useEffect(() => {
    const supabase = createClient()
    const channel = supabase
      .channel('consultant-messages-inbox')
      .on(
        'postgres_changes',
        { event: '*', schema: 'public', table: 'consultant_messages', filter: `consultant_id=eq.${userId}` },
        (payload) => {
          if (payload.eventType === 'INSERT') {
            const msg = payload.new as ConsultantMessage
            setMessages(prev => {
              if (prev.some(m => m.id === msg.id)) return prev // duplicate guard
              return [msg, ...prev]
            })
            setManagerId(prev => prev ?? msg.manager_id)
          } else if (payload.eventType === 'DELETE') {
            const deletedId = (payload.old as { id: string }).id
            setMessages(prev => prev.filter(m => m.id !== deletedId))
            setExpandedId(prev => (prev === deletedId ? null : prev))
          }
        }
      )
    .subscribe()
  })
}

```

```

    return () => { supabase.removeChannel(channel) }
  }, [userId])

function handleReply(msg: ConsultantMessage) {
  setReplyToMsg(msg)
  setShowCompose(true)
  setMsgTitle(prev => prev.trim() ? prev : `Re: ${msg.title}`)
}

async function handleExpand(msg: ConsultantMessage) {
  const isAlreadyOpen = expandedId === msg.id
  setExpandedId(isAlreadyOpen ? null : msg.id)

  if (!isAlreadyOpen && msg.sent_by_manager && !msg.read_at) {
    const res = await fetch(`/api/consultant-messages/${msg.id}/read`, { method: 'POST' })
    if (res.ok) {
      const { read_at } = await res.json()
      setMessages(prev => prev.map(m => m.id === msg.id ? { ...m, read_at } : m))
    }
  }
}

async function handleDownload(msg: ConsultantMessage, attachment: { name: string; path: string }) {
  const res = await fetch(`/api/consultant-messages/${msg.id}/download`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ path: attachment.path, name: attachment.name }),
  })
  if (!res.ok) { alert('Impossibile scaricare il file'); return }

  const { signedUrl, downloaded_at } = await res.json()

  if (downloaded_at) {
    setMessages(prev => prev.map(m => m.id === msg.id ? { ...m, downloaded_at } : m))
  }

  const a = document.createElement('a')
  a.href = signedUrl
  a.download = attachment.name
  document.body.appendChild(a)
  a.click()
  document.body.removeChild(a)
}

function handleFileChange(e: React.ChangeEvent<HTMLInputElement>) {
  const selected = Array.from(e.target.files ?? [])
  const total = selected.reduce((sum, f) => sum + f.size, 0)
  if (total > MAX_BYTES) {
    setSendError('Il limite massimo per gli allegati è di 10 MB totali')
    e.target.value = ''
    return
  }
  setSendError(null)
  setFiles(selected)
}

async function handleSend() {
  if (!msgTitle.trim() || !msgBody.trim()) return
  if (!managerId) { setSendError('Nessun manager trovato'); return }

  const total = files.reduce((sum, f) => sum + f.size, 0)
  if (total > MAX_BYTES) {
    setSendError('Il limite massimo per gli allegati è di 10 MB totali')
    return
  }

  setSending(true)
  setSendError(null)
  try {
    const attachments: Array<{ name: string; path: string }> = []

    for (const file of files) {
      const formData = new FormData()

```



```

    formData.append('file', file)
    formData.append('consultantId', userId)

    const uploadRes = await fetch('/api/consultant-messages/upload', {
      method: 'POST',
      body: formData,
    })
    if (!uploadRes.ok) {
      const err = await uploadRes.json()
      throw new Error(err.error ?? 'Errore upload file')
    }
    const uploaded = await uploadRes.json()
    attachments.push({ name: file.name, path: uploaded.path })
  }

  const res = await fetch('/api/consultant-messages', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      consultantId: managerId,
      title: msgTitle,
      body: msgBody,
      attachments,
      reply_to_id: replyToMsg?.id ?? null,
    }),
  })
  if (!res.ok) {
    const err = await res.json()
    throw new Error(err.error ?? 'Errore invio')
  }
  setMsgTitle(''); setMsgBody(''); setFiles([]); setShowCompose(false); setReplyToMsg(null)
  await loadMessages()
} catch (e) {
  setSendError(e instanceof Error ? e.message : 'Errore invio')
} finally {
  setSending(false)
}
}

if (loading) {
  return <p className="text-sm text-muted-foreground">Caricamento messaggi...</p>
}

return (
  <div className="max-w-2xl space-y-4">
    <div className="flex items-center justify-between">
      <h2 className="text-sm font-semibold">Messaggi con il Manager</h2>
      <Button size="sm" variant="outline" onClick={() => setShowCompose(v => !v)}>
        <Send className="w-3.5 h-3.5 mr-1.5" />
        Nuovo messaggio
      </Button>
    </div>

    {showCompose && (
      <div className="border border-border rounded-md p-4 space-y-3 bg-card">
        {/* Reply context banner */}
        {replyToMsg && (
          <div className="flex items-start gap-2 rounded-sm bg-primary/5 border border-primary/20 px-3 py-2 text-xs">
            <CornerDownLeft className="w-3 h-3 text-primary shrink-0 mt-0.5" />
            <div className="flex-1 min-w-0">
              <span className="text-muted-foreground">In risposta a: </span>
              <span className="font-medium">&ldquo;{replyToMsg.title}&rdquo;</span>
              <p className="text-muted-foreground truncate mt-0.5">
                {replyToMsg.body.length > 80 ? replyToMsg.body.slice(0, 80) + '...' : replyToMsg.body}
              </p>
            </div>
            <button
              onClick={() => setReplyToMsg(null)}
              className="shrink-0 text-muted-foreground hover:text-foreground"
              title="Annulla risposta"
            >
              <X className="w-3 h-3" />
            </button>
          </div>
        )}
      </div>
    )}
  </div>
)

```

```

    </div>
  })
  <div className="space-y-1.5">
    <Label>Oggetto</Label>
    <Input value={msgTitle} onChange={e => setMsgTitle(e.target.value)} placeholder="Oggetto del
messaggio" className="h-9 rounded-sm" />
  </div>
  <div className="space-y-1.5">
    <Label>Messaggio</Label>
    <textarea
      value={msgBody}
      onChange={e => setMsgBody(e.target.value)}
      placeholder="Scrivi qui..."
      rows={4}
      className="w-full rounded-sm border border-input bg-background px-3 py-2 text-sm resize-none focus-
visible:outline-none focus-visible:ring-2 focus-visible:ring-ring"
    />
  </div>
  <div className="space-y-1.5">
    <Label>
      Allegati <span className="text-muted-foreground font-normal">(opzionale, max 10 MB totali)</span>
    </Label>
    <input
      type="file"
      multiple
      onChange={handleFileChange}
      className="text-sm file:mr-3 file:py-1 file:px-3 file:rounded file:border-0 file:bg-primary
file:text-primary-foreground"
    />
    {files.length > 0 && (
      <p className="text-xs text-muted-foreground">
        {files.length} file{files.length > 1 ? ' ' : ''} · {(files.reduce((s, f) => s + f.size, 0) /
1024).toFixed(0)} KB
      </p>
    )}
  </div>
  {sendError && <p className="text-xs text-destructive">{sendError}</p>}
  <div className="flex gap-2 justify-end">
    <Button variant="outline" size="sm" onClick={() => setShowCompose(false)}>Annulla</Button>
    <Button size="sm" onClick={handleSend} disabled={sending || !msgTitle.trim() || !msgBody.trim()}>
      {sending ? 'Invio...' : 'Invia'}
    </Button>
  </div>
</div>
))

{messages.length === 0 && (
  <p className="text-sm text-muted-foreground">Nessun messaggio ancora. Il manager ti contatterà da qui.</p>
)}

{(() => {
  const topLevel = messages
    .filter(m => !m.reply_to_id)
    .sort((a, b) => new Date(b.created_at).getTime() - new Date(a.created_at).getTime())
  const replyMap: Record<string, ConsultantMessage[]> = {}
  messages.filter(m => !!m.reply_to_id).forEach(m => {
    const pid = m.reply_to_id!
    if (!replyMap[pid]) replyMap[pid] = []
    replyMap[pid].push(m)
  })
  Object.values(replyMap).forEach(arr =>
    arr.sort((a, b) => new Date(a.created_at).getTime() - new Date(b.created_at).getTime())
  )
  return topLevel.flatMap(msg => [
    { msg, isReply: false },
    ...(replyMap[msg.id] ?? []).map(r => ({ msg: r, isReply: true })),
  ])
})().map(({ msg, isReply }) => {
  const isExpanded = expandedId === msg.id
  const fromMe = !msg.sent_by_manager
  return (
    <div key={msg.id} className={isReply ? 'ml-6 border-l-2 border-primary/20 pl-3' : ''}>
      <MessageRow
        msg={msg}

```

```

        isExpanded={isExpanded}
        fromMe={fromMe}
        isReply={isReply}
        onExpand={() => handleExpand(msg)}
        onDownload={att => handleDownload(msg, att)}
        onReply={() => handleReply(msg)}
      />
    </div>
  )
  }
}
</div>
)
}

function MessageRow({
  msg, isExpanded, fromMe, isReply, onExpand, onDownload, onReply,
}): {
  msg: ConsultantMessage
  isExpanded: boolean
  fromMe: boolean
  isReply?: boolean
  onExpand: () => void
  onDownload: (att: { name: string; path: string }) => void
  onReply: () => void
} {
  const unread = msg.sent_by_manager && !msg.read_at && !fromMe

  return (
    <div className={`border rounded-md overflow-hidden transition-colors ${unread ? 'border-primary/50 bg-primary/5' : 'border-border bg-card'}`}>
      <div className="flex items-center gap-1 pr-2">
        <button
          onClick={onExpand}
          className="flex-1 min-w-0 flex items-center gap-3 px-4 py-3 text-left hover:bg-accent/50 transition-colors"
        >
          <div className="flex-1 min-w-0">
            <div className="flex items-center gap-2">
              {unread && <span className="w-2 h-2 rounded-full bg-primary shrink-0" />}
              {isReply && <CornerDownLeft className="w-3.5 h-3.5 text-primary/50 shrink-0" />}
              <p className="text-sm font-medium truncate">{msg.title}</p>
            </div>
            <p className="text-xs text-muted-foreground mt-0.5">
              {fromMe ? 'Inviato da te' : 'Dal Manager'} · {fmtDate(msg.created_at)}
            </p>
          </div>
          <div className="flex items-center gap-1.5 shrink-0">
            {msg.attachments?.length > 0 && <Paperclip className="w-3.5 h-3.5 text-muted-foreground" />}
            {isExpanded ? <ChevronUp className="w-4 h-4 text-muted-foreground" /> : <ChevronDown className="w-4 h-4 text-muted-foreground" />}
          </div>
        </button>
        {/* Reply button */}
        <button
          onClick={onReply}
          className="shrink-0 p-1.5 rounded transition-colors text-muted-foreground hover:text-primary"
          title="Rispondi"
        >
          <CornerDownLeft className="w-4 h-4" />
        </button>
      </div>

      {isExpanded && (
        <div className="px-4 pb-4 space-y-3 border-t border-border pt-3">
          <p className="text-sm whitespace-pre-wrap">{msg.body}</p>

          {msg.attachments?.length > 0 && (
            <div className="space-y-1.5">
              <p className="text-xs font-medium text-muted-foreground uppercase tracking-wide">Allegati</p>
              {msg.attachments.map((att, i) => (
                <button
                  key={i}
                  onClick={() => onDownload(att)}
                  className="flex items-center gap-2 text-sm text-primary hover:underline"

```

```
        >
        <Paperclip className="w-3.5 h-3.5" />
        {att.name}
      </button>
    )}
  </div>
)}

<div className="space-y-0.5 pt-1">
  {msg.read_at && !fromMe && (
    <p className="text-[11px] text-muted-foreground">Letto il: {fmtDate(msg.read_at)}</p>
  )}
  {msg.downloaded_at && !fromMe && (
    <p className="text-[11px] text-muted-foreground">Allegato scaricato il: {fmtDate(msg.downloaded_at)}
</p>
  )}
</div>
</div>
)}
</div>
)
}
```



```

'use client'
import { useState, useMemo, useEffect, useCallback, useRef } from 'react'
import { createClient } from '@lib/supabase/client'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import { Card, CardContent } from '@components/ui/card'
import { FileSpreadsheet, Download } from 'lucide-react'
import { formatInTimeZone, fromZonedTime } from 'date-fns-tz'
import { getDaysInMonth, differenceInMinutes } from 'date-fns'
import { it } from 'date-fns/locale'
import type { Restaurant, AbsenceType } from '@types'
import { ABSENCE_CODES } from '@types'

const TZ = 'Europe/Rome'
const TARGET_HOURS_PER_DAY = 8.5

interface Props {
  restaurants: Pick<Restaurant, 'id' | 'name'>[]
  canViewHours: boolean
}

type EmployeeRec = { id: string; full_name: string; restaurant_id: string | null }
type AttendanceRec = { id: string; user_id: string; restaurant_id: string | null; check_in: string; check_out: string | null }
type AbsenceRec = { id: string; user_id: string; restaurant_id: string | null; type: AbsenceType; start_date: string; end_date: string }

type PreviewRow = {
  id: string
  full_name: string
  cells: Record<number, string>
  totalMins?: number
  totalLabel?: string
  diffLabel?: string
  diffMins?: number
}

function minutesToLabel(minutes: number): string {
  const h = Math.floor(minutes / 60)
  const m = Math.round(minutes % 60)
  return `${h}h ${String(m).padStart(2, '0')}m`
}

// — Identical colour mapping to ReportClient —————
const PRESENZE_CELL_BG: Record<string, string> = {
  P: 'bg-green-100 text-green-900 dark:bg-green-900/30 dark:text-green-300',
  PP: 'bg-green-200 text-green-900 dark:bg-green-800/40 dark:text-green-200',
  F: 'bg-violet-100 text-violet-900 dark:bg-violet-900/30 dark:text-violet-300',
  M: 'bg-blue-100 text-blue-900 dark:bg-blue-900/30 dark:text-blue-300',
  R: 'bg-red-100 text-red-900 dark:bg-red-900/30 dark:text-red-300',
  AI: 'bg-purple-100 text-purple-900 dark:bg-purple-900/30 dark:text-purple-300',
}

const ORE_CELL_BG: Record<string, string> = {
  F: 'bg-violet-100 text-violet-900 dark:bg-violet-900/30 dark:text-violet-300',
  M: 'bg-blue-100 text-blue-900 dark:bg-blue-900/30 dark:text-blue-300',
  R: 'bg-red-100 text-red-900 dark:bg-red-900/30 dark:text-red-300',
  AI: 'bg-purple-100 text-purple-900 dark:bg-purple-900/30 dark:text-purple-300',
}

const ORE_ABSENCE_CODES = new Set(['F', 'M', 'R', 'AI'])

// — CSS classes (no interactive class – cells are strictly read-only) —————
const thCls = 'px-2 py-1.5 text-center font-semibold bg-zinc-900 text-white dark:bg-zinc-800 whitespace-nowrap'
const tdCls = 'px-1.5 py-1 text-center text-xs border border-zinc-200 dark:border-zinc-700 whitespace-nowrap tabular-nums'
const tdNameCls = 'px-2 py-1 text-left text-xs font-medium border border-zinc-200 dark:border-zinc-700 whitespace-nowrap sticky left-0 bg-white dark:bg-zinc-950 z-10'

export function ConsultantReportClient({ restaurants, canViewHours }: Props) {
  const [selectedMonth, setSelectedMonth] = useState(() => formatInTimeZone(new Date(), TZ, 'yyyy-MM'))
  const [selectedRestaurants, setSelectedRestaurants] = useState<string[]>()
  restaurants.length === 1 ? [restaurants[0].id] : []

```

```

)
const [loading, setLoading] = useState<'presenze' | 'ore' | null>(null)
const [previewPresenze, setPreviewPresenze] = useState<PreviewRow[]>([])
const [previewOre, setPreviewOre] = useState<PreviewRow[]>([])
const [previewLoading, setPreviewLoading] = useState(false)
const genRef = useRef(0)

function toggleRestaurant(id: string) {
  setSelectedRestaurants(prev =>
    prev.includes(id) ? prev.filter(r => r !== id) : [...prev, id]
  )
}

const effectiveRestaurants = selectedRestaurants.length > 0
  ? selectedRestaurants
  : restaurants.map(r => r.id)

// — Identical preview logic to ReportClient (read-only: no editor refs needed) —
const loadPreview = useCallback(async (month: string, restaurantIds: string[]) => {
  const gen = ++genRef.current
  setPreviewLoading(true)

  const [year, monthNum] = month.split('-').map(Number)
  const daysCount = getDaysInMonth(new Date(year, monthNum - 1))
  const monthStart = `${month}-01`
  const monthEnd = `${month}-${String(daysCount).padStart(2, '0')}`
  const rangeStart = fromZonedTime(`${monthStart}T00:00:00`, TZ).toISOString()
  const rangeEnd = fromZonedTime(`${monthEnd}T23:59:59`, TZ).toISOString()

  const supabase = createClient()
  const [
    { data: employees },
    { data: attendances },
    { data: absences },
  ] = await Promise.all([
    supabase.from('profiles')
      .select('id, full_name, restaurant_id')
      .in('role', ['dipendente', 'capo_servizio'])
      .in('restaurant_id', restaurantIds)
      .order('full_name'),
    supabase.from('attendances')
      .select('id, user_id, restaurant_id, check_in, check_out')
      .in('restaurant_id', restaurantIds)
      .gte('check_in', rangeStart)
      .lte('check_in', rangeEnd),
    supabase.from('absences')
      .select('id, user_id, restaurant_id, type, start_date, end_date')
      .in('restaurant_id', restaurantIds)
      .eq('status', 'approved')
      .lte('start_date', monthEnd)
      .gte('end_date', monthStart),
  ])

  if (gen !== genRef.current) return

  const emps = (employees ?? []) as EmployeeRec[]
  const atts = (attendances ?? []) as AttendanceRec[]
  const abss = (absences ?? []) as AbsenceRec[]

  const presRows: PreviewRow[] = []
  const oreRows: PreviewRow[] = []

  for (const emp of emps) {
    const presCells: Record<number, string> = {}
    const oreCells: Record<number, string> = {}
    let totalMins = 0
    let workDays = 0

    for (let day = 1; day <= daysCount; day++) {
      const dateStr = `${month}-${String(day).padStart(2, '0')}`
      const absence = abss.find(a =>
        a.user_id === emp.id && a.start_date <= dateStr && a.end_date >= dateStr
      )
      if (absence) {

```

```

        const code = ABSENCE_CODES[absence.type]
        presCells[day] = code
        oreCells[day] = code
        continue
    }
    const daySessions = atts.filter(a =>
        a.user_id === emp.id &&
        formatInTimeZone(new Date(a.check_in), TZ, 'yyyy-MM-dd') === dateStr
    )
    if (daySessions.length > 0) {
        const hasOpen = daySessions.some(a => !a.check_out)
        const dayMins = daySessions.reduce((sum, a) => {
            if (!a.check_out) return sum
            return sum + differenceInMinutes(new Date(a.check_out), new Date(a.check_in))
        }, 0)
        presCells[day] = dayMins > 720 ? 'PP' : 'P'
        if (hasOpen && dayMins === 0) {
            oreCells[day] = 'In corso'
        } else {
            totalMins += dayMins
            workDays++
            oreCells[day] = minutesToLabel(dayMins)
        }
    }
}

const targetMins = workDays * TARGET_HOURS_PER_DAY * 60
const diffMins = totalMins - targetMins
const absDiff = Math.abs(diffMins)

presRows.push({ id: emp.id, full_name: emp.full_name, cells: presCells })
oreRows.push({
    id: emp.id,
    full_name: emp.full_name,
    cells: oreCells,
    totalMins,
    totallabel: minutesToLabel(totalMins),
    diffMins,
    diffLabel: `${diffMins >= 0 ? '+' : '-'}${minutesToLabel(absDiff)}`,
})
}

setPreviewPresenze(presRows)
setPreviewOre(oreRows)
setPreviewLoading(false)
}, [])

useEffect(() => {
    if (effectiveRestaurants.length === 0) return
    // eslint-disable-next-line react-hooks/set-state-in-effect
    loadPreview(selectedMonth, effectiveRestaurants)
    // eslint-disable-next-line react-hooks/exhaustive-deps
}, [selectedMonth, selectedRestaurants.join(','), loadPreview])

const { days, monthLabel } = useMemo(() => {
    const [y, m] = selectedMonth.split('-').map(Number)
    const n = getDaysInMonth(new Date(y, m - 1))
    const raw = new Intl.DateTimeFormat('it-IT', { month: 'long', year: 'numeric' }).format(new Date(y, m - 1, 1))
    return {
        days: Array.from({ length: n }, (_, i) => i + 1),
        monthLabel: raw.charAt(0).toUpperCase() + raw.slice(1),
    }
}, [selectedMonth])

async function downloadReport(type: 'presenze' | 'ore') {
    if (effectiveRestaurants.length === 0) return
    setLoading(type)
    try {
        const res = await fetch('/api/report', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ month: selectedMonth, restaurantIds: effectiveRestaurants, type }),
        })
        if (!res.ok) { const e = await res.json(); alert(e.error || 'Errore download'); return }
    }
}

```



```

const blob = await res.blob()
const url = URL.createObjectURL(blob)
const a = document.createElement('a')
a.href = url; a.download = `report-${type}-${selectedMonth}.xlsx`; a.click()
URL.revokeObjectURL(url)
} finally { setLoading(null) }
}

return (
  <div>
    <div className="space-y-4 max-w-lg mb-6">
      <div className="space-y-1.5">
        <Label>Mese</Label>
        <Input
          type="month"
          value={selectedMonth}
          onChange={e => setSelectedMonth(e.target.value)}
          className="w-auto"
        />
      </div>
    </div>

    {restaurants.length > 1 && (
      <div className="space-y-1.5">
        <Label>Ristoranti</Label>
        <div className="flex flex-wrap gap-2">
          {restaurants.map(r => (
            <button
              key={r.id}
              onClick={() => toggleRestaurant(r.id)}
              className={`px-3 py-1.5 rounded-full text-sm font-medium border transition-colors ${
                selectedRestaurants.includes(r.id)
                  ? 'bg-primary text-primary-foreground border-primary'
                  : 'bg-background border-border text-foreground hover:bg-accent'
              }`}
            >
              {r.name}
            </button>
          ))}
        </div>
      </div>
    )}

    <div className={`grid gap-4 ${canViewHours ? 'grid-cols-2' : 'grid-cols-1 max-w-[200px]}`}>
      <Card className="cursor-pointer hover:border-primary transition-colors" onClick={() =>
downloadReport('presenze')}>
        <CardContent className="pt-6 pb-5 text-center">
          <FileSpreadsheet className="w-8 h-8 mx-auto mb-3 text-emerald-500" />
          <p className="font-medium text-sm">Report Presenze</p>
          <p className="text-xs text-muted-foreground mt-1">P · F · M · R · AI</p>
          <Button size="sm" className="mt-4 w-full" disabled={loading === 'presenze'}>
            {loading === 'presenze' ? 'Generazione...' : <><Download className="w-4 h-4 mr-1" />Scarica</>}
          </Button>
        </CardContent>
      </Card>

      {canViewHours && (
        <Card className="cursor-pointer hover:border-primary transition-colors" onClick={() =>
downloadReport('ore')}>
          <CardContent className="pt-6 pb-5 text-center">
            <FileSpreadsheet className="w-8 h-8 mx-auto mb-3 text-blue-500" />
            <p className="font-medium text-sm">Report Ore</p>
            <p className="text-xs text-muted-foreground mt-1">Ore · Totale · Delta</p>
            <Button size="sm" className="mt-4 w-full" disabled={loading === 'ore'}>
              {loading === 'ore' ? 'Generazione...' : <><Download className="w-4 h-4 mr-1" />Scarica</>}
            </Button>
          </CardContent>
        </Card>
      )}
    </div>
  </div>

  </* — Read-Only Preview Tables —————
  SECURITY: cells have NO onClick, NO cursor-pointer, disabled={true} semantics.
  The consultant is a passive viewer – no editing is ever possible. */>

```

```

<div className="space-y-8">

  { /* Preview Presenze */ }
  <div>
    <div className="flex items-center gap-2 mb-2">
      <FileSpreadsheet className="w-4 h-4 text-emerald-500 shrink-0" />
      <h2 className="text-sm font-semibold">Preview Presenze / Turni</h2>
      <span className="text-xs text-muted-foreground">{{monthLabel}}</span>
      {previewLoading && <span className="text-xs text-muted-foreground animate-pulse">. Aggiornamento...</
span>}
    </div>
    <div className="w-full rounded-md border bg-muted/20 overflow-auto max-h-[340px]">
      <table className="border-collapse text-xs" style={{ minWidth: 'max-content' }}>
        <thead>
          <tr>
            <th className={` ${thCls} sticky left-0 z-20 text-left min-w-[160px]`} >Dipendente</th>
            {days.map(d => <th key={d} className={` ${thCls} min-w-[36px]`} >{d}</th>)}
            <th className={` ${thCls} min-w-[80px]`} >Note</th>
          </tr>
        </thead>
        <tbody>
          {previewLoading && previewPresenze.length === 0 ? (
            <tr><td colspan={days.length + 2} className="text-center py-6 text-muted-foreground text-
xs">Caricamento...</td></tr>
          ) : previewPresenze.length === 0 ? (
            <tr><td colspan={days.length + 2} className="text-center py-6 text-muted-foreground text-
xs">Nessun dipendente trovato</td></tr>
          ) : previewPresenze.map(row => (
            <tr key={row.id} className="even:bg-zinc-50 dark:even:bg-zinc-900/20">
              <td className={tdNameCls}>{row.full_name}</td>
              { /* SECURITY LOCK: no onClick, no cursor-pointer, disabled semantics */ }
              {days.map(d => {
                const code = row.cells[d] ?? ''
                return (
                  <td
                    key={d}
                    aria-disabled="true"
                    className={` ${tdCls} font-semibold select-none ${PRESENZE_CELL_BG[code] ?? ''}`}
                  >
                    {code}
                  </td>
                )
              })}
              <td className={tdCls} />
            </tr>
          ))}
        </tbody>
      </table>
    </div>
    {!previewLoading && previewPresenze.length > 0 && (
      <p className="text-[11px] text-muted-foreground mt-1.5">{previewPresenze.length} dipendenti · sola
lettura</p>
    )}
  </div>

  { /* Preview Ore – only if canViewHours */ }
  {canViewHours && (
    <div>
      <div className="flex items-center gap-2 mb-2">
        <FileSpreadsheet className="w-4 h-4 text-blue-500 shrink-0" />
        <h2 className="text-sm font-semibold">Preview Ore Lavorate</h2>
        <span className="text-xs text-muted-foreground">({monthLabel})</span>
        {previewLoading && <span className="text-xs text-muted-foreground animate-pulse">.
Aggiornamento...</span>}
      </div>
      <div className="w-full rounded-md border bg-muted/20 overflow-auto max-h-[340px]">
        <table className="border-collapse text-xs" style={{ minWidth: 'max-content' }}>
          <thead>
            <tr>
              <th className={` ${thCls} sticky left-0 z-20 text-left min-w-[160px]`} >Dipendente</th>
              {days.map(d => <th key={d} className={` ${thCls} min-w-[36px]`} >{d}</th>)}
              <th className={` ${thCls} min-w-[90px]`} >Totale Ore</th>
              <th className={` ${thCls} min-w-[90px]`} >Differenza</th>
              <th className={` ${thCls} min-w-[80px]`} >Note</th>
            </tr>
          </thead>
          <tbody>
            {previewLoading && previewOre.length === 0 ? (
              <tr><td colspan={days.length + 4} className="text-center py-6 text-muted-foreground text-
xs">Caricamento...</td></tr>
            ) : previewOre.length === 0 ? (
              <tr><td colspan={days.length + 4} className="text-center py-6 text-muted-foreground text-
xs">Nessun dipendente trovato</td></tr>
            ) : previewOre.map(row => (
              <tr key={row.id} className="even:bg-zinc-50 dark:even:bg-zinc-900/20">
                <td className={tdNameCls}>{row.full_name}</td>
                { /* SECURITY LOCK: no onClick, no cursor-pointer, disabled semantics */ }
                {days.map(d => {
                  const code = row.cells[d] ?? ''
                  return (
                    <td
                      key={d}
                      aria-disabled="true"
                      className={` ${tdCls} font-semibold select-none ${PRESENZE_CELL_BG[code] ?? ''}`}
                    >
                      {code}
                    </td>
                  )
                })}
                <td className={tdNameCls}>{row.totale_ore}</td>
                <td className={tdNameCls}>{row.differenza}</td>
                <td className={tdNameCls}>{row.note}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  )}

```

```

        </tr>
    </thead>
    <tbody>
        {previewLoading && previewOre.length === 0 ? (
            <tr><td colspan={days.length + 4} className="text-center py-6 text-muted-foreground text-
xs">Caricamento...</td></tr>
        ) : previewOre.length === 0 ? (
            <tr><td colspan={days.length + 4} className="text-center py-6 text-muted-foreground text-
xs">Nessun dipendente trovato</td></tr>
        ) : previewOre.map(row => (
            <tr key={row.id} className="even:bg-zinc-50 dark:even:bg-zinc-900/20">
                <td className={tdNameCls}>{row.full_name}</td>
                {/* SECURITY LOCK: no onClick, no cursor-pointer, disabled semantics */}
                {days.map(d => {
                    const val = row.cells[d] ?? ''
                    const isAbsCode = ORE_ABSENCE_CODES.has(val)
                    return (
                        <td
                            key={d}
                            aria-disabled="true"
                            className={` ${tdCls} select-none ${isAbsCode ? `font-semibold ${ORE_CELL_BG[val]} ??
''` : ''}` : ''` }
                        >
                            {val}
                        </td>
                    )
                })}
                <td className={` ${tdCls} font-medium`} >{row.totalLabel}</td>
                <td className={` ${tdCls} font-semibold ${ (row.diffMins ?? 0) >= 0 ? 'text-emerald-700
dark:text-emerald-400' : 'text-red-700 dark:text-red-400'}` >
                    {row.diffLabel}
                </td>
                <td className={tdCls} />
            </tr>
        ))}
    </tbody>
</table>
</div>
{!previewLoading && previewOre.length > 0 && (
    <p className="text-[11px] text-muted-foreground mt-1.5">{previewOre.length} dipendenti · sola
lettura</p>
)}
</div>
)}
</div>
</div>
)
}

```

`src/components/dipendente/AbsenceRequestDialog.tsx`

```

'use client'
import { useState, useEffect } from 'react'
import { createClient } from '@lib/supabase/client'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select'
import { X, CheckCircle2 } from 'lucide-react'
import type { AbsenceType } from '@types'
import { ABSENCE_LABELS } from '@types'

interface Props {
  userId: string
  restaurantId: string | null
  onClose: () => void
}

/* Classe condivisa per tutti i form field single-line – simmetria assoluta h-10
 * NB: `appearance-none` + `box-border` + `min-w-0` sono necessari per evitare che
 * <input type="date"> erediti la larghezza intrinseca del placeholder iOS Safari
 * e sbordi rispetto al contenitore. */
const fieldCls =
  'block w-full min-w-0 h-10 box-border appearance-none bg-background border border-input rounded-md px-3 text-foreground text-base ' +
  'focus:outline-none focus:ring-1 focus:ring-ring transition-colors'

const labelCls = 'text-muted-foreground text-xs font-medium uppercase tracking-wide mb-2 block'

/* Dipendenti non possono inserire "Assenza Ingiustificata" – solo il manager la assegna */
const DIPENDENTE_TYPES: AbsenceType[] = ['ferie', 'malattia', 'riposo']

export function AbsenceRequestDialog({ userId, restaurantId, onClose }: Props) {
  const [type, setType] = useState<AbsenceType>('ferie')
  const [startDate, setStartDate] = useState('')
  const [endDate, setEndDate] = useState('')
  const [notes, setNotes] = useState('')
  const [loading, setLoading] = useState(false)
  const [done, setDone] = useState(false)
  const [error, setError] = useState<string | null>(null)

  // Lock body scroll while dialog is open; restore on unmount
  useEffect(() => {
    document.body.style.overflow = 'hidden'
    return () => { document.body.style.overflow = '' }
  }, [])

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault()
    setLoading(true)
    setError(null)
    const supabase = createClient()
    const { error: insertError } = await supabase.from('absences').insert({
      user_id: userId,
      restaurant_id: restaurantId,
      type,
      start_date: startDate,
      end_date: endDate,
      notes: notes || null,
      created_by: userId,
      status: 'pending',
    })
    if (insertError) {
      setError('Errore durante l\'invio. Riprova.')
      setLoading(false)
      return
    }
    setDone(true)
    setLoading(false)
    setTimeout(onClose, 1800)
  }

  return (
    <div className="fixed inset-0 z-50 flex items-end justify-center" onClick={onClose}>
      <div className="absolute inset-0 bg-black/60 backdrop-blur-sm" />

      <div

```

```

        className="relative w-full max-w-lg bg-card border-t border-border rounded-t-md overflow-hidden"
        onClick={e => e.stopPropagation()}
        onTouchMove={e => e.stopPropagation()}
    >
    { /* Handle */ }
    <div className="flex justify-center pt-3 pb-2">
        <div className="w-10 h-1 rounded-full bg-border" />
    </div>

    <div className="px-5 pb-10">
        { /* Header */ }
        <div className="flex items-center justify-between mb-5">
            <h2 className="text-foreground text-lg font-semibold tracking-tight">Richiedi Assenza</h2>
            <button
                onClick={onClose}
                className="w-10 h-10 rounded-md bg-card border border-border flex items-center justify-center text-
muted-foreground hover:bg-accent hover:text-foreground transition-colors"
            >
                <X className="w-4 h-4" />
            </button>
        </div>

        {done ? (
            <div className="py-8 flex flex-col items-center gap-3 text-center">
                <CheckCircle2 className="w-12 h-12 text-emerald-600 dark:text-emerald-400" />
                <p className="text-foreground font-semibold text-lg">Richiesta inviata!</p>
                <p className="text-muted-foreground text-sm">Il manager riceverà la tua richiesta</p>
            </div>
        ) : (
            <form onSubmit={handleSubmit} className="space-y-4">
                { /* Tipo */ }
                <div>
                    <label className={labelCls}>Tipo di assenza</label>
                    <Select value={type} onChange={v => setType(v as AbsenceType)}>
                        <SelectTrigger className="h-10 rounded-md bg-background border-input text-foreground
focus:ring-1 focus:ring-ring">
                            <SelectValue />
                        </SelectTrigger>
                        <SelectContent>
                            {DIPENDENTE_TYPES.map(t => (
                                <SelectItem key={t} value={t}>{ABSENCE_LABELS[t]}</SelectItem>
                            ))}
                        </SelectContent>
                    </Select>
                </div>

                { /* Dal */ }
                <div>
                    <label className={labelCls}>Dal</label>
                    <input
                        type="date"
                        value={startDate}
                        onChange={e => setStartDate(e.target.value)}
                        required
                        className={fieldCls}
                    />
                </div>

                { /* Al */ }
                <div>
                    <label className={labelCls}>Al</label>
                    <input
                        type="date"
                        value={endDate}
                        onChange={e => setEndDate(e.target.value)}
                        required
                        min={startDate}
                        className={fieldCls}
                    />
                </div>

                { /* Note */ }
                <div>
                    <label className={labelCls}>

```

```

        Note <span className="normal-case text-muted-foreground/70">(opzionale)</span>
      </label>
      <textarea
        value={notes}
        onChange={e => setNotes(e.target.value)}
        rows={3}
        placeholder="Aggiungi una nota..."
        className="w-full bg-background border border-input rounded-md px-3 py-2.5 text-foreground text-
base placeholder:text-muted-foreground/60 focus:outline-none focus:ring-1 focus:ring-ring resize-none transition-
colors"
      />
    </div>

    {error && (
      <p className="text-destructive text-sm bg-destructive/10 border border-destructive/20 rounded-md
px-3 py-2.5">
        {error}
      </p>
    )}

    { /* Submit – touch target h-14 mobile */ }
    <button
      type="submit"
      disabled={loading || !startDate || !endDate}
      className="w-full h-14 rounded-md bg-primary hover:bg-primary/90 disabled:opacity-50 text-primary-
foreground font-semibold text-base transition-colors active:scale-[0.98]"
    >
      {loading ? 'Invio in corso...' : 'Invia Richiesta'}
    </button>
  </form>
</div>
</div>
</div>
)
}

```



```

'use client'
import { useState, useEffect } from 'react'
import Link from 'next/link'
import { usePathname } from 'next/navigation'
import { Home, ClipboardList, CalendarClock } from 'lucide-react'
import { createClient } from '@lib/supabase/client'
import { useBadging } from '@hooks/useBadging'
import { cn } from '@lib/utils'

const TABS = [
  { href: '/home', icon: Home, label: 'Home' },
  { href: '/home/ods', icon: ClipboardList, label: 'ODS' },
  { href: '/home/miei-turni', icon: CalendarClock, label: 'Turni' },
]

export function BottomNav() {
  const pathname = usePathname()
  const [unread, setUnread] = useState(0)
  useBadging(unread)

  useEffect(() => {
    const supabase = createClient()

    async function fetchUnread() {
      const { count } = await supabase
        .from('notifications')
        .select('id', { count: 'exact', head: true })
        .is('read_at', null)
      setUnread(count ?? 0)
    }

    fetchUnread()

    // Realtime: re-count on any INSERT or UPDATE to notifications
    const channel = supabase
      .channel('bottom_nav_notifications')
      .on(
        'postgres_changes',
        { event: '*', schema: 'public', table: 'notifications' },
        fetchUnread,
      )
      .subscribe()

    return () => { supabase.removeChannel(channel) }
  }, [])

  // When the user lands on the ODS page the badge will auto-clear
  // because OdsClient marks notifications as read (which fires UPDATE →
  // realtime triggers fetchUnread → count drops to 0)

  return (
    <nav className="fixed bottom-0 left-0 right-0 z-30 bg-card border-t border-border flex h-14">
      {TABS.map(({ href, icon: Icon, label }) => {
        const active = href === '/home' ? pathname === '/home' : pathname.startsWith(href + '/')

        const showBadge = href === '/home/ods' && unread > 0

        return (
          <Link
            key={href}
            href={href}
            className={cn(
              'flex-1 flex flex-col items-center justify-center gap-0.5 text-[10px] font-medium tracking-wide transition-colors relative',
              active ? 'text-primary' : 'text-muted-foreground hover:text-foreground'
            )}>
            >
            <div className="relative">
              <Icon className="w-5 h-5" />
              {showBadge && (
                <span className="absolute -top-1 -right-1.5 w-4 h-4 rounded-full bg-red-500 text-white text-[9px] font-bold flex items-center justify-center leading-none">
                  {unread > 9 ? '9+' : unread}
                </span>
              )}
            </div>
          </Link>
        )
      })}
    </nav>
  )
}

```

```
        </span>
      })
    </div>
    {label}
  </Link>
)
}}
</nav>
)
}
```

`src/components/dipendente/BulletinDrawer.tsx`

```

'use client'
import { useState, useEffect } from 'react'
import { createClient } from '@lib/supabase/client'
import { X, Megaphone, Globe, Store, Users, ChevronDown } from 'lucide-react'
import { formatInTimeZone } from 'date-fns-tz'

const TZ = 'Europe/Rome'

type Bulletin = {
  id: string
  title: string
  body: string
  target: 'all' | 'restaurant' | 'role' | 'users' | 'department'
  target_user_ids: string[]
  created_at: string
  author?: { full_name: string } | null
}

interface Props {
  userId: string
  onClose: () => void
}

function TargetChip({ target }: { target: Bulletin['target'] }) {
  const base = 'shrink-0 flex items-center gap-1 text-muted-foreground text-xs border border-border rounded-sm px-1.5 py-0.5 whitespace-nowrap'
  if (target === 'all') return <span className={base}><Globe className="w-3 h-3" /> Tutti</span>
  if (target === 'restaurant') return <span className={base}><Store className="w-3 h-3" /> Ristorante</span>
  if (target === 'department') return <span className={base}><Store className="w-3 h-3" /> Reparto</span>
  return <span className={base}><Users className="w-3 h-3" /> Per te</span>
}

export function BulletinDrawer({ userId, onClose }: Props) {
  const [bulletins, setBulletins] = useState<Bulletin[]>([])
  const [loading, setLoading] = useState(true)
  const [expandedId, setExpandedId] = useState<string | null>(null)
  // track IDs already inserted in this session to avoid redundant upserts
  const [sentReadIds, setSentReadIds] = useState<Set<string>>(new Set())

  // Lock body scroll while drawer is open; restore on unmount
  useEffect(() => {
    document.body.style.overflow = 'hidden'
    return () => { document.body.style.overflow = '' }
  }, [])

  useEffect(() => {
    const supabase = createClient()
    supabase
      .from('bulletins')
      .select('id, title, body, target, target_user_ids, created_at, author:profiles!created_by(full_name)')
      .order('created_at', { ascending: false })
      .then(({ data }) => {
        setBulletins((data as unknown as Bulletin[]) ?? [])
        setLoading(false)
      })
  }, [userId])

  function handleToggle(b: Bulletin) {
    const opening = expandedId !== b.id
    setExpandedId(opening ? b.id : null)

    // Only fire the read receipt on the first open of each bulletin
    if (opening && !sentReadIds.has(b.id)) {
      setSentReadIds(prev => new Set(prev).add(b.id))
      const supabase = createClient()
      supabase
        .from('bulletin_reads')
        .upsert(
          [{ bulletin_id: b.id, user_id: userId },
            { onConflict: 'bulletin_id,user_id', ignoreDuplicates: true }
          ]
        )
        .then(() => {})
    }
  }
}

```

```

}

return (
  <div className="fixed inset-0 z-50 flex items-end justify-center" onClick={onClose}>
    <div className="absolute inset-0 bg-black/60 backdrop-blur-sm" />

    <div
      className="relative w-full max-w-lg bg-card border-t border-border rounded-t-md overflow-hidden flex flex-
col max-h-[82vh]"
      onClick={e => e.stopPropagation()}
      onTouchMove={e => e.stopPropagation()}
    >
      {/* Handle */}
      <div className="flex justify-center pt-3 pb-1 shrink-0">
        <div className="w-10 h-1 rounded-full bg-border" />
      </div>

      {/* Header */}
      <div className="flex items-center justify-between px-5 py-3 border-b border-border shrink-0">
        <div className="flex items-center gap-2">
          <Megaphone className="w-4 h-4 text-muted-foreground" />
          <h2 className="text-foreground text-base font-semibold tracking-tight">Bacheca</h2>
        </div>
        <button
          onClick={onClose}
          className="w-10 h-10 rounded-md bg-card border border-border flex items-center justify-center text-
muted-foreground hover:bg-accent hover:text-foreground transition-colors"
        >
          <X className="w-4 h-4" />
        </button>
      </div>

      {/* Lista */}
      <div className="overflow-y-auto flex-1 px-5 py-4">
        {loading ? (
          <div className="space-y-3">
            {[...Array(3)].map((_, i) => (
              <div key={i} className="h-14 bg-muted rounded-md animate-pulse" />
            ))}
          </div>
        ) : bulletins.length === 0 ? (
          <div className="py-12 text-center">
            <Megaphone className="w-8 h-8 text-muted-foreground/40 mx-auto mb-3" />
            <p className="text-muted-foreground text-sm">Nessun comunicato</p>
          </div>
        ) : (
          <div className="divide-y divide-border border border-border rounded-md overflow-hidden pb-safe">
            {bulletins.map(b => {
              const isOpen = expandedId === b.id
              return (
                <div key={b.id} className="bg-muted">
                  {/* Row header – always visible, click to toggle */}
                  <button
                    onClick={() => handleToggle(b)}
                    className="w-full text-left px-4 py-3 flex items-center gap-3 hover:bg-accent/50 transition-
colors"
                  >
                    <ChevronDown
                      className={`w-4 h-4 text-muted-foreground shrink-0 transition-transform ${isOpen ?
'rotate-180' : ''}`}
                    />
                    <div className="flex-1 min-w-0">
                      <p className="text-foreground text-sm font-semibold leading-tight truncate">
                        {b.title}
                      </p>
                      <p className="text-muted-foreground text-xs mt-0.5 tabular-nums">
                        {b.author?.full_name && (
                          <span className="not-italic">{b.author.full_name} · </span>
                        )}
                        {formatInTimeZone(new Date(b.created_at), TZ, 'dd-MM-yyyy HH:mm')}
                      </p>
                    </div>
                    <TargetChip target={b.target} />
                  </button>
                </div>
              )
            })}
          </div>
        )
      </div>
    </div>
  )
)

```

```

        { /* Expanded body */
        {isOpen && (
            <div className="px-4 pb-4 pt-1 border-t border-border bg-card">
                <p className="text-foreground/80 text-sm leading-relaxed whitespace-pre-wrap">
                    {b.body}
                </p>
            </div>
        )}
    </div>
    )
    }}}}
</div>
</div>
</div>
)
}

```

`src/components/dipendente/EmployeeHomeClient.tsx`

```

'use client'
import { useState, useEffect, useCallback } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import { createClient } from '@lib/supabase/client'
import { QRScanner } from './QRScanner'
import { AbsenceRequestDialog } from './AbsenceRequestDialog'
import { BulletinDrawer } from './BulletinDrawer'
import { ThemeToggle } from '@components/shared/ThemeToggle'
import { LogOut, Camera, UserX, Megaphone, MapPin, ImageOff } from 'lucide-react'
import { formatInTimeZone } from 'date-fns-tz'
import { differenceInSeconds } from 'date-fns'
import { it } from 'date-fns/locale'
import { useRouter } from 'next/navigation'
import { usePushNotifications } from '@hooks/usePushNotifications'
import { useGeofence } from '@hooks/useGeofence'
import { PushNotificationBanner } from '@components/shared/PushNotificationBanner'
import { compressImage } from '@lib/compressImage'
import { saveToOfflineQueue } from '@lib/offlineSync'
import type { Profile, Attendance } from '@types'

const TZ = 'Europe/Rome'

interface Props {
  profile: Profile & { restaurant?: { id: string; name: string; latitude?: number | null; longitude?: number | null } | null } | null
  openAttendance: Attendance | null
  userId: string
}

function formatDuration(seconds: number): string {
  const h = Math.floor(seconds / 3600)
  const m = Math.floor((seconds % 3600) / 60)
  const s = seconds % 60
  return `${String(h).padStart(2, '0')}:${String(m).padStart(2, '0')}:${String(s).padStart(2, '0')}`
}

export function EmployeeHomeClient({ profile, openAttendance, userId }: Props) {
  const [now, setNow] = useState(new Date())
  const [attendance, setAttendance] = useState<Attendance | null>(openAttendance)
  const [showScanner, setShowScanner] = useState(false)
  const [showAbsence, setShowAbsence] = useState(false)
  const [showBulletin, setShowBulletin] = useState(false)
  const [loading, setLoading] = useState(false)
  const [message, setMessage] = useState<{ text: string; type: 'success' | 'error' } | null>(null)
  const [unreadCount, setUnreadCount] = useState(0)
  const [gpsFailed, setGpsFailed] = useState(false)
  const router = useRouter()

  const { permission: pushPermission, subscribe: subscribePush } = usePushNotifications()
  const { status: geoStatus, check: checkGeo, userCoordsRef } = useGeofence()

  useEffect(() => {
    const interval = setInterval(() => setNow(new Date()), 1000)
    return () => clearInterval(interval)
  }, [])

  useEffect(() => {
    const lastSeen = localStorage.getItem('bulletins_last_seen') ?? '1970-01-01T00:00:00Z'
    const supabase = createClient()
    supabase
      .from('bulletins')
      .select('id', { count: 'exact', head: true })
      .gt('created_at', lastSeen)
      .then(({ count }) => setUnreadCount(count ?? 0))
  }, [])

  const elapsedSeconds = attendance
    ? differenceInSeconds(now, new Date(attendance.check_in))
    : 0

  // Geofence-aware scan trigger - fail-closed
  const handleScanPress = useCallback(async () => {
    setMessage(null)

```



```

const result = await checkGeo(profile.restaurant?.latitude, profile.restaurant?.longitude)

if (result === 'outside') {
  // Parachute: GPS may read "too far" even when the employee is inside
  // (weak indoor signal). Offer the photo proof, same as a missing fix.
  setMessage({ text: 'Posizione troppo lontana. Se sei nel locale, timbra con la foto.', type: 'error' })
  setGpsFailed(true)
  return
}
if (result === 'denied' || result === 'unsupported') {
  // Fail-closed: no GPS consent or position unavailable → block
  setMessage({ text: 'Impossibile verificare il GPS. Se sei nel locale, timbra con la foto.', type: 'error' })
  setGpsFailed(true) // ← fallback only: show photo option
  return
}
// 'inside' → allow
setGpsFailed(false)
setShowScanner(true)
}, [checkGeo, profile.restaurant])

const handleScan = useCallback(async (qrSecret: string) => {
  setShowScanner(false)
  setLoading(true)
  setMessage(null)
  // Freeze timestamp NOW – before any async work – so offline queued items
  // carry the real scan time, not the later sync time.
  const frozenAt = new Date().toISOString()

  try {
    const res = await fetch('/api/scan', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        qr_secret: qrSecret,
        type: attendance ? 'out' : 'in',
        latitude: userCoordsRef.current?.latitude,
        longitude: userCoordsRef.current?.longitude,
        accuracy: userCoordsRef.current?.accuracy,
        frozenAt,
      }),
    })
    const data = await res.json()

    if (!res.ok) {
      setMessage({ text: data.error || 'Errore durante la timbratura', type: 'error' })
      return
    }

    if (attendance) {
      setAttendance(null)
      setMessage({ text: 'Uscita registrata con successo', type: 'success' })
    } else {
      setAttendance(data.attendance)
      setMessage({
        text: data.splitShift
          ? 'Turno spezzato registrato – il manager ne sarà informato'
          : 'Entrata registrata con successo',
        type: 'success',
      })
    }
    router.refresh()
  } catch (err) {
    if (err instanceof TypeError) {
      // Network unavailable – persist to IndexedDB queue with the frozen timestamp
      await saveToOfflineQueue('clock-in', {
        qr_secret: qrSecret,
        type: attendance ? 'out' : 'in',
        latitude: userCoordsRef.current?.latitude ?? null,
        longitude: userCoordsRef.current?.longitude ?? null,
        accuracy: userCoordsRef.current?.accuracy ?? null,
        frozenAt,
      }).catch(() => {})
      setMessage({

```

```

        text: 'Sei offline. Timbratura salvata sul dispositivo, si aggiornerà automaticamente.',
        type: 'success',
      })
    } else {
      setMessage({ text: 'Errore di rete, riprova', type: 'error' })
    }
  } finally {
    setLoading(false)
  }
}, [attendance, router])

async function handleFallbackPhoto(e: React.ChangeEvent<HTMLInputElement>) {
  const photo = e.target.files?.[0]
  if (!photo) return
  e.target.value = ''
  setLoading(true)
  setMessage(null)
  try {
    // Compress to max 800 px / 70 % quality before upload (~200 KB target)
    let compressed = photo
    try { compressed = await compressImage(photo, 800, 0.7) } catch { /* fallback to original */ }

    const fd = new FormData()
    fd.append('photo', compressed)
    fd.append('type', attendance ? 'out' : 'in')
    const res = await fetch('/api/clock-in-fallback', { method: 'POST', body: fd })
    const data = await res.json()
    if (!res.ok) {
      setMessage({ text: data.error ?? 'Errore timbratura di emergenza', type: 'error' })
      return
    }
    if (attendance) {
      setAttendance(null)
    } else {
      setAttendance(data.attendance)
    }
    setGpsFailed(false)
    setMessage({ text: 'Timbratura registrata. In attesa di conferma dal Manager.', type: 'success' })
    router.refresh()
  } catch {
    setMessage({ text: 'Errore di rete, riprova', type: 'error' })
  } finally {
    setLoading(false)
  }
}

async function handleLogout() {
  const supabase = createClient()
  await supabase.auth.signOut()
  router.push('/login')
}

function handleOpenBulletin() {
  setShowBulletin(true)
  localStorage.setItem('bulletins_last_seen', new Date().toISOString())
  setUnreadCount(0)
}

const timeDisplay = formatInTimeZone(now, TZ, 'HH:mm:ss')
const dateDisplay = formatInTimeZone(now, TZ, "EEEE d MMMM yyyy", { locale: it })
const isGeoChecking = geoStatus === 'checking'

return (
  <main className="h-[calc(100dvh-56px)] overflow-hidden bg-background text-foreground flex flex-col">
    { /* Header */ }
    <div className="flex items-center justify-between px-4 pt-4 pb-3">
      <div>
        <p className="text-muted-foreground text-xs capitalize tracking-wide">{dateDisplay}</p>
        <p className="font-semibold text-sm leading-tight">{profile.full_name}</p>
        {profile.restaurant?.name && (
          <p className="text-muted-foreground text-xs">{profile.restaurant.name}</p>
        )}
      </div>
    </div>
  </main>
)

```

```

<div className="flex items-center gap-1">
  <ThemeToggle />
  <button
    onClick={handleOpenBulletin}
    className="relative w-9 h-9 flex items-center justify-center rounded-md bg-card text-muted-foreground
    hover:bg-accent hover:text-foreground transition-colors border border-border"
    aria-label="Bacheca"
  >
    <Megaphone className="w-4 h-4" />
    {unreadCount > 0 && (
      <span className="absolute -top-1 -right-1 w-4 h-4 rounded-full bg-primary text-primary-foreground
      text-[10px] font-bold flex items-center justify-center leading-none">
        {unreadCount > 9 ? '9+' : unreadCount}
      </span>
    )}
  </button>
  <button
    onClick={handleLogout}
    className="w-9 h-9 flex items-center justify-center rounded-md bg-card text-muted-foreground hover:bg-
    accent hover:text-foreground transition-colors border border-border"
    aria-label="Logout"
  >
    <LogOut className="w-4 h-4" />
  </button>
</div>
</div>

<PushNotificationBanner permission={pushPermission} onSubscribe={subscribePush} />

<div className="flex-1 flex flex-col items-center justify-center gap-6 px-6">

  {/* Clock */}
  <motion.div
    initial={{ opacity: 0, y: -8 }}
    animate={{ opacity: 1, y: 0 }}
    transition={{ duration: 0.3 }}
    className="text-center"
  >
    <div className="text-7xl font-mono font-bold tracking-tight tabular-nums leading-none text-foreground">
      {timeDisplay}
    </div>
    <AnimatePresence>
      {attendance && (
        <motion.div
          initial={{ opacity: 0, height: 0 }}
          animate={{ opacity: 1, height: 'auto' }}
          exit={{ opacity: 0, height: 0 }}
          transition={{ duration: 0.25 }}
          className="mt-5 text-center overflow-hidden"
        >
          <p className="text-muted-foreground text-xs uppercase tracking-widest mb-1.5">Turno in corso</p>
          <div className="text-4xl font-mono text-emerald-600 dark:text-emerald-400 tabular-nums font-
semibold">
            {formatDuration(elapsedSeconds)}
          </div>
        </motion.div>
      )}
    </AnimatePresence>
  </motion.div>

  {/* Feedback messages */}
  <AnimatePresence mode="wait">
    {message && (
      <motion.div
        key={message.text}
        initial={{ opacity: 0, y: 6, scale: 0.97 }}
        animate={{ opacity: 1, y: 0, scale: 1 }}
        exit={{ opacity: 0, y: -6, scale: 0.97 }}
        transition={{ duration: 0.2 }}
        className={`w-full max-w-xs rounded-md px-4 py-3 text-sm text-center font-medium border ${
          message.type === 'success'
            ? 'bg-emerald-50 text-emerald-700 border-emerald-200 dark:bg-emerald-950/30 dark:text-
emerald-400 dark:border-emerald-900'
            : 'bg-destructive/10 text-destructive border-destructive/30'
        }}
      >
        {message}
      </motion.div>
    )}
  </AnimatePresence>

```

```

    }}
  >
    {message.text}
  </motion.div>
)}
</AnimatePresence>

{/* QR scan button */}
<motion.button
  initial={{ opacity: 0, y: 8 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.3, delay: 0.1 }}
  whileTap={{ scale: 0.97 }}
  onClick={handleScanPress}
  disabled={loading || isGeoChecking}
  className={`w-full max-w-xs h-14 rounded-md flex items-center justify-center gap-3 text-base font-
semibold border transition-colors active:scale-[0.98] disabled:opacity-50 ${
    attendance
      ? 'bg-amber-500 hover:bg-amber-600 text-white border-amber-600'
      : 'bg-primary hover:bg-primary/90 text-primary-foreground border-primary'
  }}
>
  {isGeoChecking
    ? <<MapPin className="w-5 h-5 animate-pulse" /> Verifica posizione...</>
    : loading
    ? 'Elaborazione...'
    : <<Camera className="w-5 h-5" />{attendance ? 'Scansiona Uscita' : 'Scansiona Ingresso'}}</>
}
</motion.button>

{process.env.NODE_ENV === 'development' && (
  <button
    onClick={() => handleScan('__SIMULATE__')}
    disabled={loading}
    className="text-xs text-muted-foreground underline"
  >
    [DEV] Simula Scansione
  </button>
)}

{/* GPS fallback - shown only when GPS denied/unavailable */}
{gpsFailed && (
  <motion.div
    initial={{ opacity: 0, y: 6 }}
    animate={{ opacity: 1, y: 0 }}
    className="w-full max-w-xs"
  >
    <label className={`w-full h-11 rounded-md flex items-center justify-center gap-2.5 text-sm font-medium
border cursor-pointer transition-colors
  ${loading ? 'opacity-50 pointer-events-none' : ''}
  border-amber-400 bg-amber-50 text-amber-800 hover:bg-amber-100
  dark:border-amber-600 dark:bg-amber-950/30 dark:text-amber-300 dark:hover:bg-amber-950/50`}
  >
      <ImageOff className="w-4 h-4 shrink-0" />
      Problemi col GPS? Timbra con foto
      <input
        type="file"
        accept="image/*"
        capture="environment"
        className="sr-only"
        onChange={handleFallbackPhoto}
        disabled={loading}
      />
    </label>
    <p className="text-[10px] text-muted-foreground text-center mt-1">
      La timbratura sarà confermata dal Manager
    </p>
  </motion.div>
)}

<motion.button
  initial={{ opacity: 0 }}
  animate={{ opacity: 1 }}
  transition={{ duration: 0.3, delay: 0.2 }}

```

```

        onClick={() => setShowAbsence(true)}
        className="flex items-center gap-2 text-muted-foreground hover:text-foreground transition-colors text-
sm"
      >
        <UserX className="w-4 h-4" />
        Richiedi Assenza
      </motion.button>
    </div>

    {showScanner && <QRScanner onScan={handleScan} onClose={() => setShowScanner(false)} />}
    {showAbsence && <AbsenceRequestDialog userId={userId} restaurantId={profile.restaurant_id} onClose={() =>
setShowAbsence(false)} />}
    {showBulletin && <BulletinDrawer userId={userId} onClose={() => setShowBulletin(false)} />}
  </main>
)
}

```

`src/components/dipendente/MieiTurniClient.tsx`

```

'use client'
import { useState, useEffect } from 'react'
import { motion } from 'framer-motion'
import { createClient } from '@lib/supabase/client'
import { CalendarClock, Clock } from 'lucide-react'
import { formatInTimeZone } from 'date-fns-tz'
import { it } from 'date-fns/locale'
import { EXTRAORDINARY_BADGE, STANDARD_BADGE, RIPOSO_BADGE } from '@lib/turnColors'
import type { Turn } from '@types'

const TZ = 'Europe/Rome'

const SPLIT_BADGE = 'bg-zinc-100 text-zinc-600 border-zinc-200 dark:bg-zinc-800/60 dark:text-zinc-300 dark:border-zinc-700'

interface Props {
  initialTurns: Turn[]
  userId: string
}

export function MieiTorniClient({ initialTurns, userId }: Props) {
  const [turns, setTurns] = useState<Turn[]>(initialTurns)

  // — Refetch al mount — la pagina è renderizzata lato server e può
  // arrivare da un layer di cache (Service Worker / Router Cache di Next)
  // con una versione vecchia della lista, in cui manca p.es. il secondo
  // segmento di un turno spezzato. Rileggiamo i turni dal client all'avvio
  // così la lista è sempre autorevole, indipendentemente dalla cache. —
  useEffect(() => {
    const supabase = createClient()
    supabase
      .from('turns')
      .select('*')
      .eq('user_id', userId)
      .order('date', { ascending: true })
      .order('start_time', { ascending: true })
      .then(({ data }) => {
        if (data) setTurns(data as unknown as Turn[])
      })
  }, [userId])

  // — Realtime — i turni assegnati dal manager/capo servizio si
  // aggiornano qui istantaneamente, senza ricaricare la pagina —————
  useEffect(() => {
    const supabase = createClient()
    const channel = supabase
      .channel('rt-miei-turni')
      .on(
        'postgres_changes',
        { event: '*', schema: 'public', table: 'turns', filter: `user_id=eq.${userId}` },
        (payload) => {
          if (payload.eventType === 'INSERT') {
            const rec = payload.new as Turn
            setTurns(prev => prev.some(t => t.id === rec.id) ? prev : [...prev, rec])
          } else if (payload.eventType === 'UPDATE') {
            const rec = payload.new as Turn
            setTurns(prev => prev.map(t => t.id === rec.id ? rec : t))
          } else if (payload.eventType === 'DELETE') {
            const deletedId = (payload.old as { id: string }).id
            setTurns(prev => prev.filter(t => t.id !== deletedId))
          }
        }
      )
      .subscribe()

    return () => { supabase.removeChannel(channel) }
  }, [userId])

  const todayStr = formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd')

  const sorted = [...turns].sort((a, b) =>
    a.date === b.date ? a.start_time.localeCompare(b.start_time) : a.date.localeCompare(b.date)
  )

```

```

// Raggruppa i turni per giornata: un turno spezzato (più fasce nello
// stesso giorno) diventa un'unica riga con tutti gli orari insieme.
const byDate = new Map<string, Turn[]>()
for (const t of sorted) {
  const list = byDate.get(t.date) ?? []
  list.push(t)
  byDate.set(t.date, list)
}
const groups = Array.from(byDate, ([date, dayTurns]) => ({ date, turns: dayTurns }))
const upcoming = groups.filter(g => g.date >= todayStr)
const past = groups.filter(g => g.date < todayStr).reverse()

return (
  <main className="min-h-screen bg-background text-foreground flex flex-col">
    <div className="flex items-center gap-3 px-4 pt-5 pb-3 border-b border-border">
      <CalendarClock className="w-5 h-5 text-muted-foreground shrink-0" />
      <div>
        <h1 className="font-semibold text-sm leading-tight">I Miei Turni</h1>
        <p className="text-muted-foreground text-xs">Turni assegnati, passati e futuri</p>
      </div>
    </div>

    <div className="flex-1 overflow-y-auto px-4 pt-4 pb-6 space-y-6">
      <section>
        <p className="text-[10px] font-semibold uppercase tracking-widest text-muted-foreground mb-2">
          Prossimi Turni
        </p>
        {upcoming.length === 0 ? (
          <EmptyState text="Nessun turno programmato" />
        ) : (
          <div className="space-y-1.5">
            {upcoming.map(g => <TurnRow key={g.date} group={g} />)}
          </div>
        )}
      </section>

      <section>
        <p className="text-[10px] font-semibold uppercase tracking-widest text-muted-foreground mb-2">
          Turni Passati
        </p>
        {past.length === 0 ? (
          <EmptyState text="Nessun turno passato" />
        ) : (
          <div className="space-y-1.5">
            {past.map(g => <TurnRow key={g.date} group={g} />)}
          </div>
        )}
      </section>
    </div>
  </main>
)
}

function EmptyState({ text }: { text: string }) {
  return (
    <motion.div initial={{ opacity: 0 }} animate={{ opacity: 1 }} className="py-10 text-center">
      <CalendarClock className="w-7 h-7 text-muted-foreground/30 mx-auto mb-2" />
      <p className="text-muted-foreground text-sm">{text}</p>
    </motion.div>
  )
}

function TurnRow({ group }: { group: { date: string; turns: Turn[] } }) {
  const dateLabel = formatInTimeZone(`${group.date}T12:00:00Z`, TZ, 'EEEE d MMMM', { locale: it })

  const isRest = group.turns.some(t => t.is_rest_day)
  const work = group.turns.filter(t => !t.is_rest_day)
  const isExtra = !isRest && work.some(t => t.is_extraordinary)
  const isSplit = work.length > 1
  const dept = work.find(t => t.department)?.department ?? null
  // Tutte le fasce della giornata su un'unica stringa (turno spezzato incluso)
  const ranges = work.map(t => `${t.start_time.slice(0, 5)} - ${t.end_time.slice(0, 5)}`).join(' / ')
  const notes = Array.from(new Set(group.turns.map(t => t.notes).filter(Boolean))).join(' · ')

```



```

const badgeBase = 'text-[10px] px-1.5 py-0.5 rounded-sm border font-medium whitespace-nowrap'

return (
  <div className="bg-card border border-border rounded-sm px-3 py-2.5 flex items-center gap-3">
    <Clock className="w-4 h-4 text-muted-foreground shrink-0" />
    <div className="flex-1 min-w-0">
      <div className="flex items-center gap-2 flex-wrap mb-0.5">
        <span className="text-sm font-medium text-foreground capitalize">{dateLabel}</span>
        {isRest ? (
          <span className={` ${badgeBase} ${RIPOSO_BADGE}`}>Riposo</span>
        ) : isExtra ? (
          <span className={` ${badgeBase} ${EXTRAORDINARY_BADGE}`}>Straordinario</span>
        ) : (
          <span className={` ${badgeBase} ${STANDARD_BADGE}`}>Standard</span>
        )}
        {isSplit && <span className={` ${badgeBase} ${SPLIT_BADGE}`}>Spezzato</span>}
      </div>
      {!isRest && (
        <p className="text-xs text-muted-foreground">
          {ranges}
          {dept ? ` · ${dept}` : ''}
        </p>
      )}
      {notes && <p className="text-xs text-muted-foreground/80 mt-0.5">{notes}</p>}
    </div>
  </div>
)
}

```

`src/components/dipendente/OdsClient.tsx`

```

'use client'
import { useState, useEffect, useCallback } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import { createClient } from '@lib/supabase/client'
import { saveToOfflineQueue } from '@lib/offlineSync'
import { ClipboardList, Check } from 'lucide-react'
import { formatInTimeZone } from 'date-fns-tz'
import { it } from 'date-fns/locale'
import { ODS_TYPE_LABELS } from '@types'
import type { OdsTask, OdsTaskType } from '@types'

const TZ = 'Europe/Rome'

const FILTERS: { key: 'tutte' | OdsTaskType; label: string }[] = [
  { key: 'tutte', label: 'Tutte' },
  { key: 'quotidiana', label: 'Quotidiane' },
  { key: 'settimanale', label: 'Settimanali' },
  { key: 'bisettimanale', label: 'Bisettimanali' },
  { key: 'straordinaria', label: 'Straordinarie' },
]

interface Props {
  tasks: OdsTask[]
  completedTaskIds: string[]
  userId: string
  userDepartment: string | null
}

export function OdsClient({ tasks, completedTaskIds, userId, userDepartment }: Props) {
  const [taskList, setTaskList] = useState<OdsTask[]>(tasks)
  const [completed, setCompleted] = useState<Set<string>>(new Set(completedTaskIds))
  const [filter, setFilter] = useState<'tutte' | OdsTaskType>('tutte')
  const [offlineMsg, setOfflineMsg] = useState<string | null>(null)

  useEffect(() => {
    if (!offlineMsg) return
    const t = setTimeout(() => setOfflineMsg(null), 5000)
    return () => clearTimeout(t)
  }, [offlineMsg])

  // — Refetch al mount — la pagina è renderizzata lato server e può
  // arrivare da un layer di cache (Service Worker / Router Cache di Next)
  // con una lista vecchia, in cui mancano gli ordini di servizio appena
  // creati dal manager. Rileggiamo i task dal client all'avvio così la
  // lista è sempre autorevole. La RLS limita ai task di competenza. —
  const reloadTasks = useCallback(async () => {
    const supabase = createClient()
    const { data } = await supabase
      .from('ods_tasks')
      .select('*', assignee:profiles!assigned_to(id, full_name)')
      .order('created_at', { ascending: false })
    if (data) setTaskList(data as unknown as OdsTask[])
  }, [])

  useEffect(() => { reloadTasks() }, [reloadTasks])

  useEffect(() => {
    const supabase = createClient()
    supabase
      .from('notifications')
      .update({ read_at: new Date().toISOString() })
      .is('read_at', null)
      .then(() => {})
  }, [])

  const todayName = formatInTimeZone(new Date(), TZ, 'EEEE', { locale: it }).toLowerCase()

  const visible = taskList.filter(t => {
    if (t.type === 'quotidiana' || t.type === 'straordinaria') return true
    return t.recurrence_days.some(d => d.toLowerCase() === todayName)
  })

  const personalTasks = visible.filter(t => t.assigned_to === userId)

```

```

const departmentTasks = visible.filter(t => t.assigned_to === null)
const filtered = filter === 'tutte'
  ? departmentTasks
  : departmentTasks.filter(t => t.type === filter)

async function handleToggle(taskId: string) {
  const frozenAt = new Date().toISOString()
  const wasCompleted = completed.has(taskId)

  // Optimistic update - UI stays responsive regardless of network
  setCompleted(prev => {
    const next = new Set(prev)
    wasCompleted ? next.delete(taskId) : next.add(taskId)
    return next
  })

  try {
    const supabase = createClient()
    if (wasCompleted) {
      await supabase.from('ods_completions').delete().eq('task_id', taskId).eq('user_id', userId)
    } else {
      await supabase.from('ods_completions').insert({ task_id: taskId, user_id: userId })
    }
  } catch (err) {
    if (err instanceof TypeError) {
      // Network down - save to IndexedDB queue with the frozen timestamp
      await saveToOfflineQueue('ods-toggle', {
        task_id: taskId,
        user_id: userId,
        action: wasCompleted ? 'uncomplete' : 'complete',
        frozenAt,
      }).catch(() => {})
      setOfflineMsg('Sei offline. Salvato sul dispositivo, si aggiornerà automaticamente.')
    } else {
      // Server/auth error - revert optimistic update
      setCompleted(prev => {
        const next = new Set(prev)
        wasCompleted ? next.add(taskId) : next.delete(taskId)
        return next
      })
    }
  }
}

return (
  <main className="min-h-screen bg-background text-foreground flex flex-col">
    <AnimatePresence>
      {offlineMsg && (
        <motion.div
          initial={{ opacity: 0, y: -8 }}
          animate={{ opacity: 1, y: 0 }}
          exit={{ opacity: 0, y: -8 }}
          transition={{ duration: 0.2 }}
          className="mx-4 mt-3 px-3 py-2 rounded-md text-xs font-medium bg-emerald-50 text-emerald-700 border
border-emerald-200 dark:bg-emerald-950/30 dark:text-emerald-400 dark:border-emerald-900"
        >
          {offlineMsg}
        </motion.div>
      )}
    </AnimatePresence>
    <div className="flex items-center gap-3 px-4 pt-5 pb-3 border-b border-border">
      <ClipboardList className="w-5 h-5 text-muted-foreground shrink-0" />
      <div>
        <h1 className="font-semibold text-sm leading-tight">Ordine di Servizio</h1>
        <p className="text-muted-foreground text-xs">{userDepartment ?? 'Tutti i reparti'}</p>
      </div>
    </div>

    <div className="flex-1 overflow-y-auto px-4 pt-4 pb-6 space-y-6">

      {personalTasks.length > 0 && (
        <section>
          <p className="text-[10px] font-semibold uppercase tracking-widest text-muted-foreground mb-2">
            Istruzioni Personali
          </p>

```

```

    </p>
    <div className="space-y-1.5">
      <AnimatePresence initial={false}>
        {personalTasks.map((t, i) => (
          <TaskRow key={t.id} index={i} task={t} completed={completed.has(t.id)} onToggle={() =>
handleToggle(t.id)} />
        ))}
      </AnimatePresence>
    </div>
  </section>
)}

<section>
  <p className="text-[10px] font-semibold uppercase tracking-widest text-muted-foreground mb-2">
    Istruzioni di Reparto
  </p>

  <div className="flex gap-1.5 flex-wrap mb-3">
    {FILTERS.map(f => (
      <button
        key={f.key}
        onClick={() => setFilter(f.key)}
        className={`text-xs px-2.5 py-1 rounded-sm border transition-colors ${
          filter === f.key
            ? 'bg-primary text-primary-foreground border-primary'
            : 'bg-card text-muted-foreground border-border hover:bg-accent hover:text-foreground'
          }`}
      >
        {f.label}
      </button>
    ))}
  </div>

  {filtered.length === 0 ? (
    <motion.div initial={{ opacity: 0 }} animate={{ opacity: 1 }} className="py-10 text-center">
      <ClipboardList className="w-7 h-7 text-muted-foreground/30 mx-auto mb-2" />
      <p className="text-muted-foreground text-sm">Nessuna istruzione</p>
    </motion.div>
  ) : (
    <div className="space-y-1.5">
      <AnimatePresence initial={false}>
        {filtered.map((t, i) => (
          <TaskRow key={t.id} index={i} task={t} completed={completed.has(t.id)} onToggle={() =>
handleToggle(t.id)} />
        ))}
      </AnimatePresence>
    </div>
  )}
</section>
</div>
</main>
)
}

function TaskRow({ task, completed, onToggle, index }: {
  task: OdsTask; completed: boolean; onToggle: () => void; index: number
}) {
  return (
    <motion.button
      initial={{ opacity: 0, y: 8 }}
      animate={{ opacity: 1, y: 0 }}
      exit={{ opacity: 0, y: -6, transition: { duration: 0.15 } }}
      transition={{ duration: 0.2, delay: index * 0.04 }}
      whileTap={{ scale: 0.985 }}
      onClick={onToggle}
      className={`w-full flex items-start gap-3 px-3 py-2.5 rounded-sm border text-left transition-colors ${
        completed
          ? 'bg-muted/50 border-border/50'
          : 'bg-card border-border hover:bg-accent/30 active:bg-accent/50'
        }`}
    >
    <motion.div
      animate={completed ? { scale: [1, 1.25, 1] } : { scale: 1 }}
      transition={{ duration: 0.18 }}

```

```

        className={`mt-0.5 w-4 h-4 rounded-sm border shrink-0 flex items-center justify-center transition-colors $
    {
        completed ? 'bg-primary border-primary' : 'border-input bg-background'
    }}
    >
    <AnimatePresence>
      {completed && (
        <motion.div
          initial={{ scale: 0, opacity: 0 }}
          animate={{ scale: 1, opacity: 1 }}
          exit={{ scale: 0, opacity: 0 }}
          transition={{ duration: 0.15 }}
        >
          <Check className="w-2.5 h-2.5 text-primary-foreground" />
        </motion.div>
      )}
    </AnimatePresence>
  </motion.div>

  <div className="flex-1 min-w-0">
    <p className={`text-sm leading-snug ${
      completed ? 'line-through text-zinc-400 dark:text-zinc-500' : 'text-foreground'
    }`}>
      {task.title}
    </p>
    {(task.type === 'settimanale' || task.type === 'bisettimanale') &&
      task.recurrence_days.length > 0 && (
        <p className="text-xs text-muted-foreground mt-0.5 capitalize">
          {task.recurrence_days.join(', ')}
        </p>
      )}
  </div>

  {task.type === 'straordinaria' && (
    <span className="text-[10px] px-1.5 py-0.5 rounded-sm bg-amber-100 text-amber-700 dark:bg-amber-950/30
      dark:text-amber-400 border border-amber-200 dark:border-amber-800 shrink-0 self-start whitespace-nowrap">
      Straord.
    </span>
  )}
</motion.button>
)
}

```

`src/components/dipendente/QRScanner.tsx`

```

'use client'
import { useEffect, useRef } from 'react'
import { X } from 'lucide-react'

interface Props {
  onScan: (result: string) => void
  onClose: () => void
}

export function QRScanner({ onScan, onClose }: Props) {
  const scannerRef = useRef<unknown>(null)
  const containerRef = useRef<HTMLDivElement>(null)

  // Lock body scroll while scanner overlay is open
  useEffect(() => {
    document.body.style.overflow = 'hidden'
    return () => { document.body.style.overflow = '' }
  }, [])

  useEffect(() => {
    let html5QrCode: unknown

    async function startScanner() {
      const { Html5Qrcode } = await import('html5-qrcode')

      if (!containerRef.current) return

      html5QrCode = new Html5Qrcode('qr-reader')
      scannerRef.current = html5QrCode

      try {
        await (html5QrCode as { start: (constraint: unknown, config: unknown, onSuccess: (text: string) => void,
onError: () => void) => Promise<void> }).start(
          { facingMode: 'environment' },
          { fps: 10, qrbox: { width: 250, height: 250 } },
          (text: string) => {
            onScan(text)
          },
          () => {}
        )
      } catch (err) {
        console.error('Camera error:', err)
      }
    }

    startScanner()

    return () => {
      const qr = scannerRef.current as { stop?: () => Promise<void>; clear?: () => Promise<void> } | null
      if (qr?.stop) {
        qr.stop().then(() => qr.clear?.()).catch(() => {})
      }
    }
  }, [onScan])

  return (
    <div className="fixed inset-0 z-50 bg-black flex flex-col">
      <div className="flex items-center justify-between p-4">
        <p className="text-white font-medium">Inquadra il QR Code del ristorante</p>
        <button
          onClick={onClose}
          className="w-10 h-10 flex items-center justify-center rounded-md bg-white/10 text-white hover:bg-white/20 transition-colors"
        >
          <X className="w-5 h-5" />
        </button>
      </div>
      <div className="flex-1 flex items-center justify-center">
        <div ref={containerRef} id="qr-reader" className="w-full max-w-sm" />
      </div>
    </div>
  )
}

```

`src/components/manager/AccountPendentClient.tsx`

```

'use client'

import { useState, useTransition } from 'react'
import { approveAccount, rejectAccount } from '@app/actions/adminActions'
import { Button } from '@components/ui/button'
import { formatDistanceToNow } from 'date-fns'
import { it } from 'date-fns/locale'
import { UserCheck, UserX, Clock, Loader2 } from 'lucide-react'

interface PendingAccount {
  id: string
  full_name: string
  username: string | null
  created_at: string
}

export function AccountPendentClient({ initialAccounts }: { initialAccounts: PendingAccount[] }) {
  const [accounts, setAccounts] = useState(initialAccounts)
  const [loadingId, setLoadingId] = useState<string | null>(null)
  const [action, setAction] = useState<'approve' | 'reject' | null>(null)
  const [isPending, startTransition] = useTransition()

  function handleApprove(id: string) {
    setLoadingId(id)
    setAction('approve')
    startTransition(async () => {
      try {
        await approveAccount(id)
        setAccounts(prev => prev.filter(a => a.id !== id))
      } catch {
        // action failed - keep card visible
      } finally {
        setLoadingId(null)
        setAction(null)
      }
    })
  }

  function handleReject(id: string) {
    setLoadingId(id)
    setAction('reject')
    startTransition(async () => {
      try {
        await rejectAccount(id)
        setAccounts(prev => prev.filter(a => a.id !== id))
      } catch {
        // action failed - keep card visible
      } finally {
        setLoadingId(null)
        setAction(null)
      }
    })
  }

  if (accounts.length === 0) {
    return (
      <div className="flex flex-col items-center justify-center py-16 text-center text-muted-foreground gap-3">
        <UserCheck className="w-10 h-10 opacity-30" />
        <p className="text-sm">Nessuna richiesta in attesa.</p>
      </div>
    )
  }

  return (
    <div className="space-y-3">
      {accounts.map(account => {
        const isLoading = isPending && loadingId === account.id
        return (
          <div
            key={account.id}
            className={`flex items-center gap-4 rounded-lg border border-border bg-card px-4 py-3 transition-opacity ${isLoading ? 'opacity-60' : ''}`}
          >

```

```

<div className="w-10 h-10 rounded-full bg-amber-100 dark:bg-amber-900/30 flex items-center justify-
center text-sm font-semibold text-amber-700 dark:text-amber-400 shrink-0">
  {account.full_name[0]?.toUpperCase()}
</div>
<div className="flex-1 min-w-0">
  <p className="text-sm font-medium truncate">{account.full_name}</p>
  <p className="text-xs text-muted-foreground truncate">{account.username}</p>
  <p className="text-xs text-muted-foreground flex items-center gap-1 mt-0.5">
    <Clock className="w-3 h-3" />
    {formatDistanceToNow(new Date(account.created_at), { addSuffix: true, locale: it })}
  </p>
</div>
<div className="flex items-center gap-2 shrink-0">
  <Button
    size="sm"
    className="gap-1.5"
    disabled={isLoading}
    onClick={() => handleApprove(account.id)}
  >
    {isLoading && action === 'approve'
      ? <Loader2 className="w-3.5 h-3.5 animate-spin" />
      : <UserCheck className="w-3.5 h-3.5" />
    }
    Approva
  </Button>
  <Button
    size="sm"
    variant="outline"
    className="gap-1.5 text-destructive border-destructive/30 hover:bg-destructive/5"
    disabled={isLoading}
    onClick={() => handleReject(account.id)}
  >
    {isLoading && action === 'reject'
      ? <Loader2 className="w-3.5 h-3.5 animate-spin" />
      : <UserX className="w-3.5 h-3.5" />
    }
    Rifiuta
  </Button>
</div>
</div>
)
}}
</div>
)
}

```



```

'use client'
import { useState, useCallback } from 'react'
import { startOfWeek, addDays, addWeeks, format } from 'date-fns'
import { it } from 'date-fns/locale'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import { Textarea } from '@components/ui/textarea'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import { ChevronLeft, ChevronRight, Sparkles, AlertTriangle, ChevronDown, ChevronUp } from 'lucide-react'
import {
  generateAiSchedule, checkExistingTurns,
  type GenerateParams,
} from '@app/actions/aiTurni'
import type { Department, AiScheduleDraft, AiScheduleDraftTurn, ExtraordinaryClosure } from '@types'
import { DEPARTMENTS, WEEK_DAYS_SHORT } from '@types'

// Re-export GenerateParams for use in parent
export type { GenerateParams }

interface Props {
  open: boolean
  onClose: () => void
  restaurantId: string
  currentDept: Department | null // null = tutti (manager/direttore)
  currentUserRole: string
  currentIsDirettore: boolean
  onDraftCreated: (draft: AiScheduleDraft & { turns: AiScheduleDraftTurn[] }) => void
}

type Step = 'params' | 'notes' | 'generating' | 'done'
type ExistingMode = 'integrate' | 'replace' | null

const DAYS_ORDERED = [1, 2, 3, 4, 5, 6, 0] as const // Lun-Dom

export function AiScheduleDialog({
  open, onClose, restaurantId, currentDept, currentUserRole, currentIsDirettore, onDraftCreated,
}: Props) {
  const isManager = currentUserRole === 'manager'
  const isDirettore = currentUserRole === 'capo_servizio' && currentIsDirettore

  // — Selezione settimana —
  const [weekOffset, setWeekOffset] = useState(1) // default: prossima settimana
  const weekStart = startOfWeek(addWeeks(new Date(), weekOffset), { weekStartsOn: 1 })
  const weekStartStr = format(weekStart, 'yyyy-MM-dd')
  const weekLabel = `${format(weekStart, 'd MMM', { locale: it })} - ${format(addDays(weekStart, 6), 'd MMM yyyy', { locale: it })}`

  // — Reparti —
  const assignableDepts: Department[] = (isManager || isDirettore)
    ? DEPARTMENTS
    : (currentDept ? [currentDept] : [])
  const [selectedDepts, setSelectedDepts] = useState<Department[]>(assignableDepts)

  function toggleDept(d: Department) {
    setSelectedDepts(prev =>
      prev.includes(d) ? prev.filter(x => x !== d) : [...prev, d]
    )
  }

  // — Turni esistenti —
  const [existingCount, setExistingCount] = useState<number | null>(null)
  const [existingMode, setExistingMode] = useState<ExistingMode>(null)

  // — Chiusure straordinarie —
  const [showClosures, setShowClosures] = useState(false)
  const [closures, setClosures] = useState<ExtraordinaryClosure[]>([])
  const [closureDate, setClosureDate] = useState('')
  const [closureDept, setClosureDept] = useState<Department | ''>('')

  function addClosure() {
    if (!closureDate) return
    setClosures(prev => [...prev, { date: closureDate, department: closureDept || undefined }])
  }

```

```

    setClosureDate(''); setClosureDept('')
  }
function removeClosure(i: number) {
  setClosures(prev => prev.filter((_, idx) => idx !== i))
}

// — Note NL —————
const [notes, setNotes] = useState('')

// — Step & status —————
const [step, setStep] = useState<Step>('params')
const [error, setError] = useState<string | null>(null)

function resetAndClose() {
  setStep('params'); setError(null); setExistingCount(null)
  setExistingMode(null); setNotes(''); setClosures([])
  setWeekOffset(1)
  setSelectedDepts(assignableDepts)
  onClose()
}

async function handleNext() {
  setError(null)
  if (step === 'params') {
    // Controlla turni esistenti
    const scope = (isManager || isDirettore) && selectedDepts.length < DEPARTMENTS.length
      ? selectedDepts : null
    const count = await checkExistingTurns(restaurantId, weekStartStr, scope)
    setExistingCount(count)
    if (count > 0 && existingMode === null) {
      // Mostra la scelta integrate/replace prima di andare avanti
      return
    }
    setStep('notes')
  } else if (step === 'notes') {
    await handleGenerate()
  }
}

async function handleGenerate() {
  setStep('generating')
  try {
    const scope = (isManager || isDirettore) && selectedDepts.length === DEPARTMENTS.length
      ? null : selectedDepts
    const draft = await generateAiSchedule({
      restaurantId,
      weekStart: weekStartStr,
      departmentScope: scope,
      existingTurnsMode: existingMode ?? 'integrate',
      extraordinaryClosures: closures,
      notes: notes.trim() || undefined,
    })
    onDraftCreated(draft)
    resetAndClose()
  } catch (e) {
    setError(e instanceof Error ? e.message : 'Errore durante la generazione')
    setStep('notes')
  }
}

const canProceed =
  selectedDepts.length > 0 &&
  (existingCount === null || existingCount === 0 || existingMode !== null)

return (
  <Dialog open={open} onOpenChange={open => { if (!open) resetAndClose() }}>
    <DialogContent className="max-h-[90vh] overflow-y-auto sm:max-w-lg">
      <DialogHeader>
        <DialogTitle className="flex items-center gap-2">
          <Sparkles className="w-4 h-4 text-primary" />
          Genera turni con IA
        </DialogTitle>
      </DialogHeader>

```

```

    /* — Step indicator ————— */
    {step !== 'generating' && (
      <div className="flex items-center gap-1.5 text-xs text-muted-foreground mb-1">
        <span className={step === 'params' ? 'text-foreground font-medium' : ''}>1 Parametri</span>
        <span>-</span>
        <span className={step === 'notes' ? 'text-foreground font-medium' : ''}>2 Note</span>
      </div>
    )}

    /* — Step 1: Parametri ————— */
    {step === 'params' && (
      <div className="space-y-5">
        /* Settimana */
        <div className="space-y-1.5">
          <Label>Settimana</Label>
          <div className="flex items-center gap-2">
            <Button variant="outline" size="icon" className="h-8 w-8"
              onClick={() => setWeekOffset(w => w - 1)}>
              <ChevronLeft className="w-4 h-4" />
            </Button>
            <span className="flex-1 text-center text-sm font-medium">{weekLabel}</span>
            <Button variant="outline" size="icon" className="h-8 w-8"
              onClick={() => setWeekOffset(w => w + 1)}>
              <ChevronRight className="w-4 h-4" />
            </Button>
          </div>
        </div>
      </div>

      /* Reparti */
      {(isManager || isDirettore) && (
        <div className="space-y-1.5">
          <Label>Reparti</Label>
          <div className="flex flex-wrap gap-2">
            {DEPARTMENTS.map(d => (
              <button
                key={d} type="button"
                onClick={() => toggleDept(d)}
                className={`px-3 py-1.5 rounded-full text-sm font-medium border transition-colors ${
                  selectedDepts.includes(d)
                    ? 'bg-primary text-primary-foreground border-primary'
                    : 'bg-background border-border text-foreground hover:bg-accent'
                }`}
              >
                {d}
              </button>
            ))}
          </div>
        </div>
      )}

      /* Turni esistenti — mostrato solo dopo il controllo */
      {existingCount !== null && existingCount > 0 && (
        <div className="rounded-md border border-amber-200 dark:border-amber-800 bg-amber-50 dark:bg-amber-950/20 p-3 space-y-2">
          <div className="flex items-start gap-2">
            <AlertTriangle className="w-4 h-4 text-amber-600 dark:text-amber-400 shrink-0 mt-0.5" />
            <p className="text-sm text-amber-800 dark:text-amber-300">
              Trovati <strong>{existingCount}</strong> turni già presenti in questa settimana.
            </p>
          </div>
          <div className="flex flex-col gap-1.5 pl-6">
            {[('integrate', 'replace') as const].map(mode => (
              <label key={mode} className="flex items-center gap-2 cursor-pointer">
                <input
                  type="radio"
                  name="existingMode"
                  value={mode}
                  checked={existingMode === mode}
                  onChange={() => setExistingMode(mode)}
                  className="accent-primary"
                />
                <span className="text-sm">
                  {mode === 'integrate'
                    ? 'Integra (riempi solo i giorni senza turni)'

```



```

        : 'Rigenera tutto (sostituisce i turni esistenti)'}
      </span>
    </label>
  )}}
</div>
</div>
))

{/* Chiusure straordinarie — collassate */}
<div className="border border-border rounded-md">
  <button
    type="button"
    onClick={() => setShowClosures(v => !v)}
    className="w-full flex items-center justify-between px-3 py-2 text-sm text-muted-foreground
    hover:text-foreground transition-colors"
  >
    <span>Chiusure straordinarie questa settimana{closures.length > 0 ? ` (${closures.length})` : ''}
    {showClosures ? <ChevronUp className="w-4 h-4" /> : <ChevronDown className="w-4 h-4" />}
  </button>
  {showClosures && (
    <div className="px-3 pb-3 space-y-3 border-t border-border pt-3">
      <p className="text-xs text-muted-foreground">
        Aggiungi giorni in cui il ristorante (o un reparto) chiude eccezionalmente.
      </p>
      {closures.map((c, i) => (
        <div key={i} className="flex items-center gap-2 text-sm">
          <span className="flex-1">
            {c.date} {c.department ? ` - ${c.department}` : ` - Tutto il locale`}
          </span>
          <button type="button" onClick={() => removeClosure(i)}
            className="text-destructive hover:text-destructive/80 text-xs">
            Rimuovi
          </button>
        </div>
      ))}
    <div className="grid grid-cols-2 gap-2">
      <div className="space-y-1">
        <Label className="text-xs">Data</Label>
        <Input type="date" value={closureDate}
          onChange={e => setClosureDate(e.target.value)}
          min={weekStartStr}
          max={format(addDays(weekStart, 6), 'yyyy-MM-dd')}
          className="h-8 text-xs"
        />
      </div>
      <div className="space-y-1">
        <Label className="text-xs">Reparto (opz.)</Label>
        <select
          value={closureDept}
          onChange={e => setClosureDept(e.target.value as Department | '')}
          className="h-8 w-full rounded-md border border-input bg-background px-2 text-xs"
        >
          <option value="">Tutto il locale</option>
          {DEPARTMENTS.map(d => <option key={d} value={d}>{d}</option>)}
        </select>
      </div>
    </div>
    <Button size="sm" variant="outline" onClick={addClosure} disabled={!closureDate}>
      Aggiungi chiusura
    </Button>
  </div>
)}
</div>
</div>
))

{/* — Step 2: Note ————— */}
{step === 'notes' && (
  <div className="space-y-3">
    <p className="text-sm text-muted-foreground">
      Hai esigenze particolari per la settimana <strong>{weekLabel}</strong>?
      Descrivile liberamente — l'algoritmo le terrà in conto.
    </p>
  </div>
)}

```

```

        <Textarea
          value={notes}
          onChange={e => setNotes(e.target.value)}
          rows={4}
          placeholder="Es: sabato sera servono almeno 2 baristi, Marco è disponibile solo la mattina, chiudere
il Bar domenica sera..."
        />
        <p className="text-xs text-muted-foreground">Facoltativo – puoi lasciarlo vuoto e procedere.</p>
        {error && (
          <p className="text-sm text-destructive bg-destructive/10 px-3 py-2 rounded-md">{error}</p>
        )}
      </div>
    )}

    {/* — Step: Generazione in corso ————— */}
    {step === 'generating' && (
      <div className="flex flex-col items-center justify-center py-12 gap-4">
        <div className="relative">
          <Sparkles className="w-10 h-10 text-primary animate-pulse" />
        </div>
        <div className="text-center">
          <p className="font-medium">Generazione in corso.</p>
          <p className="text-sm text-muted-foreground mt-1">
            Analisi delle presenze storiche e distribuzione dei turni
          </p>
        </div>
      </div>
    )}

    {step !== 'generating' && (
      <DialogFooter className="gap-2">
        <Button variant="outline" onClick={resetAndClose}>Annulla</Button>
        {step === 'notes' && (
          <Button variant="outline" onClick={() => setStep('params')}>
            <ChevronLeft className="w-4 h-4" /> Indietro
          </Button>
        )}
        <Button onClick={handleNext} disabled={!canProceed}>
          {step === 'params'
            ? existingCount !== null && existingCount > 0 && existingMode === null
              ? 'Scegli modalità ↑'
              : 'Avanti →'
            : <><Sparkles className="w-4 h-4" /> Genera turni</>}
          </Button>
        </DialogFooter>
      )}
    </DialogContent>
  </Dialog>
)
}

```

`src/components/manager/AiScheduleDraftView.tsx`

```

'use client'
import { useState } from 'react'
import { addDays, format, parseISO } from 'date-fns'
import { it } from 'date-fns/locale'
import { Button } from '@components/ui/button'
import { Badge } from '@components/ui/badge'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import { TimeInput } from '@components/ui/time-input'
import { Label } from '@components/ui/label'
import { Switch } from '@components/ui/switch'
import { X, CheckCircle2, AlertTriangle, ArrowRightLeft, Zap, Coffee } from 'lucide-react'
import { confirmAiDraft, discardAiDraft, updateDraftTurn, rejectDraftTurn } from '@app/actions/aiTurni'
import { EXTRAORDINARY_BADGE, STANDARD_BADGE, RIPOSO_BADGE } from '@lib/turnColors'
import type { AiScheduleDraft, AiScheduleDraftTurn, AiScheduleWarning } from '@types'

interface Props {
  draft: AiScheduleDraft & { turns: AiScheduleDraftTurn[] }
  staff: { id: string; full_name: string; department: string | null }[]
  onClose: () => void
  onConfirmed: () => void
}

type TurnStatus = AiScheduleDraftTurn['status']

const TURN_BADGE: Record<string, string> = {
  standard: STANDARD_BADGE,
  extraordinary: EXTRAORDINARY_BADGE,
  rest: RIPOSO_BADGE,
  cross: 'bg-purple-50 text-purple-700 border-purple-200 dark:bg-purple-950/30 dark:text-purple-400 dark:border-purple-800',
  rejected: 'opacity-30 line-through bg-zinc-100 text-zinc-400 border-zinc-200 dark:bg-zinc-800 dark:text-zinc-500',
}

function getTurnBadge(t: AiScheduleDraftTurn): string {
  if (t.status === 'rejected') return TURN_BADGE.rejected
  if (t.is_rest_day) return TURN_BADGE.rest
  if (t.is_cross_dept) return TURN_BADGE.cross
  if (t.is_extraordinary) return TURN_BADGE.extraordinary
  return TURN_BADGE.standard
}

// — thCls / tdCls — stesse classi dell'app —————
const thCls = 'px-2 py-1.5 text-center font-semibold bg-zinc-900 text-white dark:bg-zinc-800 whitespace-nowrap text-xs'
const tdCls = 'px-1 py-1 text-center text-xs border border-zinc-200 dark:border-zinc-700 align-top'
const tdNameCls = 'px-2 py-1 text-left text-xs font-medium border border-zinc-200 dark:border-zinc-700 whitespace-nowrap sticky left-0 bg-white dark:bg-zinc-950 z-10'

export function AiScheduleDraftView({ draft, staff, onClose, onConfirmed }: Props) {
  const [turns, setTurns] = useState<AiScheduleDraftTurn[]>(draft.turns)
  const [confirming, setConfirming] = useState(false)
  const [confirmError, setConfirmError] = useState<string | null>(null)

  // — Edit modal —————
  const [editingTurn, setEditingTurn] = useState<AiScheduleDraftTurn | null>(null)
  const [editStart, setEditStart] = useState('')
  const [editEnd, setEditEnd] = useState('')
  const [editIsRest, setEditIsRest] = useState(false)
  const [editSaving, setEditSaving] = useState(false)

  function openEdit(t: AiScheduleDraftTurn) {
    setEditingTurn(t)
    setEditStart(t.start_time.slice(0, 5))
    setEditEnd(t.end_time.slice(0, 5))
    setEditIsRest(t.is_rest_day)
  }

  async function handleEditSave() {
    if (!editingTurn) return
    setEditSaving(true)
    try {
      await updateDraftTurn(editingTurn.id, {

```

```

        start_time: editIsRest ? '00:00' : editStart,
        end_time: editIsRest ? '00:00' : editEnd,
        is_rest_day: editIsRest,
        status: 'modified',
    })
    setTurns(prev => prev.map(t => t.id === editingTurn.id
    ? { ...t, start_time: editIsRest ? '00:00' : editStart, end_time: editIsRest ? '00:00' : editEnd,
is_rest_day: editIsRest, status: 'modified' }
    : t
    ))
    setEditingTurn(null)
  } finally {
    setEditSaving(false)
  }
}

async function handleReject(turnId: string) {
  await rejectDraftTurn(turnId)
  setTurns(prev => prev.map(t => t.id === turnId ? { ...t, status: 'rejected' } : t))
}

async function handleRestore(turnId: string) {
  await updateDraftTurn(turnId, { status: 'pending' })
  setTurns(prev => prev.map(t => t.id === turnId ? { ...t, status: 'pending' } : t))
}

async function handleConfirm() {
  setConfirming(true); setConfirmError(null)
  try {
    await confirmAiDraft(draft.id)
    onConfirmed()
  } catch (e) {
    setConfirmError(e instanceof Error ? e.message : 'Errore durante la conferma')
  } finally {
    setConfirming(false)
  }
}

async function handleDiscard() {
  if (!confirm('Scartare questa bozza? L\'azione non può essere annullata.')) return
  await discardAiDraft(draft.id)
  onClose()
}

// — Dati per la griglia —
const weekDays = Array.from({ length: 7 }, (_, i) =>
  addDays(parseISO(draft.week_start + 'T00:00:00'), i)
)

const turnsByUserDate: Record<string, AiScheduleDraftTurn[]> = {}
for (const t of turns) {
  const key = `${t.user_id}|${t.date}`
  if (!turnsByUserDate[key]) turnsByUserDate[key] = []
  turnsByUserDate[key].push(t)
}

// Staff con almeno un turno nella bozza (o presenti nello staff)
const draftUserIds = new Set(turns.map(t => t.user_id))
const gridStaff = staff.filter(s => draftUserIds.has(s.id))

const activeWarnings: AiScheduleWarning[] = draft.warnings ?? []
const pendingCount = turns.filter(t => t.status !== 'rejected').length

return (
  <div className="space-y-5">
    </* Header bozza */>
    <div className="flex items-start justify-between gap-3 flex-wrap">
      <div>
        <h2 className="text-lg font-semibold">Bozza turni IA</h2>
        <p className="text-sm text-muted-foreground mt-0.5">
          Settimana {format(parseISO(draft.week_start + 'T00:00:00'), 'd MMM', { locale: it })} -{' ' }
          {format(addDays(parseISO(draft.week_start + 'T00:00:00'), 6), 'd MMM yyyy', { locale: it })}
          {' ' }· {pendingCount} turni
        </p>
      </div>
    </div>
  </div>

```

```

</div>
<div className="flex gap-2">
  <Button variant="outline" size="sm" onClick={handleDiscard}>Scarta bozza</Button>
  <Button size="sm" onClick={handleConfirm} disabled={confirming || pendingCount === 0}>
    <CheckCircle2 className="w-4 h-4" />
    {confirming ? 'Pubblicazione...' : 'Conferma e Pubblica'}
  </Button>
</div>
</div>
</div>

{confirmError && (
  <p className="text-sm text-destructive bg-destructive/10 px-3 py-2 rounded-md">{confirmError}</p>
)}

{/* Warning panel */}
{activeWarnings.length > 0 && (
  <div className="rounded-md border border-amber-200 dark:border-amber-800 bg-amber-50 dark:bg-amber-950/20
p-3 space-y-1.5">
    <div className="flex items-center gap-1.5 text-sm font-medium text-amber-800 dark:text-amber-300">
      <AlertTriangle className="w-4 h-4" />
      {activeWarnings.length} avvisi di copertura
    </div>
    <ul className="space-y-1 pl-5">
      {activeWarnings.map((w, i) => (
        <li key={i} className="text-xs text-amber-700 dark:text-amber-400">
          <span className="font-medium">{w.day} · {w.department} · {w.slot_name}</span> {w.message}
        </li>
      ))}
    </ul>
  </div>
)}

{/* Legenda */}
<div className="flex flex-wrap items-center gap-3 text-[11px] text-muted-foreground">
  <span className="flex items-center gap-1.5">
    <span className={`inline-block w-3 h-3 rounded-sm border ${TURN_BADGE.standard}`} /> Turno
  </span>
  <span className="flex items-center gap-1.5">
    <span className={`inline-block w-3 h-3 rounded-sm border ${TURN_BADGE.extraordinary}`} /> Straordinario
  </span>
  <span className="flex items-center gap-1.5">
    <ArrowRightLeft className="w-3 h-3 text-purple-600" /> Jolly (altro reparto)
  </span>
  <span className="flex items-center gap-1.5">
    <span className={`inline-block w-3 h-3 rounded-sm border ${TURN_BADGE.rest}`} /> Riposo
  </span>
  <span className="flex items-center gap-1.5 ml-auto text-[10px] italic">
    Click cella → modifica · × → rimuovi
  </span>
</div>

{/* Griglia – stessa struttura di TurniManagerClient */}
<div className="w-full rounded-md border bg-card overflow-auto">
  <table className="border-collapse text-xs w-full" style={{ minWidth: 'max-content' }}>
    <thead>
      <tr>
        <th className={` ${thCls} sticky left-0 z-20 text-left min-w-[150px]`} >Dipendente</th>
        {weekDays.map(day => (
          <th key={format(day, 'yyyy-MM-dd')} className={` ${thCls} min-w-[100px]`} >
            <div className="capitalize">{format(day, 'EEE', { locale: it })}</div>
            <div className="font-normal opacity-80">{format(day, 'd/MM', { locale: it })}</div>
          </th>
        ))}
      </tr>
    </thead>
    <tbody>
      {gridStaff.length === 0 ? (
        <tr>
          <td colspan={8} className="text-center py-6 text-muted-foreground text-xs">
            Nessun turno generato
          </td>
        </tr>
      ) : gridStaff.map(member => (
        <tr key={member.id} className="even:bg-zinc-50 dark:even:bg-zinc-900/20">

```

```

        <td className={tdNameCls}>
          {member.full_name}
          {member.department && (
            <span className="block text-[10px] font-normal text-muted-foreground">{member.department}</
span>
          )}
        </td>
      {weekDays.map(day => {
        const dateStr = format(day, 'yyyy-MM-dd')
        const cellTurns = turnsByUserData[`${member.id}|${dateStr}`] ?? []
        return (
          <td key={dateStr} className={` ${tdCls} p-1`>
            <div className="flex flex-col gap-1 items-stretch min-w-[80px]">
              {cellTurns.map(turn => (
                <div
                  key={turn.id}
                  className={`flex items-center justify-between gap-0.5 rounded-sm border px-1 py-0.5
text-[10px] font-medium cursor-pointer group ${getTurnBadge(turn)} `}
                >
                  <span onClick={() => openEdit(turn)} className="flex-1 flex items-center gap-0.5 min-
w-0">
                    {turn.is_cross_dept && <ArrowRightLeft className="w-2.5 h-2.5 shrink-0" />}
                    {turn.is_extraordinary && <Zap className="w-2.5 h-2.5 shrink-0" />}
                    {turn.is_rest_day ? 'Riposo' : `${turn.start_time.slice(0, 5)}-$
{turn.end_time.slice(0, 5)} `}
                    {turn.status === 'modified' && <span className="opacity-60">*</span>}
                  </span>
                  {turn.status !== 'rejected' ? (
                    <button
                      type="button"
                      onClick={e => { e.stopPropagation(); handleReject(turn.id) }}
                      className="opacity-0 group-hover:opacity-60 hover:!opacity-100 hover:text-
destructive transition-opacity"
                      aria-label="Rimuovi"
                    >
                      <X className="w-2.5 h-2.5" />
                    </button>
                  ) : (
                    <button
                      type="button"
                      onClick={e => { e.stopPropagation(); handleRestore(turn.id) }}
                      className="text-[9px] opacity-60 hover:opacity-100"
                    >
                      ↺
                    </button>
                  )}
                </div>
              )}
            </div>
          )}
        </div>
      )}
    </tr>
  )}
</tbody>
</table>
</div>

/* — Edit turn modal ————— */
<Dialog open={!editingTurn} onOpenChange={open => { if (!open) setEditingTurn(null) }}>
  <DialogContent className="max-w-sm">
    <DialogHeader>
      <DialogTitle>Modifica turno</DialogTitle>
      {editingTurn && (
        <p className="text-sm text-muted-foreground mt-0.5">
          {staff.find(s => s.id === editingTurn.user_id)?.full_name} · {editingTurn.date}
        </p>
      )}
    </DialogHeader>
    <div className="space-y-4">
      <div className="flex items-center justify-between rounded-sm border border-border px-3 py-2.5">
        <Label>Giorno di riposo</Label>
        <Switch checked={editIsRest} onCheckedChange={setEditIsRest} />
      </div>

```

```

    {!editIsRest && (
      <div className="grid grid-cols-2 gap-3">
        <div className="space-y-1.5">
          <Label>Ora inizio</Label>
          <TimeInput value={editStart} onChange={setEditStart} />
        </div>
        <div className="space-y-1.5">
          <Label>Ora fine</Label>
          <TimeInput value={editEnd} onChange={setEditEnd} />
        </div>
      </div>
    )}
  </div>
  <DialogFooter>
    <Button variant="outline" onClick={() => setEditingTurn(null)}>Annulla</Button>
    <Button onClick={handleEditSave} disabled={editSaving || (!editIsRest && (!editStart || !editEnd))}>
      {editSaving ? 'Salvataggio...' : 'Salva'}
    </Button>
  </DialogFooter>
</DialogContent>
</Dialog>
</div>
)
}

```

`src/components/manager/ApprovazioniClient.tsx`

```

'use client'
import { useState, useEffect } from 'react'
import { useRouter } from 'next/navigation'
import { Button } from '@components/ui/button'
import { Card, CardContent } from '@components/ui/card'
import { Check, X, CalendarX } from 'lucide-react'
import { formatDate } from '@lib/Utils'
import type { Absence, AbsenceType } from '@types'
import { ABSENCE_LABELS } from '@types'
import { createClient } from '@lib/supabase/client'

type RequestWithRelations = Absence & {
  profile?: { id: string; full_name: string } | null
  restaurant?: { id: string; name: string } | null
}

interface Props {
  initialRequests: RequestWithRelations[]
}

const typeBadgeClass: Record<AbsenceType, string> = {
  ferie: 'bg-violet-100 text-violet-800 dark:bg-violet-900/30 dark:text-violet-300',
  malattia: 'bg-blue-100 text-blue-800 dark:bg-blue-900/30 dark:text-blue-300',
  riposo: 'bg-red-100 text-red-800 dark:bg-red-900/30 dark:text-red-300',
  assenza_ingiustificata: 'bg-purple-100 text-purple-800 dark:bg-purple-900/30 dark:text-purple-300',
}

export function ApprovazioniClient({ initialRequests }: Props) {
  const [requests, setRequests] = useState<RequestWithRelations[]>(initialRequests)
  const [processing, setProcessing] = useState<string | null>(null)
  const router = useRouter()

  // Supabase Realtime – keep the pending list in sync across all sessions
  useEffect(() => {
    const supabase = createClient()
    const channel = supabase
      .channel('rt-approvazioni-absences')
      .on(
        'postgres_changes',
        { event: '*', schema: 'public', table: 'absences' },
        async (payload) => {
          if (payload.eventType === 'INSERT') {
            const rec = payload.new as { id: string; status: string }
            if (rec.status !== 'pending') return
            // Fetch with joins – payload.new doesn't include profile/restaurant
            const { data } = await supabase
              .from('absences')
              .select('*', profile:profiles(id, full_name), restaurant:restaurants(id, name))
              .eq('id', rec.id)
              .single()
            if (!data) return
            setRequests(prev =>
              prev.some(r => r.id === data.id)
                ? prev
                : [data as RequestWithRelations, ...prev]
            )
          } else if (payload.eventType === 'UPDATE') {
            const rec = payload.new as { id: string; status: string }
            if (rec.status !== 'pending') {
              // Approved or rejected elsewhere → remove from queue
              setRequests(prev => prev.filter(r => r.id !== rec.id))
            }
          } else if (payload.eventType === 'DELETE') {
            const deletedId = (payload.old as { id: string }).id
            setRequests(prev => prev.filter(r => r.id !== deletedId))
          }
        }
      )
    .subscribe()

    return () => { supabase.removeChannel(channel) }
  }, [])

```

```

async function handleAction(id: string, action: 'approve' | 'reject') {
  setProcessing(id)
  const res = await fetch('/api/assenze', {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ id, action }),
  })
  if (res.ok) {
    setRequests(rs => rs.filter(r => r.id !== id))
    // Re-render server components on this page (FallbackApprovalSection)
    router.refresh()
  }
  setProcessing(null)
}

return (
  <div>
    <div className="flex items-center justify-between mb-6">
      <div>
        <h1 className="text-2xl font-bold">Approvazioni</h1>
        <p className="text-muted-foreground text-sm mt-1">
          {requests.length} richieste in attesa
        </p>
      </div>
    </div>

    {requests.length === 0 ? (
      <Card>
        <CardContent className="py-16 flex flex-col items-center text-muted-foreground gap-3">
          <CalendarX className="w-12 h-12 opacity-30" />
          <p>Nessuna richiesta in attesa</p>
        </CardContent>
      </Card>
    ) : (
      <div className="space-y-3">
        {requests.map(r => (
          <Card key={r.id}>
            <CardContent className="p-5">
              <div className="flex items-start justify-between gap-4">
                <div className="flex-1 min-w-0">
                  <div className="flex items-center gap-2 flex-wrap mb-1">
                    <span className="font-semibold">{r.profile?.full_name ?? '-'}</span>
                    <span className={`text-xs px-2 py-0.5 rounded-full font-medium ${typeBadgeClass[r.type]}`}>
                      {ABSENCE_LABELS[r.type]}
                    </span>
                  </div>
                  <p className="text-sm text-muted-foreground">
                    Dal {formatDate(r.start_date)} al {formatDate(r.end_date)}
                  </p>
                  {r.restaurant && (
                    <p className="text-xs text-muted-foreground mt-0.5">{r.restaurant.name}</p>
                  )}
                  {r.notes && (
                    <p className="text-sm mt-2 text-foreground/80 bg-muted rounded px-3 py-2">{r.notes}</p>
                  )}
                </div>
                <div className="flex gap-2 shrink-0">
                  <Button
                    size="sm"
                    onClick={() => handleAction(r.id, 'approve')}
                    disabled={processing === r.id}
                    className="bg-emerald-600 hover:bg-emerald-700 text-white"
                  >
                    <Check className="w-4 h-4" />
                  </Button>
                  <Button
                    variant="outline"
                    size="sm"
                    onClick={() => handleAction(r.id, 'reject')}
                    disabled={processing === r.id}
                    className="text-destructive hover:text-destructive border-destructive/30 dark:text-red-400
                    dark:hover:bg-red-500/10 dark:hover:text-red-300 dark:border-red-500/30"
                  >
                    <X className="w-4 h-4" />
                  </Button>
                </div>
              </div>
            </CardContent>
          </Card>
        )}
      </div>
    )}
  </div>
)

```

```
        </Button>
      </div>
    </div>
  </CardContent>
</Card>
  )}
</div>
  )
}
```

```

'use client'
import { useState, useCallback, useEffect } from 'react'
import { createClient } from '@lib/supabase/client'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select'
import { Textarea } from '@components/ui/textarea'
import { Card, CardContent } from '@components/ui/card'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import { Badge } from '@components/ui/badge'
import { Plus, Trash2, Pencil } from 'lucide-react'
import { LoadingDots } from '@components/shared/LoadingDots'
import { formatInTimeZone } from 'date-fns-tz'
import { formatDate } from '@lib/utis'
import type { Absence, Restaurant, AbsenceType, AbsenceStatus } from '@types'
import { ABSENCE_LABELS } from '@types'

const TZ = 'Europe/Rome'

const typeBadgeClass: Record<AbsenceType, string> = {
  ferie: 'bg-violet-100 text-violet-800 dark:bg-violet-900/30 dark:text-violet-300',
  malattia: 'bg-blue-100 text-blue-800 dark:bg-blue-900/30 dark:text-blue-300',
  riposo: 'bg-red-100 text-red-800 dark:bg-red-900/30 dark:text-red-300',
  assenza_ingiustificata: 'bg-purple-100 text-purple-800 dark:bg-purple-900/30 dark:text-purple-300',
}

const statusBadgeClass: Record<AbsenceStatus, string> = {
  pending: 'bg-amber-100 text-amber-800 dark:bg-amber-900/30 dark:text-amber-300',
  approved: 'bg-emerald-100 text-emerald-800 dark:bg-emerald-900/30 dark:text-emerald-300',
  rejected: 'bg-slate-100 text-slate-700 dark:bg-slate-800 dark:text-slate-400',
}

const statusLabels: Record<AbsenceStatus, string> = {
  pending: 'In attesa',
  approved: 'Approvata',
  rejected: 'Rifiutata',
}

interface DipProfile { id: string; full_name: string; restaurant_id: string | null }

interface Props {
  initialAbsences: (Absence & {
    profile?: { id: string; full_name: string } | null
    restaurant?: { id: string; name: string } | null
  })[]
  restaurants: Pick<Restaurant, 'id' | 'name'>[]
  dipendenti: DipProfile[]
  currentUserRole: string
  currentRestaurantId: string | null
  isDirettore: boolean
}

export function AssenzeClient({ initialAbsences, restaurants, dipendenti, currentUserRole, currentRestaurantId, isDirettore }: Props) {
  const [absences, setAbsences] = useState(initialAbsences)
  const [selectedMonth, setSelectedMonth] = useState(() => formatInTimeZone(new Date(), TZ, 'yyyy-MM'))
  const [selectedRestaurant, setSelectedRestaurant] = useState(currentRestaurantId ?? 'all')
  const [loading, setLoading] = useState(false)
  const [showForm, setShowForm] = useState(false)
  const [editing, setEditing] = useState<typeof initialAbsences[0] | null>(null)

  // Form state
  const [userId, setUserId] = useState('')
  const [type, setType] = useState<AbsenceType>('ferie')
  const [startDate, setStartDate] = useState('')
  const [endDate, setEndDate] = useState('')
  const [certCode, setCertCode] = useState('')
  const [notes, setNotes] = useState('')
  const [saving, setSaving] = useState(false)
  const [saveError, setSaveError] = useState<string | null>(null)

  const loadAbsences = useCallback(async (month: string, restaurantId: string, silent = false) => {

```

```

    if (!silent) setLoading(true)
    const [year, m] = month.split('-').map(Number)
    const start = new Date(Date.UTC(year, m - 1, 1)).toISOString().split('T')[0]
    const end = new Date(Date.UTC(year, m, 0)).toISOString().split('T')[0]

    const supabase = createClient()
    let query = supabase
      .from('absences')
      .select('*', profile:profiles!user_id(id, full_name), restaurant:restaurants(id, name)')
      .lte('start_date', end)
      .gte('end_date', start)
      .order('start_date', { ascending: false })

    if (restaurantId !== 'all') query = query.eq('restaurant_id', restaurantId)

    const { data, error } = await query
    if (error) console.error('[assenze] loadAbsences error:', error.message)
    setAbsences(data ?? [])
    if (!silent) setLoading(false)
  }, [])

  // — Realtime — una richiesta di assenza inviata da un dipendente, o una
  // modifica fatta da un altro responsabile, ricarica la lista (filtrata
  // sul mese/ristorante correnti) senza ricaricare la pagina. Refetch
  // silenzioso per non far lampeggiare lo skeleton.
  useEffect(() => {
    const supabase = createClient()
    const channel = supabase
      .channel('rt-assenze')
      .on(
        'postgres_changes',
        { event: '*', schema: 'public', table: 'absences' },
        () => { loadAbsences(selectedMonth, selectedRestaurant, true) }
      )
      .subscribe()
    return () => { supabase.removeChannel(channel) }
  }, [selectedMonth, selectedRestaurant, loadAbsences])

  function openCreate() {
    setEditing(null)
    setUserId('')
    setType('ferie')
    setStartDate('')
    setEndDate('')
    setCertCode('')
    setNotes('')
    setShowForm(true)
  }

  function openEdit(a: typeof initialAbsences[0]) {
    setEditing(a)
    setUserId(a.user_id)
    setType(a.type)
    setStartDate(a.start_date)
    setEndDate(a.end_date)
    setCertCode(a.certificate_code ?? '')
    setNotes(a.notes ?? '')
    setShowForm(true)
  }

  async function handleSave() {
    if (!canEdit) { showUnauthorized(); return }
    setSaving(true)
    setSaveError(null)
    const supabase = createClient()

    const selectedDip = dipendenti.find(d => d.id === userId)
    const restaurantId = selectedDip?.restaurant_id ?? null

    if (editing) {
      const { data, error } = await supabase
        .from('absences')
        .update({
          type, start_date: startDate, end_date: endDate,

```

```

        certificate_code: type === 'malattia' ? certCode || null : null,
        notes: notes || null,
      })
      .eq('id', editing.id)
      .select('*', profile:profiles!user_id(id, full_name), restaurant:restaurants(id, name)')
      .single()
    if (error) { setSaveError(error.message); setSaving(false); return }
    if (data) setAbsences(as => as.map(a => a.id === data.id ? data : a))
  } else {
    const { data: { user } } = await supabase.auth.getUser()
    const { data, error } = await supabase
      .from('absences')
      .insert({
        user_id: userId, restaurant_id: restaurantId, type,
        start_date: startDate, end_date: endDate,
        certificate_code: type === 'malattia' ? certCode || null : null,
        notes: notes || null, created_by: user!.id, status: 'approved',
      })
      .select('*', profile:profiles!user_id(id, full_name), restaurant:restaurants(id, name)')
      .single()
    if (error) { setSaveError(error.message); setSaving(false); return }
    if (data) setAbsences(as => [data, ...as])
  }

  setShowForm(false)
  setSaving(false)
}

async function handleDelete(id: string) {
  if (!canEdit) { showUnauthorized(); return }
  if (!confirm('Eliminare questa assenza?')) return
  const supabase = createClient()
  await supabase.from('absences').delete().eq('id', id)
  setAbsences(as => as.filter(a => a.id !== id))
}

const isManager = currentUserRole === 'manager'
const canEdit = isManager || (currentUserRole === 'capo_servizio' && isDirettore)

const [unauthorizedMsg, setUnauthorizedMsg] = useState(false)
function showUnauthorized() {
  setUnauthorizedMsg(true)
  setTimeout(() => setUnauthorizedMsg(false), 3000)
}

return (
  <div>
    <div className="flex items-center justify-between mb-6">
      <h1 className="text-2xl font-bold">Assenze</h1>
      {canEdit && (
        <Button onClick={openCreate} size="sm">
          <Plus className="w-4 h-4" /> Nuova
        </Button>
      )}
    </div>

    <div className="flex flex-wrap gap-3 mb-6">
      <Input
        type="month"
        value={selectedMonth}
        onChange={e => { setSelectedMonth(e.target.value); loadAbsences(e.target.value, selectedRestaurant) }}
        className="w-auto"
      />
      {isManager && (
        <Select value={selectedRestaurant} onValueChange={v => { setSelectedRestaurant(v);
loadAbsences(selectedMonth, v) }}>
          <SelectTrigger className="w-48"><SelectValue placeholder="Tutti i ristoranti" /></SelectTrigger>
          <SelectContent>
            <SelectItem value="all">Tutti i ristoranti</SelectItem>
            {restaurants.map(r => <SelectItem key={r.id} value={r.id}>{r.name}</SelectItem>)}
          </SelectContent>
        </Select>
      )}
    </div>

```



```

    {loading ? (
      <div className="space-y-2">{[...Array(4)].map((_, i) => <div key={i} className="h-16 bg-muted animate-
pulse rounded-lg" />)}</div>
    ) : absences.length === 0 ? (
      <Card><CardContent className="py-12 text-center text-muted-foreground">Nessuna assenza nel periodo</
CardContent></Card>
    ) : (
      <div className="space-y-2">
        {absences.map(a => (
          <Card key={a.id}>
            <CardContent className="p-4 flex items-center gap-4">
              <div className="flex-1 min-w-0">
                <div className="flex items-center gap-2 flex-wrap mb-1">
                  <span className="font-medium text-sm">{a.profile?.full_name ?? '-'}</span>
                  <span className={`text-xs px-2 py-0.5 rounded-full font-medium ${typeBadgeClass[a.type]}`}>
                    {ABSENCE_LABELS[a.type]}
                  </span>
                  <span className={`text-xs px-2 py-0.5 rounded-full font-medium ${statusBadgeClass[a.status]}`}>
                    {statusLabels[a.status]}
                  </span>
                </div>
                <p className="text-sm text-muted-foreground">
                  {formatDate(a.start_date)} → {formatDate(a.end_date)}
                  {a.certificate_code && <span className="ml-2 text-xs">Cert: {a.certificate_code}</span>}
                </p>
                <div>
                  {canEdit && (
                    <div className="flex gap-1 shrink-0">
                      <Button variant="ghost" size="icon" onClick={() => openEdit(a)} className="text-muted-
foreground hover:text-foreground dark:text-zinc-400 dark:hover:text-zinc-200 dark:hover:bg-zinc-700/50"><Pencil
className="w-4 h-4" /></Button>
                      <Button variant="ghost" size="icon" onClick={() => handleDelete(a.id)} className="text-
destructive hover:text-destructive dark:text-red-400 dark:hover:bg-red-500/10 dark:hover:text-red-300"><Trash2
className="w-4 h-4" /></Button>
                    </div>
                  )}
                </div>
              </CardContent>
            </Card>
          )}
        </div>
      )}
    )}

<Dialog open={showForm} onOpenChange={setShowForm}>
  <DialogContent className="max-h-[90vh] overflow-y-auto" onInteractOutside={e => e.preventDefault()}>
    <DialogHeader><DialogTitle>{editing ? 'Modifica Assenza' : 'Nuova Assenza'}</DialogTitle></DialogHeader>
    <div className="space-y-4">
      {!editing && (
        <div className="space-y-2">
          <Label>Dipendente *</Label>
          <Select value={userId} onValueChange={setUserId}>
            <SelectTrigger><SelectValue placeholder="Seleziona dipendente" /></SelectTrigger>
            <SelectContent>
              {dipendenti.map(d => <SelectItem key={d.id} value={d.id}>{d.full_name}</SelectItem>)}
            </SelectContent>
          </Select>
        </div>
      )}
      <div className="space-y-2">
        <Label>Tipo *</Label>
        <Select value={type} onValueChange={v => setType(v as AbsenceType)}>
          <SelectTrigger><SelectValue /></SelectTrigger>
          <SelectContent>
            {(Object.keys(ABSENCE_LABELS) as AbsenceType[]).map(t => (
              <SelectItem key={t} value={t}>{ABSENCE_LABELS[t]}</SelectItem>
            ))}
          </SelectContent>
        </Select>
      </div>
      <div className="grid grid-cols-2 gap-3">
        <div className="space-y-2"><Label>Dal *</Label><Input type="date" value={startDate} onChange={e =>
setStartDate(e.target.value)} required /></div>
        <div className="space-y-2"><Label>Al *</Label><Input type="date" value={endDate} onChange={e =>

```

```

setEndDate(e.target.value)} required /></div>
</div>
{type === 'malattia' && (
  <div className="space-y-2">
    <Label>Codice Certificato Medico</Label>
    <Input value={certCode} onChange={e => setCertCode(e.target.value)} placeholder="INPS-XXXX" />
  </div>
)}
<div className="space-y-2">
  <Label>Note</Label>
  <Textarea value={notes} onChange={e => setNotes(e.target.value)} rows={3} />
</div>
</div>
{saveError && (
  <p className="text-sm text-destructive bg-destructive/10 px-3 py-2 rounded-md">{saveError}</p>
)}
<DialogFooter>
  <Button variant="outline" onClick={() => setShowForm(false)}>Annulla</Button>
  <Button onClick={handleSave} disabled={saving || (!editing && !userId) || !startDate || !endDate}>
    {saving ? <>Salvataggio<LoadingDots /></> : 'Salva'}
  </Button>
</DialogFooter>
</DialogContent>
</Dialog>

{/* Unauthorized toast */}
{unauthorizedMsg && (
  <div className="fixed bottom-4 right-4 z-50 flex items-center gap-2 bg-destructive text-destructive-foreground px-4 py-3 rounded-md shadow-lg text-sm font-medium animate-in slide-in-from-bottom-2">
    Azione riservata alla direzione
  </div>
)}
</div>
)
}

```



```

'use client'
import { useState, useEffect } from 'react'
import { createClient } from '@lib/supabase/client'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select'
import { Textarea } from '@components/ui/textarea'
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import { Badge } from '@components/ui/badge'
import { Plus, Trash2, Globe, Store, Users, Eye, ChevronDown } from 'lucide-react'
import { LoadingDots } from '@components/shared/LoadingDots'
import { formatInTimeZone } from 'date-fns-tz'
import type { Bulletin, BulletinRead, BulletinTarget, Restaurant, Role } from '@types'
import { ROLE_LABELS, DEPARTMENTS } from '@types'

const TZ = 'Europe/Rome'

type DipProfile = { id: string; full_name: string; role: string }

type BulletinWithRelations = Bulletin & {
  restaurant?: { id: string; name: string } | null
  author?: { id: string; full_name: string } | null
}

interface Props {
  initialBulletins: BulletinWithRelations[]
  restaurants: Pick<Restaurant, 'id' | 'name'>[]
  dipendenti: DipProfile[]
  currentUserId: string
  currentUserRole: string
  currentRestaurantId: string | null
  canPost: boolean
  isDirettore?: boolean
}

const SELECTABLE_ROLES: { value: string; label: string }[] = [
  { value: 'capo_servizio', label: 'Capi Servizio' },
  { value: 'dipendente', label: 'Dipendenti' },
]

export function BachecaClient({
  initialBulletins, restaurants, dipendenti,
  currentUserId, currentUserRole, currentRestaurantId, canPost, isDirettore = false,
}: Props) {
  const [bulletins, setBulletins] = useState<BulletinWithRelations[]>(initialBulletins)

  // New bulletin form
  const [showForm, setShowForm] = useState(false)
  const [title, setTitle] = useState('')
  const [body, setBody] = useState('')
  const [targetType, setTargetType] = useState<BulletinTarget>('all')
  const [restaurantId, setRestaurantId] = useState(currentRestaurantId ?? '')
  const [targetRole, setTargetRole] = useState('capo_servizio')
  const [targetDepartment, setTargetDepartment] = useState(DEPARTMENTS[0])
  const [selectedUserIds, setSelectedUserIds] = useState<string[]>([])
  const [saving, setSaving] = useState(false)

  // Read receipt dialog
  const [viewingReadsFor, setViewingReadsFor] = useState<BulletinWithRelations | null>(null)
  const [reads, setReads] = useState<BulletinRead[]>([])
  const [loadingReads, setLoadingReads] = useState(false)

  // Contatori lettura per ogni bulletin (aggiornati al mount e dopo ogni nuova pubblicazione)
  const [readCounts, setReadCounts] = useState<Record<string, number>>({})

  // Espansione e tracciamento letture per capo_servizio
  const [expandedId, setExpandedId] = useState<string | null>(null)
  const [sentReadIds, setSentReadIds] = useState<Set<string>>(new Set())

  useEffect(() => {
    if (!bulletins.length) return

```

```

const supabase = createClient()
supabase
  .from('bulletin_reads')
  .select('bulletin_id')
  .in('bulletin_id', bulletins.map(b => b.id))
  .then(({ data }) => {
    const counts: Record<string, number> = {}
    data?.forEach(r => {
      counts[r.bulletin_id] = (counts[r.bulletin_id] ?? 0) + 1
    })
    setReadCounts(counts)
  })
}, [bulletins])

// — Realtime — un comunicato pubblicato o rimosso da un altro utente
// (manager o capo servizio) compare/scompare qui all'istante, senza
// ricaricare la pagina. Stesso pattern di Turni/Presenze. La RLS fa sì
// che si ricevano solo i comunicati a noi destinati. —————
useEffect(() => {
  const supabase = createClient()
  const channel = supabase
    .channel('rt-bacheca')
    .on(
      'postgres_changes',
      { event: '*', schema: 'public', table: 'bulletins' },
      async (payload) => {
        if (payload.eventType === 'DELETE') {
          const deletedId = (payload.old as { id: string }).id
          setBulletins(bs => bs.filter(b => b.id !== deletedId))
          return
        }
        // INSERT/UPDATE: il payload non include le relazioni (autore/
        // ristorante), quindi rileggiamo la riga completa.
        const rec = payload.new as { id: string }
        const { data } = await supabase
          .from('bulletins')
          .select('*', restaurant:restaurants(id, name), author:profiles!created_by(id, full_name)')
          .eq('id', rec.id)
          .single()
        if (!data) return
        setBulletins(bs => {
          const exists = bs.some(b => b.id === data.id)
          return exists
            ? bs.map(b => b.id === data.id ? data as BulletinWithRelations : b)
            : [data as BulletinWithRelations, ...bs]
        })
      }
    )
    .subscribe()

  return () => { supabase.removeChannel(channel) }
}, [])

function handleToggle(b: BulletinWithRelations) {
  const opening = expandedId !== b.id
  setExpandedId(opening ? b.id : null)

  if (opening && !sentReadIds.has(b.id)) {
    setSentReadIds(prev => new Set(prev).add(b.id))
    const supabase = createClient()
    supabase
      .from('bulletin_reads')
      .upsert(
        [{ bulletin_id: b.id, user_id: currentUser.id },
        { onConflict: 'bulletin_id,user_id', ignoreDuplicates: true }
      ]
      .then(() => {})
    )
  }
}

function resetForm() {
  setTitle(''); setBody(''); setTargetType('all')
  setRestaurantId(currentRestaurantId ?? '')
  setTargetRole('capo_servizio'); setSelectedUserIds([])
}

```

```

    setTargetDepartment(DEPARTMENTS[0])
  }

  async function handleSave() {
    setSaving(true)
    const supabase = createClient()
    const { data: { user } } = await supabase.auth.getUser()

    const { data } = await supabase
      .from('bulletins')
      .insert({
        title,
        body,
        target: targetType,
        restaurant_id: targetType === 'restaurant'
          ? (restaurantId || null)
          : (targetType === 'department' ? currentRestaurantId : null),
        target_roles: targetType === 'role' ? [targetRole] : [],
        target_user_ids: targetType === 'users' ? selectedUserIds : [],
        target_department: targetType === 'department' ? targetDepartment : null,
        created_by: user!.id,
      })
      .select('*', restaurant:restaurants(id, name), author:profiles!created_by(id, full_name)')
      .single()

    if (data) {
      setBulletins(bs => [data, ...bs])
      setReadCounts(prev => ({ ...prev, [data.id]: 0 }))
      // Fire-and-forget: invia notifica push agli utenti target
      fetch('/api/push/send', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ bulletinId: data.id }),
      }).catch(() => {})
    }
    resetForm()
    setShowForm(false)
    setSaving(false)
  }

  async function handleDelete(id: string) {
    if (!confirm('Eliminare questo comunicato?')) return
    const supabase = createClient()
    await supabase.from('bulletins').delete().eq('id', id)
    setBulletins(bs => bs.filter(b => b.id !== id))
  }

  async function openReads(bulletin: BulletinWithRelations) {
    setViewingReadsFor(bulletin)
    setReads([])
    setLoadingReads(true)
    const supabase = createClient()
    const { data } = await supabase
      .from('bulletin_reads')
      .select('*', profile:profiles!user_id(id, full_name)')
      .eq('bulletin_id', bulletin.id)
      .order('read_at', { ascending: false })
    setReads((data as BulletinRead[]) ?? [])
    setLoadingReads(false)
  }

  function toggleUserId(id: string) {
    setSelectedUserIds(ids =>
      ids.includes(id) ? ids.filter(x => x !== id) : [...ids, id]
    )
  }

  const canSave = !!title.trim() && !!body.trim() &&
    (targetType !== 'users' || selectedUserIds.length > 0)

  function TargetBadge({ b }: { b: BulletinWithRelations }) {
    if (b.target === 'all') return (
      <Badge variant="outline" className="gap-1 text-xs"><Globe className="w-3 h-3" /> Tutti</Badge>
    )
  }

```

```

    if (b.target === 'restaurant') return (
      <Badge variant="outline" className="gap-1 text-xs"><Store className="w-3 h-3" /> {b.restaurant?.name ??
'Ristorante'}</Badge>
    )
    if (b.target === 'role') return (
      <Badge variant="outline" className="gap-1 text-xs">
        <Users className="w-3 h-3" />
        {b.target_roles.map(r => ROLE_LABELS[r as Role] ?? r).join(', ')}
      </Badge>
    )
    if (b.target === 'department') return (
      <Badge variant="outline" className="gap-1 text-xs">
        <Store className="w-3 h-3" /> Reparto {b.target_department}
      </Badge>
    )
    return (
      <Badge variant="outline" className="gap-1 text-xs">
        <Users className="w-3 h-3" /> {b.target_user_ids.length} destinatari
      </Badge>
    )
  }
}

return (
  <div>
    <div className="flex items-center justify-between mb-6">
      <h1 className="text-2xl font-bold">Bacheca</h1>
      {canPost && (
        <Button onClick={() => { resetForm(); setShowForm(true) }} size="sm">
          <Plus className="w-4 h-4" /> Nuovo
        </Button>
      )}
    </div>

    {bulletins.length === 0 ? (
      <Card>
        <CardContent className="py-12 text-center text-muted-foreground">
          Nessun comunicato pubblicato
        </CardContent>
      </Card>
    ) : (
      <div className="space-y-4">
        {bulletins.map(b => {
          const isManager = currentUserRole === 'manager'
          const isOpen = isManager || expandedId === b.id
          const canViewReads = isManager || (currentUserRole === 'capo_servizio' && canPost)
          return (
            <Card key={b.id}>
              <CardHeader className={isOpen ? 'pb-2' : 'pb-3'}>
                <div className="flex items-start justify-between gap-3">
                  {!isManager ? (
                    <button
                      onClick={() => handleToggle(b)}
                      className="flex-1 text-left flex items-start gap-2 min-w-0"
                    >
                      <ChevronDown
                        className={`w-4 h-4 text-muted-foreground shrink-0 mt-0.5 transition-transform $
{isOpen ? 'rotate-180' : ''}`}
                      />
                    <div className="flex-1 min-w-0">
                      <div className="flex items-center gap-2 flex-wrap mb-1">
                        <CardTitle className="text-base">{b.title}</CardTitle>
                        <TargetBadge b={b} />
                      </div>
                      <p className="text-xs text-muted-foreground">
                        {b.author?.full_name && <>{b.author.full_name} · </>}
                        {formatInTimeZone(new Date(b.created_at), TZ, 'dd-MM-yyyy HH:mm')}
                      </p>
                    </div>
                  </button>
                ) : (
                  <div className="flex-1 min-w-0">
                    <div className="flex items-center gap-2 flex-wrap mb-1">
                      <CardTitle className="text-base">{b.title}</CardTitle>
                      <TargetBadge b={b} />
                    </div>
                  </div>
                )
              </CardHeader>
            </Card>
          )
        })}
      </div>
    )
  )
}

```

```

        </div>
        <p className="text-xs text-muted-foreground">
            {b.author?.full_name && <>{b.author.full_name} · </>}
            {formatInTimeZone(new Date(b.created_at), TZ, 'dd-MM-yyyy HH:mm')}
        </p>
        {(readCounts[b.id] ?? 0) > 0 && (
            <p className="text-xs text-muted-foreground/70 mt-0.5">
                Letto da {readCounts[b.id]} {readCounts[b.id] === 1 ? 'persona' : 'persone'}
            </p>
        )}
    </div>
    )}
    <div className="flex items-center gap-1 shrink-0">
        {canViewReads && (
            <Button
                variant="ghost" size="icon"
                onClick={() => openReads(b)}
                className="text-muted-foreground hover:text-foreground dark:text-zinc-400
dark: hover: bg-zinc-700/50"
                title="Visualizza lettura"
            >
                <Eye className="w-4 h-4" />
            </Button>
        )}
        {isManager && (
            <Button
                variant="ghost" size="icon"
                onClick={() => handleDelete(b.id)}
                className="text-destructive hover:text-destructive dark:text-red-400 dark: hover: bg-
red-500/10 dark: hover: text-red-300"
            >
                <Trash2 className="w-4 h-4" />
            </Button>
        )}
    </div>
</div>
</CardHeader>
{isOpen && (
    <CardContent className="pt-0">
        <p className="text-sm whitespace-pre-wrap">{b.body}</p>
    </CardContent>
)}
</Card>
)
}}
</div>
)}

{/* New bulletin dialog */}
<Dialog open={showForm} onOpenChange={setShowForm}>
    <DialogContent className="max-h-[90vh] overflow-y-auto">
        <DialogHeader><DialogTitle>Nuovo Comunicato</DialogTitle></DialogHeader>
        <div className="space-y-4">
            <div className="space-y-2">
                <Label>Titolo *</Label>
                <Input value={title} onChange={e => setTitle(e.target.value)} placeholder="Aggiornamento turni..." />
            </div>
            <div className="space-y-2">
                <Label>Messaggio *</Label>
                <Textarea value={body} onChange={e => setBody(e.target.value)} rows={5} placeholder="Scrivi il tuo comunicato..." />
            </div>
            <div className="space-y-2">
                <Label>Destinatari</Label>
                <Select value={targetType} onValueChange={v => setTargetType(v as BulletinTarget)}>
                    <SelectTrigger><SelectValue /></SelectTrigger>
                    <SelectContent>
                        {isDirettore ? (
                            <>
                                <SelectItem value="all">Tutto il Ristorante</SelectItem>
                                <SelectItem value="department">Reperto Specifico</SelectItem>
                                <SelectItem value="users">Singolo Dipendente</SelectItem>
                            </>
                        ) : (
                            <SelectItem value="all">Tutto il Ristorante</SelectItem>
                        )}
                    </SelectContent>
                </Select>
            </div>
        </div>
    </DialogContent>
</Dialog>

```



```

    ) : (
      <>
        <SelectItem value="all">Tutti</SelectItem>
        <SelectItem value="role">Solo per ruolo</SelectItem>
        <SelectItem value="users">Dipendenti specifici</SelectItem>
        {restaurants.length > 0 && <SelectItem value="restaurant">Ristorante specifico</SelectItem>}
      </>
    )}
  </SelectContent>
</Select>
</div>

{targetType === 'restaurant' && (
  <div className="space-y-2">
    <Label>Ristorante</Label>
    <Select value={restaurantId} onValueChange={setRestaurantId}>
      <SelectTrigger><SelectValue placeholder="Seleziona ristorante" /></SelectTrigger>
      <SelectContent>
        {restaurants.map(r => <SelectItem key={r.id} value={r.id}>{r.name}</SelectItem>)}
      </SelectContent>
    </Select>
  </div>
)}

{targetType === 'department' && (
  <div className="space-y-2">
    <Label>Reparto</Label>
    <Select value={targetDepartment} onValueChange={v => setTargetDepartment(v as typeof
targetDepartment)}>
      <SelectTrigger><SelectValue /></SelectTrigger>
      <SelectContent>
        {DEPARTMENTS.map(d => <SelectItem key={d} value={d}>{d}</SelectItem>)}
      </SelectContent>
    </Select>
  </div>
)}

{targetType === 'role' && (
  <div className="space-y-2">
    <Label>Ruolo</Label>
    <Select value={targetRole} onValueChange={setTargetRole}>
      <SelectTrigger><SelectValue /></SelectTrigger>
      <SelectContent>
        {SELECTABLE_ROLES.map(r => <SelectItem key={r.value} value={r.value}>{r.label}</SelectItem>)}
      </SelectContent>
    </Select>
  </div>
)}

{targetType === 'users' && (
  <div className="space-y-2">
    <Label>
      Dipendenti
      {selectedUserIds.length > 0 && (
        <span className="ml-2 text-xs font-normal text-muted-foreground">
          {selectedUserIds.length} selezionati
        </span>
      )}
    </Label>
    <div className="border border-input rounded-md divide-y divide-border max-h-48 overflow-y-auto">
      {dipendenti.map(d => (
        <label
          key={d.id}
          className="flex items-center gap-3 px-3 h-10 cursor-pointer hover:bg-muted transition-
colors"
        >
          <input
            type="checkbox"
            checked={selectedUserIds.includes(d.id)}
            onChange={() => toggleUserId(d.id)}
            className="shrink-0"
          />
          <span className="text-sm flex-1 truncate">{d.full_name}</span>
          <span className="text-xs text-muted-foreground shrink-0">

```

```

                {ROLE_LABELS[d.role as Role] ?? d.role}
            </span>
        </label>
    )}}
</div>
</div>
))}
</div>
<DialogFooter>
    <Button variant="outline" onClick={() => setShowForm(false)}>Annulla</Button>
    <Button onClick={handleSave} disabled={saving || !canSave}>
        {saving ? <>Pubblicazione<LoadingDots /></> : 'Pubblica'}
    </Button>
</DialogFooter>
</DialogContent>
</Dialog>

{/* Read receipts dialog */}
<Dialog open={!viewingReadsFor} onOpenChange={() => setViewingReadsFor(null)}>
    <DialogContent className="max-h-[80vh] overflow-y-auto">
        <DialogHeader>
            <DialogTitle>Chi ha letto</DialogTitle>
            {viewingReadsFor && (
                <p className="text-sm text-muted-foreground mt-0.5 truncate">{viewingReadsFor.title}</p>
            )}
        </DialogHeader>
        {loadingReads ? (
            <div className="space-y-2 py-2">
                {[...Array(3)].map((_, i) => <div key={i} className="h-9 bg-muted animate-pulse rounded" />)}
            </div>
        ) : reads.length === 0 ? (
            <p className="text-sm text-muted-foreground py-4 text-center">
                Nessuno ha ancora letto questo comunicato.
            </p>
        ) : (
            <div className="divide-y divide-border">
                {reads.map(r => (
                    <div key={r.id} className="flex items-center justify-between py-2.5 gap-4">
                        <span className="text-sm font-medium">{r.profile?.full_name ?? '-'}</span>
                        <span className="text-xs text-muted-foreground shrink-0 tabular-nums">
                            {formatInTimeZone(new Date(r.read_at), TZ, 'dd-MM-yyyy HH:mm')}
                        </span>
                    </div>
                ))}
            </div>
        )}
    </DialogContent>
    <DialogFooter>
        <Button variant="outline" onClick={() => setViewingReadsFor(null)}>Chiudi</Button>
    </DialogFooter>
</Dialog>
</div>
)
}

```

`src/components/manager/CapoServizioTimbraturaSection.tsx`

```

'use client'
import { useState, useEffect, useCallback } from 'react'
import { Camera, ImageOff } from 'lucide-react'
import { differenceInSeconds } from 'date-fns'
import { QRScanner } from '@components/dipendente/QRScanner'
import { usePushNotifications } from '@hooks/usePushNotifications'
import { useGeofence } from '@hooks/useGeofence'
import { PushNotificationBanner } from '@components/shared/PushNotificationBanner'
import { compressImage } from '@lib/compressImage'
import type { Attendance } from '@types'

interface Props {
  initialOpenAttendance: Attendance | null
  restaurantLat?: number | null
  restaurantLng?: number | null
}

function formatDuration(s: number): string {
  const h = Math.floor(s / 3600)
  const m = Math.floor((s % 3600) / 60)
  const sec = s % 60
  return `${String(h).padStart(2, '0')}:${String(m).padStart(2, '0')}:${String(sec).padStart(2, '0')}`
}

export function CapoServizioTimbraturaSection({ initialOpenAttendance, restaurantLat, restaurantLng }: Props) {
  const [attendance, setAttendance] = useState<Attendance | null>(initialOpenAttendance)
  const [showScanner, setShowScanner] = useState(false)
  const [loading, setLoading] = useState(false)
  const [message, setMessage] = useState<{ text: string; type: 'success' | 'error' } | null>(null)
  const [now, setNow] = useState(new Date())
  const [gpsFailed, setGpsFailed] = useState(false)
  const { status: geoStatus, check: checkGeo, userCoordsRef } = useGeofence()

  // Tick only while a shift is open
  useEffect(() => {
    if (!attendance) return
    const id = setInterval(() => setNow(new Date()), 1000)
    return () => clearInterval(id)
  }, [attendance])

  const elapsed = attendance ? differenceInSeconds(now, new Date(attendance.check_in)) : 0

  // Geofence-aware scan trigger - fail-closed
  const handleScanPress = useCallback(async () => {
    setMessage(null)
    const result = await checkGeo(restaurantLat, restaurantLng)
    if (result === 'outside') {
      // Parachute: a weak indoor GPS fix can read "too far" even when inside.
      setMessage({ text: 'Posizione troppo lontana. Se sei nel locale, timbra con la foto.', type: 'error' })
      setGpsFailed(true)
      return
    }
    if (result === 'denied' || result === 'unsupported') {
      setMessage({ text: 'Impossibile verificare il GPS. Se sei nel locale, timbra con la foto.', type: 'error' })
      setGpsFailed(true) // - fallback only: show photo option
      return
    }
    setGpsFailed(false)
    setShowScanner(true)
  }, [checkGeo, restaurantLat, restaurantLng])

  const handleScan = useCallback(async (qrSecret: string) => {
    setShowScanner(false)
    setLoading(true)
    setMessage(null)
    try {
      const res = await fetch('/api/scan', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          qr_secret: qrSecret,
          type: attendance ? 'out' : 'in',
          latitude: userCoordsRef.current?.latitude,

```

```

        longitude: userCoordsRef.current?.longitude,
        accuracy: userCoordsRef.current?.accuracy,
      )),
    })
  })
  const data = await res.json()
  if (!res.ok) {
    setMessage({ text: data.error ?? 'Errore durante la timbratura', type: 'error' })
    return
  }
  if (attendance) {
    setAttendance(null)
    setMessage({ text: 'Uscita registrata con successo', type: 'success' })
  } else {
    setAttendance(data.attendance)
    setMessage({
      text: data.splitShift ? 'Turno spezzato registrato' : 'Entrata registrata con successo',
      type: 'success',
    })
  }
} catch {
  setMessage({ text: 'Errore di rete, riprova', type: 'error' })
} finally {
  setLoading(false)
}
}, [attendance])

async function handleFallbackPhoto(e: React.ChangeEvent<HTMLInputElement>) {
  const photo = e.target.files?.[0]
  if (!photo) return
  e.target.value = ''
  setLoading(true)
  setMessage(null)
  try {
    // Compress to max 800 px / 70 % quality before upload (~200 KB target)
    let compressed = photo
    try { compressed = await compressImage(photo, 800, 0.7) } catch { /* fallback to original */ }

    const fd = new FormData()
    fd.append('photo', compressed)
    fd.append('type', attendance ? 'out' : 'in')
    const res = await fetch('/api/clock-in-fallback', { method: 'POST', body: fd })
    const data = await res.json()
    if (!res.ok) {
      setMessage({ text: data.error ?? 'Errore timbratura di emergenza', type: 'error' })
      return
    }
    if (attendance) {
      setAttendance(null)
    } else {
      setAttendance(data.attendance)
    }
    setGpsFailed(false)
    setMessage({ text: 'Timbratura registrata. In attesa di conferma dal Manager.', type: 'success' })
  } catch {
    setMessage({ text: 'Errore di rete, riprova', type: 'error' })
  } finally {
    setLoading(false)
  }
}

const { permission: pushPermission, subscribe: subscribePush } = usePushNotifications()
const isOut = !!attendance
const isGeoChecking = geoStatus === 'checking'

return (
  <>
    <PushNotificationBanner permission={pushPermission} onSubscribe={subscribePush} />
    <div className="mt-4 bg-card border border-border rounded-md p-4 flex items-center gap-4">
      <div className="flex-1 min-w-0">
        <p className="text-sm font-semibold tracking-tight">Timbratura Presenza</p>
        {attendance ? (
          <p className="text-xs text-emerald-600 dark:text-emerald-400 tabular-nums mt-0.5">

```

```

        Turno in corso · {formatDuration(elapsed)}
      </p>
    ) : (
      <p className="text-xs text-muted-foreground mt-0.5">Nessun turno aperto</p>
    )
  }

  {message && (
    <p className={`text-xs mt-1 font-medium ${
      message.type === 'success'
        ? 'text-emerald-600 dark:text-emerald-400'
        : 'text-destructive'
    }`}>
      {message.text}
    </p>
  )}
</div>

{/* Right: action button */}
<div className="flex items-center gap-2 shrink-0">
  {process.env.NODE_ENV === 'development' && (
    <button
      onClick={() => handleScan('__SIMULATE__')}
      disabled={loading}
      className="text-xs text-muted-foreground underline disabled:opacity-50"
    >
      [DEV]
    </button>
  )}
  <button
    onClick={handleScanPress}
    disabled={loading || isGeoChecking}
    className={`h-10 px-4 rounded-sm flex items-center gap-2 text-sm font-semibold border transition-
colors disabled:opacity-50 ${
      isOut
        ? 'bg-amber-500 hover:bg-amber-600 text-white border-amber-600'
        : 'bg-primary hover:bg-primary/90 text-primary-foreground border-primary'
    }`}
  >
    <Camera className="w-4 h-4" />
    {isGeoChecking ? 'Verifica posizione...' : loading ? 'Elaborazione...' : isOut ? 'Timbra Uscita' :
'Timbra Ingresso'}
  </button>
</div>
</div>

{/* GPS fallback – shown only when GPS denied/unavailable */}
{gpsFailed && (
  <label className={`mt-2 flex items-center gap-2.5 justify-center h-10 px-4 rounded-sm text-sm font-medium
border cursor-pointer transition-colors
${loading ? 'opacity-50 pointer-events-none' : ''}
border-amber-400 bg-amber-50 text-amber-800 hover:bg-amber-100
dark:border-amber-600 dark:bg-amber-950/30 dark:text-amber-300 dark:hover:bg-amber-950/50`}
  >
    <ImageOff className="w-4 h-4 shrink-0" />
    Problemi col GPS? Timbra con foto
    <input
      type="file"
      accept="image/*"
      capture="environment"
      className="sr-only"
      onChange={handleFallbackPhoto}
      disabled={loading}
    />
  </label>
)}

{showScanner && (
  <QRScanner onScan={handleScan} onClose={() => setShowScanner(false)} />
)}
</>
)
}

```

`src/components/manager/ConsulenteLavoroManager.tsx`

```

'use client'
import { useState, useEffect, useCallback } from 'react'
import {
  Plus, ChevronDown, ChevronUp, Trash2, Pencil, Paperclip, X,
  Clock, BriefcaseBusiness, Send, Eye, EyeOff, CornerDownLeft,
} from 'lucide-react'
import { formatInTimeZone } from 'date-fns-tz'
import { it } from 'date-fns/locale'
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import { Button } from '@components/ui/button'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import type { Profile, Restaurant, ConsultantMessage } from '@types'
import { generateUnifiedPDF } from '@lib/generateUnifiedPDF'
import { createClient } from '@lib/supabase/client'

const TZ = 'Europe/Rome'
const MAX_BYTES = 10 * 1024 * 1024

function fmtDate(iso: string | null) {
  if (!iso) return 'Mai'
  return formatInTimeZone(new Date(iso), TZ, 'dd-MM-yyyy HH:mm', { locale: it })
}

interface ConsultantProfile extends Pick<Profile, 'id' | 'full_name' | 'username' | 'last_active_at' |
'consultant_restaurant_ids' | 'can_view_hours'> {}

interface Props {
  managerId: string
  restaurants: Pick<Restaurant, 'id' | 'name'>[]
}

export function ConsulenteLavoroManager({ managerId, restaurants }: Props) {
  const [consultants, setConsultants] = useState<ConsultantProfile[]>([])
  const [loadingList, setLoadingList] = useState(true)
  const [expandedId, setExpandedId] = useState<string | null>(null)

  // Create / edit modal
  const [modalOpen, setModalOpen] = useState(false)
  const [editTarget, setEditTarget] = useState<ConsultantProfile | null>(null)
  const [formName, setFormName] = useState('')
  const [formUsername, setFormUsername] = useState('')
  const [formPassword, setFormPassword] = useState('')
  const [formNewPassword, setFormNewPassword] = useState('')
  const [showNewPwd, setShowNewPwd] = useState(false)
  const [formRestaurants, setFormRestaurants] = useState<string[]>([])
  const [formHours, setFormHours] = useState(false)
  const [formSaving, setFormSaving] = useState(false)
  const [formError, setFormError] = useState<string | null>(null)

  // Delete confirm
  const [deleteTarget, setDeleteTarget] = useState<ConsultantProfile | null>(null)
  const [deleteLoading, setDeleteLoading] = useState(false)

  // Per-consultant thread state
  const [threads, setThreads] = useState<Record<string, ConsultantMessage[]>>({})
  const [threadLoading, setThreadLoading] = useState<Record<string, boolean>>({})

  // Compose message state (per consultant id → open)
  const [composeOpen, setComposeOpen] = useState<Record<string, boolean>>({})
  const [composeTitle, setComposeTitle] = useState<Record<string, string>>({})
  const [composeBody, setComposeBody] = useState<Record<string, string>>({})
  const [composeFiles, setComposeFiles] = useState<Record<string, File[]>>({})
  const [composeSending, setComposeSending] = useState<Record<string, boolean>>({})
  const [composeMerging, setComposeMerging] = useState<Record<string, boolean>>({})
  const [composeError, setComposeError] = useState<Record<string, string | null>>({})

  // Message expand / delete / reply state
  const [msgExpanded, setMsgExpanded] = useState<Record<string, string | null>>({})
  const [msgDeleting, setMsgDeleting] = useState<Record<string, boolean>>({})
  const [deleteMsgTarget, setDeleteMsgTarget] = useState<{ consultantId: string; msg: ConsultantMessage } |
null>(null)
  const [replyTo, setReplyTo] = useState<Record<string, ConsultantMessage | null>>({})

```



```

const loadConsultants = useCallback(async () => {
  setLoadingList(true)
  const res = await fetch('/api/users?role=consulente_lavoro')
  if (res.ok) {
    const data = await res.json()
    setConsultants(data)
  }
  setLoadingList(false)
}, [])

useEffect(() => { loadConsultants() }, [loadConsultants])

// Supabase Realtime – listen to all consultant_messages for this manager
useEffect(() => {
  const supabase = createClient()
  const channel = supabase
    .channel('consultant-messages-manager')
    .on(
      'postgres_changes',
      { event: '*', schema: 'public', table: 'consultant_messages', filter: `manager_id=eq.${managerId}` },
      (payload) => {
        if (payload.eventType === 'INSERT') {
          const msg = payload.new as ConsultantMessage
          setThreads(prev => {
            const cId = msg.consultant_id
            if (!(cId in prev)) return prev // thread not yet loaded – skip
            if (prev[cId].some(m => m.id === msg.id)) return prev // duplicate guard
            return { ...prev, [cId]: [msg, ...prev[cId]] }
          })
        } else if (payload.eventType === 'DELETE') {
          const deletedId = (payload.old as { id: string }).id
          setThreads(prev => {
            const next: Record<string, ConsultantMessage[]> = {}
            for (const [cId, msgs] of Object.entries(prev)) {
              next[cId] = msgs.filter(m => m.id !== deletedId)
            }
            return next
          })
          setMsgExpanded(prev => {
            const next = { ...prev }
            for (const [cId, openId] of Object.entries(prev)) {
              if (openId === deletedId) next[cId] = null
            }
            return next
          })
        }
      }
    )
    .subscribe()

  return () => { supabase.removeChannel(channel) }
}, [managerId])

async function loadThread(consultantId: string) {
  setThreadLoading(prev => ({ ...prev, [consultantId]: true }))
  const res = await fetch(`/api/consultant-messages?consultantId=${consultantId}`)
  if (res.ok) {
    const data: ConsultantMessage[] = await res.json()
    setThreads(prev => ({ ...prev, [consultantId]: data }))
  }
  setThreadLoading(prev => ({ ...prev, [consultantId]: false }))
}

function handleAccordionToggle(id: string) {
  if (expandedId === id) {
    setExpandedId(null)
  } else {
    setExpandedId(id)
    if (!threads[id]) loadThread(id)
  }
}

// — Create / edit consultant —

```

```

function openCreate() {
  setEditTarget(null)
  setFormName(''); setFormUsername(''); setFormPassword(''); setFormNewPassword('')
  setShowNewPwd(false)
  setFormRestaurants([]); setFormHours(false)
  setFormError(null)
  setModalOpen(true)
}

function openEdit(c: ConsultantProfile) {
  setEditTarget(c)
  setFormName(c.full_name)
  setFormUsername(c.username ?? '')
  setFormPassword('')
  setFormNewPassword('')
  setShowNewPwd(false)
  setFormRestaurants(c.consultant_restaurant_ids ?? [])
  setFormHours(c.can_view_hours)
  setFormError(null)
  setModalOpen(true)
}

function toggleFormRestaurant(id: string) {
  setFormRestaurants(prev =>
    prev.includes(id) ? prev.filter(r => r !== id) : [...prev, id]
  )
}

async function handleSave() {
  setFormSaving(true)
  setFormError(null)
  try {
    if (editTarget) {
      const res = await fetch('/api/users', {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          id: editTarget.id,
          full_name: formName,
          role: 'consulente_lavoro',
          consultant_restaurant_ids: formRestaurants,
          can_view_hours: formHours,
          can_post_bulletin: false,
        }),
      })
      if (!res.ok) { const e = await res.json(); throw new Error(e.error) }

      if (formNewPassword.trim()) {
        if (formNewPassword.length < 6) throw new Error('La password deve essere di almeno 6 caratteri')
        const pwRes = await fetch('/api/users/password', {
          method: 'PUT',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ id: editTarget.id, password: formNewPassword }),
        })
        if (!pwRes.ok) { const e = await pwRes.json(); throw new Error(e.error) }
      }
    } else {
      if (!formUsername.trim() || !formPassword.trim() || !formName.trim()) {
        throw new Error('Username, password e nome sono obbligatori')
      }
      const res = await fetch('/api/users', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          username: formUsername,
          password: formPassword,
          full_name: formName,
          role: 'consulente_lavoro',
          consultant_restaurant_ids: formRestaurants,
          can_view_hours: formHours,
          can_post_bulletin: false,
        }),
      })
      if (!res.ok) { const e = await res.json(); throw new Error(e.error) }
    }
  }
}

```

```

    }
    setModalOpen(false)
    await loadConsultants()
  } catch (e) {
    setFormError(e instanceof Error ? e.message : 'Errore salvataggio')
  } finally {
    setFormSaving(false)
  }
}

async function handleDelete() {
  if (!deleteTarget) return
  setDeleteLoading(true)
  const res = await fetch(`/api/users?id=${deleteTarget.id}`, { method: 'DELETE' })
  if (res.ok) {
    setDeleteTarget(null)
    await loadConsultants()
  }
  setDeleteLoading(false)
}

// — Messaging —————
function handleReply(consultantId: string, msg: ConsultantMessage) {
  setReplyTo(prev => ({ ...prev, [consultantId]: msg }))
  setComposeOpen(prev => ({ ...prev, [consultantId]: true }))
  // Pre-fill subject only if compose is blank
  setComposeTitle(prev => ({
    ...prev,
    [consultantId]: prev[consultantId]?.trim() ? prev[consultantId] : `Re: ${msg.title}`,
  }))
}

async function handleSendMessage(consultantId: string) {
  const title = composeTitle[consultantId] ?? ''
  const body = composeBody[consultantId] ?? ''
  if (!title.trim() || !body.trim()) return

  const filesToSend = composeFiles[consultantId] ?? []
  const total = filesToSend.reduce((sum, f) => sum + f.size, 0)
  if (total > MAX_BYTES) {
    setComposeError(prev => ({ ...prev, [consultantId]: 'Il limite massimo per gli allegati è di 10 MB totali' }))
    return
  }

  setComposeError(prev => ({ ...prev, [consultantId]: null }))

  try {
    const attachments: Array<{ name: string; path: string }> = []

    if (filesToSend.length > 0) {
      // Phase 1: merge & compress all files into one PDF client-side
      setComposeMerging(prev => ({ ...prev, [consultantId]: true }))
      const unifiedFile = await generateUnifiedPDF(filesToSend)
      setComposeMerging(prev => ({ ...prev, [consultantId]: false }))

      // Phase 2: upload the single unified PDF
      setComposeSending(prev => ({ ...prev, [consultantId]: true }))
      const fd = new FormData()
      fd.append('file', unifiedFile)
      fd.append('consultantId', consultantId)
      const uploadRes = await fetch('/api/consultant-messages/upload', { method: 'POST', body: fd })
      if (!uploadRes.ok) {
        const e = await uploadRes.json()
        throw new Error(e.error ?? 'Errore upload')
      }
      const uploaded = await uploadRes.json()
      attachments.push({ name: unifiedFile.name, path: uploaded.path })
    } else {
      setComposeSending(prev => ({ ...prev, [consultantId]: true }))
    }
  }

  const res = await fetch('/api/consultant-messages', {
    method: 'POST',
  })
}

```

```

        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            consultantId,
            title,
            body,
            attachments,
            reply_to_id: replyTo[consultantId]?.id ?? null,
        }),
    ),
})
if (!res.ok) {
    const e = await res.json()
    throw new Error(e.error ?? 'Errore invio')
}
setComposeTitle(prev => ({ ...prev, [consultantId]: '' }))
setComposeBody(prev => ({ ...prev, [consultantId]: '' }))
setComposeFiles(prev => ({ ...prev, [consultantId]: [] }))
setComposeOpen(prev => ({ ...prev, [consultantId]: false }))
setReplyTo(prev => ({ ...prev, [consultantId]: null }))
await loadThread(consultantId)
} catch (e) {
    setComposeMerging(prev => ({ ...prev, [consultantId]: false }))
    setComposeError(prev => ({ ...prev, [consultantId]: e instanceof Error ? e.message : 'Errore' }))
} finally {
    setComposeSending(prev => ({ ...prev, [consultantId]: false }))
}
}

```

```

async function handleMsgExpand(consultantId: string, msg: ConsultantMessage) {
    const currentOpen = msgExpanded[consultantId]
    const isAlreadyOpen = currentOpen === msg.id

    setMsgExpanded(prev => ({ ...prev, [consultantId]: isAlreadyOpen ? null : msg.id }))

    // Mark as read if manager is reading a consultant reply
    if (!isAlreadyOpen && !msg.sent_by_manager && !msg.read_at) {
        const res = await fetch(`/api/consultant-messages/${msg.id}/read`, { method: 'POST' })
        if (res.ok) {
            const { read_at } = await res.json()
            setThreads(prev => ({
                ...prev,
                [consultantId]: (prev[consultantId] ?? []).map(m =>
                    m.id === msg.id ? { ...m, read_at } : m
                ),
            }))
        }
    }
}

```

```

async function handleMsgDelete(consultantId: string, msgId: string) {
    setMsgDeleting(prev => ({ ...prev, [msgId]: true }))
    const res = await fetch(`/api/consultant-messages?id=${msgId}`, { method: 'DELETE' })
    if (res.ok) {
        setThreads(prev => ({
            ...prev,
            [consultantId]: (prev[consultantId] ?? []).filter(m => m.id !== msgId),
        )))
        // Collapse if the deleted message was open
        setMsgExpanded(prev => ({
            ...prev,
            [consultantId]: prev[consultantId] === msgId ? null : prev[consultantId],
        )))
    }
    setMsgDeleting(prev => ({ ...prev, [msgId]: false }))
    setDeleteMsgTarget(null)
}

```

```

async function handleMsgDownload(consultantId: string, msg: ConsultantMessage, att: { name: string; path:
string }) {
    const res = await fetch(`/api/consultant-messages/${msg.id}/download`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ path: att.path, name: att.name }),
    })
    if (!res.ok) { alert('Impossibile generare il link'); return }
}

```

```

const { signedUrl, downloaded_at } = await res.json()
if (downloaded_at) {
  setThreads(prev => ({
    ...prev,
    [consultantId]: (prev[consultantId] ?? []).map(m =>
      m.id === msg.id ? { ...m, downloaded_at } : m
    ),
  }))
}
const a = document.createElement('a')
a.href = signedUrl
a.download = att.name
document.body.appendChild(a)
a.click()
document.body.removeChild(a)
}

return (
  <div className="mt-6">
    { /* Section header */ }
    <div className="flex items-center justify-between mb-3">
      <div className="flex items-center gap-2">
        <BriefcaseBusiness className="w-4 h-4 text-muted-foreground" />
        <h2 className="text-sm font-semibold">Consulenti del Lavoro</h2>
      </div>
      <button
        onClick={openCreate}
        className="flex items-center gap-1.5 h-9 px-3 rounded-sm text-sm font-medium border border-primary text-primary hover:bg-primary hover:text-primary-foreground transition-colors"
      >
        <Plus className="w-3.5 h-3.5" />
        Nuovo Consulente
      </button>
    </div>

    {loadingList && (
      <p className="text-sm text-muted-foreground">Caricamento...</p>
    )}
    {!loadingList && consultants.length === 0 && (
      <p className="text-sm text-muted-foreground">Nessun consulente creato.</p>
    )}

    { /* Consultant list */ }
    <div className="space-y-2">
      {consultants.map(c => {
        const isOpen = expandedId === c.id
        const thread = threads[c.id] ?? []
        const hasUnread = thread.some(m => !m.sent_by_manager && !m.read_at)

        return (
          <div key={c.id} className="border border-border rounded-md overflow-hidden bg-card">
            { /* Row */ }
            <div className="flex items-center gap-3 px-4 py-3">
              { /* Left: name + meta */ }
              <button
                className="flex-1 min-w-0 flex flex-col items-start text-left"
                onClick={() => handleAccordionToggle(c.id)}
              >
                <div className="flex items-center gap-2">
                  {hasUnread && <span className="w-2 h-2 rounded-full bg-primary shrink-0" />}
                  <span className="text-sm font-medium truncate">{c.full_name}</span>
                  <span className="text-xs text-muted-foreground">({c.username})</span>
                </div>
                <span className="text-xs text-muted-foreground flex items-center gap-1 mt-0.5">
                  <Clock className="w-3 h-3" />
                  Ultimo accesso: {fmtDate(c.last_active_at)}
                </span>
              </button>

              { /* Right: edit / delete / expand */ }
              <div className="flex items-center gap-1 shrink-0">
                <button
                  onClick={() => openEdit(c)}
                  className="p-1.5 rounded hover:bg-accent transition-colors text-muted-foreground"
                >

```

```

        title="Modifica"
      >
      <Pencil className="w-3.5 h-3.5" />
    </button>
    <button
      onClick={() => setDeleteTarget(c)}
      className="p-1.5 rounded hover:bg-destructive/10 transition-colors text-destructive"
      title="Elimina"
    >
      <Trash2 className="w-3.5 h-3.5" />
    </button>
    <button
      onClick={() => handleAccordionToggle(c.id)}
      className="p-1.5 rounded hover:bg-accent transition-colors text-muted-foreground"
    >
      {isOpen ? <ChevronUp className="w-4 h-4" /> : <ChevronDown className="w-4 h-4" />}
    </button>
  </div>
</div>

{/* Accordion - private message thread */}
{isOpen && (
  <div className="border-t border-border bg-muted/30 p-4 space-y-4">
    {/* Compose toggle */}
    <div className="flex items-center justify-between">
      <p className="text-xs font-semibold text-muted-foreground uppercase tracking-wide">Bacheca
Privata</p>

      <button
        onClick={() => setComposeOpen(prev => ({ ...prev, [c.id]: !prev[c.id] })))}
        className="flex items-center gap-1.5 text-xs text-primary hover:underline"
      >
        <Send className="w-3 h-3" />
        Nuovo messaggio
      </button>
    </div>

    {/* Compose form */}
    {composeOpen[c.id] && (
      <div className="border border-border rounded-md p-3 space-y-2.5 bg-card">
        {/* Reply context banner */}
        {replyTo[c.id] && (
          <div className="flex items-start gap-2 rounded-sm bg-primary/5 border border-primary/20
px-3 py-2 text-xs">
            <CornerDownLeft className="w-3 h-3 text-primary shrink-0 mt-0.5" />
            <div className="flex-1 min-w-0">
              <span className="text-muted-foreground">In risposta a: </span>
              <span className="font-medium">&ldquo;{replyTo[c.id]!.title}&rdquo;</span>
              <p className="text-muted-foreground truncate mt-0.5">
                {replyTo[c.id]!.body.length > 80
                  ? replyTo[c.id]!.body.slice(0, 80) + '...'
                  : replyTo[c.id]!.body}
              </p>
            </div>
          </div>
          <button
            onClick={() => setReplyTo(prev => ({ ...prev, [c.id]: null })))}
            className="shrink-0 text-muted-foreground hover:text-foreground"
            title="Annulla risposta"
          >
            <X className="w-3 h-3" />
          </button>
        </div>
      </div>
    )}
  <div className="space-y-1">
    <Label className="text-xs">Oggetto</Label>
    <Input
      value={composeTitle[c.id] ?? ''}
      onChange={e => setComposeTitle(prev => ({ ...prev, [c.id]: e.target.value })))}
      placeholder="Oggetto"
      className="h-8 text-sm rounded-sm"
    />
  </div>
  <div className="space-y-1">
    <Label className="text-xs">Messaggio</Label>
    <textarea

```

```

        value={composeBody[c.id] ?? ''}
        onChange={e => setComposeBody(prev => ({ ...prev, [c.id]: e.target.value })))}
        rows={3}
        placeholder="Scrivi qui..."
        className="w-full rounded-sm border border-input bg-background px-3 py-2 text-sm resize-
none focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring"
      />
    </div>
    <div className="space-y-1">
      <Label className="text-xs">Allegati <span className="text-muted-foreground font-
normal">(opzionale, max 10 MB totali – unificati in un unico PDF)</span></Label>
      <input
        type="file"
        multiple
        disabled={composeMerging[c.id] || composeSending[c.id]}
        onChange={e => {
          const incoming = Array.from(e.target.files ?? [])
          // Reset immediately so the same file can be re-selected and
          // subsequent picks don't accumulate inside the DOM input itself.
          e.target.value = ''
          const accumulated = [...(composeFiles[c.id] ?? []), ...incoming]
          const total = accumulated.reduce((sum, f) => sum + f.size, 0)
          if (total > MAX_BYTES) {
            setComposeError(prev => ({ ...prev, [c.id]: `Il limite massimo è 10 MB (attuale: $
{(total / 1024 / 1024).toFixed(1)} MB)` })))
            return
          }
          setComposeError(prev => ({ ...prev, [c.id]: null })))
          setComposeFiles(prev => ({ ...prev, [c.id]: accumulated })))
        }}
        className="text-xs file:mr-2 file:py-1 file:px-2 file:rounded file:border-0 file:bg-
primary file:text-primary-foreground disabled:opacity-50"
      />
      {(composeFiles[c.id]?.length ?? 0) > 0 && !composeMerging[c.id] && (
        <ul className="mt-1 space-y-1">
          {composeFiles[c.id].map((file, idx) => (
            <li key={idx} className="flex items-center justify-between gap-2 rounded bg-muted/60
px-2 py-1">
              <div className="flex items-center gap-1.5 min-w-0">
                <Paperclip className="w-3 h-3 shrink-0 text-muted-foreground" />
                <span className="text-[10px] truncate">{file.name}</span>
                <span className="text-[10px] text-muted-foreground shrink-0">{(file.size /
1024).toFixed(0)} KB</span>
              </div>
              <button
                type="button"
                onClick={() => setComposeFiles(prev => ({
                  ...prev,
                  [c.id]: (prev[c.id] ?? []).filter((_, i) => i !== idx),
                })))}
                className="shrink-0 p-0.5 rounded hover:bg-destructive/10 text-muted-foreground
hover:text-destructive transition-colors"
                title="Rimuovi"
              >
                <X className="w-3 h-3" />
              </button>
            </li>
          )})}
          <p className="text-[10px] text-muted-foreground pt-0.5">
            Totale: {(composeFiles[c.id].reduce((s, f) => s + f.size, 0) / 1024).toFixed(0)} KB
            · verranno unificati in un PDF
          </p>
        </ul>
      )}
      {composeMerging[c.id] && (
        <p className="text-[10px] text-amber-600 dark:text-amber-400 animate-pulse">
          Elaborazione e unificazione file in corso...
        </p>
      )}
    </div>
    {composeError[c.id] && <p className="text-xs text-destructive">{composeError[c.id]}</p>}
    <div className="flex gap-2 justify-end">
      <button
        onClick={() => setComposeOpen(prev => ({ ...prev, [c.id]: false })))}

```

```

disabled={composeMerging[c.id] || composeSending[c.id]}
className="text-xs text-muted-foreground hover:text-foreground disabled:opacity-50
disabled:cursor-not-allowed"
>
  Annulla
</button>
<Button
  size="sm"
  onClick={() => handleSendMessage(c.id)}
  disabled={composeMerging[c.id] || composeSending[c.id] || !composeTitle[c.id]?.trim()
|| !composeBody[c.id]?.trim()}
  className="h-7 text-xs"
>
  {composeMerging[c.id] ? 'Elaborazione...' : composeSending[c.id] ? 'Invio...' : 'Invia'}
</Button>
</div>
</div>
))

{/* Message thread */}
{threadLoading[c.id] ? (
  <p className="text-xs text-muted-foreground">Caricamento messaggi...</p>
) : thread.length === 0 ? (
  <p className="text-xs text-muted-foreground">Nessun messaggio ancora.</p>
) : (() => {
  // Build threaded display order: top-level newest-first,
  // replies oldest-first directly below their parent.
  const topLevel = thread
    .filter(m => !m.reply_to_id)
    .sort((a, b) => new Date(b.created_at).getTime() - new Date(a.created_at).getTime())
  const replyMap: Record<string, ConsultantMessage[]> = {}
  thread.filter(m => !!m.reply_to_id).forEach(m => {
    const pid = m.reply_to_id!
    if (!replyMap[pid]) replyMap[pid] = []
    replyMap[pid].push(m)
  })
  Object.values(replyMap).forEach(arr =>
    arr.sort((a, b) => new Date(a.created_at).getTime() - new Date(b.created_at).getTime())
  )
  const displayList = topLevel.flatMap(msg => [
    { msg, isReply: false },
    ...(replyMap[msg.id] ?? []).map(r => ({ msg: r, isReply: true })),
  ])

  return (
    <div className="space-y-2">
      {displayList.map(({ msg, isReply }) => {
        const isExpanded = msgExpanded[c.id] === msg.id
        const fromConsultant = !msg.sent_by_manager
        const unread = fromConsultant && !msg.read_at

        return (
          <div
            key={msg.id}
            className={isReply ? 'ml-6 pl-3 border-l-2 border-primary/20' : ''}
          >
            <div className={`border rounded overflow-hidden ${unread ? 'border-primary/40 bg-primary/5' : 'border-border bg-background'}`>
              <div className="flex items-center gap-1 pr-2">
                {/* Expand trigger */}
                <button
                  onClick={() => handleMsgExpand(c.id, msg)}
                  className="flex-1 min-w-0 flex items-center gap-2 px-3 py-2.5 text-left
                  hover:bg-accent/50 transition-colors"
                >
                  <div className="flex-1 min-w-0">
                    <div className="flex items-center gap-1.5">
                      {unread && <span className="w-1.5 h-1.5 rounded-full bg-primary
                      shrink-0" />}

                      {isReply && <CornerDownLeft className="w-3 h-3 text-primary/50 shrink-0" />}

                    <p className="text-xs font-medium truncate">{msg.title}</p>
                  </div>
                  <p className="text-[11px] text-muted-foreground">

```



```

        {fromConsultant ? `Da ${c.full_name}` : 'Da te'} .
{fmtDate(msg.created_at)}
        </p>
      </div>
      <div className="flex items-center gap-1 shrink-0">
        {msg.attachments?.length > 0 && <Paperclip className="w-3 h-3 text-muted-
foreground" />}
        {isExpanded ? <ChevronUp className="w-3.5 h-3.5 text-muted-foreground" /> :
<ChevronDown className="w-3.5 h-3.5 text-muted-foreground" />}
      </div>
    </button>
    { /* Reply */ }
    <button
      onClick={() => handleReply(c.id, msg)}
      className="shrink-0 p-1.5 rounded transition-colors text-muted-foreground
hover:text-primary"
      title="Rispondi"
    >
      <CornerDownLeft className="w-3.5 h-3.5" />
    </button>
    { /* Delete */ }
    <button
      onClick={() => setDeleteMsgTarget({ consultantId: c.id, msg })}
      disabled={msgDeleting[msg.id]}
      className="shrink-0 p-1.5 rounded transition-colors disabled:opacity-50 text-
destructive hover:text-destructive dark:text-red-400 dark:hover:bg-red-500/10 dark:hover:text-red-300"
      title="Elimina messaggio"
    >
      <Trash2 className="w-3.5 h-3.5" />
    </button>
  </div>

  {isExpanded && (
    <div className="px-3 pb-3 pt-2 border-t border-border space-y-2">
      <p className="text-xs whitespace-pre-wrap">{msg.body}</p>

      {msg.attachments?.length > 0 && (
        <div className="space-y-1">
          <p className="text-[10px] font-medium text-muted-foreground uppercase
tracking-wide">Allegati</p>

          {msg.attachments.map((att, i) => (
            <button
              key={i}
              onClick={() => handleMessageDownload(c.id, msg, att)}
              className="flex items-center gap-1.5 text-xs text-primary
hover:underline"
            >
              <Paperclip className="w-3 h-3" />
              {att.name}
            </button>
          ))}
        </div>
      )}

      <div className="space-y-0.5 pt-0.5">
        {msg.read_at && msg.sent_by_manager && (
          <p className="text-[10px] text-muted-foreground">Letto il:
{fmtDate(msg.read_at)}</p>

          )}
        {msg.downloaded_at && msg.sent_by_manager && (
          <p className="text-[10px] text-muted-foreground">Allegato scaricato il:
{fmtDate(msg.downloaded_at)}</p>

          )}
        </div>
      </div>
    )}
  </div>
)
  )}()
</div>

```

```

    })
  </div>
)
}}
</div>

{/* Create / Edit modal */}
<Dialog open={modalOpen} onOpenChange={open => { if (!open) setModalOpen(false) }}>
  <DialogContent onInteractOutside={e => e.preventDefault()}>
    <DialogHeader>
      <DialogTitle>{editTarget ? 'Modifica Consulente' : 'Nuovo Consulente del Lavoro'}</DialogTitle>
    </DialogHeader>

    <div className="space-y-4">
      <div className="space-y-1.5">
        <Label>Nome completo</Label>
        <Input value={formName} onChange={e => setFormName(e.target.value)} placeholder="Es. Mario Rossi"
className="h-10 rounded-sm" />
      </div>

      {!editTarget && (
        <>
          <div className="space-y-1.5">
            <Label>Username</Label>
            <Input value={formUsername} onChange={e => setFormUsername(e.target.value)} placeholder="Es.
mario.rossi" className="h-10 rounded-sm" />
          </div>
          <div className="space-y-1.5">
            <Label>Password</Label>
            <Input type="password" value={formPassword} onChange={e => setFormPassword(e.target.value)}
placeholder="Min. 6 caratteri" className="h-10 rounded-sm" />
          </div>
        </>
      )}

      {editTarget && (
        <>
          <p className="text-xs text-muted-foreground">Username: <span className="font-
mono">{editTarget.username}</span> (non modificabile)</p>
          <div className="space-y-1.5">
            <Label>Nuova password <span className="text-muted-foreground font-normal">(opzionale)</span></div>
            <div className="relative">
              <Input
                type={showNewPwd ? 'text' : 'password'}
                value={formNewPassword}
                onChange={e => setFormNewPassword(e.target.value)}
                placeholder="Lascia vuoto per non modificare"
                className="h-10 rounded-sm pr-10"
              />
              <button
                type="button"
                onClick={() => setShowNewPwd(v => !v)}
                className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground hover:text-
foreground"
                tabIndex={-1}
              >
                {showNewPwd ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
              </button>
            </div>
          </div>
        </>
      )}

      <div className="space-y-1.5">
        <Label>Ristoranti autorizzati</Label>
        <div className="flex flex-wrap gap-2">
          {restaurants.map(r => (
            <button
              key={r.id}
              type="button"
              onClick={() => toggleFormRestaurant(r.id)}
              className={`px-3 py-1.5 rounded-full text-sm font-medium border transition-colors ${
                formRestaurants.includes(r.id)

```

```

        ? 'bg-primary text-primary-foreground border-primary'
        : 'bg-background border-border text-foreground hover:bg-accent'
      )}
    }
  }
  >
    {r.name}
  </button>
  )}
</div>
{formRestaurants.length === 0 && (
  <p className="text-xs text-muted-foreground">Nessun ristorante selezionato</p>
)}
</div>

<label className="flex items-center gap-3 cursor-pointer">
  <input
    type="checkbox"
    checked={formHours}
    onChange={e => setFormHours(e.target.checked)}
    className="w-4 h-4 rounded accent-primary"
  />
  <span className="text-sm">Visualizzazione report ore</span>
</label>
</div>

{formError && (
  <p className="text-sm text-destructive bg-destructive/10 px-3 py-2 rounded-md">{formError}</p>
)}

<DialogFooter>
  <Button variant="outline" onClick={() => setModalOpen(false)} disabled={formSaving} className="h-10
rounded-sm">
    Annulla
  </Button>
  <Button onClick={handleSave} disabled={formSaving} className="h-10 rounded-sm">
    {formSaving ? 'Salvataggio...' : editTarget ? 'Salva modifiche' : 'Crea consulente'}
  </Button>
</DialogFooter>
</DialogContent>
</Dialog>

/* Delete message confirmation */
<Dialog open={!!deleteMsgTarget} onOpenChange={open => { if (!open) setDeleteMsgTarget(null) }}>
  <DialogContent>
    <DialogHeader>
      <DialogTitle>Elimina messaggio</DialogTitle>
    </DialogHeader>
    <p className="text-sm text-muted-foreground">
      Sei sicuro di voler eliminare <strong>&ldquo;{deleteMsgTarget?.msg.title}&rdquo;</strong>?
      {(deleteMsgTarget?.msg.attachments?.length ?? 0) > 0
        ? " L'allegato verrà rimosso definitivamente."
        : " Questa azione è irreversibile."
      }
    </p>
    <DialogFooter>
      <Button
        variant="outline"
        onClick={() => setDeleteMsgTarget(null)}
        disabled={!!deleteMsgTarget && msgDeleting[deleteMsgTarget.msg.id]}
        className="h-10 rounded-sm"
      >
        Annulla
      </Button>
      <Button
        variant="destructive"
        onClick={() => deleteMsgTarget && handleMsgDelete(deleteMsgTarget.consultantId,
deleteMsgTarget.msg.id)}
        disabled={!!deleteMsgTarget && msgDeleting[deleteMsgTarget.msg.id]}
        className="h-10 rounded-sm"
      >
        {deleteMsgTarget && msgDeleting[deleteMsgTarget.msg.id] ? 'Eliminazione...' : 'Conferma'}
      </Button>
    </DialogFooter>
  </DialogContent>
</Dialog>

```

```

    /* Delete confirm dialog */
    <Dialog open={!deleteTarget} onOpenChange={open => { if (!open) setDeleteTarget(null) }}>
      <DialogContent>
        <DialogHeader>
          <DialogTitle>Elimina consulente</DialogTitle>
        </DialogHeader>
        <p className="text-sm text-muted-foreground">
          Stai per eliminare <strong>{deleteTarget?.full_name}</strong>. Questa azione è irreversibile e
          rimuoverà anche tutti i messaggi.
        </p>
        <DialogFooter>
          <Button variant="outline" onClick={() => setDeleteTarget(null)} disabled={deleteLoading}
            className="h-10 rounded-sm">
            Annulla
          </Button>
          <Button variant="destructive" onClick={handleDelete} disabled={deleteLoading} className="h-10 rounded-
            sm">
            {deleteLoading ? 'Eliminazione...' : 'Elimina'}
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  </div>
)
}

```

[src/components/manager/DemoBanner.tsx](#)

```

'use client'
import { AlertTriangle } from 'lucide-react'

export function DemoBanner() {
  return (
    <div className="w-full bg-amber-500 dark:bg-amber-600 text-white px-4 py-2.5 flex items-center gap-2.5 text-sm
      font-medium">
      <AlertTriangle className="w-4 h-4 shrink-0" />
      <span>
        Account in attesa di approvazione – stai esplorando una <strong>demo</strong>.
        Alcune funzionalità sono disabilitate fino all'attivazione del servizio.
      </span>
    </div>
  )
}

```

`src/components/manager/DipendentiClient.tsx`

```

'use client'
import { useState } from 'react'
import { useRouter, usePathname } from 'next/navigation'
import { createClient } from '@lib/supabase/client'
import { useAccountStatus } from '@contexts/AccountStatusContext'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select'
import { Switch } from '@components/ui/switch'
import { Card, CardContent } from '@components/ui/card'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import {
  DropdownMenu, DropdownMenuContent, DropdownMenuItem,
  DropdownMenuSeparator, DropdownMenuTrigger,
} from '@components/ui/dropdown-menu'
import { Plus, User, MoreVertical, Pencil, KeyRound, Trash2, CheckCircle2 } from 'lucide-react'
import type { Profile, Restaurant, Role, Department, SecondaryDepartment, ShiftSlot } from '@types'
import { ROLE_LABELS, DEPARTMENTS, WEEK_DAYS_SHORT } from '@types'

const USERNAME_RE = /^[a-z0-9._-]+$/

function timeToMinutes(t: string): number {
  const [h, m] = t.split(':').map(Number)
  return h * 60 + m
}

const roleColors: Record<Role, string> = {
  manager: 'bg-purple-100 text-purple-800 dark:bg-purple-900/30 dark:text-purple-300',
  capo_servizio: 'bg-blue-100 text-blue-800 dark:bg-blue-900/30 dark:text-blue-300',
  dipendente: 'bg-slate-100 text-slate-700 dark:bg-slate-800 dark:text-slate-300',
  consulente_lavoro: 'bg-amber-100 text-amber-800 dark:bg-amber-900/30 dark:text-amber-300',
}

const deptColors: Record<Department, string> = {
  Sala: 'bg-amber-100 text-amber-800 dark:bg-amber-900/30 dark:text-amber-300',
  Pizzeria: 'bg-orange-100 text-orange-800 dark:bg-orange-900/30 dark:text-orange-300',
  Bar: 'bg-cyan-100 text-cyan-800 dark:bg-cyan-900/30 dark:text-cyan-300',
  Cucina: 'bg-red-100 text-red-800 dark:bg-red-900/30 dark:text-red-300',
}

type DipWithRestaurant = Profile & { restaurant?: { id: string; name: string } | null }

interface Props {
  initialDipendenti: DipWithRestaurant[]
  restaurants: Pick<Restaurant, 'id' | 'name'>[]
  currentUserRole: string
  currentIsDirettore?: boolean
  currentRestaurantId?: string | null
  currentRestaurantFilter?: string | null
}

export function DipendentiClient({
  initialDipendenti, restaurants, currentUserRole,
  currentIsDirettore = false, currentRestaurantId = null,
  currentRestaurantFilter = null,
}: Props) {
  const { isPending } = useAccountStatus()
  const router = useRouter()
  const pathname = usePathname()

  const [dipendenti, setDipendenti] = useState(initialDipendenti)
  const [search, setSearch] = useState('')

  // — Edit dialog —————
  const [showForm, setShowForm] = useState(false)
  const [editing, setEditing] = useState<DipWithRestaurant | null>(null)
  const [formLoading, setFormLoading] = useState(false)
  const [formError, setFormError] = useState<string | null>(null)

  const [username, setUsername] = useState('')
  const [usernameError, setUsernameError] = useState<string | null>(null)
  const [password, setPassword] = useState('')

```

```

const [fullName, setFullName] = useState('')
const [role, setRole] = useState<Role>('dipendente')
const [department, setDepartment] = useState<Department | ''>('')
const [restaurantId, setRestaurantId] = useState('none')
const [canPostBulletin, setCanPostBulletin] = useState(false)
const [isDirettore, setIsDirettore] = useState(false)

// — AI scheduling fields —————
const [weeklyRestDays, setWeeklyRestDays] = useState(1)
const [preferredRestDay, setPreferredRestDay] = useState<string>('none')
const [primarySlotIds, setPrimarySlotIds] = useState<string[]>([])
const [secondaryDepartments, setSecondaryDepartments] = useState<SecondaryDepartment[]>([])
const [weeklyHoursTarget, setWeeklyHoursTarget] = useState<string>('')
const [canSubstituteCapoServizio, setCanSubstituteCapoServizio] = useState(false)
const [restaurantSlots, setRestaurantSlots] = useState<ShiftSlot[]>([])

// — Password-reset dialog —————
const [pwTarget, setPwTarget] = useState<DipWithRestaurant | null>(null)
const [newPassword, setNewPassword] = useState('')
const [pwLoading, setPwLoading] = useState(false)
const [pwError, setPwError] = useState<string | null>(null)
const [pwSuccess, setPwSuccess] = useState(false)

// — Restaurant filter (manager only, URL-based — SSR-friendly) —————
function handleRestaurantFilter(value: string) {
  const params = new URLSearchParams()
  if (value !== 'all') params.set('restaurant_id', value)
  const qs = params.toString()
  router.push(qs ? `${pathname}?${qs}` : pathname)
}

// — Helpers —————
function validateUsername(val: string) {
  if (!val) { setUsernameError(null); return }
  setUsernameError(USERNAME_RE.test(val)
    ? null
    : 'Solo lettere minuscole, numeri, punti, trattini e underscore')
}

function openCreate() {
  setEditing(null)
  setUsername(''); setUsernameError(null); setPassword('')
  setFullName(''); setRole('dipendente'); setDepartment('')
  setRestaurantId(currentUserRole === 'manager' ? 'none' : (currentRestaurantId ?? 'none'))
  setCanPostBulletin(false); setIsDirettore(false)
  setWeeklyRestDays(1); setPreferredRestDay('none')
  setPrimarySlotIds([]); setSecondaryDepartments([])
  setWeeklyHoursTarget(''); setCanSubstituteCapoServizio(false)
  setRestaurantSlots([])
  setFormError(null); setShowForm(true)
}

async function openEdit(p: DipWithRestaurant) {
  setEditing(p)
  setUsername(''); setUsernameError(null); setPassword('')
  setFullName(p.full_name); setRole(p.role)
  setDepartment((p.department as Department | null) ?? '')
  setRestaurantId(p.restaurant_id ?? 'none')
  setCanPostBulletin(p.can_post_bulletin)
  setIsDirettore(p.is_direttore ?? false)
  setWeeklyRestDays(p.weekly_rest_days ?? 1)
  setPreferredRestDay(p.preferred_rest_day != null ? String(p.preferred_rest_day) : 'none')
  setPrimarySlotIds((p as Profile).primary_slot_ids ?? [])
  setSecondaryDepartments((p.secondary_departments ?? []) as SecondaryDepartment[])
  setWeeklyHoursTarget(p.weekly_hours_target != null ? String(p.weekly_hours_target) : '')
  setCanSubstituteCapoServizio(p.can_substitute_capo_servizio ?? false)
  setRestaurantSlots([])
  setFormError(null); setShowForm(true)

  // Carica fasce orarie del ristorante in background
  const rid = p.restaurant_id
  if (rid) {
    const supabase = createClient()
    const { data } = await supabase

```

```

        .from('shift_slots')
        .select('*')
        .eq('restaurant_id', rid)
        .order('department')
        .order('start_time')
        setRestaurantSlots((data ?? []) as ShiftSlot[])
    }
}

function openPasswordReset(p: DipWithRestaurant) {
    setPwTarget(p); setNewPassword('')
    setPwError(null); setPwSuccess(false)
}

function closePwDialog() {
    setPwTarget(null); setNewPassword('')
    setPwError(null); setPwSuccess(false)
}

// — Save (create / edit) —————
async function handleSave() {
    setFormLoading(true); setFormError(null)
    try {
        if (editing) {
            const res = await fetch('/api/users', {
                method: 'PATCH',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({
                    id: editing.id, full_name: fullName, role,
                    department: department || null,
                    restaurant_id: restaurantId === 'none' ? null : restaurantId,
                    can_post_bulletin: canPostBulletin,
                    is_direttore: role === 'capo_servizio' ? isDirettore : false,
                    weekly_rest_days: weeklyRestDays,
                    preferred_rest_day: preferredRestDay === 'none' ? null : parseInt(preferredRestDay),
                    primary_slot_ids: primarySlotIds,
                    secondary_departments: secondaryDepartments,
                    weekly_hours_target: weeklyHoursTarget ? parseInt(weeklyHoursTarget) : null,
                    can_substitute_capo_servizio: canSubstituteCapoServizio,
                }),
            })
            const data = await res.json()
            if (!res.ok) throw new Error(data.error)
            setDipendenti(ds => ds.map(d => d.id === data.id ? { ...d, ...data } : d))
        } else {
            const res = await fetch('/api/users', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({
                    username, password, full_name: fullName, role,
                    department: department || null,
                    restaurant_id: restaurantId === 'none' ? null : restaurantId,
                    can_post_bulletin: canPostBulletin,
                    is_direttore: role === 'capo_servizio' ? isDirettore : false,
                }),
            })
            const data = await res.json()
            if (!res.ok) throw new Error(data.error)
            setDipendenti(ds => [...ds, data])
        }
        setShowForm(false)
    } catch (e) {
        setFormError(e instanceof Error ? e.message : 'Errore durante il salvataggio')
    } finally {
        setFormLoading(false)
    }
}

// — Reset password —————
async function handleResetPassword() {
    if (!pwTarget) return
    setPwLoading(true); setPwError(null)
    try {
        const res = await fetch('/api/users/password', {

```



```

        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ id: pwTarget.id, password: newPassword }),
    })
    const data = await res.json()
    if (!res.ok) throw new Error(data.error)
    setPwSuccess(true)
    // Auto-close after 1.2 s so the user sees the success confirmation
    setTimeout(closePwDialog, 1200)
  } catch (e) {
    setPwError(e instanceof Error ? e.message : 'Errore durante il salvataggio')
  } finally {
    setPwLoading(false)
  }
}

async function handleDelete(id: string) {
  if (!confirm('Eliminare questo utente? Questa azione è irreversibile.')) return
  const res = await fetch(`/api/users?id=${id}`, { method: 'DELETE' })
  if (res.ok) setDipendenti(ds => ds.filter(d => d.id !== id))
}

const filtered = dipendenti.filter(d =>
  d.full_name.toLowerCase().includes(search.toLowerCase()) ||
  (d.username ?? '').toLowerCase().includes(search.toLowerCase())
)

const isManager = currentUserRole === 'manager'
// A "Direttore" (capo_servizio + is_direttore) can manage users within its
// own restaurant, with manager-equivalent powers confined to that locale.
const canManageUsers = isManager || (currentUserRole === 'capo_servizio' && currentIsDirettore)
// Direttore cannot create/assign the manager role.
// consulente_lavoro is managed via the dedicated ConsulenteLavoroManager section
const assignableRoles = (Object.keys(ROLE_LABELS) as Role[])
  .filter(r => r !== 'consulente_lavoro' && (isManager || r !== 'manager'))
const needsDept = role !== 'manager'
const canSave = !!fullName.trim() &&
  (editing
    ? !usernameError
    : !!username.trim() && !usernameError && !!password.trim()) &&
  (!needsDept || !!department)

return (
  <div>
    {/* Header */}
    <div className="flex items-center justify-between mb-6">
      <div>
        <h1 className="text-2xl font-bold">Dipendenti</h1>
        <p className="text-muted-foreground text-sm mt-1">{dipendenti.length} utenti</p>
      </div>
      {canManageUsers && (
        <Button onClick={openCreate} size="sm" disabled={isPending} title={isPending ? 'Disponibile dopo
l\'attivazione' : undefined}>
          <Plus className="w-4 h-4" /> Nuovo
        </Button>
      )}
    </div>

    <div className="flex flex-col sm:flex-row gap-3 mb-4">
      <Input
        placeholder="Cerca per nome o username..."
        value={search}
        onChange={e => setSearch(e.target.value)}
        className="sm:max-w-sm"
      />
      {/* Restaurant filter – manager only. Capo servizio is locked server-side. */}
      {isManager && (
        <Select
          value={currentRestaurantFilter ?? 'all'}
          onChange={handleRestaurantFilter}
        >
          <SelectTrigger className="sm:w-56">
            <SelectValue placeholder="Tutti i ristoranti" />
          </SelectTrigger>

```

```

        <SelectContent>
          <SelectItem value="all">Tutti i ristoranti</SelectItem>
          {restaurants.map(r => (
            <SelectItem key={r.id} value={r.id}>{r.name}</SelectItem>
          ))}
        </SelectContent>
      </Select>
    )}
  </div>

  {/* User list */}
  <div className="space-y-2">
    {filtered.map(d => (
      <Card key={d.id}>
        <CardContent className="p-4 flex items-center gap-4">
          <div className="w-10 h-10 rounded-full bg-primary/10 flex items-center justify-center shrink-0">
            <User className="w-5 h-5 text-primary" />
          </div>
          <div className="flex-1 min-w-0">
            <div className="flex items-center gap-2 flex-wrap mb-0.5">
              <p className="font-medium">{d.full_name}</p>
              <span className={`text-xs px-2 py-0.5 rounded-full font-medium ${roleColors[d.role]}`}>
                {ROLE_LABELS[d.role]}
              </span>
            </div>
            <div>
              {d.department && (
                <span className={`text-xs px-2 py-0.5 rounded-full font-medium ${deptColors[d.department as
Department]]}`}>
                  {d.department}
                </span>
              )}
            </div>
          </div>
          <p className="text-xs text-muted-foreground truncate">
            {d.username ? `@${d.username}` : ''}
            {d.username && d.restaurant && ' · '}
            {d.restaurant?.name}
          </p>
        </div>

        {/* Actions dropdown – manager / direttore */}
        {canManageUsers && (
          <DropdownMenu>
            <DropdownMenuTrigger asChild>
              <Button variant="ghost" size="icon" className="shrink-0">
                <MoreVertical className="w-4 h-4" />
              </Button>
            </DropdownMenuTrigger>
            <DropdownMenuContent align="end" className="w-44">
              <DropdownMenuItem onClick={() => openEdit(d)}>
                <Pencil className="w-4 h-4" /> Modifica
              </DropdownMenuItem>

              {/* "Cambia Password" is hidden for other managers */}
              {d.role !== 'manager' && (
                <DropdownMenuItem onClick={() => openPasswordReset(d)}>
                  <KeyRound className="w-4 h-4" /> Cambia Password
                </DropdownMenuItem>
              )}

              <DropdownMenuSeparator />
              <DropdownMenuItem
                onClick={() => handleDelete(d.id)}
                className="text-destructive focus:text-destructive dark:text-red-400 dark:focus:text-red-300
dark:focus:bg-red-500/10"
              >
                <Trash2 className="w-4 h-4" /> Elimina
              </DropdownMenuItem>
            </DropdownMenuContent>
          </DropdownMenu>
        )}
      </CardContent>
    </Card>
  )})
  {filtered.length === 0 && (
    <Card>

```

```

        <CardContent className="py-12 text-center text-muted-foreground">
            Nessun utente trovato
        </CardContent>
    </Card>
    )}
</div>

{/* — Edit / Create dialog ————— */}
<Dialog open={showForm} onOpenChange={setShowForm}>
    <DialogContent className="max-h-[90vh] overflow-y-auto">
        <DialogHeader>
            <DialogTitle>{editing ? 'Modifica Utente' : 'Nuovo Utente'}</DialogTitle>
        </DialogHeader>
        <div className="space-y-4">
            {!editing && (
                <div className="grid grid-cols-2 gap-3">
                    <div className="space-y-2">
                        <Label>Username *</Label>
                        <Input
                            value={username}
                            onChange={e => { setUsername(e.target.value.toLowerCase());
validateUsername(e.target.value.toLowerCase()) }}
                            placeholder="mario.rossi"
                            autoCapitalize="none" autoComplete="off" spellCheck={false}
                        />
                        {usernameError && <p className="text-xs text-destructive">{usernameError}</p>}
                    </div>
                    <div className="space-y-2">
                        <Label>Password *</Label>
                        <Input type="password" value={password} onChange={e => setPassword(e.target.value)}
placeholder="Min. 6 caratteri" />
                    </div>
                </div>
            )}
            <div className="space-y-2">
                <Label>Nome completo *</Label>
                <Input value={fullName} onChange={e => setFullName(e.target.value)} placeholder="Mario Rossi" />
            </div>
            <div className="grid grid-cols-2 gap-3">
                <div className="space-y-2">
                    <Label>Ruolo *</Label>
                    <Select value={role} onValueChange={v => { setRole(v as Role); if (v === 'manager')
setDepartment('') }}>
                        <SelectTrigger><SelectValue /></SelectTrigger>
                        <SelectContent>
                            {assignableRoles.map(r => (
                                <SelectItem key={r} value={r}>{ROLE_LABELS[r]}</SelectItem>
                            ))}
                        </SelectContent>
                    </Select>
                </div>
                <div>
                    <div className="space-y-2">
                        <Label>Reparto {needsDept ? '*' : ''}</Label>
                        <Select value={department} onValueChange={v => setDepartment(v as Department)} disabled={!
needsDept}>
                            <SelectTrigger><SelectValue placeholder="Seleziona reparto" /></SelectTrigger>
                            <SelectContent>
                                {DEPARTMENTS.map(d => <SelectItem key={d} value={d}>{d}</SelectItem>)}
                            </SelectContent>
                        </Select>
                    </div>
                </div>
            </div>
            {/* Restaurant field — manager only. For capo servizio / direttore the
                value is auto-set to its own restaurant in openCreate/openEdit and
                hidden, so it can never assign a user to another locale. */}
            {isManager && (
                <div className="space-y-2">
                    <Label>Ristorante</Label>
                    <Select value={restaurantId} onValueChange={setRestaurantId}>
                        <SelectTrigger><SelectValue placeholder="Nessun ristorante" /></SelectTrigger>
                        <SelectContent>
                            <SelectItem value="none">Nessun ristorante</SelectItem>
                            {restaurants.map(r => <SelectItem key={r.id} value={r.id}>{r.name}</SelectItem>)}
                        </SelectContent>
                    </Select>
                </div>
            )}
        </div>
    </DialogContent>
</Dialog>

```

```

        </Select>
      </div>
    )}
    {role === 'capo_servizio' && (
      <div>
        {/* Direttore: global access to all departments of the restaurant.
           Only a manager can grant this flag. */}
        <div className="flex items-center justify-between gap-4">
          <div className="space-y-0.5">
            <Label>Direttore (Accesso Globale al Locale)</Label>
            <p className="text-xs text-muted-foreground">
              Gestisce ODS e utenti di tutti i reparti del ristorante.
            </p>
          </div>
          <Switch
            checked={isDirettore}
            onCheckedChange={setIsDirettore}
            disabled={!isManager}
          />
        </div>
        <div className="flex items-center justify-between">
          <Label>Può pubblicare in bacheca</Label>
          <Switch checked={canPostBulletin} onCheckedChange={setCanPostBulletin} />
        </div>
      </div>
    )}
    {/* — Campi AI scheduling – solo in modifica (non creazione) — */}
    {editing && (
      <div className="border-t border-border pt-4 space-y-3">
        <p className="text-xs font-semibold text-muted-foreground uppercase tracking-wide">
          Pianificazione turni
        </p>

        <div className="grid grid-cols-2 gap-3">
          <div className="space-y-1.5">
            <Label className="text-sm">Riposi/settimana</Label>
            <input
              type="number" min={0} max={7}
              value={weeklyRestDays}
              onChange={e => setWeeklyRestDays(parseInt(e.target.value) || 0)}
              className="flex h-9 w-full rounded-md border border-input bg-background px-3 py-1 text-sm
shadow-sm"
            />
          </div>
          <div className="space-y-1.5">
            <Label className="text-sm">Ore/settimana <span className="font-normal text-muted-foreground">(vuoto = full-time)</span></Label>
            <input
              type="number" min={1} max={60} placeholder="es. 20"
              value={weeklyHoursTarget}
              onChange={e => setWeeklyHoursTarget(e.target.value)}
              className="flex h-9 w-full rounded-md border border-input bg-background px-3 py-1 text-sm
shadow-sm"
            />
          </div>
        </div>

        <div className="space-y-1.5">
          <Label className="text-sm">Giorno di riposo preferito</Label>
          <select
            value={preferredRestDay}
            onChange={e => setPreferredRestDay(e.target.value)}
            className="flex h-9 w-full rounded-md border border-input bg-background px-3 py-1 text-sm
shadow-sm"
          >
            <option value="none">Nessuna preferenza</option>
            {[1,2,3,4,5,6,0].map(d => (
              <option key={d} value={String(d)}>{WEEK_DAYS_SHORT[d]}</option>
            ))}
          </select>
        </div>

        <div className="flex items-center justify-between rounded-sm border border-border px-3 py-2.5">
          <div className="space-y-0.5">

```

```

    <Label className="text-sm">Può fare da senior / sostituire capo servizio</Label>
    <p className="text-xs text-muted-foreground">
      L'IA garantisce almeno un senior per slot. Chi non lo è non può stare da solo.
    </p>
  </div>
  <Switch checked={canSubstituteCapoServizio} onCheckedChange={setCanSubstituteCapoServizio} />
</div>

{/* Fasce principali (slot del reparto principale) */}
{(() => {
  const empDept = department || (editing as DipWithRestaurant).department
  const primarySlots = restaurantSlots.filter(s => s.department === empDept)
  if (!empDept || restaurantSlots.length === 0) return null
  return (
    <div className="space-y-2">
      <Label className="text-sm">
        Fasce principali <span className="font-normal text-muted-foreground">({empDept})</span>
      </Label>
      {primarySlots.length === 0 ? (
        <p className="text-xs text-muted-foreground">Nessuna fascia configurata per {empDept}</p>
      ) : (
        <div className="space-y-1">
          {primarySlots.map(s => {
            const overnight = timeToMinutes(s.end_time) <= timeToMinutes(s.start_time)
            return (
              <label key={s.id} className="flex items-center gap-2 cursor-pointer">
                <input
                  type="checkbox"
                  checked={primarySlotIds.includes(s.id)}
                  onChange={e => {
                    if (e.target.checked) setPrimarySlotIds(prev => [...prev, s.id])
                    else setPrimarySlotIds(prev => prev.filter(id => id !== s.id))
                  }}
                  className="accent-primary"
                />
                <span className="text-sm">{s.name}</span>
                <span className="text-xs text-muted-foreground tabular-nums">
                  {s.start_time.slice(0,5)}-{s.end_time.slice(0,5)}
                  {overnight && <span className="text-amber-600 dark:text-amber-400 ml-0.5">(+1)</span>
span>}
                </span>
              </label>
            )
          })}
        </div>
      )}
    </div>
  )
})()

{/* Fasce jolly (slot di altri reparti) – solo manager */}
{isManager && (() => {
  const empDept = department || (editing as DipWithRestaurant).department
  const jollySlots = restaurantSlots.filter(s => s.department !== empDept)
  if (restaurantSlots.length === 0) return (
    <p className="text-xs text-muted-foreground">Caricamento fasce...</p>
  )
  if (jollySlots.length === 0) return null

  // Raggruppa per reparto
  const byDept = jollySlots.reduce<Record<string, ShiftSlot[]>>((acc, s) => {
    ;acc[s.department] ??= [].push(s)
    return acc
  }, {})

  return (
    <div className="space-y-2">
      <Label className="text-sm">Fasce jolly <span className="font-normal text-muted-foreground">(se necessario lavora in altro reparto)</span></Label>
      <div className="space-y-3">
        {Object.entries(byDept).map(([dept, deptSlots]) => (
          <div key={dept}>
            <p className="text-xs font-medium text-muted-foreground mb-1">{dept}</p>
            <div className="space-y-1 pl-2">

```



```

    })
  </DialogHeader>

  {pwSuccess ? (
    <div className="flex flex-col items-center gap-3 py-4 text-center">
      <CheckCircle2 className="w-10 h-10 text-emerald-500" />
      <p className="text-sm font-medium text-emerald-600 dark:text-emerald-400">
        Password aggiornata con successo
      </p>
    </div>
  ) : (
    <>
      <div className="space-y-2">
        <Label htmlFor="new-password">Nuova password *</Label>
        <Input
          id="new-password"
          type="password"
          value={newPassword}
          onChange={e => setNewPassword(e.target.value)}
          placeholder="Min. 6 caratteri"
          autoComplete="new-password"
        />
        {newPassword.length > 0 && newPassword.length < 6 && (
          <p className="text-xs text-muted-foreground">Almeno 6 caratteri</p>
        )}
      </div>
      {pwError && (
        <p className="text-sm text-destructive bg-destructive/10 px-3 py-2 rounded-md">{pwError}</p>
      )}
      <DialogFooter>
        <Button variant="outline" onClick={closePwDialog} disabled={pwLoading}>Annulla</Button>
        <Button
          onClick={handleResetPassword}
          disabled={pwLoading || newPassword.trim().length < 6}
        >
          {pwLoading ? 'Salvataggio...' : 'Salva'}
        </Button>
      </DialogFooter>
    </>
  )}
</DialogContent>
</Dialog>
</div>
)
}

```

`src/components/manager/FallbackApprovalSection.tsx`


```

'use client'
import { useState } from 'react'
import { createClient } from '@lib/supabase/client'
import { ShieldAlert, Check, X, Image as ImageIcon } from 'lucide-react'
import { Button } from '@components/ui/button'
import { formatInTimeZone } from 'date-fns-tz'
import { it } from 'date-fns/locale'

const TZ = 'Europe/Rome'
const BUCKET = 'clock_in_proofs'

export interface PendingItem {
  id: string
  user_id: string
  check_in: string
  check_out: string | null
  fallback_photo_path: string
  restaurant_id: string | null
  profile?: { full_name: string } | null
  restaurant?: { name: string } | null
}

interface Props {
  initialPending: PendingItem[]
}

export function FallbackApprovalSection({ initialPending }: Props) {
  const [pending, setPending] = useState(initialPending)
  const [loadingId, setLoadingId] = useState<string | null>(null)
  const [previewUrl, setPreviewUrl] = useState<string | null>(null)
  const [previewLoadId, setPreviewLoadId] = useState<string | null>(null)

  async function handleAction(attendanceId: string, action: 'approve' | 'reject') {
    setLoadingId(attendanceId)
    const res = await fetch('/api/presenze/fallback-approve', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ attendanceId, action }),
    })
    if (res.ok) {
      setPending(prev => prev.filter(p => p.id !== attendanceId))
    }
    setLoadingId(null)
  }

  async function handlePreview(item: PendingItem) {
    if (previewLoadId === item.id) return
    setPreviewLoadId(item.id)
    const supabase = createClient()
    const { data } = await supabase.storage
      .from(BUCKET)
      .createSignedUrl(item.fallback_photo_path, 300)
    if (data?.signedUrl) setPreviewUrl(data.signedUrl)
    setPreviewLoadId(null)
  }

  if (pending.length === 0) return null

  return (
    <div className="mb-6 border border-amber-200 dark:border-amber-800 rounded-md overflow-hidden">
      <div className="flex items-center gap-2 bg-amber-50 dark:bg-amber-950/30 px-4 py-2.5 border-b border-amber-200 dark:border-amber-800">
        <ShieldAlert className="w-4 h-4 text-amber-600 dark:text-amber-400 shrink-0" />
        <h2 className="text-sm font-semibold text-amber-800 dark:text-amber-300">
          Timbrature di emergenza in attesa ({pending.length})
        </h2>
      </div>

      <div className="divide-y divide-border">
        {pending.map(item => {
          const isWorking = loadingId === item.id
          const isLoadingPrev = previewLoadId === item.id

```

```

const checkIn = formatInTimeZone(new Date(item.check_in), TZ, 'dd/MM/yyyy HH:mm', { locale: it })
const checkOut = item.check_out
  ? formatInTimeZone(new Date(item.check_out), TZ, 'HH:mm', { locale: it })
  : null

return (
  /* — Mobile: colonna / Desktop: riga ————— */
  <div
    key={item.id}
    className="flex flex-col gap-3 px-4 py-4 bg-background
      md:flex-row md:items-center md:justify-between md:gap-4"
  >
    /* Info — occupa tutto su mobile, si restringe su desktop */
    <div className="min-w-0">
      <p className="text-sm font-medium">{item.profile?.full_name ?? '-'}</p>
      <p className="text-xs text-muted-foreground mt-0.5">
        {item.restaurant?.name && (
          <span className="mr-2">{item.restaurant.name}</span>
        )}
        {checkOut ? `${checkIn} → ${checkOut}` : `Ingresso ${checkIn}`}
      </p>
    </div>

    /* Azioni — wrap automatico su mobile */
    <div className="flex flex-wrap items-center gap-2 md:shrink-0">
      <button
        onClick={() => handlePreview(item)}
        disabled={isLoadingPrev}
        className="flex items-center gap-1.5 text-xs text-primary hover:underline disabled:opacity-50"
      >
        <ImageIcon className="w-3.5 h-3.5" />
        {isLoadingPrev ? 'Caricamento...' : 'Vedi foto'}
      </button>

      <Button
        size="sm"
        className="h-8 px-3 text-xs bg-emerald-600 hover:bg-emerald-700 text-white rounded-sm"
        onClick={() => handleAction(item.id, 'approve')}
        disabled={isWorking}
      >
        <Check className="w-3.5 h-3.5 mr-1" />
        Approva
      </Button>

      <Button
        size="sm"
        variant="destructive"
        className="h-8 px-3 text-xs rounded-sm"
        onClick={() => handleAction(item.id, 'reject')}
        disabled={isWorking}
      >
        <X className="w-3.5 h-3.5 mr-1" />
        Rifiuta
      </Button>
    </div>
  </div>
)
)}}
</div>

/* Full-screen photo preview overlay */
{previewUrl && (
  <div
    className="fixed inset-0 z-50 bg-black/80 flex items-center justify-center p-4"
    onClick={() => setPreviewUrl(null)}
  >
    <div
      className="relative max-w-lg w-full"
      onClick={e => e.stopPropagation()}
    >
      /* eslint-disable-next-line @next/next/no-img-element */
      <img
        src={previewUrl}
        alt="Prova di timbratura"
      />
    </div>
  </div>
)}

```

```

        className="w-full rounded-lg shadow-2xl"
      />
      <button
        onClick={() => setPreviewUrl(null)}
        className="absolute -top-3 -right-3 w-8 h-8 rounded-full bg-white text-black flex items-center
justify-center shadow-lg text-sm font-bold"
      >
        x
      </button>
    </div>
  </div>
)}
</div>
)
}

```

[src/components/manager/LiveShiftCounter.tsx](#)

```

'use client'
import { useState, useEffect } from 'react'
import { differenceInMinutes } from 'date-fns'

function formatDuration(minutes: number): string {
  const h = Math.floor(minutes / 60)
  const m = minutes % 60
  return `${h}h ${String(m).padStart(2, '0')}m`
}

interface Props {
  checkInTime: string
  baseMinutes?: number
}

export function LiveShiftCounter({ checkInTime, baseMinutes = 0 }: Props) {
  const [now, setNow] = useState(() => new Date())

  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 60_000)
    return () => clearInterval(id)
  }, [])

  const elapsed = Math.max(0, differenceInMinutes(now, new Date(checkInTime)))
  return <>{formatDuration(baseMinutes + elapsed)}</>
}

```

`src/components/manager/ManagerSidebar.tsx`

```

'use client'
import { useState, useEffect } from 'react'
import Link from 'next/link'
import { usePathname } from 'next/navigation'
import { createClient } from '@lib/supabase/client'
import { useRouter } from 'next/navigation'
import {
  LayoutDashboard, Store, Users, Clock, CalendarX,
  CheckSquare, MessageSquare, FileSpreadsheet, Logout,
  Menu, X, Bell, ClipboardList, CalendarClock, UserCheck
} from 'lucide-react'
import { cn } from '@lib/utils'
import { ThemeToggle } from '@components/shared/ThemeToggle'
import { useBadging } from '@hooks/useBadging'
import type { Profile } from '@types'
import { ROLE_LABELS } from '@types'

// `direttoreOnly: true` → visibile anche a capo_servizio con is_direttore === true,
// oltre ai ruoli elencati in `roles`.
const navItems = [
  { href: '/dashboard', icon: LayoutDashboard, label: 'Dashboard', roles: ['manager', 'capo_servizio'] },
  { href: '/turni', icon: CalendarClock, label: 'Turni', roles: ['manager', 'capo_servizio'] },
  { href: '/ristoranti', icon: Store, label: 'Ristoranti', roles: ['manager'] },
  { href: '/dipendenti', icon: Users, label: 'Dipendenti', roles: ['manager'], direttoreOnly: true },
  { href: '/presenze', icon: Clock, label: 'Presenze', roles: ['manager'] },
  { href: '/assenze', icon: CalendarX, label: 'Assenze', roles: ['manager'], direttoreOnly: true },
  { href: '/approvazioni', icon: CheckSquare, label: 'Approvazioni', roles: ['manager'], direttoreOnly: true },
  { href: '/bacheca', icon: MessageSquare, label: 'Bacheca', roles: ['manager', 'capo_servizio'] },
  { href: '/ods', icon: ClipboardList, label: 'ODS', roles: ['manager', 'capo_servizio'] },
  { href: '/report', icon: FileSpreadsheet, label: 'Report', roles: ['manager', 'capo_servizio'] },
  { href: '/account-pendenti', icon: UserCheck, label: 'Account Pendenti', roles: ['manager'], platformOwnerOnly: true },
]

interface Props {
  profile: Profile & { restaurant?: { id: string; name: string } | null }
}

export function ManagerSidebar({ profile }: Props) {
  const [open, setOpen] = useState(false)
  const [unreadBulletins, setUnreadBulletins] = useState(0)
  const [unreadOds, setUnreadOds] = useState(0)
  const [pendingCount, setPendingCount] = useState(0)
  const pathname = usePathname()
  const router = useRouter()
  useBadging(unreadOds)

  // Conteggio comunicati non letti (solo capo_servizio)
  useEffect(() => {
    if (profile.role !== 'capo_servizio') return
    const lastSeen = localStorage.getItem('bulletins_last_seen') ?? '1970-01-01T00:00:00Z'
    const supabase = createClient()
    supabase
      .from('bulletins')
      .select('id', { count: 'exact', head: true })
      .gt('created_at', lastSeen)
      .then(({ count }) => setUnreadBulletins(count ?? 0))
  }, [profile.role])

  // Azzerare il badge quando si visita /bacheca
  useEffect(() => {
    if (profile.role !== 'capo_servizio') return
    if (pathname !== '/bacheca') return
    localStorage.setItem('bulletins_last_seen', new Date().toISOString())
    setUnreadBulletins(0)
  }, [pathname, profile.role])

  // Notifiche ODS non lette (solo capo_servizio) con realtime
  useEffect(() => {
    if (profile.role !== 'capo_servizio') return
    const supabase = createClient()

```

```

    async function fetchUnread() {
      const { count } = await supabase
        .from('notifications')
        .select('id', { count: 'exact', head: true })
        .is('read_at', null)
        .setUnread0ds(count ?? 0)
    }

    fetchUnread()

    const channel = supabase
      .channel('sidebar_ods_notifications')
      .on('postgres_changes', { event: '*', schema: 'public', table: 'notifications' }, fetchUnread)
      .subscribe()

    return () => { supabase.removeChannel(channel) }
  }, [profile.role])

  // Conta account pendenti (solo platform owner)
  useEffect(() => {
    if (profile.role !== 'manager' || profile.managed_restaurant_ids !== null) return
    const supabase = createClient()
    supabase
      .from('profiles')
      .select('id', { count: 'exact', head: true })
      .eq('role', 'manager')
      .eq('account_status', 'pending')
      .then(({ count }) => setPendingCount(count ?? 0))
  }, [profile.role, profile.managed_restaurant_ids])

  // Lock body scroll while mobile drawer is open
  useEffect(() => {
    if (open) {
      document.body.style.overflow = 'hidden'
    } else {
      document.body.style.overflow = ''
    }
    return () => { document.body.style.overflow = '' }
  }, [open])

  const isDirettore = profile.role === 'capo_servizio' && profile.is_direttore === true
  const isPlatformOwner = profile.role === 'manager' && profile.managed_restaurant_ids === null
  const visibleItems = navItems.filter(item =>
    (item.roles.includes(profile.role) || (item.direttoreOnly === true && isDirettore)) &&
    (!('platformOwnerOnly' in item) || (item.platformOwnerOnly === true && isPlatformOwner))
  )

  async function handleLogout() {
    const supabase = createClient()
    await supabase.auth.signOut()
    router.push('/login')
  }

  const SidebarContent = () => (
    <div className="flex flex-col h-full">
      <div className="p-6 border-b border-border">
        <h1 className="text-xl font-bold">inTurno</h1>
        <p className="text-xs text-muted-foreground mt-0.5">Turni, Presenze e ODS</p>
      </div>

      <nav className="flex-1 overflow-y-auto p-4 space-y-1">
        {visibleItems.map(({ href, icon: Icon, label }) => (
          <Link
            key={href}
            href={href}
            onClick={() => setOpen(false)}
            className={cn(
              'flex items-center gap-3 px-3 py-2.5 rounded-lg text-sm font-medium transition-colors',
              pathname === href || pathname.startsWith(href + '/')
                ? 'bg-primary text-primary-foreground'
                : 'text-muted-foreground hover:bg-accent hover:text-foreground'
            )}
          >
            <Icon className="w-4 h-4 shrink-0" />

```

```

        <span className="flex-1">{label}</span>
        {href === '/bacheca' && profile.role === 'capo_servizio' && unreadBulletins > 0 && (
          <span className="ml-auto w-5 h-5 rounded-full bg-red-500 text-white text-[10px] font-bold flex
items-center justify-center leading-none">
            {unreadBulletins > 9 ? '9+' : unreadBulletins}
          </span>
        )}
        {href === '/ods' && profile.role === 'capo_servizio' && unreadOds > 0 && (
          <span className="ml-auto w-5 h-5 rounded-full bg-red-500 text-white text-[10px] font-bold flex
items-center justify-center leading-none">
            {unreadOds > 9 ? '9+' : unreadOds}
          </span>
        )}
        {href === '/account-pendenti' && isPlatformOwner && pendingCount > 0 && (
          <span className="ml-auto w-5 h-5 rounded-full bg-amber-500 text-white text-[10px] font-bold flex
items-center justify-center leading-none">
            {pendingCount > 9 ? '9+' : pendingCount}
          </span>
        )}
      </Link>
    )}
  </nav>

  <div className="p-4 border-t border-border">
    <div className="flex items-center gap-3 mb-3">
      <div className="w-8 h-8 rounded-full bg-primary/10 flex items-center justify-center text-sm font-
semibold text-primary">
        {profile.full_name[0]?.toUpperCase()}
      </div>
      <div className="flex-1 min-w-0">
        <p className="text-sm font-medium truncate">{profile.full_name}</p>
        <p className="text-xs text-muted-foreground">{ROLE_LABELS[profile.role]}</p>
      </div>
    </div>
    <div className="flex items-center justify-between">
      <button
        onClick={handleLogout}
        className="flex items-center gap-2 text-sm text-muted-foreground hover:text-foreground transition-
colors"
      >
        <Logout className="w-4 h-4" />
        Esci
      </button>
      <ThemeToggle />
    </div>
  </div>
)

return (
  <>
    { /* Desktop sidebar */ }
    <aside className="hidden lg:flex w-64 h-full flex-col border-r border-border bg-card shrink-0">
      <SidebarContent />
    </aside>

    { /* Mobile header + drawer */ }
    <div className="lg:hidden fixed top-0 left-0 right-0 z-40 flex items-center h-14 px-4 border-b border-border
bg-background">
      <button
        onClick={() => setOpen(true)}
        className="p-2 rounded-md hover:bg-accent"
      >
        <Menu className="w-5 h-5" />
      </button>
      <span className="ml-3 font-semibold flex-1">
        {navItems.find(item => pathname === item.href || pathname.startsWith(item.href + '/'))?.label ??
'inTurno'}
      </span>
      {profile.role === 'capo_servizio' && (
        <Link href="/bacheca" className="relative p-2 rounded-md hover:bg-accent text-muted-foreground">
          <Bell className="w-5 h-5" />
          {unreadBulletins > 0 && (
            <span className="absolute top-1 right-1 w-4 h-4 rounded-full bg-red-500 text-white text-[10px] font-

```

```

bold flex items-center justify-center leading-none">
    {unreadBulletins > 9 ? '9+' : unreadBulletins}
    </span>
  })
</Link>
})
</div>

{open && (
  <div className="lg:hidden fixed inset-0 z-50 flex">
    <div className="absolute inset-0 bg-black/50" onClick={() => setOpen(false)} />
    <aside
      className="relative w-72 bg-card border-r border-border flex flex-col"
      onTouchMove={(e) => e.stopPropagation()}
    >
      <button
        onClick={() => setOpen(false)}
        className="absolute top-4 right-4 p-1 rounded hover:bg-accent"
      >
        <X className="w-5 h-5" />
      </button>
      <SidebarContent />
    </aside>
  </div>
)}

</>
)
}

```



```

'use client'
import { useState, useEffect } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import { createClient } from '@lib/supabase/client'
import { createOdsTask } from '@app/actions/ods'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import { Textarea } from '@components/ui/textarea'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import { Badge } from '@components/ui/badge'
import { Plus, Trash2, User, Users, Check } from 'lucide-react'
import { formatInTimeZone } from 'date-fns-tz'
import { it } from 'date-fns/locale'
import { DEPARTMENTS, ODS_TYPE_LABELS, ODS_DAYS_IT } from '@types'
import { LoadingDots } from '@components/shared/LoadingDots'
import type { OdsTask, OdsCompletion, OdsTaskType, Department } from '@types'

const TZ = 'Europe/Rome'

type StaffMember = { id: string; full_name: string; department: string | null }
type RestaurantItem = { id: string; name: string }
type CompletionWithProfile = OdsCompletion & {
  profile?: { id: string; full_name: string } | null
}

interface Props {
  initialTasks: OdsTask[]
  completions: CompletionWithProfile[]
  staff: StaffMember[]
  restaurants: RestaurantItem[]
  currentUserId: string
  currentUserRole: string
  currentDepartment: string | null
  currentRestaurantId: string | null
  currentIsDirettore?: boolean
}

const TYPE_BADGE_CLASS: Record<OdsTaskType, string> = {
  quotidiana: 'bg-blue-50 text-blue-700 border-blue-200 dark:bg-blue-950/30 dark:text-blue-400 dark:border-blue-800',
  settimanale: 'bg-violet-50 text-violet-700 border-violet-200 dark:bg-violet-950/30 dark:text-violet-400 dark:border-violet-800',
  bisettimanale: 'bg-cyan-50 text-cyan-700 border-cyan-200 dark:bg-cyan-950/30 dark:text-cyan-400 dark:border-cyan-800',
  straordinaria: 'bg-amber-50 text-amber-700 border-amber-200 dark:bg-amber-950/30 dark:text-amber-400 dark:border-amber-800',
}

export function OdsManagerClient({
  initialTasks, completions, staff, restaurants,
  currentUserId, currentUserRole, currentDepartment, currentRestaurantId,
  currentIsDirettore = false,
}: Props) {
  const isManager = currentUserRole === 'manager'
  // Direttore (capo_servizio): operates across all departments of its restaurant.
  const isDirettore = currentUserRole === 'capo_servizio' && currentIsDirettore
  // Whether the department field is locked to the caller's own department.
  const deptLocked = !!currentDepartment && !isDirettore

  const [tasks, setTasks] = useState<OdsTask[]>(initialTasks)
  const [deptFilter, setDeptFilter] = useState<string>('tutti')
  const [restFilter, setRestFilter] = useState<string>('tutti')

  // Mark ODS notifications as read when this page is mounted
  useEffect(() => {
    const supabase = createClient()
    supabase
      .from('notifications')
      .update({ read_at: new Date().toISOString() })
      .is('read_at', null)
      .then(() => {})
  })

```

```

}, [])
const [showForm, setShowForm] = useState(false)
const [saving, setSaving] = useState(false)

// Read receipt dialog
const [viewingTask, setViewingTask] = useState<OdsTask | null>(null)

// Form state
const [fTitle, setFTitle] = useState('')
const [fDept, setFDept] = useState<string>(currentDepartment ?? '')
const [fType, setFType] = useState<OdsTaskType>('quotidiana')
const [fDays, setFDays] = useState<string[]>([])
const [fAssignedTo, setFAssignedTo] = useState<string>('__none__')
const [fRestaurantId, setFRestaurantId] = useState<string>(currentRestaurantId ?? '')

// Build completion map: taskId → list of completors
const completionMap: Record<string, CompletionWithProfile[]> = {}
completions.forEach(c => {
  if (!completionMap[c.task_id]) completionMap[c.task_id] = []
  completionMap[c.task_id].push(c)
})

// Manager restaurant filter – applied before department filter
const tasksByRestaurant = (isManager && restFilter !== 'tutti')
  ? tasks.filter(t => t.restaurant_id === restFilter)
  : tasks

// Visible departments derive from the already-restaurant-filtered list
const allDepts = Array.from(new Set(tasksByRestaurant.map(t => t.department))).sort()

const filteredTasks = deptFilter === 'tutti'
  ? tasksByRestaurant
  : tasksByRestaurant.filter(t => t.department === deptFilter)

// Staff scoped to selected restaurant (for the create form).
// A direttore can assign to anyone in the currently-selected department.
const scopedStaff = isDirettore
  ? (fDept ? staff.filter(s => s.department === fDept) : staff)
  : fRestaurantId
  ? staff.filter(s => !currentDepartment || s.department === fDept || s.department === currentDepartment)
  : staff

function resetForm() {
  setFTitle('')
  setFDept(currentDepartment ?? '')
  setFType('quotidiana')
  setFDays([])
  setFAssignedTo('__none__')
  setFRestaurantId(currentRestaurantId ?? '')
}

function toggleDay(day: string) {
  setFDays(prev => prev.includes(day) ? prev.filter(d => d !== day) : [...prev, day])
}

async function handleCreate() {
  if (!fTitle.trim() || !fDept) return
  setSaving(true)
  try {
    const task = await createOdsTask({
      title: fTitle.trim(),
      department: fDept,
      restaurant_id: (fRestaurantId || currentRestaurantId) ?? '',
      type: fType,
      recurrence_days: (fType === 'settimanale' || fType === 'bisettimanale') ? fDays : [],
      assigned_to: fAssignedTo === '__none__' ? null : fAssignedTo,
    })
    setTasks(prev => [task, ...prev])
    resetForm()
    setShowForm(false)
  } catch (err) {
    console.error('Errore creazione ODS:', err)
  } finally {
    setSaving(false)
  }
}

```

```

    }
  }

  async function handleDelete(id: string) {
    if (!confirm('Eliminare questo ordine di servizio?')) return
    const supabase = createClient()
    await supabase.from('ods_tasks').delete().eq('id', id)
    setTasks(prev => prev.filter(t => t.id !== id))
  }

  const canDelete = (task: OdsTask) =>
    isManager ||
    (currentUserRole === 'capo_servizio' &&
      (isDirettore || !currentDepartment || task.department === currentDepartment))

  const todayName = formatInTimeZone(new Date(), TZ, 'EEEE', { locale: it }).toLowerCase()

  return (
    <div>
      { /* Header */ }
      <div className="flex items-center justify-between mb-5">
        <h1 className="text-xl font-semibold tracking-tight">Ordini di Servizio</h1>
        <Button size="sm" onClick={() => { resetForm(); setShowForm(true) }}>
          <Plus className="w-4 h-4" /> Nuovo Ordine
        </Button>
      </div>

      { /* Restaurant filter - manager only */ }
      {isManager && restaurants.length > 0 && (
        <div className="mb-4">
          <Select
            value={restFilter}
            onChange={v => { setRestFilter(v); setDeptFilter('tutti') }}
          >
            <SelectTrigger className="h-8 w-56 text-xs rounded-sm">
              <SelectValue placeholder="Tutti i ristoranti" />
            </SelectTrigger>
            <SelectContent>
              <SelectItem value="tutti">Tutti i ristoranti</SelectItem>
              {restaurants.map(r => (
                <SelectItem key={r.id} value={r.id}>{r.name}</SelectItem>
              ))}
            </SelectContent>
          </Select>
        </div>
      )}

      { /* Department filter pills */ }
      <div className="flex gap-1.5 flex-wrap mb-5">
        {[ 'tutti', ...allDepts ].map(dept => (
          <button
            key={dept}
            onClick={() => setDeptFilter(dept)}
            className={`text-xs px-2.5 py-1 rounded-sm border transition-colors capitalize ${
              deptFilter === dept
                ? 'bg-primary text-primary-foreground border-primary'
                : 'bg-card text-muted-foreground border-border hover:bg-accent hover:text-foreground'
            }`}
          >
            {dept === 'tutti' ? 'Tutti i reparti' : dept}
          </button>
        ))}
      </div>

      { /* Task list */ }
      {filteredTasks.length === 0 ? (
        <motion.div
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
          className="py-14 text-center text-muted-foreground text-sm border border-border rounded-md"
        >
          Nessun ordine di servizio
        </motion.div>
      ) : (

```

```

<div className="space-y-2">
  <AnimatePresence initial={false}>
    {filteredTasks.map((task, i) => {
      const taskCompletions = completionMap[task.id] ?? []
      const isVisibleToday =
        task.type === 'quotidiana' || task.type === 'straordinaria' ||
        task.recurrence_days.some(d => d.toLowerCase() === todayName)

      return (
        <motion.div
          key={task.id}
          initial={{ opacity: 0, y: 6 }}
          animate={{ opacity: 1, y: 0 }}
          exit={{ opacity: 0, y: -6, transition: { duration: 0.15 } }}
          transition={{ duration: 0.2, delay: i * 0.03 }}
          className={`bg-card border border-border rounded-sm px-4 py-3 flex items-start gap-3 ${
            isVisibleToday ? 'opacity-50' : ''
          }`}
        >
          {/* Completion indicator */}
          <div className={`mt-0.5 w-4 h-4 rounded-sm border shrink-0 flex items-center justify-center ${
            taskCompletions.length > 0
              ? 'bg-emerald-500 border-emerald-500'
              : 'border-input bg-background'
            }`}
            >
            {taskCompletions.length > 0 && <Check className="w-2.5 h-2.5 text-white" />}
          </div>

          {/* Content */}
          <div className="flex-1 min-w-0">
            <div className="flex items-center gap-2 flex-wrap mb-0.5">
              <span className="text-sm font-medium text-foreground leading-snug">
                {task.title}
              </span>
              <span className={`text-[10px] px-1.5 py-0.5 rounded-sm border font-medium whitespace-nowrap $
{TYPE_BADGE_CLASS[task.type]}`}
                >
                {ODS_TYPE_LABELS[task.type]}
              </span>
              <Badge variant="outline" className="text-[10px] px-1.5 py-0.5 h-auto rounded-sm capitalize">
                {task.department}
              </Badge>
            </div>

            {/* Assignee or days */}
            {task.assignee ? (
              <p className="text-xs text-muted-foreground flex items-center gap-1 mt-0.5">
                <User className="w-3 h-3" /> {task.assignee.full_name}
              </p>
            ) : (task.type === 'settimanale' || task.type === 'bisettimanale') &&
            task.recurrence_days.length > 0 ? (
              <p className="text-xs text-muted-foreground mt-0.5 capitalize">
                {task.recurrence_days.join(', ')}
              </p>
            ) : null}

            {/* Completions count - clickable */}
            {taskCompletions.length > 0 && (
              <button
                onClick={() => setViewingTask(task)}
                className="text-xs text-muted-foreground/70 hover:text-muted-foreground mt-0.5 flex items-
center gap-1"
              >
                <Users className="w-3 h-3" />
                Completato da {taskCompletions.length}{' '}
                {taskCompletions.length === 1 ? 'persona' : 'persone'}
              </button>
            )}
          </div>

          {/* Delete */}
          {canDelete(task) && (
            <Button
              variant="ghost" size="icon"
              onClick={() => handleDelete(task.id)}

```



```

    {(fType === 'settimanale' || fType === 'bisettimanale') && (
      <div className="space-y-2">
        <Label>
          Giorni
          {fType === 'settimanale' && ' (1 giorno)'}
          {fType === 'bisettimanale' && ' (2 giorni)'}
        </Label>
        <div className="flex flex-wrap gap-1.5">
          {ODS_DAYS_IT.map(day => (
            <button
              key={day}
              type="button"
              onClick={() => {
                if (fType === 'settimanale') {
                  setFDays(fDays[0] === day ? [] : [day])
                } else {
                  toggleDay(day)
                }
              }}
              className={`text-xs px-2.5 py-1 rounded-sm border capitalize transition-colors ${
                fDays.includes(day)
                  ? 'bg-primary text-primary-foreground border-primary'
                  : 'bg-card text-muted-foreground border-border hover:bg-accent'
              }`}
            >
              {day}
            </button>
          ))}
        </div>
      </div>
    )}

    {/ * Assigned to */}
    <div className="space-y-2">
      <Label>Assegna a (opzionale)</Label>
      <Select value={fAssignedTo} onChange={setFAssignedTo}>
        <SelectTrigger><SelectValue placeholder="Nessuna assegnazione specifica" /></SelectTrigger>
        <SelectContent>
          <SelectItem value="__none__">Tutto il reparto</SelectItem>
          {scopedStaff.map(s => (
            <SelectItem key={s.id} value={s.id}>{s.full_name}</SelectItem>
          ))}
        </SelectContent>
      </Select>
    </div>
  </div>

  <DialogFooter>
    <Button variant="outline" onClick={() => setShowForm(false)}>Annulla</Button>
    <Button
      onClick={handleCreate}
      disabled={saving || !fTitle.trim() || !fDept || (isManager && !fRestaurantId)}
    >
      {saving ? <Salvataggio><LoadingDots /></> : 'Crea Ordine'}
    </Button>
  </DialogFooter>
</DialogContent>
</Dialog>

{/ * — Completions dialog ————— */}
<Dialog open={!viewingTask} onOpenChange={() => setViewingTask(null)}>
  <DialogContent className="max-h-[80vh] overflow-y-auto">
    <DialogHeader>
      <DialogTitle>Chi ha completato</DialogTitle>
      {viewingTask && (
        <p className="text-sm text-muted-foreground mt-0.5 truncate">{viewingTask.title}</p>
      )}
    </DialogHeader>
    {viewingTask && (
      <div className="divide-y divide-border">
        {(completionMap[viewingTask.id] ?? []).map(c => (
          <div key={c.id} className="flex items-center justify-between py-2.5 gap-4">
            <span className="text-sm font-medium">{c.profile?.full_name ?? '-'}</span>
            <span className="text-xs text-muted-foreground shrink-0 tabular-nums">

```

```

        {formatInTimeZone(new Date(c.completed_at), TZ, 'dd-MM-yyyy HH:mm')}
      </span>
    </div>
  )}
</div>
})
<DialogFooter>
  <Button variant="outline" onClick={() => setViewingTask(null)}>Chiudi</Button>
</DialogFooter>
</DialogContent>
</Dialog>
</div>
)
}

```

[src/components/manager/PageSkeleton.tsx](#)

```

export function PageSkeleton({ rows = 6 }: { rows?: number }) {
  return (
    <div className="p-6 space-y-5 animate-pulse">
      <div className="flex items-center justify-between">
        <div className="space-y-2">
          <div className="h-6 w-36 bg-muted rounded" />
          <div className="h-3.5 w-24 bg-muted rounded" />
        </div>
        <div className="h-9 w-24 bg-muted rounded" />
      </div>
      <div className="h-9 w-56 bg-muted rounded" />
      <div className="border rounded-md overflow-hidden">
        <div className="h-10 bg-muted/60 border-b" />
        {Array.from({ length: rows }).map((_, i) => (
          <div key={i} className="h-12 border-b last:border-0 bg-background flex items-center px-4 gap-4">
            <div className="h-3.5 w-1/4 bg-muted rounded" />
            <div className="h-3.5 w-1/5 bg-muted rounded" />
            <div className="h-3.5 w-1/6 bg-muted rounded" />
          </div>
        ))}
      </div>
    </div>
  )
}

```



```

'use client'
import { useState, useCallback, useEffect, useMemo, useRef } from 'react'
import { createClient } from '@lib/supabase/client'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { TimeInput } from '@components/ui/time-input'
import { Label } from '@components/ui/label'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select'
import { Card, CardContent } from '@components/ui/card'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import { Badge } from '@components/ui/badge'
import { Clock, Trash2, AlertTriangle, Plus } from 'lucide-react'
import { LoadingDots } from '@components/shared/LoadingDots'
import { formatInTimeZone } from 'date-fns-tz'
import { differenceInMinutes } from 'date-fns'
import { it } from 'date-fns/locale'
import { LiveShiftCounter } from './LiveShiftCounter'
import type { Attendance, Restaurant, AbsenceType } from '@types'
import { ABSENCE_LABELS } from '@types'

const TZ = 'Europe/Rome'

// Absence badge colors matching AssenzeClient
const ABSENCE_BADGE: Record<string, string> = {
  ferie: 'bg-violet-100 text-violet-800 dark:bg-violet-900/30 dark:text-violet-300',
  malattia: 'bg-blue-100 text-blue-800 dark:bg-blue-900/30 dark:text-blue-300',
  riposo: 'bg-red-100 text-red-800 dark:bg-red-900/30 dark:text-red-300',
  assenza_ingiustificata: 'bg-purple-100 text-purple-800 dark:bg-purple-900/30 dark:text-purple-300',
}

type AttendanceRow = Attendance & {
  profile?: { id: string; full_name: string; role: string } | null
  restaurant?: { id: string; name: string } | null
}

export type AbsenceItem = {
  id: string
  user_id: string
  restaurant_id: string | null
  type: string
  start_date: string
  end_date: string
  profile?: { id: string; full_name: string } | null
}

interface Props {
  initialPresenze: AttendanceRow[]
  initialAbsences: AbsenceItem[]
  restaurants: Pick<Restaurant, 'id' | 'name'>[]
  dipendenti: { id: string; full_name: string; role: string }[]
  currentUserRole: string
  currentRestaurantId: string | null
  isDirettore: boolean
}

function formatDuration(minutes: number): string {
  const h = Math.floor(minutes / 60)
  const m = minutes % 60
  return `${h}h ${String(m).padStart(2, '0')}m`
}

export function PresenzeClient({
  initialPresenze, initialAbsences,
  restaurants, dipendenti, currentUserRole, currentRestaurantId, isDirettore,
}: Props) {
  const [presenze, setPresenze] = useState(initialPresenze)
  const [absences, setAbsences] = useState(initialAbsences)
  const [selectedMonth, setSelectedMonth] = useState(() => formatInTimeZone(new Date(), TZ, 'yyyy-MM'))
  const [selectedRestaurant, setSelectedRestaurant] = useState(currentRestaurantId ?? 'all')
  const [loading, setLoading] = useState(false)
  const [isLive, setIsLive] = useState(false)

  // Form unico Aggiungi/Modifica - stessi campi in entrambi i casi; in

```

```

// modifica il dipendente arriva preselezionato (e bloccato), in
// aggiunta va scelto. Stesso pattern di PresenzePreviewClient.
const [showForm, setShowForm] = useState(false)
const [editing, setEditing] = useState<AttendanceRow | null>(null)
const [confirmingDelete, setConfirmingDelete] = useState(false)
const [deleting, setDeleting] = useState(false)
const [fUserId, setFUserId] = useState('')
const [fDate, setFDate] = useState(() => formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd'))
const [fCheckIn, setFCheckIn] = useState('')
const [fCheckOut, setFCheckOut] = useState('')
const [formError, setFormError] = useState<string | null>(null)
const [formSaving, setFormSaving] = useState(false)

// Refs so the realtime handler always reads current filter values without re-subscribing
const monthRef = useRef(selectedMonth)
const restaurantRef = useRef(selectedRestaurant)
useEffect(() => { monthRef.current = selectedMonth }, [selectedMonth])
useEffect(() => { restaurantRef.current = selectedRestaurant }, [selectedRestaurant])

// Supabase Realtime subscription – wired once on mount
useEffect(() => {
  const supabase = createClient()

  const channel = supabase
    .channel('rt-presenze')
    .on(
      'postgres_changes',
      { event: '*', schema: 'public', table: 'attendances' },
      async (payload) => {
        if (payload.eventType === 'DELETE') {
          const deletedId = (payload.old as { id: string }).id
          setPresenze(ps => ps.filter(p => p.id !== deletedId))
          return
        }

        const rec = payload.new as { id: string; check_in: string; restaurant_id: string | null }

        const [y, m] = monthRef.current.split('-').map(Number)
        const monthStart = new Date(Date.UTC(y, m - 1, 1))
        const monthEnd = new Date(Date.UTC(y, m, 0, 23, 59, 59))
        const checkIn = new Date(rec.check_in)
        if (checkIn < monthStart || checkIn > monthEnd) return

        if (restaurantRef.current !== 'all' && rec.restaurant_id !== restaurantRef.current) return

        const { data } = await supabase
          .from('attendances')
          .select('*', profile:profiles(id, full_name, role), restaurant:restaurants(id, name))
          .eq('id', rec.id)
          .single()

        if (!data) return

        if (payload.eventType === 'INSERT') {
          setPresenze(ps =>
            ps.some(p => p.id === data.id)
              ? ps.map(p => p.id === data.id ? data : p)
              : [data, ...ps]
          )
        } else {
          setPresenze(ps => ps.map(p => p.id === data.id ? data : p))
        }
      }
    )
    .subscribe(status => { setIsLive(status === 'SUBSCRIBED') })

  return () => { supabase.removeChannel(channel) }
}, [])

// Load presenze + absences for the given month / restaurant filter
const loadData = useCallback(async (month: string, restaurantId: string) => {
  setLoading(true)
  const [year, m] = month.split('-').map(Number)
  const startIso = new Date(Date.UTC(year, m - 1, 1)).toISOString()

```

```

const endIso = new Date(Date.UTC(year, m, 0, 23, 59, 59)).toISOString()
const monthStart = `${month}-01`
const monthEnd = `${month}-${String(new Date(Date.UTC(year, m, 0)).getDate()).padStart(2, '0')}`

const supabase = createClient()
let presQ = supabase
  .from('attendances')
  .select('*', profile:profiles(id, full_name, role), restaurant:restaurants(id, name))
  .gte('check_in', startIso)
  .lte('check_in', endIso)
  .order('check_in', { ascending: false })

let absQ = supabase
  .from('absences')
  .select('id, user_id, restaurant_id, type, start_date, end_date, profile:profiles!user_id(id, full_name)')
  .eq('status', 'approved')
  .lte('start_date', monthEnd)
  .gte('end_date', monthStart)

if (restaurantId !== 'all') {
  presQ = presQ.eq('restaurant_id', restaurantId)
  absQ = absQ.eq('restaurant_id', restaurantId)
}

const [{ data: p }, { data: a }] = await Promise.all([presQ, absQ])
setPresenze(p ?? [])
setAbsences((a ?? []) as unknown as AbsenceItem[])
setLoading(false)
}, [])

function handleMonthChange(month: string) {
  setSelectedMonth(month)
  loadData(month, selectedRestaurant)
}

function handleRestaurantChange(restaurantId: string) {
  setSelectedRestaurant(restaurantId)
  loadData(selectedMonth, restaurantId)
}

function resetForm() {
  setEditing(null)
  setConfirmingDelete(false)
  setFUserId('')
  setFDate(formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd'))
  setFCheckIn('')
  setFCheckOut('')
  setFormError(null)
}

function openCreate() {
  resetForm()
  setShowForm(true)
}

function openEdit(p: AttendanceRow) {
  setEditing(p)
  setConfirmingDelete(false)
  setFUserId(p.user_id)
  setFDate(formatInTimeZone(new Date(p.check_in), TZ, 'yyyy-MM-dd'))
  setFCheckIn(formatInTimeZone(new Date(p.check_in), TZ, 'HH:mm'))
  setFCheckOut(p.check_out ? formatInTimeZone(new Date(p.check_out), TZ, 'HH:mm') : '')
  setFormError(null)
  setShowForm(true)
}

function closeForm() {
  setShowForm(false)
  resetForm()
}

// Un solo salvataggio per aggiunta e modifica: POST se sto creando,
// PATCH se sto modificando. L'API gestisce da sola il turno notturno
// (uscita il giorno dopo se l'orario è numericamente prima dell'ingresso).

```

```

async function handleSave() {
  if (!canEdit) { showUnauthorized(); return }
  if (!fUserId || !fDate || !fCheckIn) return
  setFormSaving(true)
  setFormError(null)

  try {
    const res = await fetch('/api/presenze', {
      method: editing ? 'PATCH' : 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(
        editing
          ? { id: editing.id, date: fDate, checkIn: fCheckIn, checkOut: fCheckOut || null }
          : { userId: fUserId, date: fDate, checkIn: fCheckIn, checkOut: fCheckOut || undefined }
      ),
    })
    const data = await res.json()

    if (!res.ok) {
      setFormError(data.error || 'Errore durante il salvataggio')
      setFormSaving(false)
      return
    }

    const saved = data.attendance as AttendanceRow
    const [y, m] = selectedMonth.split('-').map(Number)
    const monthStart = new Date(Date.UTC(y, m - 1, 1))
    const monthEnd = new Date(Date.UTC(y, m, 0, 23, 59, 59))
    const checkInDate = new Date(saved.check_in)
    const matchesMonth = checkInDate >= monthStart && checkInDate <= monthEnd
    const matchesRest = selectedRestaurant === 'all' || saved.restaurant_id === selectedRestaurant

    if (editing) {
      // Se la modifica sposta la presenza fuori dal mese/ristorante
      // filtrato, la rimuoviamo dalla lista invece di mostrarla stonata.
      setPresenze(ps => matchesMonth && matchesRest
        ? ps.map(p => p.id === saved.id ? saved : p)
        : ps.filter(p => p.id !== saved.id))
    } else if (matchesMonth && matchesRest) {
      setPresenze(ps => [saved, ...ps])
    }

    closeForm()
  } catch {
    setFormError('Errore di rete, riprova')
  } finally {
    setFormSaving(false)
  }
}

async function handleDelete() {
  if (!canEdit) { showUnauthorized(); return }
  if (!editing) return
  setDeleting(true)
  const supabase = createClient()
  const { error } = await supabase.from('attendances').delete().eq('id', editing.id)
  if (!error) {
    setPresenze(ps => ps.filter(p => p.id !== editing.id))
    closeForm()
  }
  setDeleting(false)
}

// — Grouped presenze + absent employees —————
// Depends on presenze, absences, and selectedMonth (for "show today" logic).
const groupedPresenze = useMemo(() => {
  type PresentRow = {
    userId: string
    userName: string
    restaurantName: string | null
    blocks: AttendanceRow[]
    totalMinutes: number
    hasOpen: boolean
    hasSplit: boolean
  }

```

```

    isAbsent: false
  }
  type AbsentRow = {
    userId: string
    userName: string
    isAbsent: true
    absenceType: string
  }
  type Row = PresentRow | AbsentRow

  // Build present rows (existing logic)
  const byDay = new Map<string, Map<string, PresentRow>>()

  for (const p of presence) {
    const dayKey = formatInTimeZone(new Date(p.check_in), TZ, 'yyyy-MM-dd')
    let dayMap = byDay.get(dayKey)
    if (!dayMap) { dayMap = new Map(); byDay.set(dayKey, dayMap) }

    let row = dayMap.get(p.user_id)
    if (!row) {
      row = {
        userId: p.user_id,
        userName: p.profile?.full_name ?? '-',
        restaurantName: p.restaurant?.name ?? null,
        blocks: [],
        totalMinutes: 0,
        hasOpen: false,
        hasSplit: false,
        isAbsent: false,
      }
      dayMap.set(p.user_id, row)
    }

    row.blocks.push(p)
    if (!p.check_out) row.hasOpen = true
    else row.totalMinutes += differenceInMinutes(new Date(p.check_out), new Date(p.check_in))
  }

  // Sort blocks and derive hasSplit dynamically from actual block count.
  // Reading is_split_shift from the DB is unreliable after a block is deleted
  // (the flag on the surviving block stays true). Counting real blocks is authoritative.
  for (const dayMap of byDay.values()) {
    for (const row of dayMap.values()) {
      row.blocks.sort((a, b) => new Date(a.check_in).getTime() - new Date(b.check_in).getTime())
      row.hasSplit = row.blocks.length > 1
    }
  }

  // Ensure today appears if the selected month is the current month and there
  // are approved absences for today – even if nobody has clocked in yet.
  const todayKey = formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd')
  const todayMonth = todayKey.substring(0, 7)
  if (todayMonth === selectedMonth) {
    const hasAbsencesToday = absences.some(a => a.start_date <= todayKey && a.end_date >= todayKey)
    if (hasAbsencesToday && !byDay.has(todayKey)) {
      byDay.set(todayKey, new Map())
    }
  }

  // Build absent rows: approved absence covers the day AND employee has no presence that day
  const byDayAbsent = new Map<string, Map<string, AbsentRow>>()
  for (const [dayKey, presentMap] of byDay.entries()) {
    const absentMap = new Map<string, AbsentRow>()
    for (const absence of absences) {
      if (absence.start_date > dayKey || absence.end_date < dayKey) continue
      const uid = absence.user_id
      if (presentMap.has(uid) || absentMap.has(uid)) continue
      absentMap.set(uid, {
        userId: uid,
        userName: absence.profile?.full_name ?? '-',
        isAbsent: true,
        absenceType: absence.type,
      })
    }
  }
}

```

```

    byDayAbsent.set(dayKey, absentMap)
  }

  // Sort and merge present + absent per day
  return Array.from(byDay.entries())
    .sort((a, b) => b[0].localeCompare(a[0]))
    .map(([day, dayMap]) => {
      const presentRows = Array.from(dayMap.values())
        .sort((a, b) => a.userName.localeCompare(b.userName, 'it'))
      const absentRows = Array.from((byDayAbsent.get(day) ?? new Map()).values())
        .sort((a, b) => a.userName.localeCompare(b.userName, 'it'))
      return [day, [...presentRows, ...absentRows] as Row[]] as const
    })
  }, [presenze, absences, selectedMonth])

function formatDayHeader(dayKey: string): string {
  const [y, m, d] = dayKey.split('-').map(Number)
  const date = new Date(y, m - 1, d, 12, 0, 0)
  const label = formatInTimeZone(date, TZ, 'EEEE d MMMM yyyy', { locale: it })
  return label.charAt(0).toUpperCase() + label.slice(1)
}

const isManager = currentUserRole === 'manager'
const canEdit = isManager || (currentUserRole === 'capo_servizio' && isDirettore)

const [unauthorizedMsg, setUnauthorizedMsg] = useState(false)
function showUnauthorized() {
  setUnauthorizedMsg(true)
  setTimeout(() => setUnauthorizedMsg(false), 3000)
}

return (
  <div>
    <div className="flex items-center justify-between mb-6">
      <div>
        <h1 className="text-xl font-semibold tracking-tight">Presenze</h1>
        <p className="text-muted-foreground text-sm mt-0.5">{presenze.length} timbrature</p>
      </div>
      <div className="flex items-center gap-2">
        {canEdit && (
          <Button onClick={openCreate} size="sm" className="h-8 rounded-sm px-3 text-xs gap-1.5">
            <Plus className="w-3.5 h-3.5" />
            Aggiungi Presenza
          </Button>
        )}
        <div className={`flex items-center gap-1.5 text-xs font-medium px-2.5 py-1 rounded-md border ${
          isLive
            ? 'text-emerald-600 bg-emerald-50 border-emerald-200 dark:text-emerald-400 dark:bg-emerald-950/30
dark:border-emerald-800'
            : 'text-muted-foreground bg-muted border-border'
        }`}>
          <span className={`w-1.5 h-1.5 rounded-full ${isLive ? 'bg-emerald-500 animate-pulse' : 'bg-muted-foreground'}`}> />
          {isLive ? 'Live' : 'Connessione...'}
        </div>
      </div>
    </div>

    {/* Filtri */}
    <div className="flex flex-wrap gap-3 mb-6">
      <Input
        type="month"
        value={selectedMonth}
        onChange={e => handleMonthChange(e.target.value)}
        className="w-auto"
      />
      {isManager && (
        <Select value={selectedRestaurant} onChange={handleRestaurantChange}>
          <SelectTrigger className="w-48">
            <SelectValue placeholder="Tutti i ristoranti" />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="all">Tutti i ristoranti</SelectItem>
            {restaurants.map(r => (

```

```

        <SelectItem key={r.id} value={r.id}>{r.name}</SelectItem>
      )))
    </SelectContent>
  </Select>
)
</div>

{/* Lista */}
{loading ? (
  <div className="space-y-2">
    {[...Array(5)].map((_, i) => (
      <div key={i} className="h-16 bg-muted animate-pulse rounded-md" />
    ))}
  </div>
) : groupedPresenze.length === 0 ? (
  <Card>
    <CardContent className="py-12 text-center text-muted-foreground">
      Nessuna presenza nel periodo selezionato
    </CardContent>
  </Card>
) : (
  <div className="space-y-6">
    {groupedPresenze.map(([dayKey, rows]) => (
      <section key={dayKey}>
        <div className="flex items-baseline justify-between border-b border-border pb-1.5 mb-2">
          <h2 className="text-xs font-semibold uppercase tracking-wide text-zinc-500 dark:text-zinc-400">
            {formatDayHeader(dayKey)}
          </h2>
          <span className="text-xs text-muted-foreground">
            {rows.length} {rows.length === 1 ? 'dipendente' : 'dipendenti'}
          </span>
        </div>

        <div className="space-y-2">
          {rows.map(row => {
            // — Absent employee row —————
            if (row.isAbsent) {
              return (
                <Card>
                  <CardContent className="p-4">
                    <div className="flex items-center gap-4">
                      <div className="flex-1 min-w-0 flex items-center gap-2 flex-wrap">
                        <span className="font-medium text-sm text-muted-foreground">
                          {row.userName}
                        </span>
                        <span className={`text-xs px-2 py-0.5 rounded-full font-medium $
{ABSENCE_BADGE[row.absenceType]} ?? ''`}>
                          {ABSENCE_LABELS[row.absenceType as AbsenceType] ?? row.absenceType}
                        </span>
                      </div>
                      <span className="shrink-0 text-xs text-muted-foreground">Assente</span>
                    </div>
                  </CardContent>
                </Card>
              )
            }

            // — Present employee row (existing logic) —————
            const openBlock = row.hasOpen ? row.blocks.find(b => !b.check_out) : undefined
            return (
              <Card key={` ${dayKey}-${row.userId}`} className="rounded-sm">
                <CardContent className="p-4">
                  <div className="flex items-center gap-4">
                    <div className="flex-1 min-w-0">
                      <div className="flex items-center gap-2 flex-wrap mb-2">
                        <span className="font-medium text-sm">{row.userName}</span>
                        {row.hasOpen && (
                          <Badge className="bg-emerald-100 text-emerald-700 dark:bg-emerald-900/30
dark:text-emerald-400 text-xs">
                            In corso
                          </Badge>

```



```

    ))
    {row.hasSplit && (
      <Badge className="bg-orange-100 text-orange-700 dark:bg-orange-900/30 dark:text-
orange-400 text-xs">
        Spezzato
      </Badge>
    )}
    {row.restaurantName && (
      <span className="text-xs text-muted-foreground">{row.restaurantName}</span>
    )}
  </div>

  <div className="flex items-center gap-1.5 flex-wrap">
    {row.blocks.map(b => {
      const isOpen = !b.check_out
      const checkIn = formatInTimeZone(new Date(b.check_in), TZ, 'HH:mm')
      const checkOut = b.check_out
      ? formatInTimeZone(new Date(b.check_out), TZ, 'HH:mm')
      : '...'
      return canEdit ? (
        <button
          key={b.id}
          onClick={() => openEdit(b)}
          title="Modifica turno"
          className={`h-10 px-3 inline-flex items-center justify-center text-xs font-
medium tabular-nums rounded-sm border cursor-pointer transition-colors ${
            isOpen
              ? 'bg-emerald-50 border-emerald-200 text-emerald-700 hover:bg-emerald-100
dark:bg-emerald-950/30 dark:border-emerald-900 dark:text-emerald-400'
              : 'bg-zinc-100 border-border text-foreground hover:bg-zinc-200 dark:bg-
zinc-900 dark:hover:bg-zinc-800'
            }`}
        >
          {checkIn} → {checkOut}
        </button>
      ) : (
        <span
          key={b.id}
          className={`h-10 px-3 inline-flex items-center justify-center text-xs font-
medium tabular-nums rounded-sm border ${
            isOpen
              ? 'bg-emerald-50 border-emerald-200 text-emerald-700 dark:bg-
emerald-950/30 dark:border-emerald-900 dark:text-emerald-400'
              : 'bg-zinc-100 border-border text-foreground dark:bg-zinc-900'
            }`}
        >
          {checkIn} → {checkOut}
        </span>
      )
    )}}
  </div>
</div>

<div className="shrink-0 text-right">
  <div className="text-sm font-semibold tabular-nums flex items-center gap-1.5 justify-
end">
    <Clock className="w-3.5 h-3.5 text-muted-foreground" />
    {openBlock ? (
      <LiveShiftCounter
        checkInTime={openBlock.check_in}
        baseMinutes={row.totalMinutes}
      />
    ) : (
      formatDuration(row.totalMinutes)
    )}
  </div>
  <div className="text-[11px] text-muted-foreground mt-0.5">
    {row.blocks.length} {row.blocks.length === 1 ? 'turno' : 'turni'}
  </div>
</div>
</CardContent>
</Card>
)

```

```

    }}}
  </div>
</section>
)}}
</div>
))

{/* Dialog Aggiungi/Modifica – stesso form in entrambi i casi: in
   modifica il dipendente è preselezionato e bloccato, in aggiunta
   va scelto. */}
<Dialog open={showForm} onOpenChange={open => !open && closeForm()}>
  <DialogContent onInteractOutside={e => e.preventDefault()}>
    <DialogHeader>
      <DialogTitle>
        {confirmingDelete ? 'Elimina Timbratura' : editing ? 'Modifica Presenza' : 'Aggiungi Presenza'}
      </DialogTitle>
    </DialogHeader>

    {confirmingDelete ? (
      <div className="space-y-4">
        <div className="flex gap-3 items-start rounded-md border border-destructive/30 bg-destructive/10
px-4 py-3">
          <AlertTriangle className="w-5 h-5 text-destructive shrink-0 mt-0.5" />
          <div className="text-sm">
            <p className="font-medium text-foreground">Sei sicuro di voler eliminare questa timbratura?</p>
            <p className="text-muted-foreground mt-1">L'azione è irreversibile.</p>
          </div>
        </div>
        <div>
          {editing && (
            <div className="text-xs text-muted-foreground border-l-2 border-border pl-3">
              <div>{editing.profile?.full_name ?? '-'}</div>
              <div>
                {formatInTimeZone(new Date(editing.check_in), TZ, 'dd-MM-yyyy HH:mm')} → {editing.check_out ?
formatInTimeZone(new Date(editing.check_out), TZ, 'HH:mm') : '-'}
              </div>
            </div>
          )}
        </div>
      </div>
    ) : (
      <div className="space-y-4">
        <div className="space-y-2">
          <Label>Dipendente</Label>
          <Select value={fUserId} onValueChange={setFUserId} disabled={!editing}>
            <SelectTrigger className="h-10 rounded-sm">
              <SelectValue placeholder="Seleziona dipendente..." />
            </SelectTrigger>
            <SelectContent>
              {dipendenti.map(d => (
                <SelectItem key={d.id} value={d.id}>{d.full_name}</SelectItem>
              ))}
            </SelectContent>
          </Select>
        </div>
        <div className="space-y-2">
          <Label>Data</Label>
          <Input
            type="date"
            value={fDate}
            onChange={e => setFDate(e.target.value)}
            className="h-10 rounded-sm"
          />
        </div>
      </div>
    ) : (
      <div className="grid grid-cols-2 gap-3">

```

```

        <div className="space-y-2">
          <Label>Ora Ingresso</Label>
          <TimeInput
            value={fCheckIn}
            onChange={setFCheckIn}
            className="h-10 rounded-sm"
          />
        </div>
        <div className="space-y-2">
          <Label>Ora Uscita</Label>
          <TimeInput
            value={fCheckOut}
            onChange={setFCheckOut}
            className="h-10 rounded-sm"
          />
        </div>
      </div>
      <p className="text-xs text-muted-foreground -mt-2">
        Se l'uscita è prima dell'ingresso viene registrata il giorno successivo (turno
notturmo).
      </p>
      {formError && (
        <p className="text-sm text-destructive">{formError}</p>
      )}
    </div>
    <DialogFooter className={editing ? 'sm:justify-between gap-2' : undefined}>
      {editing && (
        <Button
          variant="destructive"
          onClick={() => setConfirmingDelete(true)}
          disabled={formSaving}
          className="h-10 rounded-sm"
        >
          <Trash2 className="w-4 h-4" />
          Elimina
        </Button>
      )}
      <div className="flex gap-2">
        <Button variant="outline" onClick={closeForm} disabled={formSaving} className="h-10 rounded-sm">
          Annulla
        </Button>
        <Button
          onClick={handleSave}
          disabled={formSaving || !fUserId || !fDate || !fCheckIn}
          className="h-10 rounded-sm"
        >
          {formSaving ? <Salvataggio<LoadingDots /></> : editing ? 'Salva' : 'Aggiungi'}
        </Button>
      </div>
    </DialogFooter>
  </>
  </DialogContent>
</Dialog>

  {/* Unauthorized toast */}
  {unauthorizedMsg && (
    <div className="fixed bottom-4 right-4 z-50 flex items-center gap-2 bg-destructive text-destructive-foreground px-4 py-3 rounded-md shadow-lg text-sm font-medium animate-in slide-in-from-bottom-2">
      Azione riservata alla direzione
    </div>
  )}
</div>
)
}

```

`src/components/manager/PresenzePreviewClient.tsx`

```

'use client'
import { useState } from 'react'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { TimeInput } from '@components/ui/time-input'
import { Label } from '@components/ui/label'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import { Badge } from '@components/ui/badge'
import { Plus, UserCheck } from 'lucide-react'
import { formatInTimeZone } from 'date-fns-tz'

const TZ = 'Europe/Rome'

export type PresenzaPreviewRow = {
  id: string
  user_id: string
  check_in: string
  check_out: string | null
  profile?: { id: string; full_name: string } | null
}

interface Props {
  initialRows: PresenzaPreviewRow[]
  dipendenti: { id: string; full_name: string }[]
  isDirettore: boolean
}

export function PresenzePreviewClient({ initialRows, dipendenti, isDirettore }: Props) {
  const [rows, setRows] = useState<PresenzaPreviewRow[]>(initialRows)
  const [showForm, setShowForm] = useState(false)
  const [editing, setEditing] = useState<PresenzaPreviewRow | null>(null)
  const [saving, setSaving] = useState(false)
  const [error, setError] = useState<string | null>(null)

  const [fUserId, setFUserId] = useState('')
  const [fDate, setFDate] = useState(formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd'))
  const [fCheckIn, setFCheckIn] = useState('')
  const [fCheckOut, setFCheckOut] = useState('')

  function resetForm() {
    setEditing(null)
    setFUserId('')
    setFDate(formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd'))
    setFCheckIn('')
    setFCheckOut('')
    setError(null)
  }

  function openCreate() {
    resetForm()
    setShowForm(true)
  }

  function openEdit(row: PresenzaPreviewRow) {
    if (!isDirettore) return
    setEditing(row)
    setFUserId(row.user_id)
    setFDate(formatInTimeZone(row.check_in, TZ, 'yyyy-MM-dd'))
    setFCheckIn(formatInTimeZone(row.check_in, TZ, 'HH:mm'))
    setFCheckOut(row.check_out ? formatInTimeZone(row.check_out, TZ, 'HH:mm') : '')
    setError(null)
    setShowForm(true)
  }

  async function handleSave() {
    if (!fUserId || !fDate || !fCheckIn) return
    setSaving(true)
    setError(null)
    try {
      const res = await fetch('/api/presenze', {
        method: editing ? 'PATCH' : 'POST',
        headers: { 'Content-Type': 'application/json' },

```

```

        body: JSON.stringify(
          editing
            ? { id: editing.id, date: fDate, checkIn: fCheckIn, checkOut: fCheckOut || null }
            : { userId: fUserId, date: fDate, checkIn: fCheckIn, checkOut: fCheckOut || null }
        ),
      })
    const json = await res.json()
    if (!res.ok) throw new Error(json.error ?? 'Errore sconosciuto')

    const saved = json.attendance as PresenzaPreviewRow
    setRows(prev => editing
      ? prev.map(r => r.id === saved.id ? saved : r)
      : [...prev, saved]
    )
    resetForm()
    setShowForm(false)
  } catch (err) {
    setError(err instanceof Error ? err.message : 'Errore sconosciuto')
  } finally {
    setSaving(false)
  }
}

return (
  <div className="mt-6 bg-card border border-border rounded-md p-4">
    <div className="flex items-center justify-between mb-3">
      <h2 className="text-sm font-semibold flex items-center gap-2">
        <UserCheck className="w-4 h-4 text-muted-foreground" />
        Presenze di Oggi
      </h2>
      {isDirettore && (
        <Button size="sm" variant="outline" onClick={openCreate}>
          <Plus className="w-4 h-4" /> Aggiungi
        </Button>
      )}
    </div>

    {rows.length === 0 ? (
      <p className="text-xs text-muted-foreground py-4 text-center">Nessuna presenza registrata oggi</p>
    ) : (
      <div className={`space-y-1.5 ${isDirettore ? '' : 'pointer-events-none select-none'}`}>
        {rows.map(row => (
          <div
            key={row.id}
            onClick={() => openEdit(row)}
            className={`flex items-center justify-between px-3 py-2 rounded-sm border border-border bg-background ${
              isDirettore ? 'cursor-pointer hover:bg-accent transition-colors' : ''
            }`}
          >
            <span className="text-sm font-medium text-foreground">{row.profile?.full_name ?? '-'}</span>
            <div className="flex items-center gap-2">
              <span className="text-xs text-muted-foreground">
                Entrata {formatInTimeZone(row.check_in, TZ, 'HH:mm')}
              </span>
              {row.check_out ? (
                <Badge variant="outline" className="text-[10px] px-1.5 py-0.5 h-auto rounded-sm">
                  Uscito
                </Badge>
              ) : (
                <Badge className="text-[10px] px-1.5 py-0.5 h-auto rounded-sm bg-emerald-50 text-emerald-700 border-emerald-200 dark:bg-emerald-950/30 dark:text-emerald-400 dark:border-emerald-800">
                  In corso
                </Badge>
              )}
            </div>
          </div>
        )}
      </div>
    )}

    {/* — Add / edit dialog — direttore only ————— */}
    {isDirettore && (
      <Dialog open={showForm} onOpenChange={setShowForm}>

```

```

<DialogContent>
  <DialogHeader>
    <DialogTitle>{editing ? 'Modifica Presenza' : 'Aggiungi Presenza'}</DialogTitle>
  </DialogHeader>
  <div className="space-y-4">
    <div className="space-y-2">
      <Label>Dipendente *</Label>
      <Select value={fUserId} onChange={setFUserId} disabled={!editing}>
        <SelectTrigger><SelectValue placeholder="Seleziona dipendente" /></SelectTrigger>
        <SelectContent>
          {dipendenti.map(d => <SelectItem key={d.id} value={d.id}>{d.full_name}</SelectItem>)}
        </SelectContent>
      </Select>
    </div>
    <div className="space-y-2">
      <Label>Data *</Label>
      <Input type="date" value={fDate} onChange={e => setFDate(e.target.value)} />
    </div>
    <div className="grid grid-cols-2 gap-3">
      <div className="space-y-2">
        <Label>Ora ingresso *</Label>
        <TimeInput value={fCheckIn} onChange={setFCheckIn} />
      </div>
      <div className="space-y-2">
        <Label>Ora uscita</Label>
        <TimeInput value={fCheckOut} onChange={setFCheckOut} />
      </div>
    </div>
    {error && <p className="text-xs text-destructive">{error}</p>}
  </div>
  <DialogFooter>
    <Button variant="outline" onClick={() => setShowForm(false)}>Annulla</Button>
    <Button onClick={handleSave} disabled={saving || !fUserId || !fDate || !fCheckIn}>
      {saving ? 'Salvataggio...' : editing ? 'Salva modifiche' : 'Aggiungi'}
    </Button>
  </DialogFooter>
</DialogContent>
</Dialog>
  </div>
)
}

```



```

'use client'
import { useState, useMemo, useEffect, useCallback, useRef } from 'react'
import { useRouter } from 'next/navigation'
import { createClient } from '@lib/supabase/client'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { TimeInput } from '@components/ui/time-input'
import { Label } from '@components/ui/label'
import { Card, CardContent } from '@components/ui/card'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import { FileSpreadsheet, Download, Plus, Trash2, AlertTriangle } from 'lucide-react'
import { LoadingDots } from '@components/shared/LoadingDots'
import { formatInTimeZone, fromZonedTime } from 'date-fns-tz'
import { getDaysInMonth, differenceInMinutes } from 'date-fns'
import { it } from 'date-fns/locale'
import type { Restaurant, AbsenceType } from '@types'
import { ABSENCE_CODES, ABSENCE_LABELS } from '@types'

const TZ = 'Europe/Rome'
const TARGET_HOURS_PER_DAY = 8.5

interface Props {
  restaurants: Pick<Restaurant, 'id' | 'name'>[]
  currentUserRole: string
  currentRestaurantId: string | null
  currentUserId: string
  isDirectore: boolean
}

// — Raw record shapes kept in state for the cell editor —————
type EmployeeRec = { id: string; full_name: string; restaurant_id: string | null }
type AttendanceRec = { id: string; user_id: string; restaurant_id: string | null; check_in: string; check_out: string | null }
type AbsenceRec = { id: string; user_id: string; restaurant_id: string | null; type: AbsenceType; start_date: string; end_date: string; certificate_code: string | null; notes: string | null }

// — Preview row types —————
type PreviewRow = {
  id: string
  full_name: string
  cells: Record<number, string> // day (1..31) → cell code / value
  totalMins?: number
  totalLabel?: string
  diffLabel?: string
  diffMins?: number
}

// Shift draft used inside the editor
type ShiftDraft = { id?: string; checkIn: string; checkOut: string }

function minutesToLabel(minutes: number): string {
  const h = Math.floor(minutes / 60)
  const m = Math.round(minutes % 60)
  return `${h}h ${String(m).padStart(2, '0')}m`
}

// — Cell styles – identical to Excel colour mapping —————
const PRESENCE_CELL_BG: Record<string, string> = {
  P: 'bg-green-100 text-green-900 dark:bg-green-900/30 dark:text-green-300',
  PP: 'bg-green-200 text-green-900 dark:bg-green-800/40 dark:text-green-200',
  F: 'bg-violet-100 text-violet-900 dark:bg-violet-900/30 dark:text-violet-300',
  M: 'bg-blue-100 text-blue-900 dark:bg-blue-900/30 dark:text-blue-300',
  R: 'bg-red-100 text-red-900 dark:bg-red-900/30 dark:text-red-300',
  AI: 'bg-purple-100 text-purple-900 dark:bg-purple-900/30 dark:text-purple-300',
}

const ORE_CELL_BG: Record<string, string> = {
  F: 'bg-violet-100 text-violet-900 dark:bg-violet-900/30 dark:text-violet-300',
  M: 'bg-blue-100 text-blue-900 dark:bg-blue-900/30 dark:text-blue-300',
  R: 'bg-red-100 text-red-900 dark:bg-red-900/30 dark:text-red-300',
  AI: 'bg-purple-100 text-purple-900 dark:bg-purple-900/30 dark:text-purple-300',
}

```

```

const thCls = 'px-2 py-1.5 text-center font-semibold bg-zinc-900 text-white dark:bg-zinc-800 whitespace-nowrap'
const tdCls = 'px-1.5 py-1 text-center text-xs border border-zinc-200 dark:border-zinc-700 whitespace-nowrap tabular-nums'
const tdNameCls = 'px-2 py-1 text-left text-xs font-medium border border-zinc-200 dark:border-zinc-700 whitespace-nowrap sticky left-0 bg-white dark:bg-zinc-950 z-10'
// Interactive day cell: keeps the colour bg, signals clickability via an inset ring
const cellInteractive = 'cursor-pointer transition-shadow hover:ring-2 hover:ring-inset hover:ring-primary/50'

// Absence codes that should get coloured cells in the "Ore" table
const ORE_ABSENCE_CODES = new Set(['F', 'M', 'R', 'AI'])

const DIPENDENTE_ABSENCE_TYPES: AbsenceType[] = ['ferie', 'malattia', 'riposo', 'assenza_ingiustificata']

export function ReportClient({ restaurants, currentUserRole, currentRestaurantId, currentUserId, isDirettore }: Props) {
  const router = useRouter()
  const [selectedMonth, setSelectedMonth] = useState(() => formatInTimeZone(new Date(), TZ, 'yyyy-MM'))
  const [selectedRestaurants, setSelectedRestaurants] = useState<string[]>() {
    currentRestaurantId ? [currentRestaurantId] : []
  }
  const [loading, setLoading] = useState<'presenze' | 'ore' | null>(null)

  // Preview state
  const [previewPresenze, setPreviewPresenze] = useState<PreviewRow[]>([])
  const [previewOre, setPreviewOre] = useState<PreviewRow[]>([])
  const [previewLoading, setPreviewLoading] = useState(false)
  // Generation counter: prevents stale responses from overwriting newer ones
  const genRef = useRef(0)

  // Raw records (kept so a cell click can resolve the underlying record IDs)
  const empMapRef = useRef<Map<string, EmployeeRec>>(new Map())
  const attRef = useRef<AttendanceRec[]>([])
  const absRef = useRef<AbsenceRec[]>([])

  const isManager = currentUserRole === 'manager'
  // canEdit: manager always yes; capo_servizio only when is_direttore = true
  const canEdit = isManager || (currentUserRole === 'capo_servizio' && isDirettore)
  // canSeeHours: i capi servizio non direttori non devono vedere il dato "ore lavorate"
  const canSeeHours = isManager || isDirettore

  const [unauthorizedMsg, setUnauthorizedMsg] = useState(false)
  function showUnauthorized() {
    setUnauthorizedMsg(true)
    setTimeout(() => setUnauthorizedMsg(false), 3000)
  }

  // — Cell editor dialog state —
  const [editorOpen, setEditorOpen] = useState(false)
  const [editorEmp, setEditorEmp] = useState<EmployeeRec | null>(null)
  const [editorDate, setEditorDate] = useState('') // YYYY-MM-DD
  const [editorMode, setEditorMode] = useState<'presenza' | 'assenza'>('presenza')
  const [shifts, setShifts] = useState<ShiftDraft[]>([])
  const [origShiftIds, setOrigShiftIds] = useState<string[]>([])
  const [absId, setAbsId] = useState<string | null>(null)
  const [absType, setAbsType] = useState<AbsenceType>('ferie')
  const [absCert, setAbsCert] = useState('')
  const [absNotes, setAbsNotes] = useState('')
  const [absStart, setAbsStart] = useState('')
  const [absEnd, setAbsEnd] = useState('')
  const [editorSaving, setEditorSaving] = useState(false)
  const [editorError, setEditorError] = useState<string | null>(null)
  const [confirmDeleteAbs, setConfirmDeleteAbs] = useState(false)

  function toggleRestaurant(id: string) {
    setSelectedRestaurants(prev =>
      prev.includes(id) ? prev.filter(r => r !== id) : [...prev, id]
    )
  }

  const currentTargets = useCallback(() => (
    isManager
    ? (selectedRestaurants.length > 0 ? selectedRestaurants : restaurants.map(r => r.id))

```

```

    : (currentRestaurantId ? [currentRestaurantId] : [])
  ), [isManager, selectedRestaurants, restaurants, currentRestaurantId])

async function downloadReport(type: 'presenze' | 'ore') {
  const targets = currentTargets()
  if (targets.length === 0) return

  setLoading(type)
  try {
    const res = await fetch('/api/report', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ month: selectedMonth, restaurantIds: targets, type }),
    })

    if (!res.ok) {
      const err = await res.json()
      alert(err.error || 'Errore nel download del report')
      return
    }

    const blob = await res.blob()
    const url = URL.createObjectURL(blob)
    const a = document.createElement('a')
    a.href = url
    a.download = `report-${type}-${selectedMonth}.xlsx`
    a.click()
    URL.revokeObjectURL(url)
  } finally {
    setLoading(null)
  }
}

// — Preview data fetch —
// Mirrors the logic in /api/report/route.ts, executed client-side so the
// preview stays in sync with whatever the Excel would contain.
const loadPreview = useCallback(async (month: string, restaurantIds: string[]) => {
  const gen = ++genRef.current
  setPreviewLoading(true)

  const [year, monthNum] = month.split('-').map(Number)
  const daysCount = getDaysInMonth(new Date(year, monthNum - 1))
  const monthStart = `${month}-01`
  const monthEnd = `${month}-${String(daysCount).padStart(2, '0')}`
  const rangeStart = fromZonedTime(`${monthStart}T00:00:00`, TZ).toISOString()
  const rangeEnd = fromZonedTime(`${monthEnd}T23:59:59`, TZ).toISOString()

  const supabase = createClient()

  const [
    { data: employees },
    { data: attendances },
    { data: absences },
  ] = await Promise.all([
    supabase
      .from('profiles')
      .select('id, full_name, restaurant_id')
      .in('role', ['dipendente', 'capo_servizio'])
      .in('restaurant_id', restaurantIds)
      .order('full_name'),
    supabase
      .from('attendances')
      .select('id, user_id, restaurant_id, check_in, check_out')
      .in('restaurant_id', restaurantIds)
      .gte('check_in', rangeStart)
      .lte('check_in', rangeEnd),
    supabase
      .from('absences')
      .select('id, user_id, restaurant_id, type, start_date, end_date, certificate_code, notes')
      .in('restaurant_id', restaurantIds)
      .eq('status', 'approved')
      .lte('start_date', monthEnd)
      .gte('end_date', monthStart),
  ])
})

```

```

// Discard if a newer request has started
if (gen !== genRef.current) return

const emps = (employees ?? []) as EmployeeRec[]
const atts = (attendances ?? []) as AttendanceRec[]
const abss = (absences ?? []) as AbsenceRec[]

// Store raw records for the editor
empMapRef.current = new Map(emps.map(e => [e.id, e]))
attRef.current = atts
absRef.current = abss

if (!emps.length) {
  setPreviewPresenze([])
  setPreviewOre([])
  setPreviewLoading(false)
  return
}

const presRows: PreviewRow[] = []
const oreRows: PreviewRow[] = []

for (const emp of emps) {
  const presCells: Record<number, string> = {}
  const oreCells: Record<number, string> = {}
  let totalMins = 0
  let workDays = 0

  for (let day = 1; day <= daysCount; day++) {
    const dateStr = `${month}-${String(day).padStart(2, '0')}`

    // Approved absence takes priority over attendance
    const absence = abss.find(a =>
      a.user_id === emp.id &&
      a.start_date <= dateStr &&
      a.end_date >= dateStr
    )
    if (absence) {
      const code = ABSENCE_CODES[absence.type]
      presCells[day] = code
      oreCells[day] = code
      continue
    }

    // Sessions for this employee on this Rome-local day
    const daySessions = atts.filter(a => {
      if (a.user_id !== emp.id) return false
      return formatInTimeZone(new Date(a.check_in), TZ, 'yyyy-MM-dd') === dateStr
    })

    if (daySessions.length > 0) {
      const hasOpen = daySessions.some(a => !a.check_out)
      const dayMins = daySessions.reduce((sum, a) => {
        if (!a.check_out) return sum
        return sum + differenceInMinutes(new Date(a.check_out), new Date(a.check_in))
      }, 0)

      presCells[day] = dayMins > 720 ? 'PP' : 'P'

      if (hasOpen && dayMins === 0) {
        oreCells[day] = 'In corso'
      } else {
        totalMins += dayMins
        workDays++
        oreCells[day] = minutesToLabel(dayMins)
      }
    }
  }
}

const targetMins = workDays * TARGET_HOURS_PER_DAY * 60
const diffMins = totalMins - targetMins
const absDiff = Math.abs(diffMins)

```

```

    presRows.push({ id: emp.id, full_name: emp.full_name, cells: presCells })
    oreRows.push({
      id: emp.id,
      full_name: emp.full_name,
      cells: oreCells,
      totalMins,
      totalLabel: minutesToLabel(totalMins),
      diffMins,
      diffLabel: `${diffMins >= 0 ? '+' : '-'}${minutesToLabel(absDiff)}`,
    })
  }

  setPreviewPresenze(presRows)
  setPreviewOre(oreRows)
  setPreviewLoading(false)
}, [])

// Re-fetch preview whenever month or restaurant selection changes.
// This is a legitimate "synchronize with external system" effect (the DB):
// loadPreview owns its own loading/data state, so the set-state-in-effect
// warning is expected here.
useEffect(() => {
  const targets = currentTargets()
  if (targets.length === 0) return
  // eslint-disable-next-line react-hooks/set-state-in-effect
  loadPreview(selectedMonth, targets)
  // selectedRestaurants.join is a stable primitive derived from the array
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [selectedMonth, selectedRestaurants.join(','), isManager, loadPreview])

// Compute preview column structure from selectedMonth
const { days, monthLabel } = useMemo(() => {
  const [y, m] = selectedMonth.split('-').map(Number)
  const n = getDaysInMonth(new Date(y, m - 1))
  const raw = new Intl.DateTimeFormat('it-IT', { month: 'long', year: 'numeric' }).format(new Date(y, m - 1, 1))
  return {
    days: Array.from({ length: n }, (_, i) => i + 1),
    monthLabel: raw.charAt(0).toUpperCase() + raw.slice(1),
  }
}, [selectedMonth])

// — Open the editor for a given employee + day —————
function openCell(employeeId: string, day: number) {
  if (!canEdit) return // read-only for non-direttore capo_servizio
  const emp = empMapRef.current.get(employeeId)
  if (!emp) return
  const dateStr = `${selectedMonth}-${String(day).padStart(2, '0')}`

  setEditorEmp(emp)
  setEditorDate(dateStr)
  setEditorError(null)
  setConfirmDeleteAbs(false)

  // Existing approved absence covering this day?
  const absence = absRef.current.find(a =>
    a.user_id === employeeId && a.start_date <= dateStr && a.end_date >= dateStr
  )

  if (absence) {
    setEditorMode('assenza')
    setAbsId(absence.id)
    setAbsType(absence.type)
    setAbsCert(absence.certificate_code ?? '')
    setAbsNotes(absence.notes ?? '')
    setAbsStart(absence.start_date)
    setAbsEnd(absence.end_date)
    setShifts([])
    setOrigShiftIds([])
  } else {
    // Existing attendance blocks for this day
    const blocks = attRef.current
      .filter(a => a.user_id === employeeId && formatInTimeZone(new Date(a.check_in), TZ, 'yyyy-MM-dd') ===
dateStr)
      .sort((a, b) => new Date(a.check_in).getTime() - new Date(b.check_in).getTime())

```

```

setEditorMode('presenza')
setAbsId(null)
setAbsType('ferie'); setAbsCert(''); setAbsNotes('')
setAbsStart(dateStr); setAbsEnd(dateStr)

if (blocks.length > 0) {
  setShifts(blocks.map(b => ({
    id: b.id,
    checkIn: formatInTimeZone(new Date(b.check_in), TZ, 'HH:mm'),
    checkOut: b.check_out ? formatInTimeZone(new Date(b.check_out), TZ, 'HH:mm') : '',
  })))
  setOrigShiftIds(blocks.map(b => b.id))
} else {
  setShifts([{ checkIn: '', checkOut: '' }])
  setOrigShiftIds([])
}
}

setEditorOpen(true)
}

function addShiftRow() {
  setShifts(s => [...s, { checkIn: '', checkOut: '' }])
}
function updateShift(id: number, patch: Partial<ShiftDraft>) {
  setShifts(s => s.map((row, i) => i === id ? { ...row, ...patch } : row))
}
function removeShiftRow(id: number) {
  setShifts(s => s.filter((_, i) => i !== id))
}

async function refreshAfterMutation() {
  await loadPreview(selectedMonth, currentTargets())
  // Revalidate server components so any other open route reflects the change
  router.refresh()
}

// — Save handler (create / edit / delete from a single cell) —————
async function handleEditorSave() {
  if (!canEdit) { showUnauthorized(); return }
  if (!editorEmp) return
  setEditorSaving(true)
  setEditorError(null)
  const supabase = createClient()

  try {
    if (editorMode === 'presenza') {
      // 1) Delete blocks the user removed from the list
      const keptIds = new Set(shifts.filter(s => s.id).map(s => s.id as string))
      const toDelete = origShiftIds.filter(id => !keptIds.has(id))
      if (toDelete.length > 0) {
        const { error } = await supabase.from('attendances').delete().in('id', toDelete)
        if (error) throw new Error(error.message)
      }

      // 2) Upsert each shift row that has at least a check-in
      for (const s of shifts) {
        if (!s.checkIn) continue
        if (s.id) {
          // Passa dall'API (non un update diretto) così l'uscita che
          // scavalca la mezzanotte (es. 15:00 → 01:00) viene gestita
          // correttamente come turno notturno, non come dato invertito.
          const res = await fetch('/api/presenze', {
            method: 'PATCH',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
              id: s.id,
              date: editorDate,
              checkIn: s.checkIn,
              checkOut: s.checkOut || null,
            }),
          })
          if (!res.ok) {

```

```

        const d = await res.json()
        throw new Error(d.error || 'Errore modifica presenza')
    }
} else {
    const res = await fetch('/api/presenze', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            userId: editorEmp.id,
            date: editorDate,
            checkIn: s.checkIn,
            checkOut: s.checkOut || undefined,
        }),
    })
    if (!res.ok) {
        const d = await res.json()
        throw new Error(d.error || 'Errore creazione presenza')
    }
}
}
} else {
    // Absence mode
    if (absId) {
        const { error } = await supabase
            .from('absences')
            .update({
                type: absType,
                certificate_code: absType === 'malattia' ? (absCert || null) : null,
                notes: absNotes || null,
            })
            .eq('id', absId)
        if (error) throw new Error(error.message)
    } else {
        const { error } = await supabase.from('absences').insert({
            user_id: editorEmp.id,
            restaurant_id: editorEmp.restaurant_id,
            type: absType,
            start_date: editorDate,
            end_date: editorDate,
            certificate_code: absType === 'malattia' ? (absCert || null) : null,
            notes: absNotes || null,
            status: 'approved',
            created_by: currentUserId,
        })
        if (error) throw new Error(error.message)
    }
}

await refreshAfterMutation()
setEditorOpen(false)
} catch (e) {
    setEditorError(e instanceof Error ? e.message : 'Errore durante il salvataggio')
} finally {
    setEditorSaving(false)
}
}

async function handleDeleteAbsence() {
    if (!canEdit) { showUnauthorized(); return }
    if (!absId) return
    setEditorSaving(true)
    setEditorError(null)
    try {
        const supabase = createClient()
        const { error } = await supabase.from('absences').delete().eq('id', absId)
        if (error) throw new Error(error.message)
        await refreshAfterMutation()
        setEditorOpen(false)
    } catch (e) {
        setEditorError(e instanceof Error ? e.message : "Errore durante l'eliminazione")
    } finally {
        setEditorSaving(false)
    }
}
}

```

```

const editorDateLabel = editorDate
? (() => {
  const [y, m, d] = editorDate.split('-').map(Number)
  const date = new Date(y, m - 1, d, 12, 0, 0)
  const label = formatInTimeZone(date, TZ, 'EEEE d MMMM yyyy', { locale: it })
  return label.charAt(0).toUpperCase() + label.slice(1)
})()
: ''

return (
  <div>
    <div className="mb-6">
      <h1 className="text-2xl font-bold">Report Excel</h1>
      <p className="text-muted-foreground text-sm mt-1">Esporta presenze e ore lavorate</p>
    </div>

    <div className="space-y-6 max-w-lg">
      <div className="space-y-2">
        <Label>Mese</Label>
        <Input
          type="month"
          value={selectedMonth}
          onChange={e => setSelectedMonth(e.target.value)}
          className="w-auto"
        />
      </div>

      {isManager && restaurants.length > 0 && (
        <div className="space-y-2">
          <Label>Ristoranti (seleziona uno o più, lascia vuoto per tutti)</Label>
          <div className="flex flex-wrap gap-2">
            {restaurants.map(r => (
              <button
                key={r.id}
                onClick={() => toggleRestaurant(r.id)}
                className={`px-3 py-1.5 rounded-full text-sm font-medium border transition-colors ${
                  selectedRestaurants.includes(r.id)
                    ? 'bg-primary text-primary-foreground border-primary'
                    : 'bg-background border-border text-foreground hover:bg-accent'
                }`}
              >
                {r.name}
              </button>
            ))}
          </div>
          {selectedRestaurants.length > 0 && (
            <p className="text-xs text-muted-foreground">{selectedRestaurants.length} ristoranti selezionati</p>
          )}
        </div>
      )}

      <div className={`grid gap-4 ${canSeeHours ? 'grid-cols-2' : 'grid-cols-1'}`>
        <Card className="cursor-pointer hover:border-primary transition-colors" onClick={() =>
downloadReport('presenze')}>
          <CardContent className="pt-6 pb-5 text-center">
            <FileSpreadsheet className="w-8 h-8 mx-auto mb-3 text-emerald-500" />
            <p className="font-medium text-sm">Report Presenze</p>
            <p className="text-xs text-muted-foreground mt-1">P · F · M · R · AI</p>
            <Button size="sm" className="mt-4 w-full" disabled={loading === 'presenze'}>
              {loading === 'presenze' ? 'Generazione...' : <><Download className="w-4 h-4" /> Scarica</>}
            </Button>
          </CardContent>
        </Card>

        {canSeeHours && (
          <Card className="cursor-pointer hover:border-primary transition-colors" onClick={() =>
downloadReport('ore')}>
            <CardContent className="pt-6 pb-5 text-center">
              <FileSpreadsheet className="w-8 h-8 mx-auto mb-3 text-blue-500" />
              <p className="font-medium text-sm">Report Ore</p>
              <p className="text-xs text-muted-foreground mt-1">Ore · Totale · Delta</p>
              <Button size="sm" className="mt-4 w-full" disabled={loading === 'ore'}>
                {loading === 'ore' ? 'Generazione...' : <><Download className="w-4 h-4" /> Scarica</>}
              </Button>
            </CardContent>
          </Card>
        )}
      </div>
    </div>
  )

```



```

        </Button>
      </CardContent>
    </Card>
  )}
</div>
</div>

{/* — Preview riquadri —————
Scroll confinato al container (overflow-auto + max-h).
La tabella usa minWidth: max-content per evitare che i giorni
collassino – scroll X interno, nessun leak a livello viewport.
Ogni cella giorno è cliccabile per creare/modificare presenze/assenze. */}
<div className="mt-10 space-y-8">

  {/* Riquadro 1 – Preview Presenze/Turni */}
  <div>
    <div className="flex items-center gap-2 mb-2">
      <FileSpreadsheet className="w-4 h-4 text-emerald-500 shrink-0" />
      <h2 className="text-sm font-semibold">Preview Presenze / Turni</h2>
      <span className="text-xs text-muted-foreground">({monthLabel})</span>
      {previewLoading && (
        <span className="text-xs text-muted-foreground animate-pulse">• Aggiornamento...</span>
      )}
    </div>
    <div className="w-full rounded-md border bg-muted/20 overflow-auto max-h-[340px]">
      <table
        className="border-collapse text-xs"
        style={{ minWidth: 'max-content' }}
      >
        <thead>
          <tr>
            <th className={` ${thCls} sticky left-0 z-20 text-left min-w-[160px]`} >Dipendente</th>
            {days.map(d => (
              <th key={d} className={` ${thCls} min-w-[36px]`} >{d}</th>
            ))}
            <th className={` ${thCls} min-w-[80px]`} >Note</th>
          </tr>
        </thead>
        <tbody>
          {previewLoading && previewPresenze.length === 0 ? (
            <tr>
              <td colspan={days.length + 2} className="text-center py-6 text-muted-foreground text-xs">
                Caricamento...
              </td>
            </tr>
          ) : previewPresenze.length === 0 ? (
            <tr>
              <td colspan={days.length + 2} className="text-center py-6 text-muted-foreground text-xs">
                Nessun dipendente trovato per i filtri selezionati
              </td>
            </tr>
          ) : previewPresenze.map(row => (
            <tr key={row.id} className="even:bg-zinc-50 dark:even:bg-zinc-900/20">
              <td className={tdNameCls}>{row.full_name}</td>
              {days.map(d => {
                const code = row.cells[d] ?? ''
                return (
                  <td
                    key={d}
                    onClick={canEdit ? () => openCell(row.id, d) : undefined}
                    title={canEdit ? 'Clicca per modificare' : undefined}
                    className={` ${tdCls} font-semibold ${canEdit ? cellInteractive : ''} $
{PRESENZE_CELL_BG[code] ?? ''}`}
                  >
                    {code}
                  </td>
                )
              })}
              <td className={tdCls} />
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>

```

```

    {!previewLoading && previewPresenze.length > 0 && (
      <p className="text-[11px] text-muted-foreground mt-1.5">
        {previewPresenze.length} dipendenti · dati reali{canEdit ? ' · clicca una cella per modificare' :
      ''}
    )}
  </p>
)}
</div>

{/* Riquadro 2 – Preview Ore Lavorate (nascosto al capo servizio non direttore) */}
{canSeeHours && (
  <div>
    <div className="flex items-center gap-2 mb-2">
      <FileSpreadsheet className="w-4 h-4 text-blue-500 shrink-0" />
      <h2 className="text-sm font-semibold">Preview Ore Lavorate</h2>
      <span className="text-xs text-muted-foreground">({monthLabel})</span>
      {previewLoading && (
        <span className="text-xs text-muted-foreground animate-pulse">· Aggiornamento...</span>
      )}
    </div>
    <div className="w-full rounded-md border bg-muted/20 overflow-auto max-h-[340px]">
      <table
        className="border-collapse text-xs"
        style={{ minWidth: 'max-content' }}
      >
        <thead>
          <tr>
            <th className={` ${thCls} sticky left-0 z-20 text-left min-w-[160px]`} >Dipendente</th>
            {days.map(d => (
              <th key={d} className={` ${thCls} min-w-[36px]`} >{d}</th>
            ))}
            <th className={` ${thCls} min-w-[90px]`} >Totale Ore</th>
            <th className={` ${thCls} min-w-[90px]`} >Differenza</th>
            <th className={` ${thCls} min-w-[80px]`} >Note</th>
          </tr>
        </thead>
        <tbody>
          {previewLoading && previewOre.length === 0 ? (
            <tr>
              <td colspan={days.length + 4} className="text-center py-6 text-muted-foreground text-xs">
                Caricamento...
              </td>
            </tr>
          ) : previewOre.length === 0 ? (
            <tr>
              <td colspan={days.length + 4} className="text-center py-6 text-muted-foreground text-xs">
                Nessun dipendente trovato per i filtri selezionati
              </td>
            </tr>
          ) : previewOre.map(row => (
            <tr key={row.id} className="even:bg-zinc-50 dark:even:bg-zinc-900/20">
              <td className={tdNameCls}>{row.full_name}</td>
              {days.map(d => {
                const val = row.cells[d] ?? ''
                const isAbsCode = ORE_ABSENCE_CODES.has(val)
                return (
                  <td
                    key={d}
                    onClick={canEdit ? () => openCell(row.id, d) : undefined}
                    title={canEdit ? 'Clicca per modificare' : undefined}
                    className={` ${tdCls} ${canEdit ? cellInteractive : ''} ${isAbsCode ? 'font-semibold $
{ORE_CELL_BG[val] ?? ''} ` : ''}` : ''}`
                  >
                    {val}
                  </td>
                )
              })}
              <td className={` ${tdCls} font-medium`} >{row.totalLabel}</td>
              <td className={` ${tdCls} font-semibold $
                (row.diffMins ?? 0) >= 0
                ? 'text-emerald-700 dark:text-emerald-400'
                : 'text-red-700 dark:text-red-400'
              `}>
                {row.diffLabel}
              </td>
            </tr>
          )}
        </tbody>
      </table>
    </div>
  </div>
)

```

```

        <td className={tdCls} />
      </tr>
    )}}
  </tbody>
</table>
</div>
{!previewLoading && previewOre.length > 0 && (
  <p className="text-[11px] text-muted-foreground mt-1.5">
    {previewOre.length} dipendenti · dati reali{canEdit ? ' ' · clicca una cella per modificare' : ''}
  </p>
)}
</div>
)}
</div>

/* — Cell editor dialog —————
  Backdrop close disabilitato: il form si chiude solo con Annulla / X
  o dopo un salvataggio riuscito, per non perdere dati inseriti. */
<Dialog open={editorOpen} onOpenChange={open => { if (!open) setEditorOpen(false) }}>
  <DialogContent onInteractOutside={e => e.preventDefault()}>
    <DialogHeader>
      <DialogTitle>{editorEmp?.full_name ?? '-'}</DialogTitle>
      <p className="text-sm text-muted-foreground">{editorDateLabel}</p>
    </DialogHeader>

    {/* Mode toggle */}
    <div className="grid grid-cols-2 gap-2">
      <button
        type="button"
        onClick={() => { setEditorMode('presenza'); setConfirmDeleteAbs(false) }}
        className={`h-9 rounded-md text-sm font-medium border transition-colors ${
          editorMode === 'presenza'
            ? 'bg-primary text-primary-foreground border-primary'
            : 'bg-background border-border hover:bg-accent'
          }`>
        >
        Presenza
      </button>
      <button
        type="button"
        onClick={() => { setEditorMode('assenza'); setConfirmDeleteAbs(false) }}
        className={`h-9 rounded-md text-sm font-medium border transition-colors ${
          editorMode === 'assenza'
            ? 'bg-primary text-primary-foreground border-primary'
            : 'bg-background border-border hover:bg-accent'
          }`>
        >
        Assenza
      </button>
    </div>

    {editorMode === 'presenza' ? (
      <div className="space-y-3">
        {shifts.length === 0 && (
          <p className="text-sm text-muted-foreground">Nessun turno. Aggiungine uno.</p>
        )}
        {shifts.map((s, idx) => (
          <div key={idx} className="flex items-end gap-2">
            <div className="flex-1 space-y-1">
              <Label className="text-xs">Ingresso</Label>
              <TimeInput
                value={s.checkIn}
                onChange={v => updateShift(idx, { checkIn: v })}
                className="h-9 rounded-sm"
              />
            </div>
            <div className="flex-1 space-y-1">
              <Label className="text-xs">Uscita</Label>
              <TimeInput
                value={s.checkOut}
                onChange={v => updateShift(idx, { checkOut: v })}
                className="h-9 rounded-sm"
              />
            </div>
          </div>
        ))}
      </div>
    ) : (
      <div className="space-y-3">
        {shifts.length === 0 && (
          <p className="text-sm text-muted-foreground">Nessun turno. Aggiungine uno.</p>
        )}
        {shifts.map((s, idx) => (
          <div key={idx} className="flex items-end gap-2">
            <div className="flex-1 space-y-1">
              <Label className="text-xs">Ingresso</Label>
              <TimeInput
                value={s.checkIn}
                onChange={v => updateShift(idx, { checkIn: v })}
                className="h-9 rounded-sm"
              />
            </div>
            <div className="flex-1 space-y-1">
              <Label className="text-xs">Uscita</Label>
              <TimeInput
                value={s.checkOut}
                onChange={v => updateShift(idx, { checkOut: v })}
                className="h-9 rounded-sm"
              />
            </div>
          </div>
        ))}
      </div>
    )}
  </DialogContent>
</Dialog>

```

```

        <Button
          type="button"
          variant="outline"
          size="icon"
          onClick={() => removeShiftRow(idx)}
          className="h-9 w-9 shrink-0 text-destructive dark:text-red-400 dark: hover: bg-red-500/10
dark: hover: text-red-300"
          title="Rimuovi turno"
        >
        <Trash2 className="w-4 h-4" />
        </Button>
      </div>
    )}
    <Button type="button" variant="outline" size="sm" onClick={addShiftRow} className="gap-1.5">
      <Plus className="w-3.5 h-3.5" /> Aggiungi turno
    </Button>
  </div>
) : confirmDeleteAbs ? (
  <div className="space-y-4">
    <div className="flex gap-3 items-start rounded-md border border-destructive/30 bg-destructive/10
px-4 py-3">
      <AlertTriangle className="w-5 h-5 text-destructive shrink-0 mt-0.5" />
      <div className="text-sm">
        <p className="font-medium text-foreground">Eliminare questa assenza?</p>
        <p className="text-muted-foreground mt-1">L'azione è irreversibile.</p>
      </div>
    </div>
    <DialogFooter>
      <Button variant="outline" onClick={() => setConfirmDeleteAbs(false)} disabled={editorSaving}
className="h-10 rounded-sm">
        Annulla
      </Button>
      <Button variant="destructive" onClick={handleDeleteAbsence} disabled={editorSaving}
className="h-10 rounded-sm">
        {editorSaving ? 'Eliminazione...' : 'Elimina definitivamente'}
      </Button>
    </DialogFooter>
  </div>
) : (
  <div className="space-y-4">
    <div className="space-y-2">
      <Label>Causale</Label>
      <Select value={absType} onChange={v => setAbsType(v as AbsenceType)}>
        <SelectTrigger className="h-10 rounded-sm"><SelectValue /></SelectTrigger>
        <SelectContent>
          {DIPENDENTE_ABSENCE_TYPES.map(t => (
            <SelectItem key={t} value={t}>{ABSENCE_LABELS[t]}</SelectItem>
          ))}
        </SelectContent>
      </Select>
    </div>
    {absType === 'malattia' && (
      <div className="space-y-2">
        <Label>Codice certificato</Label>
        <Input value={absCert} onChange={e => setAbsCert(e.target.value)} placeholder="Codice INPS"
className="h-10 rounded-sm" />
      </div>
    )}
    <div className="space-y-2">
      <Label>Note <span className="text-muted-foreground font-normal">(opzionale)</span></Label>
      <Input value={absNotes} onChange={e => setAbsNotes(e.target.value)} placeholder="Aggiungi una
nota..." className="h-10 rounded-sm" />
    </div>
    {absId && (absStart !== editorDate || absEnd !== editorDate) && (
      <p className="text-xs text-muted-foreground">
        Assenza dal {absStart} al {absEnd} – le modifiche si applicano all'intero periodo.
      </p>
    )}
  </div>
)}

{editorError && (
  <p className="text-sm text-destructive bg-destructive/10 px-3 py-2 rounded-md">{editorError}</p>
)}

```

```

        {!confirmDeleteAbs && (
            <DialogFooter className="sm:justify-between gap-2">
                {editorMode === 'assenza' && absId ? (
                    <Button
                        variant="destructive"
                        onClick={() => setConfirmDeleteAbs(true)}
                        disabled={editorSaving}
                        className="h-10 rounded-sm"
                    >
                        <Trash2 className="w-4 h-4" /> Elimina
                    </Button>
                ) : <span />}
                <div className="flex gap-2">
                    <Button variant="outline" onClick={() => setEditorOpen(false)} disabled={editorSaving}
className="h-10 rounded-sm">
                        Annulla
                    </Button>
                    <Button onClick={handleEditorSave} disabled={editorSaving} className="h-10 rounded-sm">
                        {editorSaving ? <>Salvataggio</><LoadingDots /></> : 'Salva'}
                    </Button>
                </div>
            </DialogFooter>
        )}
    </DialogContent>
</Dialog>

    {/* Unauthorized toast */}
    {unauthorizedMsg && (
        <div className="fixed bottom-4 right-4 z-50 flex items-center gap-2 bg-destructive text-destructive-
foreground px-4 py-3 rounded-md shadow-lg text-sm font-medium animate-in slide-in-from-bottom-2">
            Azione riservata alla direzione
        </div>
    )}
</div>
)
}

```

```

'use client'
import { useEffect, useState } from 'react'
import { Button } from '@components/ui/button'
import { QRCode, Download } from 'lucide-react'
import QRCodeLib from 'qrcode'

interface Props {
  restaurantName: string
  qrSecret: string
}

export function RestaurantQrCard({ restaurantName, qrSecret }: Props) {
  const [dataUrl, setDataUrl] = useState<string | null>(null)

  useEffect(() => {
    QRCodeLib.toDataURL(qrSecret, {
      width: 320,
      margin: 2,
      color: { dark: '#0f172a', light: '#ffffff' },
    }).then(setDataUrl)
  }, [qrSecret])

  function download() {
    if (!dataUrl) return
    const a = document.createElement('a')
    a.href = dataUrl
    a.download = `QR-${restaurantName.replace(/\s+/g, '-')}.png`
    a.click()
  }

  return (
    <div className="mt-6 bg-card border border-border rounded-md p-4 flex items-center gap-4">
      {dataUrl ? (
        // eslint-disable-next-line @next/next/no-img-element
        <img src={dataUrl} alt={`QR Code ${restaurantName}`} className="w-20 h-20 rounded-sm border border-border" />
      ) : (
        <div className="w-20 h-20 rounded-sm border border-border bg-muted animate-pulse" />
      )}
      <div className="flex-1 min-w-0">
        <h2 className="text-sm font-semibold flex items-center gap-2">
          <QRCode className="w-4 h-4 text-muted-foreground" />
          QR Code Timbratura
        </h2>
        <p className="text-xs text-muted-foreground mt-0.5">{restaurantName}</p>
      </div>
      <Button size="sm" variant="outline" onClick={download} disabled={!dataUrl}>
        <Download className="w-4 h-4" /> Scarica
      </Button>
    </div>
  )
}

```

`src/components/manager/RestaurantsClient.tsx`

```

'use client'
import { useState } from 'react'
import { createClient } from '@lib/supabase/client'
import { useAccountStatus } from '@contexts/AccountStatusContext'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { TimeInput } from '@components/ui/time-input'
import { Label } from '@components/ui/label'
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select'
import { Badge } from '@components/ui/badge'
import { Plus, QrCode, Pencil, Trash2, MapPin, Clock, X } from 'lucide-react'
import QRCode from 'qrcode'
import type { Restaurant, ShiftSlot, Department } from '@types'
import { DEPARTMENTS, WEEK_DAYS_SHORT } from '@types'

// 0=Dom..6=Sab - mostrati come Lun-Dom per convenzione
const DAY_OPTIONS = [1, 2, 3, 4, 5, 6, 0] as const

interface Props {
  initialRestaurants: Restaurant[]
}

const emptySlotForm = () => ({
  department: '' as Department | '',
  name: '',
  start_time: '',
  end_time: '',
  required_count: 1,
  days_of_week: [] as number[], // vuoto = tutti i giorni
})

export function RestaurantsClient({ initialRestaurants }: Props) {
  const { isPending } = useAccountStatus()
  const [restaurants, setRestaurants] = useState<Restaurant[]>(initialRestaurants)

  // — Restaurant form —
  const [showForm, setShowForm] = useState(false)
  const [editing, setEditing] = useState<Restaurant | null>(null)
  const [name, setName] = useState('')
  const [address, setAddress] = useState('')
  const [latitude, setLatitude] = useState('')
  const [longitude, setLongitude] = useState('')
  const [closingDays, setClosingDays] = useState<number[]>([])
  const [loading, setLoading] = useState(false)
  const [deleting, setDeleting] = useState<string | null>(null)

  // — ShiftSlots dialog —
  const [slotsRestaurant, setSlotsRestaurant] = useState<Restaurant | null>(null)
  const [slots, setSlots] = useState<ShiftSlot[]>([])
  const [slotsLoading, setSlotsLoading] = useState(false)
  const [slotForm, setSlotForm] = useState(emptySlotForm())
  const [editingSlot, setEditingSlot] = useState<ShiftSlot | null>(null)
  const [slotSaving, setSlotSaving] = useState(false)
  const [slotError, setSlotError] = useState<string | null>(null)

  // — Helpers —
  function toggleClosingDay(day: number) {
    setClosingDays(prev =>
      prev.includes(day) ? prev.filter(d => d !== day) : [...prev, day]
    )
  }

  function toggleSlotDay(day: number) {
    setSlotForm(f => ({
      ...f,
      days_of_week: f.days_of_week.includes(day)
        ? f.days_of_week.filter(d => d !== day)
        : [...f.days_of_week, day],
    }))
  }
}

```



```

function openCreate() {
  setEditing(null)
  setName(''); setAddress(''); setLatitude(''); setLongitude('')
  setClosingDays([])
  setShowForm(true)
}

function openEdit(r: Restaurant) {
  setEditing(r)
  setName(r.name)
  setAddress(r.address ?? '')
  setLatitude(r.latitude !== null ? String(r.latitude) : '')
  setLongitude(r.longitude !== null ? String(r.longitude) : '')
  setClosingDays(r.closing_days ?? [])
  setShowForm(true)
}

async function handleSave() {
  setLoading(true)
  const supabase = createClient()
  const lat = latitude !== '' ? parseFloat(latitude) : null
  const lng = longitude !== '' ? parseFloat(longitude) : null

  if (editing) {
    const { data } = await supabase
      .from('restaurants')
      .update({ name, address: address || null, latitude: lat, longitude: lng, closing_days: closingDays })
      .eq('id', editing.id)
      .select()
      .single()
    if (data) setRestaurants(rs => rs.map(r => r.id === data.id ? data : r))
  } else {
    const { data } = await supabase
      .from('restaurants')
      .insert({ name, address: address || null, latitude: lat, longitude: lng, closing_days: closingDays })
      .select()
      .single()
    if (data) setRestaurants(rs => [...rs, data])
  }

  setShowForm(false)
  setLoading(false)
}

async function handleDelete(id: string) {
  if (!confirm('Eliminare questo ristorante?')) return
  setDeleting(id)
  const supabase = createClient()
  await supabase.from('restaurants').delete().eq('id', id)
  setRestaurants(rs => rs.filter(r => r.id !== id))
  setDeleting(null)
}

async function downloadQR(restaurant: Restaurant) {
  const url = await QRCode.toDataURL(restaurant.qr_secret, {
    width: 400, margin: 2,
    color: { dark: '#0f172a', light: '#ffffff' },
  })
  const a = document.createElement('a')
  a.href = url
  a.download = `QR-${restaurant.name.replace(/\s+/g, '-')}.png`
  a.click()
}

// — Shift slots management —
async function openSlots(r: Restaurant) {
  setSlotsRestaurant(r)
  setSlotsLoading(true)
  setSlotForm(emptySlotForm())
  setEditingSlot(null)
  setSlotError(null)
  const supabase = createClient()
  const { data } = await supabase
    .from('shift_slots')

```

```

        .select('*')
        .eq('restaurant_id', r.id)
        .order('department')
        .order('start_time')
        setSlots((data ?? []) as ShiftSlot[])
        setSlotsLoading(false)
    }

    function startEditSlot(s: ShiftSlot) {
        setEditingSlot(s)
        setSlotForm({
            department: s.department,
            name: s.name,
            start_time: s.start_time.slice(0, 5),
            end_time: s.end_time.slice(0, 5),
            required_count: s.required_count,
            days_of_week: s.days_of_week,
        })
        setSlotError(null)
    }

    function cancelEditSlot() {
        setEditingSlot(null)
        setSlotForm(emptySlotForm())
        setSlotError(null)
    }

    async function handleSaveSlot() {
        if (!slotsRestaurant || !slotForm.department || !slotForm.name || !slotForm.start_time || !slotForm.end_time)
        return
        setSlotSaving(true); setSlotError(null)
        const supabase = createClient()

        if (editingSlot) {
            const { data, error } = await supabase
                .from('shift_slots')
                .update({
                    department: slotForm.department,
                    name: slotForm.name,
                    start_time: slotForm.start_time,
                    end_time: slotForm.end_time,
                    required_count: slotForm.required_count,
                    days_of_week: slotForm.days_of_week,
                })
                .eq('id', editingSlot.id)
                .select()
                .single()
            if (error) { setSlotError(error.message) }
            else if (data) {
                setSlots(prev => prev.map(s => s.id === editingSlot.id ? data as ShiftSlot : s))
                setEditingSlot(null)
                setSlotForm(emptySlotForm())
            }
        } else {
            const { data, error } = await supabase
                .from('shift_slots')
                .insert({
                    restaurant_id: slotsRestaurant.id,
                    department: slotForm.department,
                    name: slotForm.name,
                    start_time: slotForm.start_time,
                    end_time: slotForm.end_time,
                    required_count: slotForm.required_count,
                    days_of_week: slotForm.days_of_week,
                })
                .select()
                .single()
            if (error) { setSlotError(error.message) }
            else if (data) {
                setSlots(prev => [...prev, data as ShiftSlot])
                setSlotForm(emptySlotForm())
            }
        }
        setSlotSaving(false)
    }

```

```

}

async function handleDeleteSlot(id: string) {
  const supabase = createClient()
  await supabase.from('shift_slots').delete().eq('id', id)
  setSlots(prev => prev.filter(s => s.id !== id))
  if (editingSlot?.id === id) cancelEditSlot()
}

function isOvernight(start: string, end: string) {
  const [sh, sm] = start.split(':').map(Number)
  const [eh, em] = end.split(':').map(Number)
  return eh * 60 + em <= sh * 60 + sm
}

// — Render —————
return (
  <div>
    <div className="flex items-center justify-between mb-6">
      <div>
        <h1 className="text-2xl font-bold">Ristoranti</h1>
        <p className="text-muted-foreground text-sm mt-1">{restaurants.length} ristoranti</p>
      </div>
      <Button onClick={openCreate} size="sm" disabled={isPending} title={isPending ? 'Disponibile dopo
l\'attivazione' : undefined}>
        <Plus className="w-4 h-4" /> Aggiungi
      </Button>
    </div>

    {restaurants.length === 0 ? (
      <Card>
        <CardContent className="py-12 text-center text-muted-foreground">
          Nessun ristorante. Aggiungine uno per iniziare.
        </CardContent>
      </Card>
    ) : (
      <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
        {restaurants.map(r => (
          <Card key={r.id}>
            <CardHeader className="pb-3">
              <div className="flex items-start justify-between gap-2">
                <CardTitle className="text-base">{r.name}</CardTitle>
                <Badge variant="outline" className="text-xs shrink-0">Attivo</Badge>
              </div>
              <div>
                {r.address && (
                  <p className="text-sm text-muted-foreground flex items-center gap-1 mt-1">
                    <MapPin className="w-3 h-3" /> {r.address}
                  </p>
                )}
                {r.closing_days?.length > 0 && (
                  <p className="text-xs text-muted-foreground mt-1">
                    Chiuso: {r.closing_days.map(d => WEEK_DAYS_SHORT[d]).join(', ')}
                  </p>
                )}
              </div>
            </CardHeader>
            <CardContent className="pt-0">
              <div className="flex gap-2 flex-wrap">
                <Button variant="outline" size="sm" onClick={() => downloadQR(r)} className="flex-1">
                  <QrCode className="w-4 h-4" /> QR Code
                </Button>
                <Button variant="outline" size="sm" onClick={() => openSlots(r)} className="flex-1">
                  <Clock className="w-4 h-4" /> Fasce orarie
                </Button>
                <Button variant="outline" size="sm" onClick={() => openEdit(r)}>
                  <Pencil className="w-4 h-4" />
                </Button>
                <Button
                  variant="outline" size="sm"
                  onClick={() => handleDelete(r.id)}
                  disabled={deleting === r.id}
                  className="text-destructive hover:text-destructive"
                >
                  <Trash2 className="w-4 h-4" />
                </Button>
              </div>
            </CardContent>
          </Card>
        )}
      </div>
    )}
  </div>
)

```

```

        </div>
      </CardContent>
    </Card>
  )})
</div>
})

{/* — Edit / Create restaurant ————— */}
<Dialog open={showForm} onOpenChange={setShowForm}>
  <DialogContent className="max-h-[90vh] overflow-y-auto">
    <DialogHeader>
      <DialogTitle>{editing ? 'Modifica Ristorante' : 'Nuovo Ristorante'}</DialogTitle>
    </DialogHeader>
    <div className="space-y-4">
      <div className="space-y-2">
        <Label>Nome *</Label>
        <Input value={name} onChange={e => setName(e.target.value)} placeholder="La Bella Italia" />
      </div>
      <div className="space-y-2">
        <Label>Indirizzo</Label>
        <Input value={address} onChange={e => setAddress(e.target.value)} placeholder="Via Roma 1, Milano" /
    </div>
    <div className="grid grid-cols-2 gap-3">
      <div className="space-y-2">
        <Label>Latitudine</Label>
        <Input type="number" step="0.000001" value={latitude}
          onChange={e => setLatitude(e.target.value)} placeholder="41.902782" />
      </div>
      <div className="space-y-2">
        <Label>Longitudine</Label>
        <Input type="number" step="0.000001" value={longitude}
          onChange={e => setLongitude(e.target.value)} placeholder="12.496366" />
      </div>
    </div>

    {/* Giorno/i di chiusura ordinaria */}
    <div className="space-y-2">
      <Label>Giorno/i di chiusura settimanale</Label>
      <p className="text-xs text-muted-foreground -mt-1">
        Il giorno in cui il ristorante non apre. L'IA non genererà turni in questi giorni.
      </p>
      <div className="flex flex-wrap gap-2">
        {DAY_OPTIONS.map(d => (
          <button
            key={d}
            type="button"
            onClick={() => toggleClosingDay(d)}
            className={`px-3 py-1.5 rounded-full text-sm font-medium border transition-colors ${
              closingDays.includes(d)
                ? 'bg-primary text-primary-foreground border-primary'
                : 'bg-background border-border text-foreground hover:bg-accent'
            }`}
          >
            {WEEK_DAYS_SHORT[d]}
          </button>
        ))}
      </div>
    </div>
    <DialogFooter>
      <Button variant="outline" onClick={() => setShowForm(false)}>Annulla</Button>
      <Button onClick={handleSave} disabled={loading || !name.trim()}
        {loading ? 'Salvataggio...' : 'Salva'}
      </Button>
    </DialogFooter>
  </DialogContent>
</Dialog>

{/* — Fasce orarie (ShiftSlots) ————— */}
<Dialog open={!slotsRestaurant} onOpenChange={open => { if (!open) setSlotsRestaurant(null) }}>
  <DialogContent className="max-h-[90vh] overflow-y-auto sm:max-w-2xl">
    <DialogHeader>
      <DialogTitle>Fasce orarie – {slotsRestaurant?.name}</DialogTitle>

```

```

</DialogHeader>
<div className="space-y-5">
  <p className="text-sm text-muted-foreground">
    Le fasce orarie definiscono quante persone servono per reparto in ogni turno.
    L'IA le usa come riferimento per generare i turni automaticamente.
  </p>

  { /* Elenco slot esistenti */ }
  { slotsLoading ? (
    <p className="text-sm text-muted-foreground">Caricamento...</p>
  ) : slots.length === 0 ? (
    <p className="text-sm text-muted-foreground py-2">Nessuna fascia configurata.</p>
  ) : (
    <div className="w-full rounded-md border overflow-auto max-h-[240px]">
      <table className="border-collapse text-xs w-full">
        <thead>
          <tr className="bg-zinc-900 text-white dark:bg-zinc-800">
            <th className="px-2 py-1.5 text-left">Reparto</th>
            <th className="px-2 py-1.5 text-left">Nome</th>
            <th className="px-2 py-1.5 text-center">Orario</th>
            <th className="px-2 py-1.5 text-center">N.</th>
            <th className="px-2 py-1.5 text-left">Giorni</th>
            <th className="px-2 py-1.5"></th>
          </tr>
        </thead>
        <tbody>
          { slots.map(s => (
            <tr key={s.id} className={ `even:bg-zinc-50 dark:even:bg-zinc-900/20 ${editingSlot?.id ===
s.id ? 'ring-2 ring-inset ring-primary/30' : ''}` }>
              <td className="px-2 py-1 border border-zinc-200 dark:border-zinc-700">{s.department}</td>
              <td className="px-2 py-1 border border-zinc-200 dark:border-zinc-700">{s.name}</td>
              <td className="px-2 py-1 border border-zinc-200 dark:border-zinc-700 text-center
whitespace-nowrap tabular-nums">
                {s.start_time.slice(0, 5)}-{s.end_time.slice(0, 5)}
                {isOvernight(s.start_time, s.end_time) && (
                  <span className="ml-0.5 text-[10px] text-amber-600 dark:text-amber-400" title="Termina
il giorno successivo">(+1)</span>
                )}
              </td>
              <td className="px-2 py-1 border border-zinc-200 dark:border-zinc-700 text-
center">{s.required_count}</td>
              <td className="px-2 py-1 border border-zinc-200 dark:border-zinc-700 text-xs text-muted-
foreground">
                {s.days_of_week.length === 0
                  ? 'Tutti'
                  : [1,2,3,4,5,6,0].filter(d => s.days_of_week.includes(d)).map(d =>
WEEK_DAYS_SHORT[d]).join(', ')}
                </td>
              <td className="px-2 py-1 border border-zinc-200 dark:border-zinc-700 text-center">
                <div className="flex items-center justify-center gap-0.5">
                  <Button
                    variant="ghost" size="icon"
                    onClick={() => startEditSlot(s)}
                    className="h-6 w-6 text-muted-foreground hover:text-foreground"
                  >
                    <Pencil className="w-3 h-3" />
                  </Button>
                  <Button
                    variant="ghost" size="icon"
                    onClick={() => handleDeleteSlot(s.id)}
                    className="text-destructive hover:text-destructive h-6 w-6"
                  >
                    <Trash2 className="w-3.5 h-3.5" />
                  </Button>
                </div>
              </td>
            </tr>
          ) ) }
        </tbody>
      </table>
    </div>
  ) }
}

```

```

    /* Form aggiunta/modifica slot */
    <div className="border-t border-border pt-4 space-y-3">
      <div className="flex items-center justify-between">
        <Label className="text-sm font-semibold">
          {editingSlot ? `Modifica: ${editingSlot.name}` : 'Aggiungi fascia oraria'}
        </Label>
        <div>
          {editingSlot && (
            <Button variant="ghost" size="sm" onClick={cancelEditSlot} className="h-7 text-xs gap-1 text-
muted-foreground">
              <X className="w-3 h-3" /> Annulla
            </Button>
          )}
        </div>
      </div>
      <div className="grid grid-cols-2 gap-3">
        <div className="space-y-1.5">
          <Label className="text-xs">Reparto *</Label>
          <Select
            value={slotForm.department}
            onChange={v => setSlotForm(f => ({ ...f, department: v as Department })))}
          >
            <SelectTrigger className="h-9 text-sm">
              <SelectValue placeholder="Seleziona reparto" />
            </SelectTrigger>
            <SelectContent>
              {DEPARTMENTS.map(d => <SelectItem key={d} value={d}>{d}</SelectItem>)}
            </SelectContent>
          </Select>
        </div>
        <div className="space-y-1.5">
          <Label className="text-xs">Nome fascia *</Label>
          <Input
            value={slotForm.name}
            onChange={e => setSlotForm(f => ({ ...f, name: e.target.value })))}
            placeholder="es. Pranzo, Cena, Apertura"
            className="h-9 text-sm"
          />
        </div>
      </div>
      <div>
        <div className="grid grid-cols-3 gap-3">
          <div className="space-y-1.5">
            <Label className="text-xs">Ora inizio *</Label>
            <TimeInput value={slotForm.start_time}
              onChange={v => setSlotForm(f => ({ ...f, start_time: v })))}
              className="h-9 text-sm" />
          </div>
          <div className="space-y-1.5">
            <Label className="text-xs">Ora fine *</Label>
            <TimeInput value={slotForm.end_time}
              onChange={v => setSlotForm(f => ({ ...f, end_time: v })))}
              className="h-9 text-sm" />
          </div>
          <div className="space-y-1.5">
            <Label className="text-xs">Persone *</Label>
            <Input type="number" min={1} max={20} value={slotForm.required_count}
              onChange={e => setSlotForm(f => ({ ...f, required_count: parseInt(e.target.value) || 1 })))}
              className="h-9 text-sm" />
          </div>
        </div>
        <div>
          <div className="space-y-1.5">
            <Label className="text-xs">Giorni <span className="font-normal text-muted-foreground">(vuoto =
tutti)</span></Label>
            <div className="flex flex-wrap gap-1.5">
              {DAY_OPTIONS.map(d => (
                <button
                  key={d} type="button"
                  onClick={() => toggleSlotDay(d)}
                  className={`px-2.5 py-1 rounded-full text-xs font-medium border transition-colors ${
                    slotForm.days_of_week.includes(d)
                      ? 'bg-primary text-primary-foreground border-primary'
                      : 'bg-background border-border text-foreground hover:bg-accent'
                  }`}
                >
                  {WEEK_DAYS_SHORT[d]}
                </button>
              )}
            </div>
          </div>
        </div>
      </div>
    </div>

```

```

    ))}
  </div>
</div>
{slotForm.start_time && slotForm.end_time && isOvernight(slotForm.start_time, slotForm.end_time) &&
(
  <p className="text-xs text-amber-600 dark:text-amber-400">
    △ Turno notturno: termina il giorno successivo (+1)
  </p>
)}
{slotError && <p className="text-xs text-destructive">{slotError}</p>}
<Button
  size="sm" onClick={handleSaveSlot}
  disabled={slotSaving || !slotForm.department || !slotForm.name || !slotForm.start_time || !
slotForm.end_time}
>
  {editingSlot
    ? <><Pencil className="w-4 h-4" /> {slotSaving ? 'Salvataggio...' : 'Salva modifiche'}</>
    : <><Plus className="w-4 h-4" /> {slotSaving ? 'Salvataggio...' : 'Aggiungi fascia'}</>
  }
</Button>
</div>
</div>
<DialogFooter>
  <Button variant="outline" onClick={() => setSlotsRestaurant(null)}>Chiudi</Button>
</DialogFooter>
</DialogContent>
</Dialog>
</div>
)
}

```

`src/components/manager/TelegramLinkButton.tsx`


```

'use client'
import { useEffect, useState } from 'react'
import { Button } from '@components/ui/button'
import {
  Dialog, DialogContent, DialogHeader, DialogTitle, DialogDescription, DialogFooter,
} from '@components/ui/dialog'
import { Send, CheckCircle2, Unlink } from 'lucide-react'
import { generateTelegramLink, getTelegramLinkStatus, unlinkTelegram } from '@app/actions/telegram'

export function TelegramLinkButton() {
  const [loading, setLoading] = useState(true)
  const [linked, setLinked] = useState(false)
  const [username, setUsername] = useState<string | null>(null)
  const [busy, setBusy] = useState(false)
  const [error, setError] = useState<string | null>(null)
  const [link, setLink] = useState<{ deepLink: string; pin: string; botUsername: string | null } | null>(null)

  useEffect(() => {
    getTelegramLinkStatus()
      .then(({ linked, telegramUsername }) => {
        setLinked(linked)
        setUsername(telegramUsername)
      })
      .catch(() => {})
      .finally(() => setLoading(false))
  }, [])

  async function handleConnect() {
    setBusy(true)
    setError(null)
    try {
      const { deepLink, pin, username: botUsername } = await generateTelegramLink()
      if (deepLink) setLink({ deepLink, pin, botUsername })
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Errore durante la generazione del collegamento')
    } finally {
      setBusy(false)
    }
  }

  async function handleUnlink() {
    setBusy(true)
    setError(null)
    try {
      await unlinkTelegram()
      setLinked(false)
      setUsername(null)
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Errore durante lo scollegamento')
    } finally {
      setBusy(false)
    }
  }

  if (loading) return null

  return (
    <>
      <div className="mt-6 bg-card border border-border rounded-md p-4 flex items-center gap-4">
        <div className="w-10 h-10 rounded-sm border border-border bg-muted flex items-center justify-center shrink-0">
          <Send className="w-5 h-5 text-sky-500" />
        </div>
        <div className="flex-1 min-w-0">
          <h2 className="text-sm font-semibold flex items-center gap-2">
            Bot Telegram
            {linked && <CheckCircle2 className="w-4 h-4 text-emerald-600" />}
          </h2>
          <p className="text-xs text-muted-foreground mt-0.5">
            {linked
              ? `Collegato a ${username} ? ` come @${username}` : ''}. Gestisci turni, ODS e presenze in chat.`
              : 'Collega il tuo account per gestire turni, ODS e presenze direttamente da Telegram.'}
          </p>
        </div>
      </div>
    </>
  )
}

```

```

      {error && <p className="text-xs text-red-600 mt-0.5">{error}</p>}
    </div>
    {linked ? (
      <Button size="sm" variant="outline" onClick={handleUnlink} disabled={busy}>
        <Unlink className="w-4 h-4" /> Scollega
      </Button>
    ) : (
      <Button size="sm" onClick={handleConnect} disabled={busy}>
        <Send className="w-4 h-4" /> Collega Telegram
      </Button>
    )}
  </div>

  <Dialog open={!link} onOpenChange={({open}) => { if (!open) setLink(null) }}>
    <DialogContent>
      <DialogHeader>
        <DialogTitle>Collega Telegram</DialogTitle>
        <DialogDescription>
          Apri la chat con il bot e invia il comando di avvio. Il codice è valido per 5 minuti.
        </DialogDescription>
      </DialogHeader>

      {link && (
        <div className="space-y-4">
          <a
            href={link.deepLink}
            target="_blank"
            rel="noopener noreferrer"
            className="inline-flex w-full items-center justify-center gap-2 rounded-md bg-primary text-
primary-foreground text-sm font-medium h-9 px-4"
          >
            <Send className="w-4 h-4" /> Apri {link.botUsername ? `@${link.botUsername}` : 'il bot'} su
Telegram
          </a>

          <div className="text-xs text-muted-foreground space-y-1">
            <p>Se il link non si apre automaticamente, apri tu stesso il bot su Telegram e invia:</p>
            <p className="font-mono text-sm bg-muted rounded-sm px-2 py-1.5 text-center text-foreground
tracking-wide">
              /start {link.pin}
            </p>
          </div>
        </div>
      )}

      <DialogFooter>
        <Button variant="outline" onClick={() => setLink(null)}>Chiudi</Button>
      </DialogFooter>
    </DialogContent>
  </Dialog>
</>
)
}

```

`src/components/manager/TurniManagerClient.tsx`

```

'use client'
import { useState, useEffect } from 'react'
import { createClient } from '@lib/supabase/client'
import { useAccountStatus } from '@contexts/AccountStatusContext'
import {
  createTurn, updateTurn, deleteTurn, createTurnsBulk,
  upsertStandardShift, deleteStandardShift, populateFromStandard,
  type TurnInput, type BulkTurnInput,
} from '@app/actions/turni'

import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { TimeInput } from '@components/ui/time-input'
import { Label } from '@components/ui/label'
import { Textarea } from '@components/ui/textarea'
import { Switch } from '@components/ui/switch'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from '@components/ui/dialog'
import { Plus, Trash2, ChevronLeft, ChevronRight, CalendarRange, X, Sparkles } from 'lucide-react'
import { formatInTimeZone } from 'date-fns-tz'
import { startOfWeek, addDays, addWeeks, format, parseISO } from 'date-fns'
import { it } from 'date-fns/locale'
import type { Turn, Department, StandardShift, AiScheduleDraft, AiScheduleDraftTurn } from '@types'
import { AiScheduleDialog } from './AiScheduleDialog'
import { AiScheduleDraftView } from './AiScheduleDraftView'
import { TurniTimeline } from './TurniTimeline'
import { EXTRAORDINARY_BADGE, STANDARD_BADGE, RIPOSO_BADGE } from '@lib/turnColors'

const TZ = 'Europe/Rome'

type StaffMember = { id: string; full_name: string; department: string | null; restaurant_id: string | null }
type RestaurantItem = { id: string; name: string }
type MultiDayEntry = {
  date: string
  is_rest_day: boolean
  start_time: string
  end_time: string
  is_extraordinary: boolean
  notes: string
  splitSegments: { start: string; end: string }[]
}

interface Props {
  initialTurns: Turn[]
  initialStandardShifts: StandardShift[]
  staff: StaffMember[]
  restaurants: RestaurantItem[]
  currentUserId: string
  currentUserRole: string
  currentDepartment: string | null
  currentRestaurantId: string | null
  currentIsDirettore: boolean
}

function timeToMinutes(t: string): number {
  const [h, m] = t.split(':').map(Number)
  return h * 60 + m
}

// Durata di una fascia oraria in minuti; gestisce il turno notturno
// (fine ≤ inizio → si estende al giorno dopo).
function segmentMinutes(start: string, end: string): number {
  const s = timeToMinutes(start)
  let e = timeToMinutes(end)
  if (e <= s) e += 24 * 60
  return e - s
}

function formatMinutes(total: number): string {
  const h = Math.floor(total / 60)
  const m = total % 60
  return m > 0 ? `${h}h ${m}m` : `${h}h`
}

```

```

// Una fascia spezzata è incompleta se ha un solo orario compilato tra i due.
function hasIncompleteSplit(segments: { start: string; end: string }[]): boolean {
  return segments.some(s => (s.start && !s.end) || (!s.start && s.end))
}

// Intestazione e colonna dipendente condividono lo stesso trattamento
// neutro (bg-card + bordo) della Timeline Giornaliera sotto, invece di un
// blocco colorato a sé stante – stesso linguaggio visivo in tutta la pagina.
const thCls = 'px-2 py-1.5 text-center font-semibold bg-card text-foreground border-b border-border whitespace-nowrap'
const tdCls = 'px-1.5 py-1 text-center text-xs border border-zinc-200 dark:border-zinc-700 whitespace-nowrap tabular-nums'
const tdNameStaticCls = 'px-2 py-1 text-left text-xs font-medium border border-zinc-200 dark:border-zinc-700 whitespace-nowrap bg-card'
const tdNameCls = 'px-2 py-1 text-left text-xs font-medium border border-zinc-200 dark:border-zinc-700 whitespace-nowrap sticky left-0 bg-card z-10'

const TURN_TYPE_OPTIONS = [
  { value: false, label: 'Turno di Lavoro' },
  { value: true, label: 'Riposo' },
]

// date-fns getDay(): 0=Dom .. 6=Sab
const WEEK_DAY_OPTIONS = [
  { value: 1, label: 'Lun' },
  { value: 2, label: 'Mar' },
  { value: 3, label: 'Mer' },
  { value: 4, label: 'Gio' },
  { value: 5, label: 'Ven' },
  { value: 6, label: 'Sab' },
  { value: 0, label: 'Dom' },
]

export function TurniManagerClient({
  initialTurns, initialStandardShifts, staff, restaurants,
  currentUserRole, currentDepartment, currentRestaurantId,
  currentIsDirettore,
}: Props) {
  const { isPending } = useAccountStatus()
  const isManager = currentUserRole === 'manager'
  const isDirettore = currentUserRole === 'capo_servizio' && currentIsDirettore

  const [turns, setTurns] = useState<Turn[]>(initialTurns)
  const [weekOffset, setWeekOffset] = useState(0)
  const [restFilter, setRestFilter] = useState<string>('tutti')
  // Multi-selezione: array vuoto = tutti i reparti
  const [deptFilter, setDeptFilter] = useState<string[]>([])

  const [showForm, setShowForm] = useState(false)
  const [saving, setSaving] = useState(false)
  const [error, setError] = useState<string | null>(null)
  const [editingTurn, setEditingTurn] = useState<Turn | null>(null)

  // Form state
  const [fUserId, setFUserId] = useState('')
  const [fRestaurantId, setFRestaurantId] = useState(currentRestaurantId ?? '')
  const [fDate, setFDate] = useState('')
  const [fEndDate, setFEndDate] = useState('')
  const [fDaysOfWeek, setFDaysOfWeek] = useState<number[]>([])
  const [fStart, setFStart] = useState('')
  const [fEnd, setFEnd] = useState('')
  const [fExtraordinary, setFExtraordinary] = useState(false)
  const [fIsRestDay, setFIsRestDay] = useState(false)
  const [fNotes, setFNotes] = useState('')
  const [bulkMode, setBulkMode] = useState(false)
  // Fasce aggiuntive dello stesso giorno (turno spezzato) – create insieme
  // alla fascia principale con un solo Salva, senza riaprire il form.
  const [fSplitSegments, setFSplitSegments] = useState<{ start: string; end: string }[]>([])
  // Multi-giorno: fDate è il giorno attualmente "in composizione"; le frecce
  // spostano avanti/indietro salvando il giorno corrente in questa coda,
  // finché non si preme Salva che inserisce tutto in blocco.
  const [multiDayMode, setMultiDayMode] = useState(false)
  const [multiDayQueue, setMultiDayQueue] = useState<MultiDayEntry[]>([])

```

```

// Turni Standard (Pattern Master)
const [standardShifts, setStandardShifts] = useState<StandardShift[]>(initialStandardShifts)
const [showStandardModal, setShowStandardModal] = useState(false)
const [sUserId, setSUserId] = useState('')
const [sDaysOfWeek, setSDaysOfWeek] = useState<number[]>([])
const [sStart, setSStart] = useState('')
const [sEnd, setSEnd] = useState('')
const [sSaving, setSSaving] = useState(false)
const [sError, setSError] = useState<string | null>(null)

// AI Schedule
const [showAiDialog, setShowAiDialog] = useState(false)
const [aiDraft, setAiDraft] = useState<AiScheduleDraft & { turns: AiScheduleDraftTurn[] }> | null>(null)

// — Realtime — REGOLA D'ORO: aggiornamento istantaneo via supabase_realtime —
useEffect(() => {
  const supabase = createClient()
  const filter = !isManager && currentRestaurantId
    ? `restaurant_id=eq.${currentRestaurantId}`
    : undefined

  const channel = supabase
    .channel('rt-turni')
    .on(
      'postgres_changes',
      { event: '*', schema: 'public', table: 'turns', ...(filter ? { filter } : {}) },
      async (payload) => {
        // Capo Servizio (non direttore): il filtro realtime copre solo il
        // ristorante — il reparto va verificato lato client.
        const inDeptScope = (row: { department: string | null }) =>
          isManager || isDirettore || !currentDepartment || row.department === currentDepartment

        if (payload.eventType === 'INSERT' || payload.eventType === 'UPDATE') {
          const rec = payload.new as Turn
          if (!inDeptScope(rec)) {
            setTurns(prev => prev.filter(t => t.id !== rec.id))
            return
          }
          const { data } = await supabase
            .from('turns')
            .select('*', profile:profiles!user_id(id, full_name), restaurant:restaurants(id, name))
            .eq('id', rec.id)
            .single()
          if (!data) return
          setTurns(prev => {
            const exists = prev.some(t => t.id === data.id)
            return exists
              ? prev.map(t => t.id === data.id ? data as unknown as Turn : t)
              : [...prev, data as unknown as Turn]
          })
        } else if (payload.eventType === 'DELETE') {
          const deletedId = (payload.old as { id: string }).id
          setTurns(prev => prev.filter(t => t.id !== deletedId))
        }
      }
    )
    .subscribe()

  return () => { supabase.removeChannel(channel) }
}, [isManager, isDirettore, currentDepartment, currentRestaurantId])

// — Week navigation —
const weekStart = startOfWeek(addWeeks(new Date(), weekOffset), { weekStartsOn: 1 })
const weekDays = Array.from({ length: 7 }, (_, i) => addDays(weekStart, i))
const weekStartStr = format(weekStart, 'yyyy-MM-dd')
const weekEndStr = format(addDays(weekStart, 6), 'yyyy-MM-dd')

const turnsByRestaurant = (isManager && restFilter !== 'tutti')
  ? turns.filter(t => t.restaurant_id === restFilter)
  : turns

const weekTurns = turnsByRestaurant.filter(t => t.date >= weekStartStr && t.date <= weekEndStr)

```

```

// — Griglia stile Excel: dipendenti × giorni della settimana —
const turnsByUserDate: Record<string, Turn[]> = {}
weekTurns.forEach(t => {
  const key = `${t.user_id}${t.date}`
  if (!turnsByUserDate[key]) turnsByUserDate[key] = []
  turnsByUserDate[key].push(t)
})
Object.values(turnsByUserDate).forEach(list => list.sort((a, b) => a.start_time.localeCompare(b.start_time)))

const staffByRestaurant = (isManager && restFilter !== 'tutti')
  ? staff.filter(s => s.restaurant_id === restFilter)
  : staff

// Reparti disponibili nel ristorante selezionato (o in tutti, se nessun
// filtro ristorante è attivo) – pannello nascosto se c'è un solo reparto.
const allDepts = Array.from(
  new Set(staffByRestaurant.map(s => s.department).filter((d): d is string => !!d))
).sort()

const gridStaff = deptFilter.length === 0
  ? staffByRestaurant
  : staffByRestaurant.filter(s => s.department && deptFilter.includes(s.department))

function toggleDeptFilter(dept: string) {
  setDeptFilter(prev => prev.includes(dept) ? prev.filter(d => d !== dept) : [...prev, dept])
}

// — Employee dropdown scoping —
// Direttore/Capo Servizio: `staff` arriva già filtrato dal server
// (scopeStaffQuery) per restaurant_id (+ department per i non-direttori).
// Manager: filtra per il ristorante selezionato nel form.
const scopedStaff = isManager
  ? (fRestaurantId ? staff.filter(s => s.restaurant_id === fRestaurantId) : staff)
  : staff

function resetForm() {
  setEditingTurn(null)
  setFUserId('')
  setFRestaurantId(currentRestaurantId ?? '')
  setFDate('')
  setFEndDate('')
  setFDaysOfWeek([])
  setFStart('')
  setFEnd('')
  setFExtraordinary(false)
  setFIsRestDay(false)
  setFNotes('')
  setBulkMode(false)
  setFSplitSegments([])
  setMultiDayMode(false)
  setMultiDayQueue([])
  setError(null)
}

function addSplitSegment() {
  setFSplitSegments(prev => [...prev, { start: '', end: '' }])
}

function updateSplitSegment(idx: number, patch: Partial<{ start: string; end: string }>) {
  setFSplitSegments(prev => prev.map((s, i) => i === idx ? { ...s, ...patch } : s))
}

function removeSplitSegment(idx: number) {
  setFSplitSegments(prev => prev.filter((_, i) => i !== idx))
}

// — Turni Multi-giorno —
// fDate rappresenta il giorno "in composizione"; le frecce salvano il
// giorno corrente nella coda e caricano il giorno successivo/precedente
// (già presente in coda, se già compilato).
function toggleMultiDayMode(v: boolean) {
  setMultiDayMode(v)
  setMultiDayQueue([])
  if (v) {
    setBulkMode(false)
  }
}

```

```

    setFDate(prev => prev || formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd'))
    setFIsRestDay(false)
    setFStart('')
    setFEnd('')
    setFExtraordinary(false)
    setFNotes('')
    setFSplitSegments([])
  }
}

function loadDayIntoComposer(dateStr: string, queue: MultiDayEntry[]) {
  const existing = queue.find(e => e.date === dateStr)
  setFDate(dateStr)
  if (existing) {
    setFIsRestDay(existing.is_rest_day)
    setFStart(existing.is_rest_day ? '' : existing.start_time.slice(0, 5))
    setFEnd(existing.is_rest_day ? '' : existing.end_time.slice(0, 5))
    setFExtraordinary(existing.is_extraordinary)
    setFNotes(existing.notes)
    setFSplitSegments(existing.splitSegments)
  } else {
    setFIsRestDay(false)
    setFStart('')
    setFEnd('')
    setFExtraordinary(false)
    setFNotes('')
    setFSplitSegments([])
  }
}

function goToDay(deltaDays: number) {
  if (!fDate) return
  if (!fIsRestDay && hasIncompleteSplit(fSplitSegments)) {
    setError('Completa tutte le fasce del turno spezzato di questo giorno prima di continuare, oppure rimuovile.')
    return
  }
  setError(null)
  const nextQueue = commitCurrentDayIfValid(multiDayQueue)
  setMultiDayQueue(nextQueue)
  const newDate = format(addDays(parseISO(`${fDate}T00:00:00`), deltaDays), 'yyyy-MM-dd')
  loadDayIntoComposer(newDate, nextQueue)
}

// Ritorna la coda aggiornata includendo il giorno attualmente in
// composizione, se compilato (riposo, oppure orario completo).
function commitCurrentDayIfValid(queue: MultiDayEntry[]): MultiDayEntry[] {
  if (!fDate || (!fIsRestDay && (!fStart || !fEnd))) return queue
  const entry: MultiDayEntry = {
    date: fDate,
    is_rest_day: fIsRestDay,
    start_time: fIsRestDay ? '00:00' : fStart,
    end_time: fIsRestDay ? '00:00' : fEnd,
    is_extraordinary: fIsRestDay ? false : fExtraordinary,
    notes: fNotes.trim(),
    splitSegments: fIsRestDay ? [] : fSplitSegments.filter(s => s.start && s.end),
  }
  const idx = queue.findIndex(e => e.date === fDate)
  const next = idx >= 0
    ? queue.map((e, i) => i === idx ? entry : e)
    : [...queue, entry]
  return next.sort((a, b) => a.date.localeCompare(b.date))
}

function removeMultiDayEntry(dateStr: string) {
  setMultiDayQueue(prev => prev.filter(e => e.date !== dateStr))
}

function openCreate() {
  resetForm()
  setShowForm(true)
}

// Creazione rapida da cella della griglia: precompila dipendente + data

```



```

function openCreateForCell(userId: string, dateStr: string) {
  resetForm()
  const member = staff.find(s => s.id === userId)
  setFUserId(userId)
  setFDate(dateStr)
  if (isManager && member?.restaurant_id) setFRestaurantId(member.restaurant_id)
  setShowForm(true)
}

function openEdit(turn: Turn) {
  setEditingTurn(turn)
  setFUserId(turn.user_id)
  setFRestaurantId(turn.restaurant_id)
  setFDate(turn.date)
  setFStart(turn.start_time.slice(0, 5))
  setFEnd(turn.end_time.slice(0, 5))
  setFExtraordinary(turn.is_extraordinary)
  setFIsRestDay(turn.is_rest_day)
  setFNotes(turn.notes ?? '')
  setBulkMode(false)
  setFSplitSegments([])
  setMultiDayMode(false)
  setMultiDayQueue([])
  setError(null)
  setShowForm(true)
}

function toggleFDayOfWeek(day: number) {
  setFDaysOfWeek(prev => prev.includes(day) ? prev.filter(d => d !== day) : [...prev, day])
}

async function handleSave() {
  if (!fUserId) return

  if (multiDayMode) {
    if (!fIsRestDay && hasIncompleteSplit(fSplitSegments)) {
      setError('Completa tutte le fasce del turno spezzato di questo giorno prima di salvare, oppure rimuovile.')
      return
    }
    const finalQueue = commitCurrentDayIfValid(multiDayQueue)
    if (finalQueue.length === 0) {
      setError('Compila almeno un giorno prima di salvare.')
      return
    }
  }
  setSaving(true)
  setError(null)
  try {
    const selected = staff.find(s => s.id === fUserId)
    const restaurant_id = selected?.restaurant_id ?? fRestaurantId ?? currentRestaurantId ?? ''
    const department = (selected?.department ?? currentDepartment) as Department | null
    const creates: Promise<Turn>[] = []
    for (const e of finalQueue) {
      const shared = {
        user_id: fUserId,
        restaurant_id,
        department,
        date: e.date,
        is_rest_day: e.is_rest_day,
        notes: e.notes || null,
      }
      creates.push(createTurn({ ...shared, start_time: e.start_time, end_time: e.end_time, is_extraordinary:
e.is_extraordinary }))
      for (const seg of e.splitSegments) {
        creates.push(createTurn({ ...shared, start_time: seg.start, end_time: seg.end, is_extraordinary:
e.is_extraordinary }))
      }
    }
    const created = await Promise.all(creates)
    setTurns(prev => [...prev, ...created])
    resetForm()
    setShowForm(false)
  } catch (err) {
    setError(err instanceof Error ? err.message : 'Errore sconosciuto')
  }
}

```

```

    } finally {
      setSaving(false)
    }
    return
  }

  if (!fIsRestDay && (!fStart || !fEnd)) return
  if (bulkMode ? (!fDate || !fEndDate || fDaysOfWeek.length === 0) : !fDate) return

  // Turno spezzato: ogni fascia aggiuntiva deve avere entrambi gli orari.
  const canHaveSplit = !fIsRestDay && !bulkMode && !editingTurn
  if (canHaveSplit && hasIncompleteSplit(fSplitSegments)) {
    setError('Completa tutte le fasce del turno spezzato prima di salvare, oppure rimuovile.')
    return
  }

  setSaving(true)
  setError(null)
  try {
    const selected = staff.find(s => s.id === fUserId)
    const baseFields = {
      user_id: fUserId,
      restaurant_id: selected?.restaurant_id ?? fRestaurantId ?? currentRestaurantId ?? '',
      department: (selected?.department ?? currentDepartment) as Department | null,
      start_time: fIsRestDay ? '00:00' : fStart,
      end_time: fIsRestDay ? '00:00' : fEnd,
      is_extraordinary: fIsRestDay ? false : fExtraordinary,
      is_rest_day: fIsRestDay,
      notes: fNotes.trim() || null,
    }

    if (bulkMode) {
      const payload: BulkTurnInput = {
        ...baseFields,
        start_date: fDate,
        end_date: fEndDate,
        days_of_week: fDaysOfWeek,
      }
      const created = await createTurnsBulk(payload)
      setTurns(prev => [...prev, ...created])
    } else {
      const payload: TurnInput = { ...baseFields, date: fDate }
      if (editingTurn) {
        const updated = await updateTurn(editingTurn.id, payload)
        setTurns(prev => prev.map(t => t.id === updated.id ? updated : t))
      } else {
        // Fascia principale + eventuali fasce spezzate, create insieme
        // in un solo Salva (stesso dipendente/data/ristorante/reparto).
        const validSegments = fSplitSegments.filter(s => s.start && s.end)
        const created = await Promise.all([
          createTurn(payload),
          ...validSegments.map(s => createTurn({ ...baseFields, date: fDate, start_time: s.start, end_time:
s.end })),
        ])
        setTurns(prev => [...prev, ...created])
      }
    }
    resetForm()
    setShowForm(false)
  } catch (err) {
    setError(err instanceof Error ? err.message : 'Errore sconosciuto')
  } finally {
    setSaving(false)
  }
}

async function handleDelete(id: string) {
  if (!confirm('Eliminare questo turno?')) return
  try {
    await deleteTurn(id)
    setTurns(prev => prev.filter(t => t.id !== id))
  } catch (err) {
    console.error('Errore eliminazione turno:', err)
  }
}

```

```

}

// — Turni Standard (Pattern Master) —————
function resetStandardForm() {
  setUserId('')
  setSDaysOfWeek([])
  setSStart('')
  setSEnd('')
  setSError(null)
}

function toggleSDayOfWeek(day: number) {
  setSDaysOfWeek(prev => prev.includes(day) ? prev.filter(d => d !== day) : [...prev, day])
}

async function handleAddStandardShift() {
  if (!sUserId || !sStart || !sEnd || sDaysOfWeek.length === 0) return
  setSSaving(true)
  setSError(null)
  try {
    const selected = staff.find(s => s.id === sUserId)
    const created: StandardShift[] = []
    for (const day of sDaysOfWeek) {
      const shift = await upsertStandardShift({
        user_id:      sUserId,
        restaurant_id: selected?.restaurant_id ?? currentRestaurantId ?? '',
        department:   (selected?.department ?? currentDepartment) as Department | null,
        day_of_week:   day,
        start_time:    sStart,
        end_time:      sEnd,
      })
      created.push(shift as unknown as StandardShift)
    }
    setStandardShifts(prev =>
      [...prev, ...created].sort((a, b) => a.day_of_week - b.day_of_week)
    )
    // Genera subito i turni reali per la settimana visualizzata, così il
    // turno fisso compare anche in tabella (il Realtime aggiorna `turns`)
    await populateFromStandard(weekStartStr, weekEndStr)
    resetStandardForm()
  } catch (err) {
    setSError(err instanceof Error ? err.message : 'Errore sconosciuto')
  } finally {
    setSSaving(false)
  }
}

async function handleDeleteStandardShift(id: string) {
  if (!confirm('Eliminare questo turno fisso?')) return
  try {
    await deleteStandardShift(id)
    setStandardShifts(prev => prev.filter(s => s.id !== id))
  } catch (err) {
    console.error('Errore eliminazione turno fisso:', err)
  }
}

// Turno spezzato disponibile in creazione (singola o multi-giorno), non in bulk/modifica
const canHaveSplit = !fIsRestDay && !bulkMode && !editingTurn
const validSplitSegments = fSplitSegments.filter(s => s.start && s.end)
const totalShiftMinutes = fStart && fEnd
  ? segmentMinutes(fStart, fEnd) + validSplitSegments.reduce((sum, s) => sum + segmentMinutes(s.start, s.end),
0)
  : 0
const hasIncompleteSplitSegment = canHaveSplit && hasIncompleteSplit(fSplitSegments)

// Multi-giorno: quanti giorni verrebbero salvati se si premesse Salva ora
// (coda + giorno corrente, se compilato e non già in coda).
const multiDayCurrentIsQueued = multiDayMode && fDate && multiDayQueue.some(e => e.date === fDate)
const multiDayCurrentIsValid = multiDayMode && !!fDate && (fIsRestDay || (!!fStart && !!fEnd))
const multiDayCount = multiDayMode
  ? multiDayQueue.length + (multiDayCurrentIsValid && !multiDayCurrentIsQueued ? 1 : 0)
  : 0

```

```

return (
  <div>
    /* Header – i tasti vanno su una riga propria sotto il titolo su
       mobile, senza mai sfiorare la larghezza dello schermo */
    <div className="mb-5">
      <h1 className="text-xl font-semibold tracking-tight mb-3">Gestione Turni</h1>
      <div className="grid grid-cols-2 sm:flex sm:items-center gap-2">
        <Button size="sm" variant="outline" onClick={() => setShowStandardModal(true)} disabled={isPending}
title={isPending ? 'Disponibile dopo l\'attivazione' : undefined}>
          <CalendarRange className="w-4 h-4" /> Turni Fissi
        </Button>
        <Button
          size="sm" variant="outline"
          onClick={() => setShowAiDialog(true)}
          disabled={isPending || (isManager && restFilter === 'tutti')}
          title={isPending ? 'Disponibile dopo l\'attivazione' : isManager && restFilter === 'tutti' ?
'Seleziona prima un ristorante dal filtro' : undefined}>
          >
            <Sparkles className="w-4 h-4" /> Genera con IA
          </Button>
          <Button size="sm" onClick={openCreate} className="col-span-2 sm:col-span-1" disabled={isPending}
title={isPending ? 'Disponibile dopo l\'attivazione' : undefined}>
            <Plus className="w-4 h-4" /> Nuovo Turno
          </Button>
        </div>
      </div>
    </div>

    /* Restaurant filter – manager only */
    {isManager && restaurants.length > 0 && (
      <div className="mb-4">
        <Select value={restFilter} onChange={v => { setRestFilter(v); setDeptFilter([]) }}>
          <SelectTrigger className="h-8 w-56 text-xs rounded-sm">
            <SelectValue placeholder="Tutti i ristoranti" />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="tutti">Tutti i ristoranti</SelectItem>
            {restaurants.map(r => (
              <SelectItem key={r.id} value={r.id}>{r.name}</SelectItem>
            ))}
          </SelectContent>
        </Select>
      </div>
    )}

    /* Department filter – multi-selezione, mostrato solo se c'è più di un reparto */
    {allDepts.length > 1 && (
      <div className="mb-4 flex flex-wrap gap-1.5">
        <button
          onClick={() => setDeptFilter([])}
          className={`text-xs px-2.5 py-1 rounded-sm border transition-colors ${
            deptFilter.length === 0
              ? 'bg-foreground text-background border-foreground'
              : 'bg-card text-muted-foreground border-border hover:bg-accent hover:text-foreground'
            }`}
          >
          Tutti i reparti
        </button>
        {allDepts.map(dept => (
          <button
            key={dept}
            onClick={() => toggleDeptFilter(dept)}
            className={`text-xs px-2.5 py-1 rounded-sm border transition-colors capitalize ${
              deptFilter.includes(dept)
                ? 'bg-foreground text-background border-foreground'
                : 'bg-card text-muted-foreground border-border hover:bg-accent hover:text-foreground'
            }`}
            >
            {dept}
          </button>
        ))}
      </div>
    )}

    /* Week navigation – frecce simmetriche ai lati, data centrata su una riga */

```

```

<div className="flex items-center justify-between gap-3 mb-5">
  <Button variant="outline" size="icon" className="h-8 w-8 shrink-0" onClick={() => setWeekOffset(w => w -
1)} aria-label="Settimana precedente">
    <ChevronLeft className="w-4 h-4" />
  </Button>
  <div className="text-sm font-medium text-foreground text-center whitespace-nowrap tabular-nums">
    {formatInTimeZone(weekStart, TZ, "d MMM", { locale: it })} - {formatInTimeZone(addDays(weekStart, 6),
TZ, "d MMM yyyy", { locale: it })}
  </div>
  <Button variant="outline" size="icon" className="h-8 w-8 shrink-0" onClick={() => setWeekOffset(w => w +
1)} aria-label="Settimana successiva">
    <ChevronRight className="w-4 h-4" />
  </Button>
</div>
{weekOffset !== 0 && (
  <div className="text-center -mt-3 mb-5">
    <button onClick={() => setWeekOffset(0)} className="text-xs text-muted-foreground hover:text-foreground
underline">
      torna a oggi
    </button>
  </div>
)}

/* Griglia Turni – stile Excel: dipendenti × giorni della settimana.
Ogni cella mostra i turni (o riposi) del dipendente in quel giorno;
click su un turno per modificarlo, "+" per aggiungerne uno nuovo. */
<div className="w-full rounded-md border bg-card overflow-auto">
  <table className="border-collapse text-xs w-full" style={{ minWidth: 'max-content' }}>
    <thead>
      <tr>
        <th className={` ${thCls} sticky left-0 z-20 text-left min-w-[150px]`} >Dipendente</th>
        {weekDays.map(day => (
          <th key={format(day, 'yyyy-MM-dd')} className={` ${thCls} min-w-[100px]`} >
            <div className="capitalize">{formatInTimeZone(day, TZ, 'EEE', { locale: it })}</div>
            <div className="font-normal opacity-80">{formatInTimeZone(day, TZ, 'd/MM', { locale: it })}</div>
          </th>
        ))}
        <th className={` ${thCls} min-w-[90px]`} >Monte Ore</th>
      </tr>
    </thead>
    <tbody>
      {gridStaff.length === 0 ? (
        <tr>
          <td colspan={9} className="text-center py-6 text-muted-foreground text-xs">
            Nessun dipendente
          </td>
        </tr>
      ) : gridStaff.map(member => {
        // Monte ore settimanale stimato: somma di tutte le fasce non-riposo
        // della settimana visualizzata, turni spezzati inclusi (ogni fascia
        // dello stesso giorno è una voce a parte in cellTurns), gestendo
        // anche i turni notturni (fine ≤ inizio – si estendono a mezzanotte).
        const weeklyMinutes = weekDays.reduce((sum, day) => {
          const dateStr = format(day, 'yyyy-MM-dd')
          const cellTurns = turnsByUserData[`${member.id}|${dateStr}`] ?? []
          return sum + cellTurns
            .filter(t => !t.is_rest_day)
            .reduce((s, t) => s + segmentMinutes(t.start_time.slice(0, 5), t.end_time.slice(0, 5)), 0)
        }, 0)

        return (
          <tr key={member.id} className="even:bg-zinc-50 dark:even:bg-zinc-900/20">
            <td className={tdNameCls}>
              {member.full_name}
              {isManager && restFilter === 'tutti' && member.restaurant_id && (
                <span className="block text-[10px] font-normal text-muted-foreground">
                  {restaurants.find(r => r.id === member.restaurant_id)?.name ?? ''}
                </span>
              )}
            </td>
            {weekDays.map(day => {
              const dateStr = format(day, 'yyyy-MM-dd')
              const cellTurns = turnsByUserData[`${member.id}|${dateStr}`] ?? []

```

```

        return (
          <td key={dateStr} className={` ${tdCls} align-top p-1`} >
            <div className="flex flex-col gap-1 items-stretch min-w-[80px]" >
              {cellTurns.map(turn => (
                <div
                  key={turn.id}
                  onClick={() => openEdit(turn)}
                  className={`flex items-center justify-center gap-1 rounded-sm border px-1 py-0.5 text-[10px] font-medium whitespace-nowrap cursor-pointer ${
                    turn.is_rest_day ? RIPOSO_BADGE : turn.is_extraordinary ? EXTRAORDINARY_BADGE : STANDARD_BADGE
                  }`}
                >
                  <span>
                    {turn.is_rest_day ? 'Riposo' : `${turn.start_time.slice(0, 5)}-${turn.end_time.slice(0, 5)}`}
                  </span>
                  <button
                    type="button"
                    onClick={e => { e.stopPropagation(); handleDelete(turn.id) }}
                    className="opacity-50 hover:opacity-100 hover:text-destructive"
                    aria-label="Elimina turno"
                  >
                    <X className="w-2.5 h-2.5" />
                  </button>
                </div>
              ))}
              <button
                type="button"
                onClick={() => openCreateForCell(member.id, dateStr)}
                className="flex items-center justify-center h-5 rounded-sm border border-dashed border-border text-muted-foreground hover:text-foreground hover:bg-accent transition-colors"
                aria-label="Aggiungi turno"
              >
                <Plus className="w-3 h-3" />
              </button>
            </div>
          </td>
        )
      )}
    <td className={` ${tdCls} font-semibold`} >
      {weeklyMinutes > 0 ? formatMinutes(weeklyMinutes) : '-'}
    </td>
  </tr>
)
}}
</tbody>
</table>
</div>

{/* Legenda */}
<div className="flex flex-wrap items-center gap-3 mt-3 text-[11px] text-muted-foreground">
  <span className="flex items-center gap-1.5">
    <span className={`inline-block w-3 h-3 rounded-sm border ${STANDARD_BADGE}`} /> Turno standard
  </span>
  <span className="flex items-center gap-1.5">
    <span className={`inline-block w-3 h-3 rounded-sm border ${EXTRAORDINARY_BADGE}`} /> Straordinario
  </span>
  <span className="flex items-center gap-1.5">
    <span className={`inline-block w-3 h-3 rounded-sm border ${RIPOSO_BADGE}`} /> Riposo
  </span>
</div>

{/* Timeline giornaliera – stessa lista dipendenti della griglia (rispetta i filtri ristorante/reparto), navigazione giorno per giorno */}
<TurniTimeline staff={gridStaff} turns={turnsByRestaurant} onEditTurn={openEdit} />

{/* — Create / edit modal ————— */}
<Dialog open={showForm} onOpenChange={setShowForm}>
  <DialogContent className="max-h-[90vh] overflow-y-auto">
    <DialogHeader>
      <DialogTitle>
        {editingTurn ? 'Modifica Turno' : bulkMode ? 'Inserimento Multiplo' : multiDayMode ? 'Turni su Più Giorni' : 'Nuovo Turno'}
      </DialogTitle>
    </DialogHeader>
  </DialogContent>
</Dialog>

```

```

</DialogTitle>
</DialogHeader>
<div className="space-y-4">
  /* Restaurant – manager only */
  {isManager && restaurants.length > 0 && (
    <div className="space-y-2">
      <Label>Ristorante *</Label>
      <Select value={fRestaurantId} onValueChange={v => { setFRestaurantId(v); setFUserId('') }}>
        <SelectTrigger><SelectValue placeholder="Seleziona ristorante" /></SelectTrigger>
        <SelectContent>
          {restaurants.map(r => <SelectItem key={r.id} value={r.id}>{r.name}</SelectItem>)}
        </SelectContent>
      </Select>
    </div>
  )}

  /* Employee */
  <div className="space-y-2">
    <Label>Dipendente *</Label>
    <Select value={fUserId} onValueChange={setFUserId}>
      <SelectTrigger><SelectValue placeholder="Seleziona dipendente" /></SelectTrigger>
      <SelectContent>
        {scopedStaff.map(s => (
          <SelectItem key={s.id} value={s.id}>
            {s.full_name}{s.department ? ` · ${s.department}` : ''}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  </div>

  /* Tipo: Turno di Lavoro o Riposo */
  <div className="space-y-2">
    <Label>Tipo *</Label>
    <div className="flex flex-wrap gap-2">
      {TURN_TYPE_OPTIONS.map(o => (
        <button
          key={String(o.value)}
          type="button"
          onClick={() => { setFIsRestDay(o.value); if (o.value) setFSplitSegments([]) }}
          className={`px-3 py-1.5 rounded-full text-sm font-medium border transition-colors ${
            fIsRestDay === o.value
              ? 'bg-primary text-primary-foreground border-primary'
              : 'bg-background border-border text-foreground hover:bg-accent'
            }`}
        >
          {o.label}
        </button>
      ))}
    </div>
  </div>

  /* Inserimento Multiplo toggle – solo in creazione, esclude Multi-giorno */
  {!editingTurn && !multiDayMode && (
    <div className="flex items-center justify-between rounded-sm border border-border px-3 py-2.5">
      <div>
        <Label className="text-sm">Inserimento Multiplo</Label>
        <p className="text-xs text-muted-foreground">Stesso orario ripetuto su un range di date, sui
giorni selezionati</p>
      </div>
      <Switch checked={bulkMode} onCheckedChange={v => { setBulkMode(v); if (v)
setFSplitSegments([]) }} />
    </div>
  )}

  /* Multi-giorno toggle – solo in creazione, esclude Inserimento Multiplo */
  {!editingTurn && !bulkMode && (
    <div className="flex items-center justify-between rounded-sm border border-border px-3 py-2.5">
      <div>
        <Label className="text-sm">Più Giorni (orari diversi)</Label>
        <p className="text-xs text-muted-foreground">Compila un turno diverso per ogni giorno, poi salva
tutto insieme</p>
      </div>
      <Switch checked={multiDayMode} onCheckedChange={toggleMultiDayMode} />
    </div>
  )}

```

```

    </div>
  )}

  {/* Date */}
  {multiDayMode ? (
    <div className="space-y-2">
      <Label>Giorno *</Label>
      <div className="flex items-center justify-between gap-2 rounded-sm border border-border px-2
py-1.5">
        <Button type="button" variant="ghost" size="icon" className="h-7 w-7 shrink-0" onClick={() =>
goToday(-1)} aria-label="Giorno precedente">
          <ChevronLeft className="w-4 h-4" />
        </Button>
        <span className="text-sm font-medium capitalize text-center flex-1">
          {fDate ? formatInTimeZone(`${fDate}T12:00:00Z`, TZ, 'EEEE d MMMM', { locale: it }) : '-'}
        </span>
        <Button type="button" variant="ghost" size="icon" className="h-7 w-7 shrink-0" onClick={() =>
goToday(1)} aria-label="Giorno successivo">
          <ChevronRight className="w-4 h-4" />
        </Button>
      </div>
      <p className="text-[11px] text-muted-foreground">
        Compila il turno di questo giorno, poi usa le frecce per passare al successivo.
      </p>
    </div>
  ) : bulkMode ? (
    <>
      <div className="grid grid-cols-2 gap-3">
        <div className="space-y-2">
          <Label>Data inizio *</Label>
          <Input type="date" value={fDate} onChange={e => setFDate(e.target.value)} />
        </div>
        <div className="space-y-2">
          <Label>Data fine *</Label>
          <Input type="date" value={fEndDate} onChange={e => setFEndDate(e.target.value)} />
        </div>
      </div>
      <div className="space-y-2">
        <Label>Giorni della settimana *</Label>
        <div className="flex flex-wrap gap-2">
          {WEEK_DAY_OPTIONS.map(d => (
            <button
              key={d.value}
              type="button"
              onClick={() => toggleFDayOfWeek(d.value)}
              className={`px-3 py-1.5 rounded-full text-sm font-medium border transition-colors ${
                fDaysOfWeek.includes(d.value)
                  ? 'bg-primary text-primary-foreground border-primary'
                  : 'bg-background border-border text-foreground hover:bg-accent'
              }`}
            >
              {d.label}
            </button>
          ))}
        </div>
      </div>
    </>
  ) : (
    <div className="space-y-2">
      <Label>Data *</Label>
      <Input type="date" value={fDate} onChange={e => setFDate(e.target.value)} />
    </div>
  )}

  {/* Giorni già compilati nel piano multi-giorno */}
  {multiDayMode && multiDayQueue.length > 0 && (
    <div className="space-y-1.5">
      <Label className="text-xs font-semibold text-muted-foreground">
        Giorni in programma ({multiDayQueue.length})
      </Label>
      {multiDayQueue.map(e => (
        <div
          key={e.date}
          className={`flex items-center justify-between rounded-sm border px-2.5 py-1.5 text-xs ${

```



```

        e.date === fDate ? 'border-primary bg-primary/5' : 'border-border'
      }`
    }
  >
  <button
    type="button"
    onClick={() => loadDayIntoComposer(e.date, multiDayQueue)}
    className="flex-1 text-left"
  >
    <span className="font-medium capitalize">
      {formatInTimeZone(`${e.date}T12:00:00Z`, TZ, 'EEE d MMM', { locale: it })}
    </span>
    { ' - ' }
    {e.is_rest_day
      ? 'Riposo'
      : `${e.start_time.slice(0, 5)}-${e.end_time.slice(0, 5)}${e.splitSegments.length > 0 ? ` +
    ${e.splitSegments.length} fascia/e` : ''}`}
  </button>
  <button
    type="button"
    onClick={() => removeMultiDayEntry(e.date)}
    className="text-muted-foreground hover:text-destructive ml-2"
    aria-label="Rimuovi giorno"
  >
    <X className="w-3 h-3" />
  </button>
</div>
)}}
</div>
)}}

{!fIsRestDay && (
  <
    { /* Times */ }
    <div className="grid grid-cols-2 gap-3">
      <div className="space-y-2">
        <Label>Ora inizio *</Label>
        <TimeInput value={fStart} onChange={setFStart} />
      </div>
      <div className="space-y-2">
        <Label>Ora fine *</Label>
        <TimeInput value={fEnd} onChange={setFEnd} />
      </div>
    </div>

    { /* Durata – somma anche le eventuali fasce spezzate qui sotto */ }
    {fStart && fEnd && (
      <p className="text-xs text-muted-foreground -mt-2">
        {validSplitSegments.length > 0 ? 'Durata totale (turno spezzato): ' : 'Durata turno: '}
        <span className="font-medium text-foreground">{formatMinutes(totalShiftMinutes)}</span>
      </p>
    )}
  )}

  { /* Turno spezzato – solo in creazione singola (non bulk, non modifica) */ }
  {canHaveSplit && (
    <div className="space-y-3">
      {fSplitSegments.map((seg, idx) => (
        <div key={idx} className="rounded-sm border border-border bg-muted/20 p-3 space-y-3">
          <div className="flex items-center justify-between">
            <Label className="text-xs font-semibold text-muted-foreground">
              Turno spezzato – fascia aggiuntiva {idx + 2}
            </Label>
            <button
              type="button"
              onClick={() => removeSplitSegment(idx)}
              className="text-muted-foreground hover:text-destructive"
              aria-label="Rimuovi fascia"
            >
              <X className="w-3.5 h-3.5" />
            </button>
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>Ora inizio *</Label>
              <TimeInput value={seg.start} onChange={v => updateSplitSegment(idx, { start: v })} />

```

```

        </div>
        <div className="space-y-2">
          <Label>Ora fine *</Label>
          <TimeInput value={seg.end} onChange={v => updateSplitSegment(idx, { end: v })} />
        </div>
      </div>
      {seg.start && seg.end && (
        <p className="text-xs text-muted-foreground">
          Durata fascia: <span className="font-medium text-foreground">{formatMinutes(segmentMinutes(seg.start, seg.end))}</span>
        </p>
      )}
    </div>
  )}
  <Button type="button" variant="outline" size="sm" onClick={addSplitSegment} className="w-full">
    <Plus className="w-3.5 h-3.5" /> Aggiungi turno spezzato
  </Button>
</div>
)}

{/* Extraordinary toggle */}
<div className="flex items-center justify-between rounded-sm border border-border px-3 py-2.5">
  <div>
    <Label className="text-sm">Turno Straordinario</Label>
    <p className="text-xs text-muted-foreground">Evidenziato in arancione nel calendario</p>
  </div>
  <Switch checked={fExtraordinary} onCheckedChange={setFExtraordinary} />
</div>
</>
)}

{/* Notes */}
<div className="space-y-2">
  <Label>Note</Label>
  <Textarea
    value={fNotes}
    onChange={e => setFNotes(e.target.value)}
    rows={2}
    placeholder="Note opzionali..."
  />
</div>

{error && (
  <p className="text-xs text-destructive">{error}</p>
)}
</div>
<DialogFooter>
  <Button variant="outline" onClick={() => setShowForm(false)}>Annulla</Button>
  <Button
    onClick={handleSave}
    disabled={
      saving || !fUserId ||
      (multiDayMode
        ? (multiDayCount === 0 || hasIncompleteSplitSegment)
        : (
            (!fIsRestDay && (!fStart || !fEnd)) ||
            (bulkMode ? (!fDate || !fEndDate || fDaysOfWeek.length === 0) : !fDate) ||
            hasIncompleteSplitSegment
          )
        )
    }
  >
    {saving
      ? 'Salvataggio...'
      : editingTurn
      ? 'Salva modifiche'
      : multiDayMode
      ? `Crea turni (${multiDayCount} giorni)`
      : bulkMode
      ? 'Crea turni'
      : 'Crea turno'}
    </Button>
</DialogFooter>

```

```

</DialogContent>
</Dialog>

{/* — Turni Fissi (Pattern Master) ————— */}
<Dialog open={showStandardModal} onOpenChange={setShowStandardModal}>
  <DialogContent className="max-h-[90vh] overflow-y-auto">
    <DialogHeader>
      <DialogTitle>Gestisci Turni Fissi</DialogTitle>
    </DialogHeader>
    <div className="space-y-4">
      <div className="w-full rounded-md border bg-muted/20 overflow-auto max-h-[280px]">
        <table className="border-collapse text-xs w-full">
          <thead>
            <tr>
              <th className={` ${thCls} text-left`} >Dipendente</th>
              <th className={thCls}>Giorno</th>
              <th className={thCls}>Orario</th>
              <th className={thCls}></th>
            </tr>
          </thead>
          <tbody>
            {standardShifts.length === 0 ? (
              <tr>
                <td colSpan={4} className="text-center py-4 text-muted-foreground text-xs">
                  Nessun turno fisso configurato
                </td>
              </tr>
            ) : standardShifts.map(s => (
              <tr key={s.id} className="even:bg-zinc-50 dark:even:bg-zinc-900/20">
                <td className={tdNameStaticCls}>{s.profile?.full_name ?? '-'}</td>
                <td className={tdCls}>{WEEK_DAY_OPTIONS.find(d => d.value === s.day_of_week)?.label}</td>
                <td className={tdCls}>{s.start_time.slice(0, 5)} - {s.end_time.slice(0, 5)}</td>
                <td className={tdCls}>
                  <Button
                    variant="ghost" size="icon"
                    onClick={() => handleDeleteStandardShift(s.id)}
                    className="text-destructive hover:text-destructive dark:text-red-400 dark:hover:bg-red-500/10 dark:hover:text-red-300 h-6 w-6"
                  >
                    <Trash2 className="w-3.5 h-3.5" />
                  </Button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>

      <div className="space-y-3 border-t border-border pt-4">
        <Label className="text-sm font-semibold">Aggiungi turno fisso</Label>
        <div className="space-y-2">
          <Label>Dipendente *</Label>
          <Select value={sUserId} onValueChange={setSUserId}>
            <SelectTrigger><SelectValue placeholder="Seleziona dipendente" /></SelectTrigger>
            <SelectContent>
              {staff.map(s => (
                <SelectItem key={s.id} value={s.id}>
                  {s.full_name}{s.department ? ` · ${s.department}` : ''}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
        </div>

        <div className="space-y-2">
          <Label>
            Giorni della settimana *
            {sDaysOfWeek.length > 0 && (
              <span className="ml-2 text-xs font-normal text-muted-foreground">
                {sDaysOfWeek.length} selezionati
              </span>
            )}
          </Label>
          <div className="flex flex-wrap gap-2">
            {WEEK_DAY_OPTIONS.map(d => (

```

```

        <button
            key={d.value}
            type="button"
            onClick={() => toggleSDayOfWeek(d.value)}
            className={`px-3 py-1.5 rounded-full text-sm font-medium border transition-colors ${
                sDaysOfWeek.includes(d.value)
                    ? 'bg-primary text-primary-foreground border-primary'
                    : 'bg-background border-border text-foreground hover:bg-accent'
            }`}
        >
            {d.label}
        </button>
    )}}
</div>
</div>
<div className="grid grid-cols-2 gap-3">
    <div className="space-y-2">
        <Label>Ora inizio *</Label>
        <TimeInput value={sStart} onChange={setSStart} />
    </div>
    <div className="space-y-2">
        <Label>Ora fine *</Label>
        <TimeInput value={sEnd} onChange={setSEnd} />
    </div>
</div>
{sError && <p className="text-xs text-destructive">{sError}</p>}
<Button
    size="sm"
    onClick={handleAddStandardShift}
    disabled={sSaving || !sUserId || !sStart || !sEnd || sDaysOfWeek.length === 0}
>
    <Plus className="w-4 h-4" /> {sSaving ? 'Salvataggio...' : 'Aggiungi'}
</Button>
</div>
</div>
<DialogFooter>
    <Button variant="outline" onClick={() => setShowStandardModal(false)}>Chiudi</Button>
</DialogFooter>
</DialogContent>
</Dialog>

{/* — AI Schedule ————— */}
<AiScheduleDialog
    open={showAiDialog}
    onClose={() => setShowAiDialog(false)}
    restaurantId={isManager ? (restFilter !== 'tutti' ? restFilter : '') : (currentRestaurantId ?? '')}
    currentDept={currentDepartment as Department | null}
    currentUserRole={currentUserRole}
    currentIsDirettore={currentIsDirettore}
    onDraftCreated={draft => { setAiDraft(draft); setShowAiDialog(false) }}
/>

{/* Vista bozza AI */}
{aiDraft && (
    <Dialog open={!aiDraft} onOpenChange={open => { if (!open) setAiDraft(null) }}>
        <DialogContent className="max-h-[95vh] overflow-y-auto max-w-5xl w-full">
            <AiScheduleDraftView
                draft={aiDraft}
                staff={staff}
                onClose={() => setAiDraft(null)}
                onConfirmed={() => { setAiDraft(null) }}
            />
        </DialogContent>
    </Dialog>
)}

</div>
)
}

```

`src/components/manager/TurniTimeline.tsx`

```

'use client'
import { useEffect, useRef, useState } from 'react'
import { Button } from '@components/ui/button'
import { ChevronLeft, ChevronRight } from 'lucide-react'
import { formatInTimeZone } from 'date-fns-tz'
import { addDays, format, parseISO } from 'date-fns'
import { it } from 'date-fns/locale'
import { createTurn, updateTurn, type TurnInput } from '@app/actions/turni'
import { EXTRAORDINARY_BADGE, STANDARD_BADGE, RIPOSO_BADGE } from '@lib/turnColors'
import type { Turn, Department } from '@types'

const TZ = 'Europe/Rome'
const HOUR_WIDTH = 72 // px per ora – spazio sufficiente per etichetta e fascia leggibile
const NAME_COL_WIDTH = 140
const DEFAULT_START_HOUR = 8
const DEFAULT_END_HOUR = 24
const SNAP_MIN = 15 // arrotondamento del trascinamento, in minuti
const MIN_SHIFT_MIN = 15 // durata minima consentita di un turno
const MIN_DRAG_CREATE_MIN = 30 // sotto questa soglia un drag "a vuoto" non crea nulla
const CLICK_THRESHOLD_PX = 5 // sotto questa soglia un trascinamento dal centro è un click (apre la modifica)
const DESKTOP_QUERY = '(min-width: 768px)' // tablet/PC – su smartphone niente drag, solo visualizzazione

// Su schermi piccoli il rischio di spostare un turno per sbaglio scorrendo
// col dito è troppo alto: il drag (ridimensiona/sposta/crea) è abilitato
// solo da tablet in su. Parte "false" (nessuna interazione) finché il
// primo effect non conferma la larghezza reale, per restare prudenti anche
// nel primissimo render lato client.
function useIsDesktop(): boolean {
  const [isDesktop, setIsDesktop] = useState(false)
  useEffect(() => {
    const mql = window.matchMedia(DESKTOP_QUERY)
    setIsDesktop(mql.matches)
    const onChange = () => setIsDesktop(mql.matches)
    mql.addEventListener('change', onChange)
    return () => mql.removeEventListener('change', onChange)
  }, [])
  return isDesktop
}

type StaffMember = { id: string; full_name: string; department: string | null; restaurant_id: string | null }

interface Props {
  staff: StaffMember[]
  turns: Turn[]
  onEditTurn: (turn: Turn) => void
}

type DragState =
  | {
    mode: 'resize'
    turn: Turn
    edge: 'start' | 'end'
    trackEl: HTMLElement
    origStart: number
    origEnd: number
    previewStart: number
    previewEnd: number
  }
  | {
    mode: 'move'
    turn: Turn
    trackEl: HTMLElement
    origStart: number
    origEnd: number
    startClientX: number
    moved: boolean // false finché il puntatore non supera la soglia di click
    previewStart: number
    previewEnd: number
  }
  | {
    mode: 'create'
    member: StaffMember
    date: string
  }

```

```

    trackEl: HTMLElement
    anchorMin: number
    previewStart: number
    previewEnd: number
  }

function timeToMinutes(t: string): number {
  const [h, m] = t.slice(0, 5).split(':').map(Number)
  return h * 60 + m
}

function minutesToHHMM(totalMin: number): string {
  const m = ((Math.round(totalMin) % 1440) + 1440) % 1440
  const h = Math.floor(m / 60)
  const mm = m % 60
  return `${String(h).padStart(2, '0')}:${String(mm).padStart(2, '0')}`
}

function clamp(v: number, min: number, max: number): number {
  return Math.min(Math.max(v, min), max)
}

function snap(v: number): number {
  return Math.round(v / SNAP_MIN) * SNAP_MIN
}

// Intervallo [inizio,fine] in minuti dalla mezzanotte del giorno selezionato;
// gestisce il turno notturno estendendo la fine oltre le 24:00.
function turnRange(t: Turn): { start: number; end: number } {
  const start = timeToMinutes(t.start_time)
  let end = timeToMinutes(t.end_time)
  if (end <= start) end += 24 * 60
  return { start, end }
}

export function TurniTimeline({ staff, turns, onEditTurn }: Props) {
  const [date, setDate] = useState(() => formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd'))
  const [nowTick, setNowTick] = useState(() => Date.now())
  const canDrag = useIsDesktop()

  // `drag` esiste SOLO tra pointerdown e pointerup (i listener globali sono
  // agganciati mentre è non-null). `frozenPreview` è uno scatto congelato
  // usato solo per continuare a mostrare la fascia nella posizione rilasciata
  // mentre la chiamata al server è in corso, senza restare agganciato ai
  // movimenti del puntatore (altrimenti un mousemove dopo il rilascio
  // continuerebbe a spostare l'anteprima).
  const [drag, setDrag] = useState<DragState | null>(null)
  const [frozenPreview, setFrozenPreview] = useState<DragState | null>(null)
  const [savingKey, setSavingKey] = useState<string | null>(null) // id turno, oppure 'new'
  const [dragError, setDragError] = useState<string | null>(null)
  const dragRef = useRef<DragState | null>(null)

  // Override ottimistici: dopo un salvataggio riuscito, il prop `turns` del
  // padre resta indietro finché non arriva l'evento realtime (~mezzo secondo).
  // Senza questi, la fascia tornerebbe per un attimo alla posizione vecchia
  // (flash). L'override tiene la fascia dove l'utente l'ha lasciata e viene
  // rimosso quando i dati veri arrivano; i turni creati col drag vivono in
  // `extraTurns` finché non compaiono nel prop.
  const [overrides, setOverrides] = useState<Record<string, { start_time: string; end_time: string }>>({})
  const [extraTurns, setExtraTurns] = useState<Turn[]>([])

  useEffect(() => {
    setOverrides(prev => {
      const stale = Object.keys(prev).filter(id => {
        const t = turns.find(x => x.id === id)
        return t && t.start_time.slice(0, 5) === prev[id].start_time && t.end_time.slice(0, 5) ===
prev[id].end_time
      })
      if (stale.length === 0) return prev
      const next = { ...prev }
      stale.forEach(id => delete next[id])
      return next
    })
    setExtraTurns(prev => prev.length ? prev.filter(e => !turns.some(t => t.id === e.id)) : prev)
  })
}

```

```

}, [turns])

// La linea "adesso" avanza ogni minuto, senza bisogno di refresh manuale.
useEffect(() => {
  const interval = setInterval(() => setNowTick(Date.now()), 60_000)
  return () => clearInterval(interval)
}, [])

useEffect(() => {
  if (!dragError) return
  const t = setTimeout(() => setDragError(null), 4000)
  return () => clearTimeout(t)
}, [dragError])

function shiftDay(delta: number) {
  setDate(prev => format(addDays(parseISO(`${prev}T00:00:00`), delta), 'yyyy-MM-dd'))
}

const nativeDateRef = useRef<HTMLInputElement>(null)
function openDatePicker() {
  const el = nativeDateRef.current
  if (!el) return
  if (typeof el.showPicker === 'function') {
    try { el.showPicker(); return } catch { /* fallback sotto */ }
  }
  el.focus()
  el.click()
}

const effectiveTurns = [
  ...turns.map(t => overrides[t.id] ? { ...t, ...overrides[t.id] } : t),
  ...extraTurns.filter(e => !turns.some(t => t.id === e.id)),
]
const dayTurns = effectiveTurns.filter(t => t.date === date)
const workTurns = dayTurns.filter(t => !t.is_rest_day)

// Asse orario: parte da min(8:00, primo turno) e arriva a max(24:00, ultimo turno)
const rawStart = Math.min(DEFAULT_START_HOUR * 60, ...workTurns.map(t => turnRange(t).start))
const rawEnd = Math.max(DEFAULT_END_HOUR * 60, ...workTurns.map(t => turnRange(t).end))
const axisStartHour = Math.floor(rawStart / 60)
const axisEndHour = Math.ceil(rawEnd / 60)
const axisStartMin = axisStartHour * 60
const axisEndMin = axisEndHour * 60
const totalHours = axisEndHour - axisStartHour
const pxPerMin = HOUR_WIDTH / 60
const totalWidth = totalHours * HOUR_WIDTH

const hourMarks = Array.from({ length: totalHours }, (_, i) => axisStartHour + i)

const todayStr = formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd')
const isToday = date === todayStr
const nowMinRaw = isToday
  ? parseInt(formatInTimeZone(nowTick, TZ, 'H'), 10) * 60 + parseInt(formatInTimeZone(nowTick, TZ, 'm'), 10)
  : null
const nowInRange = nowMinRaw !== null && nowMinRaw >= axisStartMin && nowMinRaw <= axisStartMin + totalHours *
60
const nowLeftPx = nowInRange ? (nowMinRaw! - axisStartMin) * pxPerMin : 0

const turnsByStaff: Record<string, Turn[]> = {}
workTurns.forEach(t => {
  if (!turnsByStaff[t.user_id]) turnsByStaff[t.user_id] = []
  turnsByStaff[t.user_id].push(t)
})
const restDayStaffIds = new Set(dayTurns.filter(t => t.is_rest_day).map(t => t.user_id))

// Mentre si trascina attivamente uso `drag`; dopo il rilascio, finché la
// chiamata al server non è conclusa, uso lo scatto congelato `frozenPreview`.
const activePreview = drag ?? frozenPreview

// — Drag: ridimensiona una fascia esistente oppure disegna un turno nuovo —
function minuteFromClientX(clientX: number, trackEl: HTMLElement): number {
  const rect = trackEl.getBoundingClientRect()
  return axisStartMin + (clientX - rect.left) / pxPerMin
}

```



```

function updateDrag(updater: (prev: DragState) => DragState) {
  setDrag(prev => {
    if (!prev) return prev
    const next = updater(prev)
    dragRef.current = next
    return next
  })
}

function startResize(e: React.PointerEvent, turn: Turn, edge: 'start' | 'end') {
  if (!canDrag) return
  e.stopPropagation()
  e.preventDefault()
  const trackEl = (e.currentTarget as HTMLElement).closest('[data-track]') as HTMLElement | null
  if (!trackEl) return
  const { start, end } = turnRange(turn)
  const initial: DragState = { mode: 'resize', turn, edge, trackEl, origStart: start, origEnd: end,
    previewStart: start, previewEnd: end }
  dragRef.current = initial
  setDrag(initial)
}

function startMove(e: React.PointerEvent, turn: Turn) {
  if (!canDrag) return
  e.stopPropagation()
  e.preventDefault()
  const trackEl = (e.currentTarget as HTMLElement).closest('[data-track]') as HTMLElement | null
  if (!trackEl) return
  const { start, end } = turnRange(turn)
  const initial: DragState = {
    mode: 'move', turn, trackEl, origStart: start, origEnd: end,
    startClientX: e.clientX, moved: false, previewStart: start, previewEnd: end,
  }
  dragRef.current = initial
  setDrag(initial)
}

function startCreate(e: React.PointerEvent<HTMLDivElement>, member: StaffMember) {
  if (!canDrag) return
  if (e.target !== e.currentTarget) return // click su una fascia/etichetta esistente, non sull'area vuota
  if (e.button !== 0) return
  e.preventDefault()
  const trackEl = e.currentTarget
  const anchorMin = clamp(snap(minuteFromClientX(e.clientX, trackEl)), axisStartMin, axisEndMin)
  const initial: DragState = { mode: 'create', member, date, trackEl, anchorMin, previewStart: anchorMin,
    previewEnd: anchorMin }
  dragRef.current = initial
  setDrag(initial)
}

// Mentre un trascinamento è attivo, disabilita la selezione testo su tutta
// la pagina (anche con prefisso WebKit per Safari/iOS): la selezione parte
// nell'istante del pointerdown, quindi la timeline stessa è già select-none
// in modo permanente – questo copre il resto della pagina se il puntatore
// esce dal riquadro durante il gesto.
useEffect(() => {
  if (!drag) return
  const body = document.body as HTMLElement & { style: CSSStyleDeclaration & { webkitUserSelect?: string } }
  const prevUserSelect = body.style.userSelect
  const prevWebkit = body.style.webkitUserSelect
  body.style.userSelect = 'none'
  body.style.webkitUserSelect = 'none'
  const sel = window.getSelection?().()
  sel?.removeAllRanges?().()
  return () => {
    body.style.userSelect = prevUserSelect
    body.style.webkitUserSelect = prevWebkit
  }
}, [drag !== null])

useEffect(() => {
  if (!drag) return

```

```

function onMove(e: PointerEvent) {
  updateDrag(prev => {
    const min = minuteFromClientX(e.clientX, prev.trackEl)
    if (prev.mode === 'resize') {
      if (prev.edge === 'start') {
        return { ...prev, previewStart: clamp(snap(min), prev.origEnd - 24 * 60, prev.origEnd -
MIN_SHIFT_MIN) }
      }
      return { ...prev, previewEnd: clamp(snap(min), prev.origStart + MIN_SHIFT_MIN, prev.origStart + 24 *
60) }
    }
    if (prev.mode === 'move') {
      const dxPx = Math.abs(e.clientX - prev.startClientX)
      const duration = prev.origEnd - prev.origStart
      const deltaMin = snap((e.clientX - prev.startClientX) / pxPerMin)
      const newStart = clamp(prev.origStart + deltaMin, 0, 2 * 1440 - duration)
      return {
        ...prev,
        previewStart: newStart,
        previewEnd: newStart + duration,
        moved: prev.moved || dxPx > CLICK_THRESHOLD_PX,
      }
    }
    const anchor = prev.anchorMin
    const cur = clamp(snap(min), axisStartMin, axisEndMin)
    return { ...prev, previewStart: Math.min(anchor, cur), previewEnd: Math.max(anchor, cur) }
  })
}

async function onUp() {
  const finalDrag = dragRef.current
  setDrag(null)
  dragRef.current = null
  if (!finalDrag) return

  if (finalDrag.mode === 'resize') {
    const orig = turnRange(finalDrag.turn)
    if (finalDrag.previewStart === orig.start && finalDrag.previewEnd === orig.end) return
  } else if (finalDrag.mode === 'move') {
    if (!finalDrag.moved) {
      // Nessuno spostamento reale: era un click, apri la modifica.
      onEditTurn(finalDrag.turn)
      return
    }
    if (finalDrag.previewStart === finalDrag.origStart) return
  } else if (finalDrag.previewEnd - finalDrag.previewStart < MIN_DRAG_CREATE_MIN) {
    return
  }

  // Congela la fascia nella posizione rilasciata finché il server non risponde
  setFrozenPreview(finalDrag)
  setSavingKey(finalDrag.mode === 'create' ? 'new' : finalDrag.turn.id)

  try {
    if (finalDrag.mode === 'resize' || finalDrag.mode === 'move') {
      const { turn, previewStart, previewEnd } = finalDrag
      const newStart = minutesToHHMM(previewStart)
      const newEnd = minutesToHHMM(previewEnd)
      await updateTurn(turn.id, {
        user_id: turn.user_id,
        restaurant_id: turn.restaurant_id,
        department: turn.department,
        date: turn.date,
        start_time: newStart,
        end_time: newEnd,
        is_extraordinary: turn.is_extraordinary,
        is_rest_day: false,
        notes: turn.notes,
      }) satisfies TurnInput
      // Override ottimistico PRIMA di sganciare l'anteprima congelata:
      // React applica i due set nello stesso render, quindi la fascia
      // non torna mai alla posizione vecchia in attesa del realtime.
      setOverrides(prev => ({ ...prev, [turn.id]: { start_time: newStart, end_time: newEnd } }))
    } else {

```

```

const { member, date: dayDate, previewStart, previewEnd } = finalDrag
const created = await createTurn({
  user_id: member.id,
  restaurant_id: member.restaurant_id ?? '',
  department: member.department as Department | null,
  date: dayDate,
  start_time: minutesToHHMM(previewStart),
  end_time: minutesToHHMM(previewEnd),
  is_extraordinary: false,
  is_rest_day: false,
  notes: null,
}) satisfies TurnInput
// Il turno appena creato resta visibile subito, senza aspettare il realtime
setExtraTurns(prev => [...prev, created])
}
} catch (err) {
  setDragError(err instanceof Error ? err.message : 'Errore durante il salvataggio del turno')
} finally {
  setSavingKey(null)
  setFrozenPreview(null)
}
}
}

window.addEventListener('pointermove', onMove)
window.addEventListener('pointerup', onUp)
return () => {
  window.removeEventListener('pointermove', onMove)
  window.removeEventListener('pointerup', onUp)
}
// eslint-disable-next-line react-hooks/exhaustive-deps
}, [drag !== null])

return (
  <div className="mt-6">
    <div className="flex items-center justify-between gap-3 mb-3">
      <h2 className="text-sm font-semibold text-foreground">Timeline Giornaliera</h2>
      <div className="flex items-center gap-2">
        <Button variant="outline" size="icon" className="h-7 w-7" onClick={() => shiftDay(-1)} aria-label="Giorno precedente">
          <ChevronLeft className="w-3.5 h-3.5" />
        </Button>
        <div className="relative">
          <button
            type="button"
            onClick={openDatePicker}
            title="Apri calendario"
            className="text-xs font-medium text-foreground capitalize whitespace-nowrap tabular-nums min-w-[100px] text-center hover:underline decoration-dotted underline-offset-2"
            >
              {formatInTimeZone(`${date}T12:00:00Z`, TZ, 'EEE d MMM', { locale: it })}
            </button>
            </* Input nativo invisibile: al click sulla data apre il calendario
              del sistema per saltare a un giorno lontano senza cliccare
              troppe volte sulle frecce. */>
            <input
              ref={nativeDateRef}
              type="date"
              value={date}
              onChange={e => e.target.value && setDate(e.target.value)}
              tabIndex={-1}
              aria-hidden="true"
              className="sr-only"
            />
          </div>
        </div>
        <Button variant="outline" size="icon" className="h-7 w-7" onClick={() => shiftDay(1)} aria-label="Giorno successivo">
          <ChevronRight className="w-3.5 h-3.5" />
        </Button>
        </* Sempre presente (anche su oggi) per non far "saltare" il layout
          quando appare/scompare cambiando data. */>
        <button
          onClick={() => setDate(todayStr)}
          disabled={isToday}
          className="text-[11px] underline ml-1 text-muted-foreground hover:text-foreground disabled:no-

```

```

underline disabled:opacity-40 disabled:cursor-default"
    >
      oggi
    </button>
  </div>
</div>

{dragError && (
  <p className="text-xs text-destructive mb-2">{dragError}</p>
)}

{/* Scroll confinato al proprio contenitore, stesso pattern del Report Ore */}
{/* select-none permanente (+ varianti WebKit/iOS): la selezione parte al
  pointerdown, prima che lo stato di drag esista – disattivarla solo
  durante il drag arriva sempre troppo tardi. Qui non c'è testo che
  abbia senso copiare, quindi la disattiviamo del tutto. */}
<div
  className="w-full rounded-md border bg-card overflow-auto max-h-[420px] select-none [-webkit-user-
select:none] [-webkit-touch-callout:none]"
  onDragStart={e => e.preventDefault()}
  >
    <div style={{ minWidth: totalWidth + NAME_COL_WIDTH }}>
      {/* Header ore – sticky in alto mentre si scorre verticalmente */}
      <div className="flex border-b border-border sticky top-0 bg-card z-20">
        <div className="shrink-0 sticky left-0 bg-card z-10 border-r border-border" style={{ width:
NAME_COL_WIDTH }} />
        <div className="flex" style={{ width: totalWidth }}>
          {hourMarks.map(h => (
            <div
              key={h}
              style={{ width: HOUR_WIDTH }}
              className="shrink-0 text-[10px] text-muted-foreground px-1 py-1.5 border-r border-border/50"
            >
              {String(h % 24).padStart(2, '0')}:00
            </div>
          ))}
        </div>
      </div>

      {/* Righe dipendenti */}
      {staff.length === 0 ? (
        <div className="py-6 text-center text-xs text-muted-foreground">Nessun dipendente</div>
      ) : staff.map(member => {
        const memberTurns = (turnsByStaff[member.id] ?? []).sort((a, b) =>
a.start_time.localeCompare(b.start_time))
        const isRest = restDayStaffIds.has(member.id)
        const createPreview = activePreview?.mode === 'create' && activePreview.member.id === member.id ?
activePreview : null

        return (
          <div key={member.id} className="flex border-b border-border last:border-b-0 even:bg-zinc-50
dark:even:bg-zinc-900/20">
            <div
              className="shrink-0 sticky left-0 bg-inherit z-10 border-r border-border px-2 py-2 text-xs font-
medium text-foreground flex items-center truncate"
              style={{ width: NAME_COL_WIDTH }}
            >
              {member.full_name}
            </div>
            <div
              data-track
              onPointerDown={canDrag ? e => startCreate(e, member) : undefined}
              style={{ width: totalWidth, minHeight: 44, touchAction: canDrag && drag ? 'none' : 'pan-x' }}
              className={`relative ${canDrag ? 'cursor-crosshair' : ''}`}
            >
              {/* Griglia ore verticale, solo decorativa */}
              <div className="absolute inset-0 flex pointer-events-none">
                {hourMarks.map(h => (
                  <div key={h} style={{ width: HOUR_WIDTH }} className="shrink-0 border-r border-border/30" />
                ))}
              </div>

              {isRest && memberTurns.length === 0 && (
                <div className="absolute inset-y-0 left-1.5 flex items-center pointer-events-none">

```

```

        <span className={`text-[10px] px-1.5 py-0.5 rounded-sm border font-medium ${RIPOSO_BADGE}`}
>Riposo</span>
    </div>
  )}

  {memberTurns.map(t => {
    const isPreviewing = (activePreview?.mode === 'resize' || activePreview?.mode === 'move')
    && activePreview.turn.id === t.id
    const { start, end } = isPreviewing
      ? { start: activePreview.previewStart, end: activePreview.previewEnd }
      : turnRange(t)
    const left = (start - axisStartMin) * pxPerMin
    const width = Math.max((end - start) * pxPerMin, 26)
    const isSaving = savingKey === t.id
    return (
      <div
        key={t.id}
        style={{ left, width, top: 6, height: 32 }}
        className={`absolute rounded-sm border flex items-center text-[10px] font-medium select-
none transition-opacity ${
          isSaving ? 'opacity-60' : ''
        } ${t.is_extraordinary ? EXTRAORDINARY_BADGE : STANDARD_BADGE}`}
        title={` ${member.full_name} · ${minutesToHHMM(start)}-${minutesToHHMM(end)} `}
      >
        /* Maniglie e spostamento – solo tablet/PC (canDrag).
        Su smartphone la timeline è pura visualizzazione:
        niente maniglie, niente drag, scroll sempre naturale. */
        {canDrag && (
          <div
            onPointerDown={e => startResize(e, t, 'start')}
            className="absolute left-0 inset-y-0 w-4 cursor-ew-resize"
            style={{ touchAction: 'none' }}
          />
        )}
        <div
          onPointerDown={canDrag ? e => startMove(e, t) : undefined}
          style={canDrag ? { touchAction: 'none' } : undefined}
          className={`flex-1 h-full truncate text-left px-2 flex items-center ${
            canDrag ? 'cursor-grab active:cursor-grabbing hover:opacity-80' : ''
          }`}
        >
          {minutesToHHMM(start)}-${minutesToHHMM(end)}
        </div>
        {canDrag && (
          <div
            onPointerDown={e => startResize(e, t, 'end')}
            className="absolute right-0 inset-y-0 w-4 cursor-ew-resize"
            style={{ touchAction: 'none' }}
          />
        )}
      </div>
    )
  )}}

  {createPreview && (
    <div
      style={{
        left: (createPreview.previewStart - axisStartMin) * pxPerMin,
        width: Math.max((createPreview.previewEnd - createPreview.previewStart) * pxPerMin, 4),
        top: 6,
        height: 32,
      }}
      className={`absolute rounded-sm border-2 border-dashed pointer-events-none flex items-center
justify-center text-[10px] font-medium ${STANDARD_BADGE}`}
    >
      {createPreview.previewEnd - createPreview.previewStart >= MIN_DRAG_CREATE_MIN &&
        `${minutesToHHMM(createPreview.previewStart)}-${minutesToHHMM(createPreview.previewEnd)} `}
    </div>
  )}

  {nowInRange && (
    <div className="absolute inset-y-0 w-px bg-red-500 z-10 pointer-events-none" style={{ left:
nowLeftPx }} />
  )}

```

```

        </div>
      </div>
    )
  }
}
</div>
</div>

<div className="flex flex-wrap items-center gap-3 mt-2 text-[11px] text-muted-foreground">
  <span className="flex items-center gap-1.5">
    <span className={`inline-block w-3 h-3 rounded-sm border ${STANDARD_BADGE}`} /> Turno standard
  </span>
  <span className="flex items-center gap-1.5">
    <span className={`inline-block w-3 h-3 rounded-sm border ${EXTRAORDINARY_BADGE}`} /> Straordinario
  </span>
  <span className="flex items-center gap-1.5">
    <span className={`inline-block w-3 h-3 rounded-sm border ${RIPOSO_BADGE}`} /> Riposo
  </span>
  <span className="flex items-center gap-1.5">
    <span className="inline-block w-3 h-0.5 bg-red-500" /> Adesso
  </span>
  <span className="text-muted-foreground/70 hidden md:inline">
    Trascina i bordi per allungare/accorciare · trascina dal centro per spostare · trascina un'area
    vuota per crearne uno nuovo
  </span>
  <span className="text-muted-foreground/70 md:hidden">
    Sola visualizzazione su smartphone – usa la tabella qui sopra per modificare i turni
  </span>
</div>
</div>
)
}

```

[src/components/shared/LoadingDots.tsx](#)

```

export function LoadingDots() {
  return (
    <span className="inline-flex items-center gap-[3px] ml-0.5">
      {[0, 150, 300].map(delay => (
        <span
          key={delay}
          className="block w-1 h-1 rounded-full bg-current animate-bounce"
          style={{ animationDelay: `${delay}ms`, animationDuration: '0.8s' }}
        />
      ))}
    </span>
  )
}

```



```

'use client'
import { useState } from 'react'
import { createClient } from '@lib/supabase/client'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import { useRouter } from 'next/navigation'

// Must match the domain used in /api/users POST to generate fake-email accounts.
const FAKE_DOMAIN = 'struttura.local'

function resolveEmail(input: string): string {
  // If the user typed a full email address (manager accounts), use it as-is.
  // Otherwise treat it as a username and append the internal domain.
  return input.includes('@') ? input : `${input}@${FAKE_DOMAIN}`
}

export function LoginForm() {
  const [credential, setCredential] = useState('')
  const [password, setPassword] = useState('')
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState<string | null>(null)
  const router = useRouter()

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault()
    setLoading(true)
    setError(null)

    const supabase = createClient()
    const { error } = await supabase.auth.signInWithPassword({
      email: resolveEmail(credential.trim().toLowerCase()),
      password,
    })

    if (error) {
      setError('Username o password non corretti')
      setLoading(false)
      return
    }

    router.push('/')
    router.refresh()
  }

  return (
    <form onSubmit={handleSubmit} className="space-y-4">
      <div className="space-y-2">
        <Label htmlFor="credential">Username</Label>
        <Input
          id="credential"
          type="text"
          value={credential}
          onChange={e => setCredential(e.target.value)}
          placeholder="mario.rossi"
          required
          autoComplete="username"
          autoCapitalize="none"
          autoCorrect="off"
          spellCheck={false}
        />
      </div>
      <div className="space-y-2">
        <Label htmlFor="password">Password</Label>
        <Input
          id="password"
          type="password"
          value={password}
          onChange={e => setPassword(e.target.value)}
          placeholder="....."
          required
          autoComplete="current-password"
        />
      </div>
    </form>
  )
}

```



```
</div>
{error && (
  <p className="text-sm text-destructive">{error}</p>
)}
<Button type="submit" className="w-full" disabled={loading}>
  {loading ? 'Accesso in corso...' : 'Accedi'}
</Button>
<p className="text-center text-sm text-muted-foreground">
  Non hai un account?{' '}
  <a href="/register" className="underline hover:text-foreground">
    Registrati
  </a>
</p>
</form>
)
}
```

`src/components/shared/OfflineSyncProvider.tsx`

```

'use client'
import { useEffect, useState, useCallback } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import { createClient } from '@lib/supabase/client'
import {
  getOfflineQueue,
  clearOfflineItem,
  type ClockInPayload,
  type OdsTogglePayload,
} from '@lib/offlineSync'

type Toast = { text: string; type: 'success' | 'info' }

export function OfflineSyncProvider({ children }: { children: React.ReactNode }) {
  const [toast, setToast] = useState<Toast | null>(null)

  function flash(text: string, type: Toast['type'] = 'success') {
    setToast({ text, type })
    setTimeout(() => setToast(null), 5000)
  }

  const processQueue = useCallback(async () => {
    let queue
    try { queue = await getOfflineQueue() } catch { return }
    if (queue.length === 0) return

    let synced = 0

    for (const item of queue) {
      try {
        if (item.type === 'clock-in') {
          const p = item.payload as ClockInPayload
          const res = await fetch('/api/scan', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
              qr_secret: p.qr_secret,
              type: p.type,
              latitude: p.latitude,
              longitude: p.longitude,
              accuracy: p.accuracy,
              frozenAt: p.frozenAt,
            }),
          })
          // 409 = open shift already exists – consider it handled
          if (res.ok || res.status === 409) {
            await clearOfflineItem(item.id)
            synced++
          }
        } else if (item.type === 'ods-toggle') {
          const p = item.payload as OdsTogglePayload
          const supabase = createClient()
          if (p.action === 'uncomplete') {
            const { error } = await supabase
              .from('ods_completions')
              .delete()
              .eq('task_id', p.task_id)
              .eq('user_id', p.user_id)
            if (!error) { await clearOfflineItem(item.id); synced++ }
          } else {
            const { error } = await supabase
              .from('ods_completions')
              .insert({ task_id: p.task_id, user_id: p.user_id, completed_at: p.frozenAt })
            if (!error) { await clearOfflineItem(item.id); synced++ }
          }
        }
      } catch {
        // Network still down for this item – leave in queue for next 'online' event
      }
    }

    if (synced > 0) flash('Dati offline sincronizzati con successo!')
  }, [])
}

```

```

useEffect(() => {
  // Flush any queue left from a previous session on first mount
  processQueue()
  window.addEventListener('online', processQueue)
  return () => window.removeEventListener('online', processQueue)
}, [processQueue])

return (
  <>
    {children}
    <AnimatePresence>
      {toast && (
        <motion.div
          key={toast.text}
          initial={{ opacity: 0, y: 20, scale: 0.97 }}
          animate={{ opacity: 1, y: 0, scale: 1 }}
          exit={{ opacity: 0, y: 10, scale: 0.97 }}
          transition={{ duration: 0.2 }}
          className={`fixed bottom-6 left-1/2 -translate-x-1/2 z-[9999] px-4 py-3 rounded-md text-sm font-medium
border shadow-lg max-w-xs w-[calc(100%-2rem)] text-center pointer-events-none ${
          toast.type === 'success'
            ? 'bg-emerald-50 text-emerald-700 border-emerald-200 dark:bg-emerald-950/30 dark:text-emerald-400
dark:border-emerald-900'
            : 'bg-primary/10 text-primary border-primary/30'
          }}
        >
          {toast.text}
        </motion.div>
      )}
    </AnimatePresence>
  </>
)
}

```

```
'use client'
import { BellRing, X } from 'lucide-react'
import { useState } from 'react'
import type { PushPermission } from '@/hooks/usePushNotifications'

interface Props {
  permission: PushPermission
  onSubscribe: () => Promise<void>
}

export function PushNotificationBanner({ permission, onSubscribe }: Props) {
  const [dismissed, setDismissed] = useState(false)
  const [loading, setLoading] = useState(false)

  if (permission !== 'default' || dismissed) return null

  async function handleSubscribe() {
    setLoading(true)
    await onSubscribe()
    setLoading(false)
  }

  return (
    <div className="mx-4 mb-3 flex items-center gap-3 rounded-md border border-border bg-card px-3 py-2.5">
      <BellRing className="w-4 h-4 text-primary shrink-0" />
      <p className="flex-1 text-xs text-muted-foreground leading-snug">
        Ricevi notifiche sui comunicati in bacheca
      </p>
      <button
        onClick={handleSubscribe}
        disabled={loading}
        className="shrink-0 text-xs font-semibold text-primary hover:text-primary/80 transition-colors disabled:opacity-50"
      >
        {loading ? '...' : 'Attiva'}
      </button>
      <button
        onClick={() => setDismissed(true)}
        className="shrink-0 text-muted-foreground hover:text-foreground transition-colors"
        aria-label="Chiudi"
      >
        <X className="w-3.5 h-3.5" />
      </button>
    </div>
  )
}
```



```

'use client'
import { useState } from 'react'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import Link from 'next/link'

type Step = 'form' | 'success'

export function RegisterForm() {
  const [step, setStep] = useState<Step>('form')
  const [fullName, setFullName] = useState('')
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [confirm, setConfirm] = useState('')
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState<string | null>(null)

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault()
    setError(null)

    if (password !== confirm) {
      setError('Le password non coincidono')
      return
    }
    if (password.length < 8) {
      setError('La password deve essere di almeno 8 caratteri')
      return
    }

    setLoading(true)
    try {
      const res = await fetch('/api/register', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ full_name: fullName.trim(), email: email.trim().toLowerCase(), password }),
      })
      const data = await res.json()
      if (!res.ok) {
        setError(data.error ?? 'Errore durante la registrazione')
        return
      }
      setStep('success')
    } catch {
      setError('Errore di rete. Riprova.')
    } finally {
      setLoading(false)
    }
  }

  if (step === 'success') {
    return (
      <div className="text-center space-y-4">
        <div className="w-12 h-12 rounded-full bg-green-100 dark:bg-green-900/30 flex items-center justify-center mx-auto">
          <svg className="w-6 h-6 text-green-600 dark:text-green-400" fill="none" viewBox="0 0 24 24" stroke="currentColor">
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M5 13l4 4L19 7" />
          </svg>
        </div>
        <h2 className="text-lg font-semibold">Richiesta inviata!</h2>
        <p className="text-sm text-muted-foreground">
          Il tuo account è in attesa di approvazione. Riceverai una notifica non appena
          il servizio viene attivato. Nel frattempo puoi esplorare la demo.
        </p>
        <Button asChild className="w-full">
          <Link href="/login">Accedi alla demo</Link>
        </Button>
      </div>
    )
  }
}

```

```

return (
  <form onSubmit={handleSubmit} className="space-y-4">
    <div className="space-y-2">
      <Label htmlFor="full_name">Nome completo</Label>
      <Input
        id="full_name"
        type="text"
        value={fullName}
        onChange={e => setFullName(e.target.value)}
        placeholder="Mario Bianchi"
        required
        autoComplete="name"
      />
    </div>
    <div className="space-y-2">
      <Label htmlFor="email">Email</Label>
      <Input
        id="email"
        type="email"
        value={email}
        onChange={e => setEmail(e.target.value)}
        placeholder="mario@esempio.it"
        required
        autoComplete="email"
        autoCapitalize="none"
      />
    </div>
    <div className="space-y-2">
      <Label htmlFor="password">Password</Label>
      <Input
        id="password"
        type="password"
        value={password}
        onChange={e => setPassword(e.target.value)}
        placeholder="Minimo 8 caratteri"
        required
        autoComplete="new-password"
      />
    </div>
    <div className="space-y-2">
      <Label htmlFor="confirm">Conferma password</Label>
      <Input
        id="confirm"
        type="password"
        value={confirm}
        onChange={e => setConfirm(e.target.value)}
        placeholder="Ripeti la password"
        required
        autoComplete="new-password"
      />
    </div>
    {error && (
      <p className="text-sm text-destructive">{error}</p>
    )}
    <Button type="submit" className="w-full" disabled={loading}>
      {loading ? 'Invio richiesta...' : 'Richiedi accesso'}
    </Button>
    <p className="text-center text-sm text-muted-foreground">
      Hai già un account?{' '}
      <Link href="/login" className="underline hover:text-foreground">
        Accedi
      </Link>
    </p>
  </form>
)
}

```

src/components/shared/ThemeProvider.tsx

```
'use client'
import { ThemeProvider as NextThemesProvider } from 'next-themes'
import type { ComponentProps } from 'react'

export function ThemeProvider({ children, ...props }: ComponentProps<typeof NextThemesProvider>) {
  return <NextThemesProvider {...props}>{children}</NextThemesProvider>
}
```

src/components/shared/ThemeToggle.tsx

```
'use client'
import { useTheme } from 'next-themes'
import { Sun, Moon } from 'lucide-react'
import { useEffect, useState } from 'react'
import { cn } from '@lib/utils'

export function ThemeToggle({ className }: { className?: string }) {
  const { resolvedTheme, setTheme } = useTheme()
  const [mounted, setMounted] = useState(false)
  useEffect(() => { setMounted(true) }, [])

  // Avoid hydration mismatch – render placeholder until mounted
  if (!mounted) return <div className={cn('w-9 h-9', className)} />

  return (
    <button
      onClick={() => setTheme(resolvedTheme === 'dark' ? 'light' : 'dark')}
      className={cn(
        'w-9 h-9 flex items-center justify-center rounded-md text-muted-foreground',
        'hover:text-foreground hover:bg-accent transition-colors',
        className,
      )}
      aria-label="Cambia tema"
    >
      {resolvedTheme === 'dark'
        ? <Sun className="w-4 h-4" />
        : <Moon className="w-4 h-4" />}
    </button>
  )
}
```

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { cn } from "@lib/utils"

const badgeVariants = cva(
  "inline-flex items-center rounded-full border px-2.5 py-0.5 text-xs font-semibold transition-colors focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2",
  {
    variants: {
      variant: {
        default: "border-transparent bg-primary text-primary-foreground hover:bg-primary/80",
        secondary: "border-transparent bg-secondary text-secondary-foreground hover:bg-secondary/80",
        destructive: "border-transparent bg-destructive text-destructive-foreground hover:bg-destructive/80",
        outline: "text-foreground",
      },
    },
    defaultVariants: { variant: "default" },
  }
)

export interface BadgeProps
  extends React.HTMLAttributes<HTMLDivElement>,
    VariantProps<typeof badgeVariants> {}

function Badge({ className, variant, ...props }: BadgeProps) {
  return <div className={cn(badgeVariants({ variant }), className)} {...props} />
}

export { Badge, badgeVariants }
```

```

import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva, type VariantProps } from "class-variance-authority"
import { cn } from "@/lib/utils"

const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium ring-offset-background transition-colors focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 disabled:pointer-events-none disabled:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground hover:bg-primary/90",
        destructive: "bg-destructive text-destructive-foreground hover:bg-destructive/90",
        outline: "border border-input bg-background hover:bg-accent hover:text-accent-foreground",
        secondary: "bg-secondary text-secondary-foreground hover:bg-secondary/80",
        ghost: "hover:bg-accent hover:text-accent-foreground",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default: "h-10 px-4 py-2",
        sm: "h-9 rounded-md px-3",
        lg: "h-11 rounded-md px-8",
        icon: "h-10 w-10",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
)

export interface ButtonProps
  extends React.ButtonHTMLAttributes<HTMLButtonElement>,
    VariantProps<typeof buttonVariants> {
  asChild?: boolean
}

const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
  ({ className, variant, size, asChild = false, ...props }, ref) => {
    const Comp = asChild ? Slot : "button"
    return (
      <Comp
        className={cn(buttonVariants({ variant, size, className }))}
        ref={ref}
        {...props}
      />
    )
  }
)
Button.displayName = "Button"

export { Button, buttonVariants }

```

```
import * as React from "react"
import { cn } from "@lib/utils"

const Card = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>({
  className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("rounded-lg border bg-card text-card-foreground shadow-sm", className)}
    {...props}
  />
)
Card.displayName = "Card"

const CardHeader = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>({
  className, ...props }, ref) => (
  <div ref={ref} className={cn("flex flex-col space-y-1.5 p-6", className)} {...props} />
)
CardHeader.displayName = "CardHeader"

const CardTitle = React.forwardRef<HTMLParagraphElement, React.HTMLAttributes<HTMLHeadingElement>>({
  className, ...props }, ref) => (
  <h3 ref={ref} className={cn("text-2xl font-semibold leading-none tracking-tight", className)} {...props} />
)
CardTitle.displayName = "CardTitle"

const CardDescription = React.forwardRef<HTMLParagraphElement, React.HTMLAttributes<HTMLParagraphElement>>({
  className, ...props }, ref) => (
  <p ref={ref} className={cn("text-sm text-muted-foreground", className)} {...props} />
)
CardDescription.displayName = "CardDescription"

const CardContent = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>({
  className, ...props }, ref) => (
  <div ref={ref} className={cn("p-6 pt-0", className)} {...props} />
)
CardContent.displayName = "CardContent"

const CardFooter = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>({
  className, ...props }, ref) => (
  <div ref={ref} className={cn("flex items-center p-6 pt-0", className)} {...props} />
)
CardFooter.displayName = "CardFooter"

export { Card, CardHeader, CardFooter, CardTitle, CardDescription, CardContent }
```



```

"use client"
import * as React from "react"
import * as DialogPrimitive from "@radix-ui/react-dialog"
import { X } from "lucide-react"
import { cn } from "@lib/utls"

const Dialog = DialogPrimitive.Root
const DialogTrigger = DialogPrimitive.Trigger
const DialogPortal = DialogPrimitive.Portal
const DialogClose = DialogPrimitive.Close

// Returns true when the originating DOM event happened inside a Radix popper
// layer (Select/Popover/Dropdown/Calendar content is portalled and wrapped in
// `[data-radix-popper-content-wrapper]`). Used to stop those interactions from
// being treated as an "outside click" that would close the surrounding Dialog.
function isInsideRadixPopper(
  e: { detail?: { originalEvent?: Event }; target?: EventTarget | null }
): boolean {
  const originalTarget = e.detail?.originalEvent?.target ?? e.target
  const node = originalTarget as HTMLElement | null
  return !(
    node &&
    typeof node.closest === "function" &&
    node.closest("[data-radix-popper-content-wrapper]")
  )
}

const DialogOverlay = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Overlay>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Overlay>
>(({ className, ...props }, ref) => (
  <DialogPrimitive.Overlay
    ref={ref}
    className={cn(
      "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
      className
    )}
    {...props}
  />
))
DialogOverlay.displayName = DialogPrimitive.Overlay.displayName

const DialogContent = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Content>
>(({ className, children, onPointerDownOutside, onInteractOutside, ...props }, ref) => (
  <DialogPortal>
    <DialogOverlay />
    <DialogPrimitive.Content
      ref={ref}
      onPointerDownOutside={(e) => {
        // Keep the dialog open when the pointer-down happens inside a Radix
        // popper layer (Select / Popover / Dropdown / Calendar). Clicking an
        // option in a portalled popper would otherwise count as an "outside"
        // interaction and tear down the form, losing the user's input.
        if (isInsideRadixPopper(e)) e.preventDefault()
        onPointerDownOutside?.(e)
      }}
      onInteractOutside={(e) => {
        if (isInsideRadixPopper(e)) e.preventDefault()
        onInteractOutside?.(e)
      }}
      className={cn(
        "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border
        bg-background p-6 shadow-lg duration-200 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
        data-[state=closed]:slide-out-to-left-1/2 data-[state=closed]:slide-out-to-top-[48%] data-[state=open]:slide-in-from-left-1/2 data-[state=open]:slide-in-from-top-[48%] sm:rounded-lg",
        className
      )}
      {...props}
    />
  >

```

```

    {children}
    <DialogPrimitive.Close className="absolute right-4 top-4 rounded-sm opacity-70 ring-offset-background
transition-opacity hover:opacity-100 focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2
disabled:pointer-events-none data-[state=open]:bg-accent data-[state=open]:text-muted-foreground">
      <X className="h-4 w-4" />
      <span className="sr-only">Close</span>
    </DialogPrimitive.Close>
  </DialogPrimitive.Content>
</DialogPortal>
))
DialogContent.displayName = DialogPrimitive.Content.displayName

const DialogHeader = ({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) => (
  <div className={cn("flex flex-col space-y-1.5 text-center sm:text-left", className)} {...props} />
)
DialogHeader.displayName = "DialogHeader"

const DialogFooter = ({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) => (
  <div className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)} {...props} />
)
DialogFooter.displayName = "DialogFooter"

const DialogTitle = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Title>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Title>
>(({ className, ...props }, ref) => (
  <DialogPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold leading-none tracking-tight", className)}
    {...props}
  />
))
DialogTitle.displayName = DialogPrimitive.Title.displayName

const DialogDescription = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Description>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Description>
>(({ className, ...props }, ref) => (
  <DialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props}
  />
))
DialogDescription.displayName = DialogPrimitive.Description.displayName

export {
  Dialog, DialogPortal, DialogOverlay, DialogClose, DialogTrigger,
  DialogContent, DialogHeader, DialogFooter, DialogTitle, DialogDescription,
}

```

`src/components/ui/dropdown-menu.tsx`


```

'use client'

import * as React from 'react'
import * as DropdownMenuPrimitive from '@radix-ui/react-dropdown-menu'
import { Check, ChevronRight, Circle } from 'lucide-react'
import { cn } from '@lib/utils'

const DropdownMenu = DropdownMenuPrimitive.Root
const DropdownMenuTrigger = DropdownMenuPrimitive.Trigger
const DropdownMenuGroup = DropdownMenuPrimitive.Group
const DropdownMenuPortal = DropdownMenuPrimitive.Portal
const DropdownMenuSub = DropdownMenuPrimitive.Sub
const DropdownMenuRadioGroup = DropdownMenuPrimitive.RadioGroup

const DropdownMenuSubTrigger = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.SubTrigger>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.SubTrigger> & { inset?: boolean }
>(({ className, inset, children, ...props }, ref) => (
  <DropdownMenuPrimitive.SubTrigger
    ref={ref}
    className={cn(
      'flex cursor-default select-none items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-
accent data-[state=open]:bg-accent [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0',
      inset && 'pl-8',
      className
    )}
    {...props}
  >
    {children}
    <ChevronRight className="ml-auto" />
  </DropdownMenuPrimitive.SubTrigger>
))
DropdownMenuSubTrigger.displayName = DropdownMenuPrimitive.SubTrigger.displayName

const DropdownMenuSubContent = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.SubContent>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.SubContent>
>(({ className, ...props }, ref) => (
  <DropdownMenuPrimitive.SubContent
    ref={ref}
    className={cn(
      'z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-lg data-
[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-
[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2',
      className
    )}
    {...props}
  />
))
DropdownMenuSubContent.displayName = DropdownMenuPrimitive.SubContent.displayName

const DropdownMenuContent = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Content>
>(({ className, sideOffset = 4, ...props }, ref) => (
  <DropdownMenuPrimitive.Portal>
    <DropdownMenuPrimitive.Content
      ref={ref}
      sideOffset={sideOffset}
      className={cn(
        'z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-md
data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-
[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-
from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-
from-bottom-2',
        className
      )}
      {...props}
    />
  </DropdownMenuPrimitive.Portal>
))
DropdownMenuContent.displayName = DropdownMenuPrimitive.Content.displayName

```

```

const DropdownMenuItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Item> & { inset?: boolean }
>(({ className, inset, ...props }, ref) => (
  <DropdownMenuPrimitive.Item
    ref={ref}
    className={cn(
      'relative flex cursor-default select-none items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-none
transition-colors focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-
[disabled]:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0',
      inset && 'pl-8',
      className
    )}
    {...props}
  />
))
DropdownMenuItem.displayName = DropdownMenuPrimitive.Item.displayName

const DropdownMenuCheckboxItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.CheckboxItem>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.CheckboxItem>
>(({ className, children, checked, ...props }, ref) => (
  <DropdownMenuPrimitive.CheckboxItem
    ref={ref}
    className={cn(
      'relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none
transition-colors focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-
[disabled]:opacity-50',
      className
    )}
    checked={checked}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.CheckboxItem>
))
DropdownMenuCheckboxItem.displayName = DropdownMenuPrimitive.CheckboxItem.displayName

const DropdownMenuRadioItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.RadioItem>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.RadioItem>
>(({ className, children, ...props }, ref) => (
  <DropdownMenuPrimitive.RadioItem
    ref={ref}
    className={cn(
      'relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none
transition-colors focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-
[disabled]:opacity-50',
      className
    )}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Circle className="h-2 w-2 fill-current" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.RadioItem>
))
DropdownMenuRadioItem.displayName = DropdownMenuPrimitive.RadioItem.displayName

const DropdownMenuLabel = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Label>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Label> & { inset?: boolean }
>(({ className, inset, ...props }, ref) => (
  <DropdownMenuPrimitive.Label
    ref={ref}

```

```

      className={cn('px-2 py-1.5 text-sm font-semibold', inset && 'pl-8', className)}
      {...props}
    />
  ))
  DropdownMenuLabel.displayName = DropdownMenuPrimitive.Label.displayName

  const DropdownMenuSeparator = React.forwardRef<
    React.ElementRef<typeof DropdownMenuPrimitive.Separator>,
    React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Separator>
  >(({ className, ...props }, ref) => (
    <DropdownMenuPrimitive.Separator
      ref={ref}
      className={cn('-mx-1 my-1 h-px bg-muted', className)}
      {...props}
    />
  ))
  DropdownMenuSeparator.displayName = DropdownMenuPrimitive.Separator.displayName

  const DropdownMenuShortcut = ({ className, ...props }: React.HTMLAttributes<HTMLSpanElement>) => (
    <span className={cn('ml-auto text-xs tracking-widest opacity-60', className)} {...props} />
  )
  DropdownMenuShortcut.displayName = 'DropdownMenuShortcut'

  export {
    DropdownMenu,
    DropdownMenuTrigger,
    DropdownMenuContent,
    DropdownMenuItem,
    DropdownMenuCheckboxItem,
    DropdownMenuRadioItem,
    DropdownMenuLabel,
    DropdownMenuSeparator,
    DropdownMenuShortcut,
    DropdownMenuGroup,
    DropdownMenuPortal,
    DropdownMenuSub,
    DropdownMenuSubContent,
    DropdownMenuSubTrigger,
    DropdownMenuRadioGroup,
  }

```

src/components/ui/input.tsx

```
import * as React from "react"
import { cn } from "@lib/utils"

const NATIVE_PICKERS = new Set(["date", "datetime-local", "month", "week", "time"])

const Input = React.forwardRef<HTMLInputElement, React.ComponentProps<"input">>(>
  ({ className, type, ...props }, ref) => {
    const isNativePicker = type ? NATIVE_PICKERS.has(type) : false
    return (
      <input
        type={type}
        className={cn(
          "flex h-9 w-full rounded-md border border-input bg-background px-3 py-1.5 text-base ring-offset-
background placeholder:text-muted-foreground focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-
ring focus-visible:ring-offset-1 disabled:cursor-not-allowed disabled:opacity-50 file:border-0 file:bg-transparent
file:text-base file:font-medium",
          // Native picker types (month/date/datetime-local/week/time) ship with
          // browser-injected chrome that ignores `h-9` / `w-full`, producing
          // visibly taller and wider controls than sibling SelectTriggers.
          // `appearance-none` strips that chrome; `min-w-0 box-border` lock
          // the box to the Tailwind sizing tokens. `[color-scheme:light]`
          // keeps the picker popup theme-aligned.
          isNativePicker && "[color-scheme:light] appearance-none min-w-0 box-border leading-none",
        )}
        ref={ref}
        {...props}
      />
    )
  }
)
Input.displayName = "Input"

export { Input }
```

src/components/ui/label.tsx

```
"use client"
import * as React from "react"
import * as LabelPrimitive from "@radix-ui/react-label"
import { cva, type VariantProps } from "class-variance-authority"
import { cn } from "@lib/utils"

const labelVariants = cva(
  "text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
)

const Label = React.forwardRef<
  React.ElementRef<typeof LabelPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof LabelPrimitive.Root> &
  VariantProps<typeof labelVariants>
>(({ className, ...props }, ref) => (
  <LabelPrimitive.Root
    ref={ref}
    className={cn(labelVariants(), className)}
    {...props}
  />
))
Label.displayName = LabelPrimitive.Root.displayName

export { Label }
```



```

"use client"
import * as React from "react"
import * as SelectPrimitive from "@radix-ui/react-select"
import { Check, ChevronDown, ChevronUp } from "lucide-react"
import { cn } from "@lib/utls"

const Select = SelectPrimitive.Root
const SelectGroup = SelectPrimitive.Group
const SelectValue = SelectPrimitive.Value

const SelectTrigger = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Trigger>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Trigger>
>(({ className, children, ...props }, ref) => (
  <SelectPrimitive.Trigger
    ref={ref}
    className={cn(
      "flex h-9 w-full items-center justify-between rounded-md border border-input bg-background px-3 py-1.5 text-sm ring-offset-background placeholder:text-muted-foreground focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-1 disabled:cursor-not-allowed disabled:opacity-50 [&span]:line-clamp-1",
      className
    )}
    {...props}
  >
    {children}
    <SelectPrimitive.Icon asChild>
      <ChevronDown className="h-4 w-4 opacity-50" />
    </SelectPrimitive.Icon>
  </SelectPrimitive.Trigger>
))
SelectTrigger.displayName = SelectPrimitive.Trigger.displayName

const SelectScrollUpButton = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.ScrollUpButton>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.ScrollUpButton>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.ScrollUpButton ref={ref} className={cn("flex cursor-default items-center justify-center py-1", className)} {...props}>
    <ChevronUp className="h-4 w-4" />
  </SelectPrimitive.ScrollUpButton>
))
SelectScrollUpButton.displayName = SelectPrimitive.ScrollUpButton.displayName

const SelectScrollDownButton = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.ScrollDownButton>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.ScrollDownButton>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.ScrollDownButton ref={ref} className={cn("flex cursor-default items-center justify-center py-1", className)} {...props}>
    <ChevronDown className="h-4 w-4" />
  </SelectPrimitive.ScrollDownButton>
))
SelectScrollDownButton.displayName = SelectPrimitive.ScrollDownButton.displayName

const SelectContent = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Content>
>(({ className, children, position = "popper", ...props }, ref) => (
  <SelectPrimitive.Portal>
    <SelectPrimitive.Content
      ref={ref}
      className={cn(
        "relative z-50 max-h-96 min-w-[8rem] overflow-hidden rounded-md border bg-popover text-popover-foreground shadow-md data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
        position === "popper" &&
        "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1 data-[side=right]:translate-x-1 data-[side=top]:-translate-y-1",
        className
      )}
      position={position}
    >
      {children}
    </SelectPrimitive.Content>
  </SelectPrimitive.Portal>
))

```

```

    {...props}
  >
    <SelectScrollUpButton />
    <SelectPrimitive.Viewport className={cn("p-1", position === "popper" && "h-[var(--radix-select-trigger-height)] w-full min-w-[var(--radix-select-trigger-width)]")}>
      {children}
    </SelectPrimitive.Viewport>
    <SelectScrollDownButton />
  </SelectPrimitive.Content>
</SelectPrimitive.Portal>
))
SelectContent.displayName = SelectPrimitive.Content.displayName

const SelectLabel = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Label>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Label>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.Label ref={ref} className={cn("py-1.5 pl-8 pr-2 text-sm font-semibold", className)}
  {...props} />
))
SelectLabel.displayName = SelectPrimitive.Label.displayName

const SelectItem = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Item>
>(({ className, children, ...props }, ref) => (
  <SelectPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex w-full cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-
      none focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      className
    )}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <SelectPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </SelectPrimitive.ItemIndicator>
    </span>
    <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
  </SelectPrimitive.Item>
))
SelectItem.displayName = SelectPrimitive.Item.displayName

const SelectSeparator = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Separator>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Separator>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.Separator ref={ref} className={cn("-mx-1 my-1 h-px bg-muted", className)} {...props} />
))
SelectSeparator.displayName = SelectPrimitive.Separator.displayName

export {
  Select, SelectGroup, SelectValue, SelectTrigger, SelectContent,
  SelectLabel, SelectItem, SelectSeparator, SelectScrollUpButton, SelectScrollDownButton,
}

```

`src/components/ui/separator.tsx`

```
"use client"
import * as React from "react"
import * as SeparatorPrimitive from "@radix-ui/react-separator"
import { cn } from "@lib/utils"

const Separator = React.forwardRef<
  React.ElementRef<typeof SeparatorPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof SeparatorPrimitive.Root>
>(({ className, orientation = "horizontal", decorative = true, ...props }, ref) => (
  <SeparatorPrimitive.Root
    ref={ref}
    decorative={decorative}
    orientation={orientation}
    className={cn(
      "shrink-0 bg-border",
      orientation === "horizontal" ? "h-[1px] w-full" : "h-full w-[1px]",
      className
    )}
    {...props}
  />
))
Separator.displayName = SeparatorPrimitive.Root.displayName

export { Separator }
```

`src/components/ui/skeleton.tsx`

```
import { cn } from "@lib/utils"

function Skeleton({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) {
  return (
    <div
      className={cn("skeleton-shimmer rounded-md", className)}
      {...props}
    />
  )
}

export { Skeleton }
```

```
"use client"
import * as React from "react"
import * as SwitchPrimitives from "@radix-ui/react-switch"
import { cn } from "@lib/utils"

const Switch = React.forwardRef<
  React.ElementRef<typeof SwitchPrimitives.Root>,
  React.ComponentPropsWithoutRef<typeof SwitchPrimitives.Root>
>(({ className, ...props }, ref) => (
  <SwitchPrimitives.Root
    className={cn(
      "peer inline-flex h-6 w-11 shrink-0 cursor-pointer items-center rounded-full border-2 border-transparent
      transition-colors focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-
      offset-2 focus-visible:ring-offset-background disabled:cursor-not-allowed disabled:opacity-50 data-
      [state=checked]:bg-primary data-[state=unchecked]:bg-input",
      className
    )}
    {...props}
    ref={ref}
  >
    <SwitchPrimitives.Thumb
      className={cn(
        "pointer-events-none block h-5 w-5 rounded-full bg-background shadow-lg ring-0 transition-transform data-
        [state=checked]:translate-x-5 data-[state=unchecked]:translate-x-0"
      )}
    />
  </SwitchPrimitives.Root>
))
Switch.displayName = SwitchPrimitives.Root.displayName

export { Switch }
```

```

"use client"
import * as React from "react"
import * as TabsPrimitive from "@radix-ui/react-tabs"
import { cn } from "@/lib/utils"

const Tabs = TabsPrimitive.Root

const TabsList = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.List>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.List>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.List
    ref={ref}
    className={cn("inline-flex h-10 items-center justify-center rounded-md bg-muted p-1 text-muted-foreground",
      className)}
    {...props}
  />
))
TabsList.displayName = TabsPrimitive.List.displayName

const TabsTrigger = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.Trigger>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.Trigger>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.Trigger
    ref={ref}
    className={cn(
      "inline-flex items-center justify-center whitespace-nowrap rounded-sm px-3 py-1.5 text-sm font-medium ring-
      offset-background transition-all focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-
      visible:ring-offset-2 disabled:pointer-events-none disabled:opacity-50 data-[state=active]:bg-background data-
      [state=active]:text-foreground data-[state=active]:shadow-sm",
      className
    )}
    {...props}
  />
))
TabsTrigger.displayName = TabsPrimitive.Trigger.displayName

const TabsContent = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.Content>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.Content
    ref={ref}
    className={cn("mt-2 ring-offset-background focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-
    ring focus-visible:ring-offset-2", className)}
    {...props}
  />
))
TabsContent.displayName = TabsPrimitive.Content.displayName

export { Tabs, TabsList, TabsTrigger, TabsContent }

```

`src/components/ui/textarea.tsx`

```
import * as React from "react"
import { cn } from "@lib/utils"

const Textarea = React.forwardRef<HTMLTextAreaElement, React.ComponentProps<"textarea">>(
  ({ className, ...props }, ref) => (
    <textarea
      className={cn(
        "flex min-h-[80px] w-full rounded-md border border-input bg-background px-3 py-2 text-sm ring-offset-
background placeholder:text-muted-foreground focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-
ring focus-visible:ring-offset-2 disabled:cursor-not-allowed disabled:opacity-50",
        className
      )}
      ref={ref}
      {...props}
    />
  )
)
Textarea.displayName = "Textarea"

export { Textarea }
```



```

'use client'
import { useEffect, useRef, useState } from 'react'
import { Clock } from 'lucide-react'
import { cn } from '@lib/utils'

interface TimeInputProps {
  value: string // "HH:mm" oppure ""
  onChange: (value: string) => void
  className?: string
  disabled?: boolean
}

// Su iOS Safari <input type="time"> apre SOLO la rotella nativa: non c'è
// modo di digitare le cifre da tastiera in quel controllo. Qui affianchiamo
// un campo testuale (tastiera numerica, digitazione libera con inserimento
// automatico dei ":") a un <input type="time"> nascosto che resta
// raggiungibile tramite l'icona orologio, per chi preferisce la rotella.
export function TimeInput({ value, onChange, className, disabled }: TimeInputProps) {
  const [text, setText] = useState(formatDisplay(value))
  const nativeRef = useRef<HTMLInputElement>(null)

  useEffect(() => { setText(formatDisplay(value)) }, [value])

  function formatDisplay(v: string): string {
    return v ? v.slice(0, 5) : ''
  }

  function handleTextChange(e: React.ChangeEvent<HTMLInputElement>) {
    const digits = e.target.value.replace(/\D/g, '').slice(0, 4)
    const formatted = digits.length > 2 ? `${digits.slice(0, 2)}:${digits.slice(2)}` : digits
    setText(formatted)

    if (formatted.length === 5) {
      const [h, m] = formatted.split(':').map(Number)
      if (h <= 23 && m <= 59) onChange(formatted)
    } else if (formatted === '') {
      onChange('')
    }
  }

  function handleBlur() {
    if (text.length === 5) {
      const [h, m] = text.split(':').map(Number)
      if (h <= 23 && m <= 59) return
    }
    // Valore incompleto o non valido – torna all'ultimo orario valido
    setText(formatDisplay(value))
  }

  function openNativePicker() {
    const el = nativeRef.current
    if (!el) return
    // showPicker() è il modo corretto quando supportato; su iOS Safari
    // (che non lo espone) il focus/click nel gestore del tap apre comunque
    // la rotella, essendo dentro lo stesso gesto utente.
    if (typeof el.showPicker === 'function') {
      try { el.showPicker(); return } catch { /* fallback sotto */ }
    }
    el.focus()
    el.click()
  }

  return (
    <div className="relative flex items-center w-full">
      <input
        type="text"
        inputMode="numeric"
        autoComplete="off"
        placeholder="HH:MM"
        value={text}
        onChange={handleTextChange}
        onBlur={handleBlur}
        maxLength={5}

```

```

        disabled={disabled}
        className={cn(
          "flex h-9 w-full rounded-md border border-input bg-background px-3 py-1.5 pr-9 text-base tabular-nums
          ring-offset-background placeholder:text-muted-foreground focus-visible:outline-none focus-visible:ring-2 focus-
          visible:ring-ring focus-visible:ring-offset-1 disabled:cursor-not-allowed disabled:opacity-50",
          className
        )}
      />
      <button
        type="button"
        onClick={openNativePicker}
        disabled={disabled}
        tabIndex={-1}
        aria-label="Apri selettore orario"
        className="absolute right-1 flex h-7 w-7 items-center justify-center rounded-sm text-muted-foreground
        hover:text-foreground hover:bg-accent disabled:pointer-events-none transition-colors"
      >
        <Clock className="w-4 h-4" />
      </button>
      <input
        ref={nativeRef}
        type="time"
        value={value}
        onChange={e => onChange(e.target.value)}
        disabled={disabled}
        tabIndex={-1}
        aria-hidden="true"
        className="sr-only"
      />
    </div>
  )
}

```



```

"use client"
import * as React from "react"
import * as ToastPrimitive from "@radix-ui/react-toast"
import { X } from "lucide-react"
import { cn } from "@lib/utils"

const ToastProvider = ToastPrimitive.Provider
const ToastViewport = React.forwardRef<
  React.ElementRef<typeof ToastPrimitive.Viewport>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitive.Viewport>
>(({ className, ...props }, ref) => (
  <ToastPrimitive.Viewport
    ref={ref}
    className={cn("fixed bottom-0 right-0 z-[100] flex max-h-screen w-full flex-col-reverse p-4 sm:bottom-0
sm:right-0 sm:top-auto sm:flex-col md:max-w-[420px]", className)}
    {...props}
  />
))
ToastViewport.displayName = ToastPrimitive.Viewport.displayName

const Toast = React.forwardRef<
  React.ElementRef<typeof ToastPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitive.Root> & { variant?: "default" | "destructive" }
>(({ className, variant = "default", ...props }, ref) => (
  <ToastPrimitive.Root
    ref={ref}
    className={cn(
      "group pointer-events-auto relative flex w-full items-center justify-between space-x-4 overflow-hidden
rounded-md border p-6 pr-8 shadow-lg transition-all",
      variant === "destructive" ? "border-destructive bg-destructive text-destructive-foreground" : "border bg-
background text-foreground",
      className
    )}
    {...props}
  />
))
Toast.displayName = ToastPrimitive.Root.displayName

const ToastClose = React.forwardRef<
  React.ElementRef<typeof ToastPrimitive.Close>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitive.Close>
>(({ className, ...props }, ref) => (
  <ToastPrimitive.Close
    ref={ref}
    className={cn("absolute right-2 top-2 rounded-md p-1 opacity-0 transition-opacity group-hover:opacity-100",
className)}
    {...props}
  >
    <X className="h-4 w-4" />
  </ToastPrimitive.Close>
))
ToastClose.displayName = ToastPrimitive.Close.displayName

const ToastTitle = React.forwardRef<
  React.ElementRef<typeof ToastPrimitive.Title>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitive.Title>
>(({ className, ...props }, ref) => (
  <ToastPrimitive.Title ref={ref} className={cn("text-sm font-semibold", className)} {...props} />
))
ToastTitle.displayName = ToastPrimitive.Title.displayName

const ToastDescription = React.forwardRef<
  React.ElementRef<typeof ToastPrimitive.Description>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitive.Description>
>(({ className, ...props }, ref) => (
  <ToastPrimitive.Description ref={ref} className={cn("text-sm opacity-90", className)} {...props} />
))
ToastDescription.displayName = ToastPrimitive.Description.displayName

export { ToastProvider, ToastViewport, Toast, ToastClose, ToastTitle, ToastDescription }

```

`src/contexts/AccountStatusContext.tsx`

```
'use client'
import { createContext, useContext } from 'react'

interface AccountStatusContextValue {
  isPending: boolean
}

const AccountStatusContext = createContext<AccountStatusContextValue>({ isPending: false })

export function AccountStatusProvider({
  isPending,
  children,
}: {
  isPending: boolean
  children: React.ReactNode
}) {
  return (
    <AccountStatusContext.Provider value={{ isPending }}>
      {children}
    </AccountStatusContext.Provider>
  )
}

export function useAccountStatus() {
  return useContext(AccountStatusContext)
}
```

`src/hooks/useActivityTracker.ts`

```
'use client'
import { useEffect, useRef } from 'react'

// Throttle: fire at most once every 3 minutes.
// Listens to mousemove, keydown, touchstart – stops updating when the user is idle.
const THROTTLE_MS = 3 * 60 * 1000

export function useActivityTracker() {
  const lastFiredRef = useRef<number>(0)

  useEffect(() => {
    function handleActivity() {
      const now = Date.now()
      if (now - lastFiredRef.current < THROTTLE_MS) return
      lastFiredRef.current = now
      fetch('/api/activity', { method: 'POST' }).catch(() => undefined)
    }

    const opts = { passive: true } as AddEventListenerOptions
    window.addEventListener('mousemove', handleActivity, opts)
    window.addEventListener('keydown', handleActivity, opts)
    window.addEventListener('touchstart', handleActivity, opts)

    // Ping on mount so last_active_at is set immediately on page load
    handleActivity()

    return () => {
      window.removeEventListener('mousemove', handleActivity)
      window.removeEventListener('keydown', handleActivity)
      window.removeEventListener('touchstart', handleActivity)
    }
  }, [])
}
```

```
import { useEffect } from 'react'

type BadgingNavigator = Navigator & {
  setAppBadge?(count?: number): Promise<void>
  clearAppBadge?(): Promise<void>
}

export function useBadging(count: number) {
  useEffect(() => {
    if (typeof window === 'undefined') return
    const nav = navigator as BadgingNavigator
    if (!nav.setAppBadge) return
    if (count > 0) {
      nav.setAppBadge(count).catch(() => {})
    } else {
      nav.clearAppBadge?.().catch(() => {})
    }
  }, [count])
}
```



```

import { useRef, useState } from 'react'

const RAGGIO_MAX = 100 // metres

// Indoors the GPS signal is weak and the device falls back to Wi-Fi/cell
// positioning, which reports a position with a large uncertainty radius
// (`coords.accuracy`). Ignoring that radius is what makes a person standing
// *inside* the restaurant read as "too far". We treat the timbratura as valid
// when the geofence circle and the GPS uncertainty circle overlap – i.e. the
// reported distance, minus the accuracy radius, is still within RAGGIO_MAX.
// The buffer is capped so a hopeless fix (accuracy of several km) can't let
// someone timbrare from anywhere.
const ACCURACY_BUFFER_MAX = 250 // metres

function haversine(lat1: number, lon1: number, lat2: number, lon2: number): number {
  const R = 6_371_000
  const φ1 = (lat1 * Math.PI) / 180
  const φ2 = (lat2 * Math.PI) / 180
  const Δφ = ((lat2 - lat1) * Math.PI) / 180
  const Δλ = ((lon2 - lon1) * Math.PI) / 180
  const a = Math.sin(Δφ / 2) ** 2 + Math.cos(φ1) * Math.cos(φ2) * Math.sin(Δλ / 2) ** 2
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a))
}

export type GeofenceStatus = 'idle' | 'checking' | 'inside' | 'outside' | 'denied' | 'unsupported'

export function useGeofence() {
  const [status, setStatus] = useState<GeofenceStatus>('idle')
  const [distance, setDistance] = useState<number | null>(null)
  // Last device coordinates obtained – forwarded to the server for validation
  const userCoordsRef = useRef<{ latitude: number; longitude: number; accuracy: number | null } | null>(null)

  function check(restaurantLat?: number | null, restaurantLng?: number | null): Promise<GeofenceStatus> {
    // Fail-closed: no geolocation API → block the action
    if (!('geolocation' in navigator)) {
      setStatus('denied')
      return Promise.resolve('denied')
    }

    setStatus('checking')
    return new Promise(resolve => {
      navigator.geolocation.getCurrentPosition(
        pos => {
          userCoordsRef.current = {
            latitude: pos.coords.latitude,
            longitude: pos.coords.longitude,
            accuracy: pos.coords.accuracy ?? null,
          }

          if (restaurantLat != null && restaurantLng != null) {
            const dist = haversine(pos.coords.latitude, pos.coords.longitude, restaurantLat, restaurantLng)
            setDistance(dist)
            // Give credit for GPS uncertainty (capped) so a weak indoor fix
            // doesn't reject someone who is actually inside.
            const buffer = Math.min(pos.coords.accuracy ?? 0, ACCURACY_BUFFER_MAX)
            const s: GeofenceStatus = dist <= RAGGIO_MAX + buffer ? 'inside' : 'outside'
            setStatus(s)
            resolve(s)
          } else {
            // No restaurant coordinates configured – position obtained, allow
            setDistance(null)
            setStatus('inside')
            resolve('inside')
          }
        },
        () => {
          // Fail-closed: permission denied, timeout, or position unavailable
          // all block the scan.
          setStatus('denied')
          resolve('denied')
        },
        { enableHighAccuracy: true, timeout: 8000, maximumAge: 60_000 },
      )
    })
  }
}

```

```
    })  
  }  
  
  return { status, distance, check, userCoordsRef }  
}
```

```

'use client'
import { useEffect, useState, useCallback } from 'react'

export type PushPermission = 'default' | 'granted' | 'denied' | 'unsupported'

export function usePushNotifications() {
  const [permission, setPermission] = useState<PushPermission>('default')

  // Leggi stato iniziale e registra SW silenziosamente (senza chiedere permesso)
  useEffect(() => {
    if (typeof window === 'undefined') return
    if (!('serviceWorker' in navigator) || !('PushManager' in window) || !('Notification' in window)) {
      setPermission('unsupported')
      return
    }
    setPermission(Notification.permission as PushPermission)
    // updateViaCache: 'none' - il browser non usa la cache HTTP per sw.js,
    // così una nuova versione del service worker viene scaricata subito;
    // reg.update() forza il controllo aggiornamenti ad ogni apertura.
    navigator.serviceWorker
      .register('/sw.js', { updateViaCache: 'none' })
      .then(reg => reg.update())
      .catch(() => {})

    // Quando un nuovo service worker prende il controllo (dopo un deploy),
    // ricarica la pagina UNA volta così i bundle/dati nuovi vengono caricati
    // subito, senza dover chiudere e riaprire l'app più volte. Ricarichiamo
    // solo se una versione precedente era già al controllo (utente che torna):
    // così evitiamo un reload inutile al primissimo install.
    const hadController = !!navigator.serviceWorker.controller
    let refreshing = false
    const onControllerChange = () => {
      if (refreshing || !hadController) return
      refreshing = true
      window.location.reload()
    }
    navigator.serviceWorker.addEventListener('controllerchange', onControllerChange)
    return () => navigator.serviceWorker.removeEventListener('controllerchange', onControllerChange)
  }, [])

  // Se il permesso è già concesso (utente che ritorna), assicura che la subscription esista
  useEffect(() => {
    if (typeof window === 'undefined') return
    if (!('serviceWorker' in navigator) || !('PushManager' in window)) return
    if (Notification.permission !== 'granted') return

    navigator.serviceWorker.ready.then(async (reg) => {
      const existing = await reg.pushManager.getSubscription()
      if (existing) return
      // Subscription scaduta - rinnova silenziosamente
      const sub = await reg.pushManager.subscribe({
        userVisibleOnly: true,
        applicationServerKey: process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY,
      })
      await fetch('/api/push/subscribe', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(sub),
      })
    }).catch(() => {})
  }, [])

  // Chiamato esplicitamente dal click dell'utente sul banner
  const subscribe = useCallback(async () => {
    if (typeof window === 'undefined') return
    if (!('serviceWorker' in navigator) || !('PushManager' in window)) return
    try {
      const perm = await Notification.requestPermission()
      setPermission(perm as PushPermission)
      if (perm !== 'granted') return

      const reg = await navigator.serviceWorker.ready
      let sub = await reg.pushManager.getSubscription()
    }
  }, [])

```

```

    if (!sub) {
      sub = await reg.pushManager.subscribe({
        userVisibleOnly: true,
        applicationServerKey: process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY,
      })
    }
    await fetch('/api/push/subscribe', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(sub),
    })
  } catch {
    // Browser non supporta push o utente ha negato
  }
}, [])

return { permission, subscribe }
}

```

src/lib/attendanceTime.ts

```

import { fromZonedTime } from 'date-fns-tz'
import { addDays, format, parseISO } from 'date-fns'

// Calcola i timestamp ISO di ingresso/uscita di una presenza. Se l'uscita è
// numericamente "prima" dell'ingresso (es. 15:00 → 01:00) si tratta di un
// turno notturno che scavalca la mezzanotte, non di un errore di battitura:
// l'uscita viene automaticamente spostata al giorno successivo.
export function computeAttendanceIso(
  tz: string,
  date: string,
  checkIn: string,
  checkOut?: string | null,
): { checkInIso: string; checkOutIso: string | null } {
  const checkInIso = fromZonedTime(`${date}T${checkIn}:00`, tz).toISOString()
  if (!checkOut) return { checkInIso, checkOutIso: null }

  let checkOutIso = fromZonedTime(`${date}T${checkOut}:00`, tz).toISOString()
  if (checkOutIso <= checkInIso) {
    const nextDay = format(addDays(parseISO(`${date}T00:00:00`), 1), 'yyyy-MM-dd')
    checkOutIso = fromZonedTime(`${nextDay}T${checkOut}:00`, tz).toISOString()
  }
  return { checkInIso, checkOutIso }
}

```



```
import type { SupabaseClient } from '@supabase/supabase-js'

// When an employee forgets to timbrare l'uscita, the shift stays open
// indefinitely (running counter, blocked next check-in, wrong report hours).
// We detect a forgotten check-out as a shift left open for more than
// STALE_AFTER_HOURS, and close it automatically at check_in + AUTO_DURATION_HOURS.
const STALE_AFTER_HOURS = 16
const AUTO_DURATION_HOURS = 7

/**
 * Auto-close shifts left open past the stale threshold.
 *
 * @param client Supabase client. Pass an admin client to sweep all users,
 *               or a user-scoped client (RLS) to close only the caller's own.
 * @param userId Optional - restrict the sweep to a single employee.
 * @returns number of shifts closed.
 */
export async function autoCloseStaleShifts(
  client: SupabaseClient,
  userId?: string,
): Promise<number> {
  const cutoffIso = new Date(Date.now() - STALE_AFTER_HOURS * 3_600_000).toISOString()

  let query = client
    .from('attendances')
    .select('id, check_in')
    .is('check_out', null)
    .lt('check_in', cutoffIso)

  if (userId) query = query.eq('user_id', userId)

  const { data: stale, error } = await query
  if (error || !stale || stale.length === 0) return 0

  await Promise.all(
    stale.map(row => {
      const checkOut = new Date(
        new Date(row.check_in).getTime() + AUTO_DURATION_HOURS * 3_600_000,
      ).toISOString()
      return client.from('attendances').update({ check_out: checkOut }).eq('id', row.id)
    })
  )

  return stale.length
}
```

```
// Client-side image compression using a canvas – no dependencies.
// Resizes to maxWidth keeping aspect ratio, then encodes as JPEG at the given quality.
export async function compressImage(
  file: File,
  maxWidth = 800,
  quality = 0.7,
): Promise<File> {
  return new Promise((resolve, reject) => {
    const objectUrl = URL.createObjectURL(file)
    const img = new Image()

    img.onload = () => {
      URL.revokeObjectURL(objectUrl)

      const scale = Math.min(1, maxWidth / img.naturalWidth)
      const w = Math.round(img.naturalWidth * scale)
      const h = Math.round(img.naturalHeight * scale)

      const canvas = document.createElement('canvas')
      canvas.width = w
      canvas.height = h

      const ctx = canvas.getContext('2d')
      if (!ctx) { reject(new Error('Canvas non supportato')); return }
      ctx.drawImage(img, 0, 0, w, h)

      canvas.toBlob(
        blob => {
          if (!blob) { reject(new Error('Compressione fallita')); return }
          // Preserve original name but force .jpg extension
          const name = file.name.replace(/\.[^.]*/, '.jpg')
          resolve(new File([blob], name, { type: 'image/jpeg' }))
        },
        'image/jpeg',
        quality,
      )
    }

    img.onerror = () => {
      URL.revokeObjectURL(objectUrl)
      reject(new Error("Impossibile leggere l'immagine"))
    }

    img.src = objectUrl
  })
}
```



```

import { createAdminClient } from '@lib/supabase/admin'
import { addDays, format, startOfWeek, addWeeks, subDays, getDay } from 'date-fns'

// — Restaurant configs —

const RESTAURANT_CONFIGS = [
  { name: 'Ristorante Da Marco', address: 'Via Roma 1, Milano', closing_days: [0], departments:
['Sala', 'Cucina', 'Bar'] },
  { name: 'Trattoria La Piazzetta', address: 'Corso Vittorio 22, Torino', closing_days: [0, 1], departments:
['Sala', 'Cucina'] },
  { name: 'Pizzeria Napoli DOC', address: 'Via Napoli 5, Roma', closing_days: [0], departments:
['Pizzeria', 'Sala', 'Bar'] },
  { name: 'Bistrot del Porto', address: 'Lungomare 14, Genova', closing_days: [1], departments:
['Sala', 'Bar'] },
  { name: 'Osteria dei Sapori', address: 'Via Garibaldi 8, Firenze', closing_days: [0], departments:
['Sala', 'Cucina'] },
]

// 10 unique names per restaurant
const EMPLOYEE_NAMES = [
  'Marco Rossi', 'Giulia Bianchi', 'Luca Ferrari', 'Anna Conti', 'Paolo Mancini', 'Sara Romano', 'Davide Greco',
  'Elena Russo', 'Matteo Esposito', 'Laura Fontana',
  'Andrea Colombo', 'Chiara Ricci', 'Roberto Marino', 'Valeria Bruno', 'Simone Gallo', 'Francesca Costa',
  'Emanuele Vitale', 'Alessia Lombardi', 'Fabrizio Serra', 'Marta Barbieri',
  'Antonio Ferrara', 'Carmela Esposito', 'Francesco Russo', 'Ciro Mancini', 'Salvatore Greco', 'Rosaria Conti',
  'Vincenzo Napoli', 'Maria Romano', 'Luigi Dangelo', 'Teresa Giordano',
  'Giovanni Parodi', 'Elisa Moretti', 'Stefano Barrali', 'Claudia Ferretti', 'Daniele Marini', 'Valentina Gatti',
  'Roberto Pellegrini', 'Serena Martini', 'Diego Caruso', 'Beatrice Neri',
  'Alessandro Toscani', 'Monica Fabbri', 'Enrico Pellegrino', 'Isabella Riva', 'Cristian Moro', 'Silvia
Padovani', 'Nicola Fuoco', 'Patrizia Longo', 'Filippo Angeli', 'Carmen Ruggieri',
]

// Shift times [check-in offset minutes, start, end] by department
const SHIFT_TIMES: Record<string, { morning: [string, string]; evening: [string, string] }> = {
  Sala: { morning: ['11:30', '16:00'], evening: ['18:00', '23:30'] },
  Cucina: { morning: ['10:00', '15:30'], evening: ['17:00', '23:00'] },
  Bar: { morning: ['08:00', '14:00'], evening: ['14:00', '20:00'] },
  Pizzeria: { morning: ['11:00', '15:00'], evening: ['18:00', '23:00'] },
}

// Shift slot definitions per department
const SLOT_TEMPLATES: Record<string, { name: string; start_time: string; end_time: string; required_count:
number }> = {
  Sala: [{ name: 'Pranzo', start_time: '11:30', end_time: '16:00', required_count: 2 }, { name: 'Cena',
start_time: '18:00', end_time: '23:30', required_count: 2 }],
  Cucina: [{ name: 'Pranzo', start_time: '10:00', end_time: '15:30', required_count: 1 }, { name: 'Cena',
start_time: '17:00', end_time: '23:00', required_count: 1 }],
  Bar: [{ name: 'Apertura', start_time: '08:00', end_time: '14:00', required_count: 1 }, { name:
'Pomeriggio', start_time: '14:00', end_time: '20:00', required_count: 1 }],
  Pizzeria: [{ name: 'Pranzo', start_time: '11:00', end_time: '15:00', required_count: 1 }, { name: 'Cena',
start_time: '18:00', end_time: '23:00', required_count: 2 }],
}

// ODS task templates per role: [title, type, recurrence_days, department or null]
const ODS_RESTAURANT_TASKS = [
  { title: 'Apertura e controllo cassa', type: 'quotidiana', recurrence_days: ['lunedì', 'martedì', 'mercoledì',
'giovedì', 'venerdì', 'sabato'], department: 'Sala' },
  { title: 'Controllo scorte magazzino', type: 'settimanale', recurrence_days: ['lunedì'], department: null },
  { title: 'Pulizia generale locali', type: 'quotidiana', recurrence_days: ['lunedì', 'martedì', 'mercoledì',
'giovedì', 'venerdì', 'sabato'], department: null },
]

const ODS_DEPT_TASKS: Record<string, { title: string; type: string; recurrence_days: string[] }> = {
  Sala: [{ title: 'Mise en place tavoli', type: 'quotidiana', recurrence_days:
['lunedì', 'martedì', 'mercoledì', 'giovedì', 'venerdì', 'sabato'] }, { title: 'Pulizia e riassetto sala', type:
'quotidiana', recurrence_days: ['lunedì', 'martedì', 'mercoledì', 'giovedì', 'venerdì', 'sabato'] }],
  Cucina: [{ title: 'Sanificazione piani cottura', type: 'quotidiana', recurrence_days:
['lunedì', 'martedì', 'mercoledì', 'giovedì', 'venerdì', 'sabato'] }, { title: 'Inventario frigoriferi', type:
'bisettimanale', recurrence_days: ['lunedì', 'giovedì'] }],
  Bar: [{ title: 'Pulizia macchina caffè', type: 'quotidiana', recurrence_days:
['lunedì', 'martedì', 'mercoledì', 'giovedì', 'venerdì', 'sabato'] }, { title: 'Ordine bevande', type: 'settimanale',
recurrence_days: ['lunedì'] }],
  Pizzeria: [{ title: 'Preparazione impasti', type: 'quotidiana', recurrence_days:

```

```

['lunedì', 'martedì', 'mercoledì', 'giovedì', 'venerdì', 'sabato'] }, { title: 'Pulizia forno', type: 'bisettimanale',
recurrence_days: ['mercoledì', 'sabato'] }],
}

// Bulletin templates
const BULLETIN_TEMPLATES = [
  { title: 'Benvenuti nel nuovo anno lavorativo!', body: 'Cari colleghi, con l'inizio della nuova stagione vi
diamo il benvenuto. Ricordate che il rispetto degli orari è fondamentale per garantire un servizio di qualità.
Buon lavoro a tutti!', target: 'all' as const },
  { title: 'Aggiornamento turni estivi', body: 'A partire da luglio i turni serali si estenderanno fino alle 00:00
per venire incontro alla maggiore affluenza estiva. Il calendario aggiornato è disponibile nella sezione turni.',
target: 'all' as const },
  { title: 'Novità menù stagionale', body: 'Comunichiamo che a partire dalla prossima settimana verrà introdotto
il nuovo menù stagionale. Tutti i dipendenti di sala sono invitati a prendere visione delle nuove proposte prima
del servizio.', target: 'department' as const, target_department: 'Sala' },
  { title: 'Nuove procedure HACCP cucina', body: 'In seguito al controllo sanitario del mese scorso, sono state
aggiornate le procedure HACCP. Il documento aggiornato è affisso in cucina. Si prega di firmarlo entro venerdì.',
target: 'department' as const, target_department: 'Cucina' },
  { title: 'Promemoria: divisa estiva', body: 'Dal 1° giugno è obbligatorio indossare la divisa estiva. Chi non ne
fosse ancora in possesso contatti il responsabile per il ritiro.', target: 'all' as const },
  { title: 'Comunicazione reparto bar', body: 'Si ricorda al personale del bar di aggiornare quotidianamente il
registro delle temperature frigoriferi. Non farlo costituisce infrazione alle norme igieniche.', target:
'department' as const, target_department: 'Bar' },
  { title: 'Riunione mensile staff', body: 'Si convoca riunione di tutto il personale per lunedì prossimo alle
10:00. La partecipazione è obbligatoria. Ordine del giorno: andamento stagione, proposte miglioramento, varie ed
eventuali.', target: 'role' as const, target_roles: ['capo_servizio', 'dipendente'] },
]

// — Helpers —————

function nameToUsername(name: string, uid: string, restIdx: number): string {
  return name.toLowerCase().replace(/s+/g, '.').replace(/^[a-z0-9.]/g, '') + `.r${restIdx}.${uid}`
}

// Build ISO UTC timestamp from local Italian date + time (CEST = UTC+2)
function ts(dateStr: string, localTime: string, offsetMinutes = 0): string {
  const [hh, mm] = localTime.split(':').map(Number)
  const totalMin = hh * 60 + mm + offsetMinutes - 120 // subtract CEST offset
  const h = Math.floor(((totalMin % 1440) + 1440) % 1440 / 60)
  const m = Math.abs(totalMin % 60)
  return `${dateStr}T${String(h).padStart(2, '0')}:${String(m).padStart(2, '0')}:00.000Z`
}

function getEmployeeDept(departments: string[], empIdx: number): string {
  if (empIdx === 0) return departments[0]
  const rest = empIdx - 1 // 0-8
  if (departments.length === 1) return departments[0]
  if (departments.length === 2) return rest < 5 ? departments[0] : departments[1]
  // 3 depts: 3 each
  return departments[Math.floor(rest / 3) % departments.length]
}

// — Main: create all demo data —————

export async function createDemoData(managerId: string): Promise<string[]> {
  const admin = createAdminClient()
  const uid = managerId.slice(0, 8)
  const today = new Date()

  // — 1. Create 5 demo restaurants —————
  const { data: restaurants, error: restErr } = await admin
    .from('restaurants')
    .insert(RESTAURANT_CONFIGS.map((r, i) => ({
      name: r.name,
      address: r.address,
      closing_days: r.closing_days,
      qr_secret: `demo-${managerId}-${i}`,
      is_demo: true,
      owner_id: managerId,
    })))
    .select('id, closing_days')

  if (restErr || !restaurants?.length) throw new Error('Errore creazione ristoranti demo: ' + restErr?.message)
}

```

```

const restaurantIds = restaurants.map(r => r.id)

// — 2. Create employees for each restaurant (parallel) —————
const allEmployeeProfiles: { id: string; department: string; restaurant_id: string; rest_day_offset: number }[]
= []

await Promise.all(restaurantIds.map(async (restaurantId, rIdx) => {
  const cfg = RESTAURANT_CONFIGS[rIdx]
  const names = EMPLOYEE_NAMES[rIdx]

  const createdProfiles = await Promise.all(names.map(async (empName, empIdx) => {
    const dept = getEmployeeDept(cfg.departments, empIdx)
    const role = empIdx === 0 ? 'capo_servizio' : 'dipendente'
    const username = nameToUsername(empName, uid, rIdx)
    const email = `${username}@demo.struttura.local`

    const { data: authData, error: authErr } = await admin.auth.admin.createUser({
      email,
      password: 'demo-pass-' + uid,
      email_confirm: true,
      user_metadata: { full_name: empName, role },
    })
    if (authErr || !authData?.user) return null

    await admin.from('profiles').upsert({
      id: authData.user.id,
      full_name: empName,
      username,
      role,
      department: dept,
      restaurant_id: restaurantId,
      account_status: 'active',
      is_direttore: empIdx === 0,
      can_post_bulletin: empIdx === 0,
      weekly_rest_days: 1,
      primary_slot_ids: [],
      secondary_departments: [],
      consultant_restaurant_ids: [],
      can_view_hours: false,
    })

    return { id: authData.user.id, department: dept, restaurant_id: restaurantId, rest_day_offset: empIdx % 6 }
  }))

  allEmployeeProfiles.push(...createdProfiles.filter(Boolean) as typeof allEmployeeProfiles)
}))

// — 3. Shift slots per restaurant —————
const allSlots: object[] = []
for (let rIdx = 0; rIdx < restaurantIds.length; rIdx++) {
  const cfg = RESTAURANT_CONFIGS[rIdx]
  for (const dept of cfg.departments) {
    const templates = SLOT_TEMPLATES[dept] ?? []
    for (const tpl of templates) {
      allSlots.push({ ...tpl, department: dept, restaurant_id: restaurantIds[rIdx], days_of_week: [] })
    }
  }
}
if (allSlots.length) await admin.from('shift_slots').insert(allSlots)

// — 4. Attendance records for the last 60 days —————
const attendances: object[] = []
const twoMonthsAgo = subDays(today, 60)

for (const emp of allEmployeeProfiles) {
  const cfg = RESTAURANT_CONFIGS[restaurantIds.indexOf(emp.restaurant_id)]
  const shifts = SHIFT_TIMES[emp.department] ?? SHIFT_TIMES['Sala']
  let dayOff = 0 // rest day counter within the week

  let d = new Date(twoMonthsAgo)
  while (d < today) {
    const dow = getDay(d) // 0=Sun
    const dateStr = format(d, 'yyyy-MM-dd')

```

```

// Skip closing days and Sundays (dow=0 always closed for simplicity)
if (!cfg.closing_days.includes(dow) && dow !== 0) {
  // Each employee rests 1 day per working week (Mon offset by emp)
  const weekMon = 1 + emp.rest_day_offset // 1=Mon..6=Sat
  const isRestDay = dow === weekMon

  if (!isRestDay) {
    // ~90% attendance
    const hash = (emp.id.charCodeAt(0) + dateStr.charCodeAt(5)) % 10
    if (hash > 0) { // skip ~10% days (no show / off)
      const isEvening = emp.department === 'Bar' ? false : (parseInt(dateStr.slice(-2)) % 2 === 0)
      const [start, end] = isEvening ? shifts.evening : shifts.morning
      // slight random variation: ±5 min on check-in, +5..+20 on check-out
      const inOffset = -5 + (emp.id.charCodeAt(1) % 10) // -5..+5 min
      const outOffset = 5 + (emp.id.charCodeAt(2) % 15) // +5..+20 min

      attendances.push({
        user_id: emp.id,
        restaurant_id: emp.restaurant_id,
        check_in: ts(dateStr, start, inOffset),
        check_out: ts(dateStr, end, outOffset),
        is_split_shift: false,
        needs_manager_approval: false,
      })
    }
  }
  d = addDays(d, 1)
  dayOff++
}

// Bulk insert attendances in chunks to avoid payload limits
for (let i = 0; i < attendances.length; i += 500) {
  await admin.from('attendances').insert(attendances.slice(i, i + 500))
}

// — 5. Turns for current + next 3 weeks —————
const turns: object[] = []
const weekStart = startOfWeek(today, { weekStartsOn: 1 })

for (let week = 0; week < 4; week++) {
  const wStart = addWeeks(weekStart, week)
  for (let dayOff = 0; dayOff < 6; dayOff++) {
    const d2 = addDays(wStart, dayOff)
    const dow = getDay(d2)
    const dateStr = format(d2, 'yyyy-MM-dd')

    for (const emp of allEmployeeProfiles) {
      const cfg = RESTAURANT_CONFIGS[restaurantIds.indexOf(emp.restaurant_id)]
      if (cfg.closing_days.includes(dow)) continue

      const restDow = 1 + emp.rest_day_offset
      if (dow === restDow) {
        turns.push({ user_id: emp.id, restaurant_id: emp.restaurant_id, department: emp.department, date:
dateStr, start_time: '00:00', end_time: '00:00', is_rest_day: true, is_extraordinary: false, created_by:
managerId })
        continue
      }

      const shifts = SHIFT_TIMES[emp.department] ?? SHIFT_TIMES['Sala']
      const isEvening = emp.department === 'Bar' ? false : dayOff % 2 === 0
      const [start, end] = isEvening ? shifts.evening : shifts.morning
      turns.push({ user_id: emp.id, restaurant_id: emp.restaurant_id, department: emp.department, date: dateStr,
start_time: start, end_time: end, is_rest_day: false, is_extraordinary: false, created_by: managerId })
    }
  }
}

for (let i = 0; i < turns.length; i += 500) {
  await admin.from('turns').insert(turns.slice(i, i + 500))
}

// — 6. ODS tasks —————

```

```

const odsTasks: object[] = []

for (let rIdx = 0; rIdx < restaurantIds.length; rIdx++) {
  const cfg = RESTAURANT_CONFIGS[rIdx]
  const rId = restaurantIds[rIdx]
  const rEmps = allEmployeeProfiles.filter(e => e.restaurant_id === rId)
  const capo = rEmps[0] // capo_servizio
  const emp1 = rEmps[1] // first dipendente
  const emp2 = rEmps[4] // first cucina/other dept

  // General restaurant tasks
  for (const t of ODS_RESTAURANT_TASKS) {
    const deptEmps = t.department ? rEmps.filter(e => e.department === t.department) : rEmps
    odsTasks.push({
      title: t.title, department: t.department ?? cfg.departments[0],
      restaurant_id: rId, creator_id: managerId,
      assigned_to: capo?.id ?? null,
      type: t.type, recurrence_days: t.recurrence_days,
    })
  }

  // Per-department tasks
  for (const dept of cfg.departments) {
    const deptTemplates = ODS_DEPT_TASKS[dept] ?? []
    const deptEmps = rEmps.filter(e => e.department === dept)
    for (const t of deptTemplates) {
      odsTasks.push({
        title: t.title, department: dept,
        restaurant_id: rId, creator_id: managerId,
        assigned_to: null, // department-wide (no individual)
        type: t.type, recurrence_days: t.recurrence_days,
      })
    }

    // One task assigned to specific employee in this dept
    if (deptEmps[0]) {
      odsTasks.push({
        title: `Report settimanale ${dept}`,
        department: dept, restaurant_id: rId, creator_id: managerId,
        assigned_to: deptEmps[0].id,
        type: 'settimanale', recurrence_days: ['venerdi'],
      })
    }
  }

  // Personal task for first dipendente
  if (emp1) {
    odsTasks.push({
      title: 'Inventario personale attrezzature',
      department: emp1.department, restaurant_id: rId, creator_id: managerId,
      assigned_to: emp1.id,
      type: 'bisettimanale', recurrence_days: ['lunedì', 'giovedì'],
    })
  }
}

if (odsTasks.length) await admin.from('ods_tasks').insert(odsTasks)

// — 7. Bulletins —————
const bulletins: object[] = []

for (let rIdx = 0; rIdx < restaurantIds.length; rIdx++) {
  const rId = restaurantIds[rIdx]
  const cfg = RESTAURANT_CONFIGS[rIdx]
  const rEmps = allEmployeeProfiles.filter(e => e.restaurant_id === rId)

  // General restaurant announcements (2 per restaurant)
  bulletins.push({
    title: BULLETIN_TEMPLATES[rIdx % BULLETIN_TEMPLATES.length].title,
    body: BULLETIN_TEMPLATES[rIdx % BULLETIN_TEMPLATES.length].body,
    target: 'all', target_roles: [], target_user_ids: [], target_department: null,
    restaurant_id: rId, created_by: managerId,
  })
}

```



```

bulletins.push({
  title: 'Aggiornamento procedure sicurezza',
  body: `Si informano tutti i dipendenti di ${cfg.name} che le procedure di sicurezza sono state aggiornate. Consultare l'apposita bacheca fisica per i dettagli operativi.`,
  target: 'all', target_roles: [], target_user_ids: [], target_department: null,
  restaurant_id: rId, created_by: managerId,
})

// Department-specific bulletins for each dept
for (const dept of cfg.departments) {
  const deptTpl = BULLETIN_TEMPLATES.find(b => b.target === 'department' && b.target_department === dept)
  bulletins.push({
    title: deptTpl?.title ?? `Comunicazione reparto ${dept}`,
    body: deptTpl?.body ?? `Aggiornamento operativo per il reparto ${dept}. Si prega di prenderne visione e di applicare le indicazioni nel prossimo turno.`,
    target: 'department', target_roles: [], target_user_ids: [], target_department: dept,
    restaurant_id: rId, created_by: managerId,
  })
}

// Personal bulletin for a specific employee
const targetEmp = rEmps.find(e => e.department === cfg.departments[0] && e !== rEmps[0])
if (targetEmp) {
  bulletins.push({
    title: 'Comunicazione personale',
    body: `Ti informiamo che la tua richiesta di cambio turno per il prossimo weekend è stata approvata. Ricordati di concordare la copertura con un collega.`,
    target: 'users', target_roles: [], target_user_ids: [targetEmp.id], target_department: null,
    restaurant_id: rId, created_by: managerId,
  })
}

// Capo_servizio announcement
bulletins.push({
  title: 'Riunione capiservizie',
  body: `Convoco tutti i capiservizie di ${cfg.name} per lunedì alle 09:30 per la pianificazione del mese. Portate il calendario delle disponibilità.`,
  target: 'role', target_roles: ['capo_servizio'], target_user_ids: [], target_department: null,
  restaurant_id: rId, created_by: managerId,
})
}

if (bulletins.length) await admin.from('bulletins').insert(bulletins)

return restaurantIds
}

// — Delete all demo data for a manager —————

export async function deleteDemoData(managerId: string): Promise<void> {
  const admin = createAdminClient()

  const { data: demoRests } = await admin
    .from('restaurants')
    .select('id')
    .eq('is_demo', true)
    .eq('owner_id', managerId)

  if (!demoRests?.length) return

  for (const rest of demoRests) {
    const rid = rest.id

    const { data: demoProfiles } = await admin
      .from('profiles')
      .select('id')
      .eq('restaurant_id', rid)
      .neq('id', managerId)

    // Delete ODS completions via task IDs
    const { data: odsTasks } = await admin.from('ods_tasks').select('id').eq('restaurant_id', rid)
    if (odsTasks?.length) {
      await admin.from('ods_completions').delete().in('task_id', odsTasks.map(t => t.id))
    }
  }
}

```

```

// Delete bulletins
await admin.from('bulletins').delete().eq('restaurant_id', rid)

// Delete AI drafts
const { data: drafts } = await admin.from('ai_schedule_drafts').select('id').eq('restaurant_id', rid)
if (drafts?.length) {
  await admin.from('ai_schedule_draft_turns').delete().in('draft_id', drafts.map(d => d.id))
}
await admin.from('ai_schedule_drafts').delete().eq('restaurant_id', rid)

await admin.from('ods_tasks').delete().eq('restaurant_id', rid)
await admin.from('turns').delete().eq('restaurant_id', rid)
await admin.from('shift_slots').delete().eq('restaurant_id', rid)
await admin.from('absences').delete().eq('restaurant_id', rid)
await admin.from('attendances').delete().eq('restaurant_id', rid)

if (demoProfiles?.length) {
  await admin.from('profiles').delete().in('id', demoProfiles.map(p => p.id))
  for (const p of demoProfiles) {
    await admin.auth.admin.deleteUser(p.id).catch(() => {})
  }
}

await admin.from('restaurants').delete().eq('id', rid)
}
}

```

```

import { Resend } from 'resend'

const resend = process.env.RESEND_API_KEY
  ? new Resend(process.env.RESEND_API_KEY)
  : null

const ADMIN_EMAIL = process.env.ADMIN_EMAIL ?? 'flavianoconiglio94@gmail.com'
const APP_NAME = 'inTurno'
const FROM_EMAIL = process.env.RESEND_FROM_EMAIL ?? 'noreply@inturno.app'

async function sendEmail(to: string, subject: string, html: string): Promise<void> {
  if (!resend) {
    console.warn('[email] RESEND_API_KEY non configurato - email non inviata:', subject)
    return
  }
  try {
    await resend.emails.send({ from: `${APP_NAME} <${FROM_EMAIL}>`, to, subject, html })
  } catch (err) {
    console.error('[email] Errore invio:', err)
  }
}

export async function sendNewRegistrationAlert(opts: {
  fullName: string
  email: string
  approveUrl: string
}): Promise<void> {
  await sendEmail(
    ADMIN_EMAIL,
    `[${APP_NAME}] Nuova richiesta di accesso - ${opts.fullName}`,
    `
    <h2>Nuova richiesta di accesso</h2>
    <p><strong>${opts.fullName}</strong> (${opts.email}) ha richiesto l'accesso a ${APP_NAME}.</p>
    <p>
      <a href="${opts.approveUrl}" style="display:inline-block;padding:10px
20px;background:#000;color:#fff;border-radius:6px;text-decoration:none;">
        Approva account
      </a>
    </p>
    <p style="color:#888;font-size:12px;">Puoi anche approvarlo dalla pagina Account Pendenti nell'app.</p>
    `
  )
}

export async function sendAccountActivatedEmail(opts: {
  fullName: string
  email: string
  loginUrl: string
}): Promise<void> {
  await sendEmail(
    opts.email,
    `Il tuo account ${APP_NAME} è stato attivato!`,
    `
    <h2>Benvenuto su ${APP_NAME}, ${opts.fullName}!</h2>
    <p>Il tuo account è stato approvato. Puoi accedere subito con la tua email e password.</p>
    <p>
      <a href="${opts.loginUrl}" style="display:inline-block;padding:10px 20px;background:#000;color:#fff;border-
radius:6px;text-decoration:none;">
        Accedi ora
      </a>
    </p>
    `
  )
}

```



```

// Client-only utility – uses canvas API and dynamically imports pdf-lib
// so it never runs on the server and doesn't inflate the initial bundle.

async function compressImageForPDF(
  file: File,
): Promise<{ bytes: ArrayBuffer; isPng: boolean }> {
  const MAX_DIM = 1200
  const QUALITY = 0.85

  return new Promise((resolve, reject) => {
    const url = URL.createObjectURL(file)
    const img = new Image()
    img.onload = () => {
      URL.revokeObjectURL(url)
      const scale = Math.min(1, MAX_DIM / Math.max(img.naturalWidth, img.naturalHeight))
      const w = Math.round(img.naturalWidth * scale)
      const h = Math.round(img.naturalHeight * scale)
      const canvas = document.createElement('canvas')
      canvas.width = w
      canvas.height = h
      canvas.getContext('2d')!.drawImage(img, 0, 0, w, h)
      const isPng = file.type === 'image/png'
      canvas.toBlob(
        blob => {
          if (!blob) { reject(new Error('Compressione immagine fallita')); return }
          blob.arrayBuffer().then(bytes => resolve({ bytes, isPng })).catch(reject)
        },
        isPng ? 'image/png' : 'image/jpeg',
        isPng ? undefined : QUALITY,
      )
    }
    img.onerror = () => { URL.revokeObjectURL(url); reject(new Error('Caricamento immagine fallito')) }
    img.src = url
  })
}

// Merges an array of image/PDF Files into a single PDF File.
// pdf-lib is imported lazily to keep it out of the initial JS bundle.
export async function generateUnifiedPDF(files: File[]): Promise<File> {
  const { PDFDocument } = await import('pdf-lib')
  const unified = await PDFDocument.create()

  for (const file of files) {
    if (file.type === 'application/pdf') {
      const bytes = await file.arrayBuffer()
      const srcDoc = await PDFDocument.load(bytes, { ignoreEncryption: true })
      const indices = srcDoc.getPageIndices()
      const copied = await unified.copyPages(srcDoc, indices)
      copied.forEach(p => unified.addPage(p))
    } else if (file.type.startsWith('image/')) {
      const { bytes, isPng } = await compressImageForPDF(file)
      const img = isPng
        ? await unified.embedPng(bytes)
        : await unified.embedJpg(bytes)

      // A4 page (595.28 × 841.89 pt) with 36pt margins on each side
      const PAGE_W = 595.28
      const PAGE_H = 841.89
      const MARGIN = 36
      const { width, height } = img.scaleToFit(PAGE_W - MARGIN * 2, PAGE_H - MARGIN * 2)
      const page = unified.addPage([PAGE_W, PAGE_H])
      page.drawImage(img, {
        x: (PAGE_W - width) / 2,
        y: (PAGE_H - height) / 2,
        width,
        height,
      })
    }
  }
  // Files that are neither image nor PDF are silently skipped
}

const pdfBytes = await unified.save()
// new Uint8Array(src) copies into a plain ArrayBuffer, satisfying the File constructor's BlobPart type

```

```
return new File([new Uint8Array(pdfBytes)], 'allegati_unificati.pdf', { type: 'application/pdf' })
}
```

```

// Native IndexedDB offline queue – no external dependencies.
// All functions are safe to call from client components only.

const DB_NAME = 'inturno-offline'
const STORE = 'queue'
const VERSION = 1

function openDB(): Promise<IDBDatabase> {
  return new Promise((resolve, reject) => {
    if (typeof indexedDB === 'undefined') { reject(new Error('no-idb')); return }
    const req = indexedDB.open(DB_NAME, VERSION)
    req.onupgradeneeded = e => {
      const db = (e.target as IDBOpenDBRequest).result
      if (!db.objectStoreNames.contains(STORE)) {
        db.createObjectStore(STORE, { keyPath: 'id' })
      }
    }
    req.onsuccess = e => resolve((e.target as IDBOpenDBRequest).result)
    req.onerror = e => reject((e.target as IDBOpenDBRequest).error)
  })
}

export type OfflineQueueType = 'clock-in' | 'ods-toggle'

export interface ClockInPayload {
  qr_secret: string
  type: 'in' | 'out'
  latitude: number | null
  longitude: number | null
  accuracy: number | null
  frozenAt: string // ISO – timestamp at the moment the button was pressed
}

export interface OdsTogglePayload {
  task_id: string
  user_id: string
  action: 'complete' | 'uncomplete'
  frozenAt: string
}

export interface OfflineQueueItem {
  id: string
  type: OfflineQueueType
  payload: ClockInPayload | OdsTogglePayload
  createdAt: string
}

export async function saveToOfflineQueue(
  type: OfflineQueueType,
  payload: ClockInPayload | OdsTogglePayload,
): Promise<void> {
  const db = await openDB()
  const item: OfflineQueueItem = {
    id: `${Date.now()}-${Math.random().toString(36).slice(2, 7)}`,
    type,
    payload,
    createdAt: new Date().toISOString(),
  }
  return new Promise((resolve, reject) => {
    const tx = db.transaction(STORE, 'readwrite')
    tx.objectStore(STORE).add(item)
    tx.oncomplete = () => resolve()
    tx.onerror = () => reject(tx.error)
  })
}

export async function getOfflineQueue(): Promise<OfflineQueueItem[]> {
  const db = await openDB()
  return new Promise((resolve, reject) => {
    const tx = db.transaction(STORE, 'readonly')
    const req = tx.objectStore(STORE).getAll()
    req.onsuccess = () => resolve(req.result ?? [])
    req.onerror = () => reject(req.error)
  })
}

```



```
    })
  }

export async function clearOfflineItem(id: string): Promise<void> {
  const db = await openDB()
  return new Promise((resolve, reject) => {
    const tx = db.transaction(STORE, 'readwrite')
    tx.objectStore(STORE).delete(id)
    tx.oncomplete = () => resolve()
    tx.onerror = () => reject(tx.error)
  })
}
```

`src/lib/supabase/admin.ts`

```
import { createClient } from '@supabase/supabase-js'

export function createAdminClient() {
  return createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!,
    { auth: { autoRefreshToken: false, persistSession: false } }
  )
}
```

`src/lib/supabase/client.ts`

```
import { createBrowserClient } from '@supabase/ssr'

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

```
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'

export async function createClient() {
  const cookieStore = await cookies()

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll()
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            )
          } catch {
            // Server Component – cookies set in middleware
          }
        },
      },
    },
  )
}
```



```

import { google } from '@ai-sdk/google'
import { generateText, tool, stepCountIs, type ModelMessage, type Tool } from 'ai'
import { z } from 'zod'
import { addDays } from 'date-fns'
import { formatInTimeZone, fromZonedTime } from 'date-fns-tz'
import { it } from 'date-fns/locale'
import type { Ctx } from './context'
import {
  isManager, isDirettore, isCapoServizio, canManagePresenze,
  assigneeInScope, scopeTurnsQuery, scopeStaffQuery, toScopeProfile,
} from './scope'
import { TZ, isValidTime, normalizeTime, formatDateLabel } from './format'
import { DEPARTMENTS, ODS_DAYS_IT, ODS_TYPE_LABELS, type OdsTaskType, type Department } from '@types'
import { getAiHistory, saveAiHistory } from './aiHistory'

// — Assistente AI (Gemini) – comandi e domande in linguaggio naturale —
// Attivato per i messaggi liberi (non slash-command) quando non c'è una
// sessione wizard attiva. Usa tool-calling per leggere/scrivere sui dati
// nello stesso scope RBAC dei comandi classici.

const DATE_RE = /^d{4}-d{2}-d{2}$/
const GEMINI_MODEL = process.env.GEMINI_MODEL || 'gemini-2.5-flash'
// Modello di riserva, usato automaticamente se quello principale esaurisce la quota gratuita.
const GEMINI_FALLBACK_MODEL = process.env.GEMINI_FALLBACK_MODEL || 'gemini-2.5-flash-lite'

function isRateLimitError(err: unknown): boolean {
  const message = err instanceof Error ? err.message : String(err)
  return /429|rate.?limit|quota|RESOURCE_EXHAUSTED/i.test(message)
}

function todayStr(): string {
  return formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd')
}

function addDaysToDateStr(dateStr: string, days: number): string {
  return formatInTimeZone(addDays(new Date(`${dateStr}T12:00:00Z`), days), TZ, 'yyyy-MM-dd')
}

function dayBounds(dateStr: string): { start: string; end: string } {
  return {
    start: fromZonedTime(`${dateStr}T00:00:00`, TZ).toISOString(),
    end: fromZonedTime(`${dateStr}T23:59:59.999`, TZ).toISOString(),
  }
}

function getOdsCutoff(): string {
  const now = new Date()
  const romeHour = parseInt(formatInTimeZone(now, TZ, 'H'), 10)
  const refDate = romeHour < 4 ? new Date(now.getTime() - 86_400_000) : now
  const cutoffDate = formatInTimeZone(refDate, TZ, 'yyyy-MM-dd')
  return fromZonedTime(`${cutoffDate}T04:00:00`, TZ).toISOString()
}

function resolveDepartment(input: string): Department | null {
  const norm = input.trim().toLowerCase()
  return DEPARTMENTS.find(d => d.toLowerCase() === norm) ?? null
}

// — Prompt di sistema —

function buildSystemPrompt(ctx: Ctx): string {
  const { profile } = ctx
  const now = new Date()

  const calendarLines: string[] = []
  for (let i = 0; i < 14; i++) {
    const d = addDays(now, i)
    const dateStr = formatInTimeZone(d, TZ, 'yyyy-MM-dd')
    const label = formatInTimeZone(d, TZ, 'EEEE d MMMM', { locale: it })
    const tag = i === 0 ? ' ← OGGI' : i === 1 ? ' ← DOMANI' : i === 2 ? ' ← DOPODOMANI' : ''
    calendarLines.push(`${dateStr} = ${label}${tag}`)
  }
  const oggiLine = calendarLines[0]

```

```

const domaniLine = calendarLines[1]

let scopeLabel: string
if (isManager(profile)) {
  scopeLabel = 'Manager: accesso globale a tutti i ristoranti e reparti.'
} else if (isDirettore(profile)) {
  scopeLabel = `Direttore del ristorante "${profile.restaurant?.name ?? ''}": accesso a tutti i reparti di
questo ristorante.`
} else {
  scopeLabel = `Capo Servizio del reparto "${profile.department}" nel ristorante "${profile.restaurant?.name ??
''}": accesso solo a questo reparto.`
}

return `Sei l'assistente virtuale del bot Telegram di "Turni", un'app per la gestione di turni, ODS (ordini di
servizio) e presenze in ristoranti italiani.

Stai parlando con ${profile.full_name}. ${scopeLabel}

ATTENZIONE alle date relative: ${oggiLine}. ${domaniLine}. "Domani" è SEMPRE il giorno immediatamente dopo oggi,
non quello dopo ancora: prima di usare una data negli strumenti, controlla la riga corrispondente nel calendario
qui sotto e verifica che il nome del giorno coincida con quello che stai per scrivere all'utente.

Calendario di riferimento (usa SEMPRE il formato yyyy-MM-dd negli strumenti):
${calendarLines.join('\n')}`

ISTRUZIONI:
- Rispondi sempre in italiano, con tono colloquiale, amichevole e professionale, come un collega disponibile. Usa
il "tu".
- Sii conciso: messaggi brevi, vai dritto al punto.
- Puoi usare la formattazione Markdown di Telegram (*grassetto*, _corsivo_) e qualche emoji con moderazione (
turni, ODS, presenze, ).
- Per qualsiasi azione che coinvolge un dipendente (turno, riposo, ODS, presenza), se non conosci il suo ID usa
prima lo strumento cerca_dipendenti. Se ci sono più persone con nomi simili, chiedi all'utente di specificare a
chi si riferisce: non scegliere a caso.
- Gli ID (UUID) sono identificatori interni: NON chiederli MAI all'utente (non li conosce e non deve conoscerli) e
non mostrarli nelle risposte. Quando ti serve l'ID di un dipendente o di un turno, ricavalalo TU con
cerca_dipendenti o lista_turni, oppure riusalo dai risultati degli strumenti nei messaggi precedenti.
- Se l'utente indica una persona con un nome parziale o solo il cognome (es. "accolla", "vecchi"), NON chiedere
subito il nome completo: prova prima cerca_dipendenti con quel testo. Chiedi chiarimenti solo se la ricerca
restituisce più persone o nessuna.
- Esegui un'azione (creare/eliminare/modificare) SOLO se la richiesta è chiara e contiene tutte le informazioni
necessarie. Se manca qualcosa (data, orario, dipendente, reparto...), chiedi prima di procedere: non inventare
dati.
- Per "scambiare"/"sostituire"/"spostare" i turni (es. tra due dipendenti o tra due date), usa SEMPRE prima
lista_turni per leggere i turni esistenti coinvolti, poi usa modifica_turno per aggiornare data/orari di quei
turni già esistenti (scambiando i valori tra loro). NON usare crea_turno per uno scambio: creerebbe un turno
duplicato lasciando quello vecchio invariato. Usa crea_turno solo quando il dipendente non ha ancora un turno in
quella data.
- Dopo aver eseguito un'azione, confermala riassumendo cosa hai fatto.
- Per le domande sui dati (es. "chi lavora venerdì?", "quante presenze oggi?"), usa gli strumenti di lettura e
rispondi in modo naturale, senza limitarti a riportare i dati grezzi.
- Se una richiesta non è di tua competenza, suggerisci i comandi /help, /turni, /ods, /presenze.
- Vedi gli ultimi messaggi della conversazione: usali per capire riferimenti a cose appena discusse (es. un
dipendente già identificato, una data già menzionata, un'azione proposta poco prima). Se nei messaggi precedenti
hai già trovato l'ID di un dipendente o di un turno, riusalo senza richiamare di nuovo lo strumento di ricerca, a
meno che non sia passato troppo tempo o il contesto sia cambiato.
- IMPORTANTE: non annunciare mai a parole un'azione futura o "in corso" (es. "ora recupero i turni...", "procedo a
cancellarli...", "Ok, elimino il turno...") come fosse la tua risposta finale. Se una frase descrive un'azione,
quell'azione deve essere GIÀ stata eseguita con lo strumento corrispondente, nella stessa risposta e PRIMA di
scrivere quella frase. Se devi ancora compiere un passo, chiamalo SUBITO (non limitarti a scriverlo a parole) e
continua a concatenare le chiamate agli strumenti finché la richiesta dell'utente non è completamente conclusa.
Esempio SBAGLIATO: rispondere solo "Ok, elimino il turno di sabato 20 giugno." senza aver chiamato elimina_turno.
Esempio CORRETTO: chiamare lo strumento elimina_turno e poi rispondere " Ho eliminato il turno di sabato 20
giugno."`
}

// — Tool: ricerca dipendenti —————

function cercaDipendentiTool(ctx: Ctx) {
  return tool({
    description: 'Cerca dipendenti/capi servizio nel tuo ambito per nome. Usa questo strumento per trovare l\'ID
di un dipendente prima di creare turni, riposi, ODS o presenze.',
    inputSchema: z.object({
      nome: z.string().optional().describe('Nome o parte del nome da cercare. Se omissso, restituisce tutti i

```

```

dipendenti nel tuo ambito.'),
  }},
  execute: async ({ nome }) => {
    let query = ctx.admin
      .from('profiles')
      .select('id, full_name, department, restaurant_id, restaurant:restaurants(name)')
      .in('role', ['dipendente', 'capo_servizio'])
      .order('full_name')

    if (isDirettore(ctx.profile)) {
      query = query.eq('restaurant_id', ctx.profile.restaurant_id)
    } else if (isCapoServizio(ctx.profile)) {
      query = query.eq('restaurant_id', ctx.profile.restaurant_id).eq('department', ctx.profile.department)
    }

    if (nome?.trim()) query = query.ilike('full_name', `${nome.trim()}%`)

    const { data, error } = await query
    if (error) return { error: error.message }

    return {
      dipendenti: (data ?? []).map((d: Record<string, unknown>) => ({
        id: d.id,
        nome: d.full_name,
        reparto: d.department,
        ristorante: (d.restaurant as { name: string } | null)?.name ?? null,
      })),
    },
  })
}

// — Tool: turni —————

function listaTurniTool(ctx: Ctx) {
  return tool({
    description: 'Elenca i turni di lavoro in un intervallo di date (default: prossimi 7 giorni a partire da oggi), opzionalmente filtrati per dipendente. Restituisce anche l\'ID di ciascun turno, utile per eliminarlo in seguito.',
    inputSchema: z.object({
      data_inizio: z.string().optional().describe('Data iniziale yyyy-MM-dd (default: oggi)'),
      data_fine: z.string().optional().describe('Data finale yyyy-MM-dd (default: 6 giorni dopo data_inizio)'),
      dipendente_id: z.string().optional().describe('Filtra solo i turni di questo dipendente'),
    }),
    execute: async ({ data_inizio, data_fine, dipendente_id }) => {
      const start = data_inizio && DATE_RE.test(data_inizio) ? data_inizio : todayStr()
      const end = data_fine && DATE_RE.test(data_fine) ? data_fine : addDaysToDateStr(start, 6)

      let query = ctx.admin
        .from('turns')
        .select('id, date, start_time, end_time, is_extraordinary, is_rest_day, department, user_id, profile:profiles!user_id(full_name), restaurant:restaurants(name)')
        .gte('date', start)
        .lte('date', end)
        .order('date')
        .order('start_time')
        .limit(60)

      query = scopeTurnsQuery(query, toScopeProfile(ctx.profile), ctx.profile.id)
      if (dipendente_id) query = query.eq('user_id', dipendente_id)

      const { data, error } = await query
      if (error) return { error: error.message }

      const turni = (data ?? []).map((t: Record<string, unknown>) => ({
        id: t.id,
        data: t.date,
        giorno: formatDateLabel(t.date as string),
        dipendente: (t.profile as { full_name: string } | null)?.full_name ?? null,
        reparto: t.department,
        ristorante: (t.restaurant as { name: string } | null)?.name ?? null,
        riposo: t.is_rest_day,
        straordinario: t.is_extraordinary,
        orario: t.is_rest_day ? null : `${(t.start_time as string).slice(0, 5)}-${(t.end_time as string).slice(0,

```

```

5))`,
  )))

  return { turni, totale: turni.length }
},
})
}

function creaTurnoTool(ctx: Ctx) {
  return tool({
    description: 'Crea un nuovo turno di lavoro per un dipendente in una data e fascia oraria specifiche.',
    inputSchema: z.object({
      dipendente_id: z.string().describe('ID del dipendente (ottenuto con cerca_dipendenti)'),
      data: z.string().describe('Data del turno, formato yyyy-MM-dd'),
      ora_inizio: z.string().describe('Orario di inizio, formato HH:MM (24h)'),
      ora_fine: z.string().describe('Orario di fine, formato HH:MM (24h)'),
      straordinario: z.boolean().optional().describe('true se è un turno straordinario'),
    }),
    execute: async ({ dipendente_id, data, ora_inizio, ora_fine, straordinario }) => {
      if (!DATE_RE.test(data)) return { error: 'Formato data non valido, usa yyyy-MM-dd.' }
      if (!isValidTime(ora_inizio) || !isValidTime(ora_fine)) return { error: 'Formato orario non valido, usa HH:MM.' }

      const { data: employee, error: empErr } = await ctx.admin
        .from('profiles')
        .select('id, full_name, restaurant_id, department, role')
        .eq('id', dipendente_id)
        .single()
      if (empErr || !employee) return { error: 'Dipendente non trovato.' }

      if (!assigneeInScope(ctx.profile, employee)) {
        return { error: 'Non sei autorizzato ad assegnare turni a questo dipendente.' }
      }

      const { data: existing } = await ctx.admin.from('turns').select('id, start_time, end_time, is_rest_day').eq('user_id', employee.id).eq('date', data).maybeSingle()
      if (existing) {
        return { error: `${employee.full_name} ha già ${existing.is_rest_day ? 'un riposo' : 'un turno'} (${(existing.start_time as string).slice(0, 5)}-${(existing.end_time as string).slice(0, 5)}) il ${formatDateLabel(data)} (ID turno: ${existing.id}). Usa modifica_turno per cambiarlo oppure elimina_turno per rimuoverlo prima di crearne uno nuovo.` }
      }

      const { error } = await ctx.admin.from('turns').insert({
        user_id: employee.id,
        restaurant_id: employee.restaurant_id,
        department: employee.department,
        date: data,
        start_time: normalizeTime(ora_inizio),
        end_time: normalizeTime(ora_fine),
        is_extraordinary: !!straordinario,
        is_rest_day: false,
        created_by: ctx.profile.id,
      })
      if (error) return { error: error.message }

      return {
        ok: true,
        messaggio: `Turno creato per ${employee.full_name} il ${formatDateLabel(data)} dalle ${ora_inizio} alle ${ora_fine}${straordinario ? ' (straordinario)' : ''}.`,
      },
    })
  })
}

function modificaTurnoTool(ctx: Ctx) {
  return tool({
    description: 'Modifica data e/o orari di un turno di lavoro già esistente (NON un riposo), dato il suo ID (ottenuto con lista_turni). Usa questo strumento per spostare un turno o cambiarne l\'orario, ad esempio per scambiare i turni tra due dipendenti, invece di crearne uno nuovo.',
    inputSchema: z.object({
      turno_id: z.string().describe('ID del turno da modificare (ottenuto con lista_turni)'),
      data: z.string().optional().describe('Nuova data, formato yyyy-MM-dd (omettere se non cambia)'),
      ora_inizio: z.string().optional().describe('Nuovo orario di inizio, formato HH:MM (omettere se non'

```

```

cambia)'),
  ora_fine: z.string().optional().describe('Nuovo orario di fine, formato HH:MM (omettere se non cambia)'),
  straordinario: z.boolean().optional().describe('true/false se cambia lo stato di straordinario'),
}),
execute: async ({ turno_id, data, ora_inizio, ora_fine, straordinario }) => {
  if (data !== undefined && !DATE_RE.test(data)) return { error: 'Formato data non valido, usa yyyy-MM-dd.' }
  if (ora_inizio !== undefined && !isValidTime(ora_inizio)) return { error: 'Formato orario di inizio non valido, usa HH:MM.' }
  if (ora_fine !== undefined && !isValidTime(ora_fine)) return { error: 'Formato orario di fine non valido, usa HH:MM.' }

  const update: Record<string, unknown> = {}
  if (data !== undefined) update.date = data
  if (ora_inizio !== undefined) update.start_time = normalizeTime(ora_inizio)
  if (ora_fine !== undefined) update.end_time = normalizeTime(ora_fine)
  if (straordinario !== undefined) update.is_extraordinary = straordinario
  if (Object.keys(update).length === 0) return { error: 'Nessuna modifica specificata: indica almeno data, ora_inizio, ora_fine o straordinario.' }

  let query = ctx.admin.from('turns').update(update).eq('id', turno_id).eq('is_rest_day', false)
  query = scopeTurnsQuery(query, toScopeProfile(ctx.profile), ctx.profile.id)

  const { data: rows, error } = await query.select('id, date, start_time, end_time')
  if (error) return { error: error.message }
  if (!rows?.length) return { error: 'Turno non trovato, non modificabile (es. è un riposo) o non sei autorizzato a modificarlo.' }

  const row = rows[0] as { date: string; start_time: string; end_time: string }
  return { ok: true, messaggio: `Turno aggiornato: ${formatDateLabel(row.date)} dalle ${row.start_time.slice(0, 5)} alle ${row.end_time.slice(0, 5)}.` }
},
})
}

function assegnaRiposoTool(ctx: Ctx) {
  return tool({
    description: 'Assegna un giorno di riposo a un dipendente in una data specifica.',
    inputSchema: z.object({
      dipendente_id: z.string().describe('ID del dipendente (ottenuto con cerca_dipendenti)'),
      data: z.string().describe('Data del riposo, formato yyyy-MM-dd'),
    }),
    execute: async ({ dipendente_id, data }) => {
      if (!DATE_RE.test(data)) return { error: 'Formato data non valido, usa yyyy-MM-dd.' }

      const { data: employee, error: empErr } = await ctx.admin
        .from('profiles')
        .select('id, full_name, restaurant_id, department, role')
        .eq('id', dipendente_id)
        .single()
      if (empErr || !employee) return { error: 'Dipendente non trovato.' }

      if (!assigneeInScope(ctx.profile, employee)) {
        return { error: 'Non sei autorizzato ad assegnare riposi a questo dipendente.' }
      }

      const { error } = await ctx.admin.from('turns').insert({
        user_id: employee.id,
        restaurant_id: employee.restaurant_id,
        department: employee.department,
        date: data,
        start_time: '00:00:00',
        end_time: '00:00:00',
        is_extraordinary: false,
        is_rest_day: true,
        created_by: ctx.profile.id,
      })
      if (error) return { error: error.message }

      const { data: managers } = await ctx.admin.from('profiles').select('id').eq('role', 'manager')
      if (managers?.length) {
        await ctx.admin.from('notifications').insert(
          (managers as { id: string }[]).map(m => ({
            user_id: m.id,
            title: 'Nuovo riposo assegnato',
          }))
        )
      }
    }
  })
}

```



```

        message: `${ctx.profile.full_name} ha assegnato un riposo a ${employee.full_name} per il $
{formatDateLabel(data)}`,
        link: '/turni',
    })),
    )
}

return { ok: true, messaggio: `Riposo assegnato a ${employee.full_name} per il ${formatDateLabel(data)}.` }
},
})
}

function eliminaTurnoTool(ctx: Ctx) {
    return tool({
        description: 'Elimina un turno esistente, dato il suo ID (ottenuto con lista_turni). Usa solo se l\'utente ha
chiaramente identificato il turno da eliminare.',
        inputSchema: z.object({
            turno_id: z.string().describe('ID del turno da eliminare'),
        }),
        execute: async ({ turno_id }) => {
            let query = ctx.admin.from('turns').delete().eq('id', turno_id)
            query = scopeTurnsQuery(query, toScopeProfile(ctx.profile), ctx.profile.id)

            const { data, error } = await query.select('id')
            if (error) return { error: error.message }
            if (!data?.length) return { error: 'Turno non trovato o non sei autorizzato a eliminarlo.' }

            return { ok: true, messaggio: 'Turno eliminato.' }
        },
    })
}

// — Tool: ODS —————

function listaOdsTool(ctx: Ctx) {
    return tool({
        description: 'Elenca i compiti ODS (ordini di servizio) previsti per una data (default: oggi) nel tuo ambito.
Restituisce anche l\'ID di ciascun compito, utile per segnarlo come completato.',
        inputSchema: z.object({
            data: z.string().optional().describe('Data yyyy-MM-dd (default: oggi)'),
        }),
        execute: async ({ data }) => {
            const targetDate = data && DATE_RE.test(data) ? data : todayStr()
            const isToday = targetDate === todayStr()

            let query = ctx.admin
                .from('ods_tasks')
                .select('id, title, department, type, recurrence_days, assigned_to, assignee:profiles!
assigned_to(full_name)')
                .order('department')
            query = scopeStaffQuery(query, toScopeProfile(ctx.profile))

            const { data: rows, error } = await query
            if (error) return { error: error.message }

            const dayName = formatInTimeZone(`${targetDate}T12:00:00Z`, TZ, 'EEEE', { locale: it }).toLowerCase()
            const due = (rows ?? []).filter((t: Record<string, unknown>) => {
                const type = t.type as OdsTaskType
                if (type === 'quotidiana' || type === 'straordinaria') return true
                return (t.recurrence_days as string[]).some(d => d.toLowerCase() === dayName)
            })

            let doneSet = new Set<string>()
            if (isToday && due.length) {
                const cutoff = getOdsCutoff()
                const { data: completions } = await ctx.admin
                    .from('ods_completions')
                    .select('task_id')
                    .in('task_id', due.map((t: Record<string, unknown>) => t.id))
                    .gte('completed_at', cutoff)
                doneSet = new Set((completions ?? []).map((c: { task_id: string }) => c.task_id))
            }

            return {

```

```

    data: targetDate,
    compiti: due.map((t: Record<string, unknown>) => ({
      id: t.id,
      titolo: t.title,
      reparto: t.department,
      tipo: ODS_TYPE_LABELS[t.type as OdsTaskType],
      assegnatario: (t.assignee as { full_name: string } | null)?.full_name ?? `Tutto il reparto $
{t.department}`,
      completato: isToday ? doneSet.has(t.id as string) : null,
    })),
    nota: isToday ? undefined : 'Lo stato di completamento è disponibile solo per la data di oggi.',
  },
},
})
}

function creaOdsTool(ctx: Ctx) {
  return tool({
    description: 'Crea un nuovo compito ODS (ordine di servizio) per un reparto o per un dipendente specifico.',
    inputSchema: z.object({
      titolo: z.string().describe('Titolo/descrizione del compito'),
      reparto: z.string().describe('Reparto: Sala, Pizzeria, Bar o Cucina'),
      tipo: z.enum(['quotidiana', 'settimanale', 'bisettimanale', 'straordinaria']).optional().describe('Tipo di
ricorrenza (default: quotidiana)'),
      giorni: z.array(z.string()).optional().describe('Per tipo settimanale/bisettimanale: giorni della settimana
in italiano minuscolo (es. ["lunedì", "giovedì"])'),
      dipendente_id: z.string().optional().describe('Se il compito va assegnato a un dipendente specifico invece
che a tutto il reparto'),
      ristorante: z.string().optional().describe('Nome del ristorante (necessario solo per i manager quando il
compito è per tutto un reparto, senza dipendente_id)'),
    }),
    execute: async ({ titolo, reparto, tipo, giorni, dipendente_id, ristorante }) => {
      const dept = resolveDepartment(reparto)
      if (!dept) return { error: `Reparto non valido. Valori possibili: ${DEPARTMENTS.join(', ')}.` }

      let restaurantId: string | null = null
      let assignedTo: string | null = null

      if (dipendente_id) {
        const { data: employee, error: empErr } = await ctx.admin
          .from('profiles')
          .select('id, full_name, restaurant_id, department, role')
          .eq('id', dipendente_id)
          .single()
        if (empErr || !employee) return { error: 'Dipendente non trovato.' }
        if (!assigneeInScope(ctx.profile, employee)) return { error: 'Non sei autorizzato ad assegnare compiti a
questo dipendente.' }
        restaurantId = employee.restaurant_id
        assignedTo = employee.id
      } else if (isManager(ctx.profile)) {
        if (!ristorante) return { error: 'Specifica il nome del ristorante: sei un manager e devi indicare a quale
ristorante si riferisce il compito.' }
        const { data: restaurants } = await ctx.admin.from('restaurants').select('id, name').ilike('name', `%$
{ristorante.trim()}%`)
        if (!restaurants?.length) return { error: `Nessun ristorante trovato con nome simile a "${ristorante}".` }
        if (restaurants.length > 1) return { error: `Più ristoranti trovati: ${restaurants as { name: string }
[]}.map(r => r.name).join(', ')} Specifica meglio.` }
        restaurantId = restaurants[0].id
      } else {
        restaurantId = ctx.profile.restaurant_id
        if (isCapoServizio(ctx.profile) && !isDirettore(ctx.profile) && dept !== ctx.profile.department) {
          return { error: `Puoi creare compiti solo per il reparto ${ctx.profile.department}.` }
        }
      }

      if (!restaurantId) return { error: 'Impossibile determinare il ristorante.' }

      const taskType: OdsTaskType = tipo ?? 'quotidiana'
      const recurrenceDays = (taskType === 'settimanale' || taskType === 'bisettimanale')
        ? (giorni ?? []).map(g => g.toLowerCase().trim()).filter(g => (ODS_DAYS_IT as readonly
string[]).includes(g))
        : []

      const { data: task, error } = await ctx.admin.from('ods_tasks').insert({

```

```

        title: titolo,
        department: dept,
        restaurant_id: restaurantId,
        type: taskType,
        recurrence_days: recurrenceDays,
        creator_id: ctx.profile.id,
        assigned_to: assignedTo,
    }).select('id').single()
    if (error) return { error: error.message }

    let recipients: { id: string }[] = []
    if (assignedTo) {
        recipients = [{ id: assignedTo }]
    } else {
        const { data: staff } = await ctx.admin.from('profiles').select('id')
            .eq('restaurant_id', restaurantId).eq('department', dept)
            .in('role', ['dipendente', 'capo_servizio']).neq('id', ctx.profile.id)
        recipients = staff ?? []
    }
    if (recipients.length) {
        await ctx.admin.from('notifications').insert(
            recipients.map(r => ({
                user_id: r.id,
                title: assignedTo ? 'Nuova mansione assegnata' : 'Nuova istruzione di Reparto',
                message: titolo,
                link: '/ods',
            })),
        )
    }

    return { ok: true, messaggio: `Compito "${titolo}" creato per il reparto ${dept}${assignedTo ? '' : ' (tutto il reparto)'}`, id: task.id }
},
})
}

function completaOdsTool(ctx: Ctx) {
    return tool({
        description: 'Segna un compito ODS come completato per oggi, dato il suo ID (ottenuto con lista_ods).',
        inputSchema: z.object({
            ods_id: z.string().describe('ID del compito ODS'),
        }),
        execute: async ({ ods_id }) => {
            let query = ctx.admin.from('ods_tasks').select('id, title').eq('id', ods_id)
            query = scopeStaffQuery(query, toScopeProfile(ctx.profile))
            const { data: task, error } = await query.maybeSingle()
            if (error) return { error: error.message }
            if (!task) return { error: 'Compito non trovato o non nel tuo ambito.' }

            const { error: insErr } = await ctx.admin.from('ods_completions').insert({
                task_id: ods_id,
                user_id: ctx.profile.id,
            })
            if (insErr) return { error: insErr.message }

            return { ok: true, messaggio: `Compito "${task.title}" segnato come completato.` }
        },
    })
}

// — Tool: presenze (solo manager / direttore) —————

function listaPresenzeTool(ctx: Ctx) {
    return tool({
        description: 'Elenca le presenze (timbrature) registrate in una data (default: oggi).',
        inputSchema: z.object({
            data: z.string().optional().describe('Data yyyy-MM-dd (default: oggi)'),
        }),
        execute: async ({ data }) => {
            const targetDate = data && DATE_RE.test(data) ? data : todayStr()
            const { start, end } = dayBounds(targetDate)

            let query = ctx.admin
                .from('attendances')

```

```

        .select('id, check_in, check_out, profile:profiles(full_name), restaurant:restaurants(name)')
        .gte('check_in', start)
        .lte('check_in', end)
        .order('check_in')
        .limit(60)
    if (isDirettore(ctx.profile)) query = query.eq('restaurant_id', ctx.profile.restaurant_id)

    const { data: rows, error } = await query
    if (error) return { error: error.message }

    return {
        data: targetDate,
        presenze: (rows ?? []).map((r: Record<string, unknown>) => ({
            id: r.id,
            dipendente: (r.profile as { full_name: string } | null)?.full_name ?? null,
            ristorante: (r.restaurant as { name: string } | null)?.name ?? null,
            ingresso: formatInTimeZone(r.check_in as string, TZ, 'HH:mm'),
            uscita: r.check_out ? formatInTimeZone(r.check_out as string, TZ, 'HH:mm') : null,
        })),
    },
    })
}

function creaPresenzaTool(ctx: Ctx) {
    return tool({
        description: 'Crea una presenza manuale (timbratura) per un dipendente.',
        inputSchema: z.object({
            dipendente_id: z.string().describe('ID del dipendente (ottenuto con cerca_dipendenti)'),
            data: z.string().describe('Data yyyy-MM-dd'),
            ora_ingresso: z.string().describe('Orario di ingresso HH:MM'),
            ora_uscita: z.string().optional().describe('Orario di uscita HH:MM, opzionale'),
        }),
        execute: async ({ dipendente_id, data, ora_ingresso, ora_uscita }) => {
            if (!DATE_RE.test(data)) return { error: 'Formato data non valido, usa yyyy-MM-dd.' }
            if (!isValidTime(ora_ingresso) || (ora_uscita && !isValidTime(ora_uscita))) return { error: 'Formato orario non valido, usa HH:MM.' }

            const { data: employee, error: empErr } = await ctx.admin
                .from('profiles')
                .select('id, full_name, restaurant_id, department, role')
                .eq('id', dipendente_id)
                .single()
            if (empErr || !employee) return { error: 'Dipendente non trovato.' }
            if (!assigneeInScope(ctx.profile, employee)) return { error: 'Non sei autorizzato a creare presenze per questo dipendente.' }

            const checkInIso = fromZonedTime(`${data}T${ora_ingresso}:00`, TZ).toISOString()
            const checkOutIso = ora_uscita ? fromZonedTime(`${data}T${ora_uscita}:00`, TZ).toISOString() : null
            if (checkOutIso && checkOutIso <= checkInIso) return { error: 'L\'orario di uscita deve essere dopo l\'ingresso.' }

            const { error } = await ctx.admin.from('attendances').insert({
                user_id: employee.id,
                restaurant_id: employee.restaurant_id,
                check_in: checkInIso,
                check_out: checkOutIso,
            })
            if (error) return { error: error.message }

            return { ok: true, messaggio: `Presenza creata per ${employee.full_name} il ${formatDateLabel(data)}, ingresso ${ora_ingresso}${ora_uscita ? `, uscita ${ora_uscita}` : ''}.` }
        },
    })
}

function modificaPresenzaTool(ctx: Ctx) {
    return tool({
        description: 'Modifica gli orari di ingresso/uscita di una presenza esistente, dato il suo ID (ottenuto con lista_presenze).',
        inputSchema: z.object({
            presenza_id: z.string().describe('ID della presenza'),
            ora_ingresso: z.string().optional().describe('Nuovo orario di ingresso HH:MM'),
            ora_uscita: z.string().optional().describe('Nuovo orario di uscita HH:MM'),
        })
    })
}

```

```

    }},
    execute: async ({ presenza_id, ora_ingresso, ora_uscita }) => {
      if (!ora_ingresso && !ora_uscita) return { error: 'Specifica almeno un orario da modificare.' }
      if ((ora_ingresso && !isValidTime(ora_ingresso)) || (ora_uscita && !isValidTime(ora_uscita))) {
        return { error: 'Formato orario non valido, usa HH:MM.' }
      }

      let query = ctx.admin.from('attendances').select('id, check_in, check_out, restaurant_id').eq('id',
presenza_id)
      if (isDirettore(ctx.profile)) query = query.eq('restaurant_id', ctx.profile.restaurant_id)
      const { data: att, error: getErr } = await query.maybeSingle()
      if (getErr) return { error: getErr.message }
      if (!att) return { error: 'Presenza non trovata o non nel tuo ambito.' }

      const dateStr = formatInTimeZone(att.check_in, TZ, 'yyyy-MM-dd')
      const update: Record<string, string> = {}

      if (ora_ingresso) update.check_in = fromZonedTime(`${dateStr}T${ora_ingresso}:00`, TZ).toISOString()
      if (ora_uscita) update.check_out = fromZonedTime(`${dateStr}T${ora_uscita}:00`, TZ).toISOString()

      const checkIn = update.check_in ?? att.check_in
      const checkOut = update.check_out ?? att.check_out
      if (checkOut && checkOut <= checkIn) return { error: 'L\'orario di uscita deve essere dopo l\'ingresso.' }

      const { error } = await ctx.admin.from('attendances').update(update).eq('id', presenza_id)
      if (error) return { error: error.message }

      return { ok: true, messaggio: 'Presenza aggiornata.' }
    },
  })
}

// — Esecuzione assistente —————

function buildTools(ctx: Ctx): Record<string, Tool> {
  const tools: Record<string, Tool> = {
    cerca_dipendenti: cercaDipendentiTool(ctx),
    lista_turni: listaTurniTool(ctx),
    crea_turno: creaTurnoTool(ctx),
    modifica_turno: modificaTurnoTool(ctx),
    assegna_riposo: assegnaRiposoTool(ctx),
    elimina_turno: eliminaTurnoTool(ctx),
    lista_ods: listaOdsTool(ctx),
    crea_ods: creaOdsTool(ctx),
    completa_ods: completaOdsTool(ctx),
  }

  if (canManagePresenze(ctx.profile)) {
    tools.lista_presenze = listaPresenzeTool(ctx)
    tools.crea_presenza = creaPresenzaTool(ctx)
    tools.modifica_presenza = modificaPresenzaTool(ctx)
  }

  return tools
}

// Rete di sicurezza: riconosce le risposte finali "rotte" del modello, da
// correggere con un secondo passaggio forzato. Due casi ricorrenti:
// 1. chiede all'utente gli ID interni (UUID) invece di usare cerca_dipendenti;
// 2. annuncia un'azione imminente ("ora recupero...", "ok, elimino...")
// senza aver chiamato lo strumento corrispondente.
function looksUnfinished(text: string): boolean {
  if (/^bid\b/i.test(text) && /fornir|ho bisogno di (sapere|conoscere)|puoi (dirmi|darmi|indicarmi)|mi (serve|
servono|dai|dici)|qual è /i.test(text)) {
    return true
  }
  // Se contiene una domanda è probabilmente una richiesta di conferma
  // legittima ("Vuoi che procedo?"): non va corretta automaticamente,
  // altrimenti l'azione verrebbe eseguita senza la conferma dell'utente.
  if (text.includes('?')) return false
  if (/^b(un attimo|un momento|procedo|provvedo|riprovo|sto per|vado a|mi metto a)\b/i.test(text)) return true
  // Verbo d'azione alla prima persona presente a inizio frase
  // (es. "Ok, elimino il turno...", "Ora recupero i turni...")
  return /^(^|[\s+]|b(?:ok|certo|perfetto|bene|va bene|d'accordo|allora)[,!]?\s+)(?:ora\s+|adesso\s+|subito\s+)?

```

```

(recupero|cerco|controllo|verifico|elimino|cancello|creo|aggiungo|modifico|sostituisco|scambio|sposto|assegno|
registro)\b/i.test(text)
}

const CORRECTION_NUDGE = '[Nota automatica del sistema, invisibile all\'utente] La tua ultima risposta non va
bene: non devi mai chiedere ID all\'utente né annunciare azioni senza eseguirle. Usa SUBITO gli strumenti
necessari (cerca_dipendenti per trovare le persone, lista_turni per i turni, ecc.), completa la richiesta
dell\'utente e rispondi solo con il risultato finale o con una domanda che l\'utente può davvero capire (mai sugli
ID).'
```

export async function runAiAssistant(ctx: Ctx, userText: string): Promise<string> {

```

  if (!process.env.GOOGLE_GENERATIVE_AI_API_KEY) {
    return '  L\'assistente AI non è ancora configurato. Usa /help per l\'elenco dei comandi disponibili.'
  }

  try {
    const history = await getAiHistory(ctx.admin, ctx.telegramId)
    const messages: ModelMessage[] = [...history, { role: 'user', content: userText }]
    const system = buildSystemPrompt(ctx)
    const tools = buildTools(ctx)

    const generate = async (msgs: ModelMessage[]) => {
      try {
        return await generateText({
          model: google(GEMINI_MODEL),
          system,
          messages: msgs,
          tools,
          stopWhen: stepCountIs(8),
        })
      } catch (err) {
        if (!isRateLimitError(err) || GEMINI_FALLBACK_MODEL === GEMINI_MODEL) throw err
        console.warn(`[AI] Quota esaurita per ${GEMINI_MODEL}, passo a ${GEMINI_FALLBACK_MODEL}`)
        return await generateText({
          model: google(GEMINI_FALLBACK_MODEL),
          system,
          messages: msgs,
          tools,
          stopWhen: stepCountIs(8),
        })
      }
    }

    let result = await generate(messages)
    let text = result.text?.trim()
    let finalMessages: ModelMessage[] = [...messages, ...result.response.messages]

    if (text && looksUnfinished(text)) {
      console.warn(`[AI] Risposta incompleta ("${text.slice(0, 80)}..."), forzo un passaggio correttivo`)
      const retryMessages: ModelMessage[] = [...finalMessages, { role: 'user', content: CORRECTION_NUDGE }]
      result = await generate(retryMessages)
      const retryText = result.text?.trim()
      if (retryText) {
        text = retryText
        finalMessages = [...retryMessages, ...result.response.messages]
      }
    }

    if (!text) return '  Non sono riuscito a generare una risposta. Riprova oppure usa /help per i comandi
    disponibili.'

    await saveAiHistory(ctx.admin, ctx.telegramId, finalMessages)
    return text
  } catch (err) {
    console.error('Errore assistente AI Telegram:', err instanceof Error ? err.stack ?? err.message : err)
    if (isRateLimitError(err)) {
      return '  Troppe richieste all\'assistente AI in questo momento. Aspetta qualche secondo e riprova.'
    }
    return '  Si è verificato un errore con l\'assistente AI. Riprova più tardi oppure usa /help per i comandi
    disponibili.'
  }
}

```

```

import type { createAdminClient } from '@lib/supabase/admin'
import type { ModelMessage } from 'ai'

// — Memoria conversazionale dell'assistente AI —————
// Mantiene la cronologia completa (testo + tool-call/tool-result) degli
// ultimi scambi per telegram_id, così l'assistente può ragionare in
// relazione ai messaggi appena ricevuti e riutilizzare ID restituiti
// dagli strumenti in turni precedenti (es. ID dei turni da eliminare
// dopo una conferma dell'utente).

type AdminClient = ReturnType<typeof createAdminClient>

const HISTORY_TURNS = 4 // numero di scambi utente/assistente da mantenere

export async function getAiHistory(admin: AdminClient, telegramId: number): Promise<ModelMessage[]> {
  const { data } = await admin
    .from('telegram_ai_messages')
    .select('messages')
    .eq('telegram_id', telegramId)
    .maybeSingle()

  return (data?.messages as ModelMessage[] | null) ?? []
}

export async function saveAiHistory(admin: AdminClient, telegramId: number, messages: ModelMessage[]):
Promise<void> {
  await admin
    .from('telegram_ai_messages')
    .upsert({ telegram_id: telegramId, messages: stripProviderData(trimToLastTurns(messages, HISTORY_TURNS)),
      updated_at: new Date().toISOString() })
}

export async function clearAiHistory(admin: AdminClient, telegramId: number): Promise<void> {
  await admin.from('telegram_ai_messages').delete().eq('telegram_id', telegramId)
}

// Mantiene solo gli ultimi `maxTurns` scambi, tagliando a partire dal
// messaggio "user" che apre il primo scambio da conservare (così non si
// rompono coppie tool-call/tool-result a metà).
function trimToLastTurns(messages: ModelMessage[], maxTurns: number): ModelMessage[] {
  const turnStarts: number[] = []
  messages.forEach((m, i) => { if (m.role === 'user') turnStarts.push(i) })
  if (turnStarts.length <= maxTurns) return messages
  return messages.slice(turnStarts[turnStarts.length - maxTurns])
}

// Rimuove i metadati specifici del provider (es. `thoughtSignature` di Gemini)
// prima di salvare la cronologia: sono voluminosi e, se rigiocati in una
// richiesta successiva eventualmente gestita da un modello diverso (es. dopo
// un fallback per quota esaurita), possono causare errori di validazione.
function stripProviderData(messages: ModelMessage[]): ModelMessage[] {
  return messages.map((m) => {
    const { providerOptions, ...rest } = m as ModelMessage & { providerOptions?: unknown }
    void providerOptions
    if (Array.isArray(rest.content)) {
      return {
        ...rest,
        content: rest.content.map((part) => {
          const { providerOptions, ...restPart } = part as typeof part & { providerOptions?: unknown }
          void providerOptions
          return restPart
        }),
      } as ModelMessage
    }
    return rest as ModelMessage
  })
}

```



```

import type { InlineKeyboardMarkup, TgUser } from './types'

const API_BASE = 'https://api.telegram.org'

function token(): string {
  const t = process.env.TELEGRAM_BOT_TOKEN
  if (!t) throw new Error('TELEGRAM_BOT_TOKEN non configurato')
  return t
}

async function call<T>(method: string, body?: Record<string, unknown>): Promise<T> {
  const res = await fetch(`${API_BASE}/bot${token()}/${method}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body ?? {}),
  })
  const json = await res.json()
  if (!json.ok) {
    throw new Error(`Telegram API error (${method}): ${json.description ?? 'unknown'}`)
  }
  return json.result as T
}

export interface SendMessageOptions {
  reply_markup?: InlineKeyboardMarkup
  parse_mode?: 'Markdown' | 'HTML'
}

export async function sendMessage(chatId: number, text: string, options?: SendMessageOptions) {
  return call<{ message_id: number }>('sendMessage', {
    chat_id: chatId,
    text,
    ...options,
  })
}

export async function editMessageText(
  chatId: number,
  messageId: number,
  text: string,
  options?: SendMessageOptions,
) {
  return call('editMessageText', {
    chat_id: chatId,
    message_id: messageId,
    text,
    ...options,
  })
}

export async function answerCallbackQuery(callbackQueryId: string, text?: string, showAlert?: boolean) {
  return call('answerCallbackQuery', {
    callback_query_id: callbackQueryId,
    text,
    show_alert: showAlert ?? false,
  })
}

export async function setWebhook(url: string, secretToken: string) {
  return call('setWebhook', {
    url,
    secret_token: secretToken,
    allowed_updates: ['message', 'callback_query'],
  })
}

let cachedMe: TgUser | null = null
export async function getMe(): Promise<TgUser> {
  if (cachedMe) return cachedMe
  cachedMe = await call<TgUser>('getMe')
  return cachedMe
}

```



```

import { createAdminClient } from '@lib/supabase/admin'
import type { Department, Role } from '@types'

export interface TelegramProfile {
  id: string
  full_name: string
  role: Role
  restaurant_id: string | null
  department: Department | null
  is_direttore: boolean
  restaurant?: { id: string; name: string } | null
}

// Ruoli che NON possono mai collegare Telegram (vincolo di sicurezza).
const FORBIDDEN_ROLES: Role[] = ['dipendente', 'consulente_lavoro']

export async function getTelegramUser(telegramId: number): Promise<TelegramProfile | null> {
  const admin = createAdminClient()

  const { data: link } = await admin
    .from('telegram_links')
    .select('user_id')
    .eq('telegram_id', telegramId)
    .maybeSingle()

  if (!link) return null

  const { data: profile } = await admin
    .from('profiles')
    .select('id, full_name, role, restaurant_id, department, is_direttore, restaurant:restaurants(id, name)')
    .eq('id', link.user_id)
    .single()

  if (!profile) return null

  // Difesa in profondità: se il ruolo è cambiato dopo il collegamento,
  // tratta l'utente come non collegato.
  if (FORBIDDEN_ROLES.includes(profile.role as Role)) return null

  return profile as unknown as TelegramProfile
}

export async function linkAccountByPin(
  telegramId: number,
  pin: string,
  telegramUser: { username?: string; first_name?: string },
): Promise<TelegramProfile> {
  const admin = createAdminClient()

  const { data: pinRow } = await admin
    .from('telegram_link_pins')
    .select('user_id, expires_at')
    .eq('pin', pin)
    .maybeSingle()

  if (!pinRow || new Date(pinRow.expires_at) < new Date()) {
    throw new Error('Codice non valido o scaduto. Genera un nuovo codice dall\'app.')
  }

  const { data: profile } = await admin
    .from('profiles')
    .select('id, full_name, role, restaurant_id, department, is_direttore, restaurant:restaurants(id, name)')
    .eq('id', pinRow.user_id)
    .single()

  if (!profile) throw new Error('Profilo non trovato')

  if (FORBIDDEN_ROLES.includes(profile.role as Role)) {
    throw new Error('Il tuo ruolo non è abilitato a collegare Telegram.')
  }

  // Rimuove eventuali collegamenti precedenti in conflitto (stesso utente
  // Supabase o stesso account Telegram già collegati altrove).

```

```
await admin.from('telegram_links').delete().eq('user_id', profile.id)
await admin.from('telegram_links').delete().eq('telegram_id', telegramId)

await admin.from('telegram_links').insert({
  user_id:      profile.id,
  telegram_id:  telegramId,
  telegram_username: telegramUser.username ?? null,
  telegram_first_name: telegramUser.first_name ?? null,
  linked_at:    new Date().toISOString(),
})

await admin.from('telegram_link_pins').delete().eq('pin', pin)

return profile as unknown as TelegramProfile
}
```

```

import { sendMessage } from '../api'
import { linkAccountByPin, type TelegramProfile } from '../auth'
import { isManager, isDirettore, isCapoServizio } from '../scope'
import type { Ctx } from '../context'
import { reply } from '../context'

export function helpText(profile: TelegramProfile): string {
  const lines = [ `    Ciao *${profile.full_name}*!`, '' ]

  lines.push('    *Turni*')
  lines.push('/turni – turni dei prossimi 7 giorni')
  lines.push('/nuovoturno – assegna un nuovo turno')
  lines.push('/riposo – assegna un riposo')
  lines.push('/eliminaturno – elimina un turno')
  lines.push('')

  lines.push('    *ODS*')
  lines.push('/ods – compiti di oggi')
  lines.push('/nuovoods – crea un nuovo compito')
  lines.push('/completaods – segna un compito come completato')

  if (isManager(profile) || isDirettore(profile)) {
    lines.push('')
    lines.push('    *Presenze*')
    lines.push('/presenze – presenze di oggi')
    lines.push('/modificapresenza – modifica orari presenza')
    lines.push('/nuovapresenza – crea una presenza manuale')
  }

  lines.push('')
  lines.push('/help – mostra questo messaggio')
  lines.push('/annulla – annulla l\'operazione in corso')

  lines.push('')
  lines.push('    *Assistente AI*')
  lines.push('Scrivimi anche in linguaggio naturale, senza comandi! Es. "Chi lavora venerdì in cucina?", "Segna un riposo a Mario per domani", "Crea un turno per Giulia lunedì 9-17".')

  let scopeLabel = ''
  if (isManager(profile)) scopeLabel = 'Manager – accesso globale a tutti i ristoranti'
  else if (isDirettore(profile)) scopeLabel = `Direttore – ${profile.restaurant?.name ?? 'ristorante'} (tutti i reparti)`
  else if (isCapoServizio(profile)) scopeLabel = `Capo Servizio – ${profile.restaurant?.name ?? 'ristorante'} (${profile.department ?? 'reparto'})`

  if (scopeLabel) lines.push('', `_${scopeLabel}_`)

  return lines.join('\n')
}

export async function cmdHelp(ctx: Ctx) {
  return reply(ctx, helpText(ctx.profile), { parse_mode: 'Markdown' })
}

// /start [pin] – gestito prima della risoluzione del profilo, perché un
// account non ancora collegato non ha un TelegramProfile.
export async function handleStart(
  telegramId: number,
  chatId: number,
  args: string,
  telegramUser: { username?: string; first_name?: string },
  existingProfile: TelegramProfile | null,
): Promise<void> {
  const pin = args.trim()

  if (!pin) {
    if (existingProfile) {
      await sendMessage(chatId, helpText(existingProfile), { parse_mode: 'Markdown' })
    } else {
      await sendMessage(
        chatId,
        `    Benvenuto nel bot Turni!\n\nPer collegare il tuo account, genera un codice dall\'app (sezione Dashboard → "Collega Telegram") e invialo qui.`,
      )
    }
  }
}

```

```
    )
  }
  return
}

try {
  const profile = await linkAccountByPin(telegramId, pin, telegramUser)
  await sendMessage(chatId, `  Account collegato con successo!\n\n${helpText(profile)}`, { parse_mode:
'Markdown' })
} catch (err) {
  const message = err instanceof Error ? err.message : 'Errore durante il collegamento.'
  await sendMessage(chatId, `  ${message}`)
}
}
```

```

import { formatInTimeZone, fromZonedTime } from 'date-fns-tz'
import { it } from 'date-fns/locale'
import type { Ctx } from '../context'
import { reply } from '../context'
import { setSession, clearSession } from '../session'
import { scopeStaffQuery, toScopeProfile, isManager, isDirettore, assigneeInScope } from '../scope'
import { TZ, escapeMd, buildEmployeeKeyboard, buildCancelKeyboard, buildYesNoKeyboard } from '../format'
import { listRestaurants, buildRestaurantKeyboard, listAssignableStaff } from '../wizard'
import { DEPARTMENTS, ODS_DAYS_IT, ODS_TYPE_LABELS, type OdsTaskType } from '@types'
import type { InlineKeyboardMarkup } from '../types'

function getOdsCutoff(): string {
  const now = new Date()
  const romeHour = parseInt(formatInTimeZone(now, TZ, 'H'), 10)
  const refDate = romeHour < 4 ? new Date(now.getTime() - 86_400_000) : now
  const cutoffDate = formatInTimeZone(refDate, TZ, 'yyyy-MM-dd')
  return fromZonedTime(`${cutoffDate}T04:00:00`, TZ).toISOString()
}

function todayName(): string {
  return formatInTimeZone(new Date(), TZ, 'EEEE', { locale: it }).toLowerCase()
}

type OdsRow = {
  id: string; title: string; department: string; type: OdsTaskType
  recurrence_days: string[]; assigned_to: string | null
  assignee: { id: string; full_name: string } | null
}

function isDueToday(t: { type: OdsTaskType; recurrence_days: string[] }): boolean {
  if (t.type === 'quotidiana' || t.type === 'straordinaria') return true
  return t.recurrence_days.some(d => d.toLowerCase() === todayName())
}

// — /ods — compiti di oggi nel proprio scope —————
export async function cmdOds(ctx: Ctx) {
  const { admin, profile } = ctx

  let query = admin
    .from('ods_tasks')
    .select('id, title, department, type, recurrence_days, assigned_to, assignee:profiles!assigned_to(id, full_name)')
    .order('department')
  query = scopeStaffQuery(query, toScopeProfile(profile))

  const { data, error } = await query
  if (error) return reply(ctx, `Errore: ${error.message}`)

  const tasks = ((data ?? []) as unknown as OdsRow[]).filter(isDueToday)
  if (!tasks.length) return reply(ctx, 'Nessun compito ODS previsto per oggi.')

  const cutoff = getOdsCutoff()
  const { data: completions } = await admin
    .from('ods_completions')
    .select('task_id')
    .in('task_id', tasks.map(t => t.id))
    .gte('completed_at', cutoff)

  const doneSet = new Set((completions ?? []).map((c: { task_id: string }) => c.task_id))

  const lines = [' *ODS — Oggi*\n']
  for (const t of tasks) {
    const done = doneSet.has(t.id) ? ' ' : ' '
    const who = t.assignee?.full_name ?? `Tutto il reparto ${t.department}`
    lines.push(`${done} *${escapeMd(t.title)}*\n  ${escapeMd(t.department)} · ${escapeMd(who)} · ${ODS_TYPE_LABELS[t.type]}`)
  }

  return reply(ctx, lines.join('\n\n'), { parse_mode: 'Markdown' })
}

// — /completaods — segna compiti come completati —————
export async function cmdCompletaOds(ctx: Ctx) {

```

```

const { admin, profile } = ctx

let query = admin
  .from('ods_tasks')
  .select('id, title, department, type, recurrence_days')
  .order('department')
query = scopeStaffQuery(query, toScopeProfile(profile))

const { data } = await query
const tasks = ((data ?? []) as unknown as OdsRow[]).filter(isDueToday)
if (!tasks.length) return reply(ctx, ' Nessun compito ODS previsto per oggi.')

const cutoff = getOdsCutoff()
const { data: myCompletions } = await admin
  .from('ods_completions')
  .select('task_id')
  .eq('user_id', profile.id)
  .in('task_id', tasks.map(t => t.id))
  .gte('completed_at', cutoff)

const doneSet = new Set((myCompletions ?? []).map((c: { task_id: string }) => c.task_id))
const pending = tasks.filter(t => !doneSet.has(t.id))

if (!pending.length) return reply(ctx, ' Hai già completato tutti i compiti di oggi!')

const buttons = pending.map(t => [{ text: ` ${t.title} (${t.department})`, callback_data: `co_done:${t.id}`
` }])
buttons.push([{ text: ' Annulla', callback_data: 'cancel' }])
await setSession(ctx.telegramId, 'co_select', {})
return reply(ctx, ' Seleziona il compito da segnare come completato:', { reply_markup: { inline_keyboard:
buttons } })
}

// — /nuovoods - wizard creazione compito —————
export async function cmdNuovoOds(ctx: Ctx) {
  if (isManager(ctx.profile)) {
    const restaurants = await listRestaurants(ctx.admin)
    if (!restaurants.length) return reply(ctx, 'Nessun ristorante trovato.')
    await setSession(ctx.telegramId, 'no_restaurant', {})
    return reply(ctx, ' Seleziona il ristorante:', { reply_markup: buildRestaurantKeyboard(restaurants,
'no_rest') })
  }
  await setSession(ctx.telegramId, 'no_title', {})
  return reply(ctx, ' Inserisci il titolo del nuovo compito ODS:', { reply_markup: buildCancelKeyboard() })
}

function buildTypeKeyboard(): InlineKeyboardMarkup {
  return {
    inline_keyboard: [
      [{ text: 'Quotidiana', callback_data: 'no_type:quotidiana' }, { text: 'Straordinaria', callback_data:
'no_type:straordinaria' }],
      [{ text: 'Settimanale', callback_data: 'no_type:settimanale' }, { text: 'Bisettimanale', callback_data:
'no_type:bisettimanale' }],
      [{ text: ' Annulla', callback_data: 'cancel' }],
    ],
  }
}

function buildDaysKeyboard(selected: string[]): InlineKeyboardMarkup {
  const rows = ODS_DAYS_IT.map(d => [{
    text: `${selected.includes(d) ? ' ' : ' '} ${d}`,
    callback_data: `no_day:${d}`,
  }])
  rows.push([{ text: ' Fatto', callback_data: 'no_days_done' }])
  rows.push([{ text: ' Annulla', callback_data: 'cancel' }])
  return { inline_keyboard: rows }
}

function buildTargetKeyboard(profile: Ctx['profile']): InlineKeyboardMarkup {
  const rows: InlineKeyboardMarkup['inline_keyboard'] = [{ text: ' Dipendente specifico', callback_data:
'no_target:employee' }]
  if (isDirettore(profile) || isManager(profile)) {
    for (const dept of DEPARTMENTS) {
      rows.push([{ text: ` Reparto ${dept}`, callback_data: `no_target:dept:${dept}` }])
    }
  }
}

```

```

    }
    rows.push([ { text: '    Tutti i reparti', callback_data: 'no_target:alldept' } ])
  } else {
    rows.push([ { text: `    Tutto il reparto ${profile.department}`, callback_data: `no_target:dept:${profile.department}` } ])
  }
  rows.push([ { text: '    Annulla', callback_data: 'cancel' } ])
  return { inline_keyboard: rows }
}

async function goToTargetStep(ctx: Ctx, data: Record<string, unknown>) {
  await setSession(ctx.telegramId, 'no_target', data)
  await reply(ctx, '    A chi vuoi assegnare il compito?', { reply_markup: buildTargetKeyboard(ctx.profile) })
}

// — Dispatcher callback —————
export async function handleOdsCallback(ctx: Ctx, state: string, data: Record<string, unknown>): Promise<boolean>
{
  const cb = ctx.callbackData ?? ''

  if (cb.startsWith('co_done:')) {
    const taskId = cb.split(':')[1]
    await clearSession(ctx.telegramId)
    const { error } = await ctx.admin.from('ods_completions').insert({ task_id: taskId, user_id: ctx.profile.id })
    if (error) await reply(ctx, `Errore: ${error.message}`)
    else await reply(ctx, '    Compito segnato come completato!')
    return true
  }

  if (state === 'no_restaurant' && cb.startsWith('no_rest:')) {
    const restaurantId = cb.split(':')[1]
    await setSession(ctx.telegramId, 'no_title', { restaurant_id: restaurantId })
    await reply(ctx, '    Inserisci il titolo del nuovo compito ODS:', { reply_markup: buildCancelKeyboard() })
    return true
  }

  if (state === 'no_type' && cb.startsWith('no_type:')) {
    const type = cb.split(':')[1] as OdsTaskType
    const next = { ...data, type, days: [] as string[] }
    if (type === 'settimanale' || type === 'bisettimanale') {
      await setSession(ctx.telegramId, 'no_days', next)
      await reply(ctx, '    Seleziona i giorni della settimana:', { reply_markup: buildDaysKeyboard([]) })
    } else {
      await goToTargetStep(ctx, next)
    }
    return true
  }

  if (state === 'no_days' && cb.startsWith('no_day:')) {
    const day = cb.split(':')[1]
    const days = new Set((data.days as string[]) ?? [])
    if (days.has(day)) days.delete(day)
    else days.add(day)
    const next = { ...data, days: Array.from(days) }
    await setSession(ctx.telegramId, 'no_days', next)
    await reply(ctx, '    Seleziona i giorni della settimana:', { edit: true, reply_markup:
buildDaysKeyboard(next.days) })
    return true
  }

  if (state === 'no_days' && cb === 'no_days_done') {
    const days = (data.days as string[]) ?? []
    if (!days.length) {
      await reply(ctx, '    Seleziona almeno un giorno.', { reply_markup: buildDaysKeyboard(days) })
      return true
    }
    await goToTargetStep(ctx, data)
    return true
  }

  if (state === 'no_target' && cb.startsWith('no_target:')) {
    const parts = cb.split(':')
    const kind = parts[1]
    if (kind === 'employee') {

```

```

const staff = await listAssignableStaff(ctx.admin, ctx.profile, data.restaurant_id as string | undefined)
if (!staff.length) return reply(ctx, 'Nessun dipendente trovato.').then(() => true)
await setSession(ctx.telegramId, 'no_employee', data)
await reply(ctx, '  Seleziona il dipendente:', { reply_markup: buildEmployeeKeyboard(staff, 'no_emp') })
return true
}
if (kind === 'dept') {
  const department = parts[2]
  const next = { ...data, department, assigned_to: null }
  await setSession(ctx.telegramId, 'no_confirm', next)
  await reply(ctx, summaryNuovoOds(next), { parse_mode: 'Markdown', reply_markup:
buildYesNoKeyboard('no_confirm') })
  return true
}
if (kind === 'alldept') {
  const next = { ...data, department: null, assigned_to: null, all_departments: true }
  await setSession(ctx.telegramId, 'no_confirm', next)
  await reply(ctx, summaryNuovoOds(next), { parse_mode: 'Markdown', reply_markup:
buildYesNoKeyboard('no_confirm') })
  return true
}
}

if (state === 'no_employee' && cb.startsWith('no_emp:')) {
  const employeeId = cb.split(':')[1]
  const staff = await listAssignableStaff(ctx.admin, ctx.profile, data.restaurant_id as string | undefined)
  const employee = staff.find(s => s.id === employeeId)
  if (!employee) return reply(ctx, 'Dipendente non trovato.').then(() => true)
  if (!assigneeInScope(ctx.profile, employee)) {
    await clearSession(ctx.telegramId)
    await reply(ctx, '  Non autorizzato per questo dipendente.')
    return true
  }
  const next = { ...data, department: employee.department, assigned_to: employee.id, assignee_name:
employee.full_name, restaurant_id: employee.restaurant_id }
  await setSession(ctx.telegramId, 'no_confirm', next)
  await reply(ctx, summaryNuovoOds(next), { parse_mode: 'Markdown', reply_markup:
buildYesNoKeyboard('no_confirm') })
  return true
}

if (state === 'no_confirm' && cb.startsWith('no_confirm:')) {
  const ok = cb.split(':')[1] === 'yes'
  await clearSession(ctx.telegramId)
  if (!ok) {
    await reply(ctx, '  Operazione annullata.')
    return true
  }
  await createOdsFromWizard(ctx, data)
  return true
}

return false
}

// — Dispatcher testo —————
export async function handleOdsText(ctx: Ctx, state: string, data: Record<string, unknown>): Promise<boolean> {
  if (state === 'no_title') {
    const title = ctx.text.trim()
    if (!title) {
      await reply(ctx, '  Il titolo non può essere vuoto. Riprova:')
      return true
    }
    const next = { ...data, title }
    await setSession(ctx.telegramId, 'no_type', next)
    await reply(ctx, '  Con quale frequenza?', { reply_markup: buildTypeKeyboard() })
    return true
  }
  return false
}

function summaryNuovoOds(data: Record<string, unknown>): string {
  const target = data.all_departments
  ? 'Tutti i reparti'

```

```

    : data.assigned_to
    : ? `${data.assignee_name}`
    : `Tutto il reparto ${data.department}`

const days = (data.days as string[]) ?? []
const lines = [
  ' *Riepilogo nuovo compito ODS*',
  '',
  ` Titolo: ${escapeMd(data.title as string)} `,
  ` Tipo: ${ODS_TYPE_LABELS[data.type as OdsTaskType]} `,
]
if (days.length) lines.push(` Giorni: ${days.join(', ')} `)
lines.push(` Assegnato a: ${escapeMd(target)} `)
lines.push('', 'Confermi?')
return lines.join('\n')
}

async function createOdsFromWizard(ctx: Ctx, data: Record<string, unknown>) {
  const { admin, profile } = ctx
  const restaurantId = (data.restaurant_id as string | undefined) ?? profile.restaurant_id
  if (!restaurantId) {
    await reply(ctx, 'Errore: ristorante non determinato.')
    return
  }

  const departments: string[] = data.all_departments
  ? [...DEPARTMENTS]
  : [data.department as string]

  const baseRow = {
    title: data.title as string,
    restaurant_id: restaurantId,
    type: data.type as OdsTaskType,
    recurrence_days: (data.days as string[]) ?? [],
    creator_id: profile.id,
  }

  let created = 0
  for (const department of departments) {
    const assignedTo = !data.all_departments ? (data.assigned_to as string | null) : null
    const { data: task, error } = await admin
      .from('ods_tasks')
      .insert({ ...baseRow, department, assigned_to: assignedTo })
      .select('id')
      .single()

    if (error || !task) continue
    created++

    // Notifiche non bloccanti ai destinatari
    let recipients: { id: string }[] = []
    if (assignedTo) {
      recipients = [{ id: assignedTo }]
    } else {
      const { data: staff } = await admin
        .from('profiles')
        .select('id')
        .eq('restaurant_id', restaurantId)
        .eq('department', department)
        .in('role', ['dipendente', 'capo_servizio'])
        .neq('id', profile.id)
      recipients = staff ?? []
    }
    if (recipients.length) {
      await admin.from('notifications').insert(
        recipients.map(r => ({
          user_id: r.id,
          title: assignedTo ? 'Nuova mansione assegnata' : 'Nuova istruzione di Reparto',
          message: data.title as string,
          link: '/ods',
        }))
      )
    }
  }
}

```

```
if (created === 0) {  
  await reply(ctx, 'Errore nella creazione del compito.')  
} else {  
  await reply(ctx, `  Compito ODS creato${created > 1 ? ` per ${created} reparti` : ''}.`)  
}  
}
```

`src/lib/telegram/commands/presenze.ts`


```

import { formatInTimeZone, fromZonedTime } from 'date-fns-tz'
import type { Ctx } from '../context'
import { reply } from '../context'
import { setSession, clearSession } from '../session'
import { isManager, isDirettore, assigneeInScope } from '../scope'
import { TZ, todayStr, escapeMd, isValidTime, buildDateKeyboard, buildEmployeeKeyboard, buildCancelKeyboard,
buildYesNoKeyboard, formatDateLabel } from '../format'
import { listAssignableStaff } from '../wizard'

function dayBounds(dateStr: string) {
  const start = fromZonedTime(`${dateStr}T00:00:00`, TZ).toISOString()
  const end = fromZonedTime(`${dateStr}T23:59:59.999`, TZ).toISOString()
  return { start, end }
}

// — /presenze – presenze di oggi nel proprio scope —————
export async function cmdPresenze(ctx: Ctx) {
  const { admin, profile } = ctx
  const { start, end } = dayBounds(todayStr())

  let query = admin
    .from('attendances')
    .select('id, check_in, check_out, profile:profiles(full_name), restaurant:restaurants(name)')
    .gte('check_in', start)
    .lte('check_in', end)
    .order('check_in')
    .limit(30)

  if (isDirettore(profile)) query = query.eq('restaurant_id', profile.restaurant_id)

  const { data, error } = await query
  if (error) return reply(ctx, `Errore: ${error.message}`)

  type Row = { check_in: string; check_out: string | null; profile: { full_name: string } | null; restaurant:
{ name: string } | null }
  const rows = (data ?? []) as unknown as Row[]
  if (!rows.length) return reply(ctx, ' Nessuna presenza registrata oggi.')

  const lines = [' *Presenze di oggi*\n']
  for (const r of rows) {
    const checkIn = formatInTimeZone(r.check_in, TZ, 'HH:mm')
    const checkOut = r.check_out ? formatInTimeZone(r.check_out, TZ, 'HH:mm') : '-'
    const restPart = isManager(profile) && r.restaurant?.name ? ` · ${escapeMd(r.restaurant.name)} ` : ''
    lines.push(` ${escapeMd(r.profile?.full_name ?? '-')}: ${checkIn} → ${checkOut}${restPart}`)
  }
  return reply(ctx, lines.join('\n'), { parse_mode: 'Markdown' })
}

// — /modificapresenza – wizard modifica orari presenza esistente —————
export async function cmdModificaPresenza(ctx: Ctx) {
  const { admin, profile } = ctx
  const { start, end } = dayBounds(todayStr())

  let query = admin
    .from('attendances')
    .select('id, check_in, check_out, profile:profiles(full_name)')
    .gte('check_in', start)
    .lte('check_in', end)
    .order('check_in')
    .limit(30)

  if (isDirettore(profile)) query = query.eq('restaurant_id', profile.restaurant_id)

  const { data } = await query
  type Row = { id: string; check_in: string; check_out: string | null; profile: { full_name: string } | null }
  const rows = (data ?? []) as unknown as Row[]
  if (!rows.length) return reply(ctx, ' Nessuna presenza registrata oggi da modificare.')

  const buttons = rows.map(r => [{
    text: `${r.profile?.full_name ?? '-'}: ${formatInTimeZone(r.check_in, TZ, 'HH:mm')} → ${r.check_out ?
formatInTimeZone(r.check_out, TZ, 'HH:mm') : '-'}`,
    callback_data: `mp_sel:${r.id}`,
  }])
}

```

```

    buttons.push([ { text: '  Annulla', callback_data: 'cancel' } ])
    await setSession(ctx.telegramId, 'mp_select', {})
    return reply(ctx, '  Seleziona la presenza da modificare:', { reply_markup: { inline_keyboard: buttons } })
  }

// — /nuovapresenza — wizard creazione presenza manuale —————
export async function cmdNuovaPresenza(ctx: Ctx) {
  const staff = await listAssignableStaff(ctx.admin, ctx.profile, isManager(ctx.profile) ? undefined :
ctx.profile.restaurant_id ?? undefined)
  if (!staff.length) return reply(ctx, 'Nessun dipendente trovato.')
  await setSession(ctx.telegramId, 'np_employee', {})
  return reply(ctx, '  Seleziona il dipendente:', { reply_markup: buildEmployeeKeyboard(staff, 'np_emp') })
}

// — Dispatcher callback —————
export async function handlePresenzeCallback(ctx: Ctx, state: string, data: Record<string, unknown>):
Promise<boolean> {
  const cb = ctx.callbackData ?? ''
  const { admin, profile } = ctx

  if (state === 'mp_select' && cb.startsWith('mp_sel:')) {
    const id = cb.split(':')[1]
    let query = admin.from('attendances').select('id, check_in, check_out').eq('id', id)
    if (isDirettore(profile)) query = query.eq('restaurant_id', profile.restaurant_id)
    const { data: row } = await query.maybeSingle()
    if (!row) {
      await clearSession(ctx.telegramId)
      await reply(ctx, '  Presenza non trovata o non autorizzata.')
      return true
    }
    const date = formatInTimeZone(row.check_in, TZ, 'yyyy-MM-dd')
    await setSession(ctx.telegramId, 'mp_checkin', { attendance_id: id, date })
    await reply(ctx, `  Inserisci il nuovo orario di *ingresso* (HH:MM):`, { parse_mode: 'Markdown',
reply_markup: buildCancelKeyboard() })
    return true
  }

  if (state === 'mp_checkout' && cb === 'mp_checkout:none') {
    await finalizeModificaPresenza(ctx, data, null)
    return true
  }

  if (state === 'np_employee' && cb.startsWith('np_emp:')) {
    const employeeId = cb.split(':')[1]
    const staff = await listAssignableStaff(admin, profile, isManager(profile) ? undefined :
profile.restaurant_id ?? undefined)
    const employee = staff.find(s => s.id === employeeId)
    if (!employee) return reply(ctx, 'Dipendente non trovato.').then(() => true)
    if (!assigneeInScope(profile, employee)) {
      await clearSession(ctx.telegramId)
      await reply(ctx, '  Non autorizzato per questo dipendente.')
      return true
    }
    await setSession(ctx.telegramId, 'np_date', { employee_id: employee.id, employee_name: employee.full_name,
restaurant_id: employee.restaurant_id })
    await reply(ctx, `  ${escapeMd(employee.full_name)}\n\n  Seleziona la data:`, { parse_mode: 'Markdown',
reply_markup: buildDateKeyboard('np_date') })
    return true
  }

  if (state === 'np_date' && cb.startsWith('np_date:')) {
    const date = cb.split(':')[1]
    await setSession(ctx.telegramId, 'np_checkin', { ...data, date })
    await reply(ctx, `  ${escapeMd(formatDateLabel(date))}\n\n  Inserisci l'orario di *ingresso* (HH:MM):`,
{ parse_mode: 'Markdown', reply_markup: buildCancelKeyboard() })
    return true
  }

  if (state === 'np_checkout' && cb === 'np_checkout:none') {
    await finalizeNuovaPresenza(ctx, data, null)
    return true
  }

  if (state === 'np_confirm' && cb.startsWith('np_confirm:')) {

```

```

    const ok = cb.split(':')[1] === 'yes'
    await clearSession(ctx.telegramId)
    if (!ok) {
        await reply(ctx, 'Operazione annullata.')
        return true
    }
    await insertNuovaPresenza(ctx, data)
    return true
}

return false
}

// — Dispatcher testo —————
export async function handlePresenzeText(ctx: Ctx, state: string, data: Record<string, unknown>): Promise<boolean>
{
    if (state === 'mp_checkin') {
        if (!isValidTime(ctx.text)) {
            await reply(ctx, 'Formato non valido. Inserisci l\'orario come HH:MM.')
            return true
        }
        const next = { ...data, check_in: ctx.text.trim() }
        await setSession(ctx.telegramId, 'mp_checkout', next)
        await reply(ctx, 'Inserisci il nuovo orario di *uscita* (HH:MM), oppure:', {
            parse_mode: 'Markdown',
            reply_markup: { inline_keyboard: [[{ text: 'Nessuna uscita', callback_data: 'mp_checkout:none' }]],
            [{ text: 'Annulla', callback_data: 'cancel' }]] },
        })
        return true
    }
    if (state === 'mp_checkout') {
        if (!isValidTime(ctx.text)) {
            await reply(ctx, 'Formato non valido. Inserisci l\'orario come HH:MM oppure usa il bottone.')
            return true
        }
        await finalizeModificaPresenza(ctx, data, ctx.text.trim())
        return true
    }

    if (state === 'np_checkin') {
        if (!isValidTime(ctx.text)) {
            await reply(ctx, 'Formato non valido. Inserisci l\'orario come HH:MM.')
            return true
        }
        const next = { ...data, check_in: ctx.text.trim() }
        await setSession(ctx.telegramId, 'np_checkout', next)
        await reply(ctx, 'Inserisci l\'orario di *uscita* (HH:MM), oppure:', {
            parse_mode: 'Markdown',
            reply_markup: { inline_keyboard: [[{ text: 'Nessuna uscita', callback_data: 'np_checkout:none' }]],
            [{ text: 'Annulla', callback_data: 'cancel' }]] },
        })
        return true
    }
    if (state === 'np_checkout') {
        if (!isValidTime(ctx.text)) {
            await reply(ctx, 'Formato non valido. Inserisci l\'orario come HH:MM oppure usa il bottone.')
            return true
        }
        await finalizeNuovaPresenza(ctx, data, ctx.text.trim())
        return true
    }

    return false
}

async function finalizeModificaPresenza(ctx: Ctx, data: Record<string, unknown>, checkOut: string | null) {
    const { admin, profile } = ctx
    const date = data.date as string
    const checkIn = data.check_in as string

    const checkInIso = fromZonedTime(`${date}T${checkIn}:00`, TZ).toISOString()
    const checkOutIso = checkOut ? fromZonedTime(`${date}T${checkOut}:00`, TZ).toISOString() : null

    if (checkOutIso && checkOutIso <= checkInIso) {

```

```

    await clearSession(ctx.telegramId)
    await reply(ctx, "    L'orario di uscita deve essere successivo a quello di ingresso. Operazione annullata.")
    return
}

await clearSession(ctx.telegramId)

let query = admin.from('attendances').update({ check_in: checkInIso, check_out: checkOutIso }).eq('id',
data.attendance_id as string)
if (isDirettore(profile)) query = query.eq('restaurant_id', profile.restaurant_id)
const { error } = await query

if (error) await reply(ctx, `Errore: ${error.message}`)
else await reply(ctx, '    Presenza aggiornata.')
}

async function finalizeNuovaPresenza(ctx: Ctx, data: Record<string, unknown>, checkOut: string | null) {
    const next = { ...data, check_out: checkOut }
    await setSession(ctx.telegramId, 'np_confirm', next)

    const checkOutLabel = checkOut ?? '-'
    await reply(ctx, [
        '    *Riepilogo nuova presenza*',
        '',
        `    Dipendente: ${escapeMd(data.employee_name as string)}\`,
        `    Data: ${escapeMd(formatDateLabel(data.date as string))}\`,
        `    Ingresso: ${data.check_in}\`,
        `    Uscita: ${checkOutLabel}\`,
        '',
        '    Confermi?',
    ]).join('\n'), { parse_mode: 'Markdown', reply_markup: buildYesNoKeyboard('np_confirm') })
}

async function insertNuovaPresenza(ctx: Ctx, data: Record<string, unknown>) {
    const { admin } = ctx
    const date = data.date as string
    const checkIn = data.check_in as string
    const checkOut = data.check_out as string | null

    const checkInIso = fromZonedTime(`${date}T${checkIn}:00`, TZ).toISOString()
    const checkOutIso = checkOut ? fromZonedTime(`${date}T${checkOut}:00`, TZ).toISOString() : null

    if (checkOutIso && checkOutIso <= checkInIso) {
        await reply(ctx, "    L'orario di uscita deve essere successivo a quello di ingresso. Operazione annullata.")
        return
    }

    const { error } = await admin.from('attendances').insert({
        user_id:      data.employee_id,
        restaurant_id: data.restaurant_id,
        check_in:      checkInIso,
        check_out:      checkOutIso,
    })

    if (error) await reply(ctx, `Errore: ${error.message}`)
    else await reply(ctx, `    Presenza creata per *${escapeMd(data.employee_name as string)}*.\`, { parse_mode:
'Markdown' })
}

```

`src/lib/telegram/commands/turni.ts`

```

import type { Ctx } from '../context'
import { reply } from '../context'
import { setSession, clearSession } from '../session'
import { toScopeProfile, scopeTurnsQuery, isManager, assigneeInScope } from '../scope'
import {
  formatDateLabel, todayStr, dateAfter, isValidTime, normalizeTime,
  buildDateKeyboard, buildEmployeeKeyboard, buildYesNoKeyboard, buildCancelKeyboard, escapeMd,
} from '../format'
import { listRestaurants, buildRestaurantKeyboard, listAssignableStaff } from '../wizard'

// — /turni — elenco turni nel proprio scope per i prossimi 7 giorni —
export async function cmdTurni(ctx: Ctx) {
  const { admin, profile } = ctx
  const start = todayStr()
  const end = dateAfter(6)

  let query = admin
    .from('turns')
    .select('date, start_time, end_time, is_extraordinary, is_rest_day, department, profile:profiles!
user_id(full_name), restaurant:restaurants(name)')
    .gte('date', start)
    .lte('date', end)
    .order('date')
    .order('start_time')
    .limit(40)

  query = scopeTurnsQuery(query, toScopeProfile(profile), profile.id)

  const { data, error } = await query
  if (error) return reply(ctx, `Errore: ${error.message}`)

  type Row = {
    date: string; start_time: string; end_time: string
    is_extraordinary: boolean; is_rest_day: boolean; department: string | null
    profile: { full_name: string } | null
    restaurant: { name: string } | null
  }
  const rows = (data ?? []) as unknown as Row[]

  if (!rows.length) {
    return reply(ctx, ' Nessun turno programmato nei prossimi 7 giorni.')
  }

  let lastDate = ''
  const lines: string[] = [' *Turni — prossimi 7 giorni*\n']
  for (const t of rows) {
    if (t.date !== lastDate) {
      lines.push(`\n*${escapeMd(formatDateLabel(t.date))}*`)
      lastDate = t.date
    }
    const name = escapeMd(t.profile?.full_name ?? '-')
    const restPart = isManager(profile) && t.restaurant?.name ? ` · ${escapeMd(t.restaurant.name)}` : ''
    if (t.is_rest_day) {
      lines.push(` ${name} — Riposo${restPart}`)
    } else {
      const badge = t.is_extraordinary ? ' ⚡' : ''
      const dept = t.department ? ` (${escapeMd(t.department)})` : ''
      lines.push(` ${t.start_time.slice(0, 5)}-${t.end_time.slice(0, 5)} ${name}${dept}${badge}${restPart}`)
    }
  }

  return reply(ctx, lines.join('\n'), { parse_mode: 'Markdown' })
}

// — /nuovoturno — wizard creazione turno —
export async function cmdNuovoTurno(ctx: Ctx) {
  if (isManager(ctx.profile)) {
    const restaurants = await listRestaurants(ctx.admin)
    if (!restaurants.length) return reply(ctx, 'Nessun ristorante trovato.')
    await setSession(ctx.telegramId, 'nt_restaurant', {})
    return reply(ctx, ' Seleziona il ristorante:', { reply_markup: buildRestaurantKeyboard(restaurants,
'nt_rest') })
  }
}

```

```

    await startEmployeeStep(ctx, 'nt', {})
  }

// — /riposo – wizard inserimento riposo —————
export async function cmdRiposo(ctx: Ctx) {
  if (isManager(ctx.profile)) {
    const restaurants = await listRestaurants(ctx.admin)
    if (!restaurants.length) return reply(ctx, 'Nessun ristorante trovato.')
    await setSession(ctx.telegramId, 'r_restaurant', {})
    return reply(ctx, '  Seleziona il ristorante:', { reply_markup: buildRestaurantKeyboard(restaurants, 'r_rest') })
  }
  await startEmployeeStep(ctx, 'r', {})
}

async function startEmployeeStep(ctx: Ctx, prefix: string, data: Record<string, unknown>) {
  const staff = await listAssignableStaff(ctx.admin, ctx.profile, data.restaurant_id as string | undefined)
  if (!staff.length) return reply(ctx, 'Nessun dipendente trovato nel tuo reparto/ristorante.')
  await setSession(ctx.telegramId, `${prefix}_employee`, data)
  return reply(ctx, '  Seleziona il dipendente:', { reply_markup: buildEmployeeKeyboard(staff, `${prefix}_emp`) })
}

// — /eliminaturno – wizard cancellazione turno —————
export async function cmdEliminaTurno(ctx: Ctx) {
  await setSession(ctx.telegramId, 'dt_date', {})
  return reply(ctx, '  Elimina turno\n\nSeleziona la data:', { reply_markup: buildDateKeyboard('dt_date') })
}

// — Dispatcher per i bottoni (callback_query) —————
export async function handleTurniCallback(ctx: Ctx, state: string, data: Record<string, unknown>):
Promise<boolean> {
  const cb = ctx.callbackData ?? ''

  // — Nuovo turno —
  if (state === 'nt_restaurant' && cb.startsWith('nt_rest:')) {
    const restaurantId = cb.split(':')[1]
    await startEmployeeStep(ctx, 'nt', { restaurant_id: restaurantId })
    return true
  }
  if (state === 'nt_employee' && cb.startsWith('nt_emp:')) {
    const employeeId = cb.split(':')[1]
    const staff = await listAssignableStaff(ctx.admin, ctx.profile, data.restaurant_id as string | undefined)
    const employee = staff.find(s => s.id === employeeId)
    if (!employee) return reply(ctx, 'Dipendente non trovato.').then(() => true)
    if (!assigneeInScope(ctx.profile, employee)) {
      await clearSession(ctx.telegramId)
      await reply(ctx, '  Non autorizzato per questo dipendente.')
      return true
    }
    await setSession(ctx.telegramId, 'nt_date', {
      ...data, employee_id: employee.id, employee_name: employee.full_name,
      department: employee.department, restaurant_id: employee.restaurant_id,
    })
    await reply(ctx, `  ${escapeMd(employee.full_name)}\n\n  Seleziona la data:`, {
      parse_mode: 'Markdown', reply_markup: buildDateKeyboard('nt_date'),
    })
    return true
  }
  if (state === 'nt_date' && cb.startsWith('nt_date:')) {
    const date = cb.split(':')[1]
    await setSession(ctx.telegramId, 'nt_start', { ...data, date })
    await reply(ctx, `  ${escapeMd(formatDateLabel(date))}\n\n  Inserisci l'orario di inizio (es. 09:00):`, {
      parse_mode: 'Markdown', reply_markup: buildCancelKeyboard(),
    })
    return true
  }
  if (state === 'nt_extra' && cb.startsWith('nt_extra:')) {
    const isExtra = cb.split(':')[1] === 'yes'
    const next = { ...data, is_extraordinary: isExtra }
    await setSession(ctx.telegramId, 'nt_confirm', next)
    await reply(ctx, summaryNuovoTurno(next), { parse_mode: 'Markdown', reply_markup:
      buildYesNoKeyboard('nt_confirm') })
    return true
  }
}

```

```

}
if (state === 'nt_confirm' && cb.startsWith('nt_confirm:')) {
  const ok = cb.split(':')[1] === 'yes'
  await clearSession(ctx.telegramId)
  if (!ok) {
    await reply(ctx, 'Operazione annullata.')
    return true
  }
  const { error } = await ctx.admin.from('turns').insert({
    user_id: data.employee_id,
    restaurant_id: data.restaurant_id,
    department: data.department,
    date: data.date,
    start_time: normalizeTime(data.start_time as string),
    end_time: normalizeTime(data.end_time as string),
    is_extraordinary: !!data.is_extraordinary,
    is_rest_day: false,
    created_by: ctx.profile.id,
  })
  if (error) {
    await reply(ctx, `Errore: ${error.message}`)
  } else {
    await reply(ctx, `Turno creato per *${escapeMd(data.employee_name as string)}*.`, { parse_mode:
'Markdown' })
  }
  return true
}

// — Riposo —
if (state === 'r_restaurant' && cb.startsWith('r_rest:')) {
  const restaurantId = cb.split(':')[1]
  await startEmployeeStep(ctx, 'r', { restaurant_id: restaurantId })
  return true
}
if (state === 'r_employee' && cb.startsWith('r_emp:')) {
  const employeeId = cb.split(':')[1]
  const staff = await listAssignableStaff(ctx.admin, ctx.profile, data.restaurant_id as string | undefined)
  const employee = staff.find(s => s.id === employeeId)
  if (!employee) return reply(ctx, 'Dipendente non trovato.').then(() => true)
  if (!assigneeInScope(ctx.profile, employee)) {
    await clearSession(ctx.telegramId)
    await reply(ctx, 'Non autorizzato per questo dipendente.')
    return true
  }
  await setSession(ctx.telegramId, 'r_date', {
    ...data, employee_id: employee.id, employee_name: employee.full_name,
    department: employee.department, restaurant_id: employee.restaurant_id,
  })
  await reply(ctx, ` ${escapeMd(employee.full_name)}\n\n Seleziona la data del riposo:`, {
    parse_mode: 'Markdown', reply_markup: buildDateKeyboard('r_date'),
  })
  return true
}
if (state === 'r_date' && cb.startsWith('r_date:')) {
  const date = cb.split(':')[1]
  const next = { ...data, date }
  await setSession(ctx.telegramId, 'r_confirm', next)
  await reply(
    ctx,
    ` Confermi il riposo per *${escapeMd(data.employee_name as string)}* il *${
escapeMd(formatDateLabel(date))}*?`,
    { parse_mode: 'Markdown', reply_markup: buildYesNoKeyboard('r_confirm') },
  )
  return true
}
if (state === 'r_confirm' && cb.startsWith('r_confirm:')) {
  const ok = cb.split(':')[1] === 'yes'
  await clearSession(ctx.telegramId)
  if (!ok) {
    await reply(ctx, 'Operazione annullata.')
    return true
  }
  await createRiposo(ctx, data)
  return true
}

```



```

}

// — Elimina turno —
if (state === 'dt_date' && cb.startsWith('dt_date:')) {
  const date = cb.split(':')[1]
  let query = ctx.admin
    .from('turns')
    .select('id, start_time, end_time, is_rest_day, profile:profiles!user_id(full_name)')
    .eq('date', date)
    .order('start_time')
  query = scopeTurnsQuery(query, toScopeProfile(ctx.profile), ctx.profile.id)
  const { data: turns } = await query
  type Row = { id: string; start_time: string; end_time: string; is_rest_day: boolean; profile: { full_name:
string } | null }
  const rows = (turns ?? []) as unknown as Row[]
  if (!rows.length) {
    await clearSession(ctx.telegramId)
    await reply(ctx, `Nessun turno trovato per il ${escapeMd(formatDateLabel(date))}`, { parse_mode:
'Markdown' })
    return true
  }
  const buttons = rows.map(t => [{
    text: t.is_rest_day
      ? ` ${t.profile?.full_name ?? '-'} (Riposo)`
      : `${t.start_time.slice(0, 5)}-${t.end_time.slice(0, 5)} ${t.profile?.full_name ?? '-'}`,
    callback_data: `dt_sel:${t.id}`,
  ]})
  buttons.push([ { text: ' Annulla', callback_data: 'cancel' } ])
  await setSession(ctx.telegramId, 'dt_select', { date })
  await reply(ctx, ` Turni del ${escapeMd(formatDateLabel(date))}\n\nSeleziona quello da eliminare:`, {
    parse_mode: 'Markdown', reply_markup: { inline_keyboard: buttons },
  })
  return true
}

if (state === 'dt_select' && cb.startsWith('dt_sel:')) {
  const id = cb.split(':')[1]
  await setSession(ctx.telegramId, 'dt_confirm', { ...data, turn_id: id })
  await reply(ctx, ` Confermi l'eliminazione di questo turno?`, { reply_markup:
buildYesNoKeyboard('dt_confirm' ) })
  return true
}

if (state === 'dt_confirm' && cb.startsWith('dt_confirm:')) {
  const ok = cb.split(':')[1] === 'yes'
  await clearSession(ctx.telegramId)
  if (!ok) {
    await reply(ctx, ` Operazione annullata.`)
    return true
  }
  let query = ctx.admin.from('turns').delete().eq('id', data.turn_id as string)
  query = scopeTurnsQuery(query, toScopeProfile(ctx.profile), ctx.profile.id)
  const { error } = await query
  if (error) await reply(ctx, `Errore: ${error.message}`)
  else await reply(ctx, ` Turno eliminato.`)
  return true
}

return false
}

// — Dispatcher per i messaggi di testo (input orari) —————
export async function handleTurniText(ctx: Ctx, state: string, data: Record<string, unknown>): Promise<boolean> {
  if (state === 'nt_start') {
    if (!isValidTime(ctx.text)) {
      await reply(ctx, ` Formato non valido. Inserisci l'orario come HH:MM (es. 09:00).`)
      return true
    }
    const next = { ...data, start_time: ctx.text.trim() }
    await setSession(ctx.telegramId, 'nt_end', next)
    await reply(ctx, ` Inserisci l'orario di fine (es. 17:00):`, { reply_markup: buildCancelKeyboard() })
    return true
  }
  if (state === 'nt_end') {
    if (!isValidTime(ctx.text)) {
      await reply(ctx, ` Formato non valido. Inserisci l'orario come HH:MM (es. 17:00).`)
    }
  }
}

```

```

        return true
    }
    const next = { ...data, end_time: ctx.text.trim() }
    await setSession(ctx.telegramId, 'nt_extra', next)
    await reply(ctx, ' È un turno straordinario?', { reply_markup: buildYesNoKeyboard('nt_extra') })
    return true
}
return false
}

function summaryNuovoTurno(data: Record<string, unknown>): string {
    return [
        ' *Riepilogo nuovo turno*',
        '',
        ` Dipendente: ${escapeMd(data.employee_name as string)}`,
        ` Data: ${escapeMd(formatDateLabel(data.date as string))}`,
        ` Orario: ${data.start_time}-${data.end_time}`,
        ` Straordinario: ${data.is_extraordinary ? 'Sì' : 'No'}`,
        '',
        'Confermi?',
    ].join('\n')
}

async function createRiposo(ctx: Ctx, data: Record<string, unknown>) {
    const { admin, profile } = ctx
    const { error } = await admin.from('turns').insert({
        user_id:      data.employee_id,
        restaurant_id: data.restaurant_id,
        department:   data.department,
        date:          data.date,
        start_time:    '00:00:00',
        end_time:      '00:00:00',
        is_extraordinary: false,
        is_rest_day:    true,
        created_by:     profile.id,
    })

    if (error) {
        await reply(ctx, `Errore: ${error.message}`)
        return
    }

    await reply(ctx, ` Riposo registrato per *${escapeMd(data.employee_name as string)}* il *${escapeMd(formatDateLabel(data.date as string))}*`, { parse_mode: 'Markdown' })

    // Notifica non bloccante ai manager
    const { data: managers } = await admin.from('profiles').select('id').eq('role', 'manager')
    if (managers?.length) {
        await admin.from('notifications').insert(
            managers.map(m => ({
                user_id: m.id,
                title:   'Nuovo riposo assegnato',
                message: `${profile.full_name} ha assegnato un riposo a ${data.employee_name} per il ${formatDateLabel(data.date as string)}`,
                link:     '/turni',
            }))
        )
    }
}

```

```
import { createAdminClient } from '@lib/supabase/admin'
import { editMessageText, sendMessage } from './api'
import type { TelegramProfile } from './auth'
import type { InlineKeyboardMarkup } from './types'

export interface Ctx {
  telegramId: number
  chatId: number
  profile: TelegramProfile
  admin: ReturnType<typeof createAdminClient>
  /** Testo completo del messaggio (comandi o input liberi). */
  text: string
  /** Testo dopo il comando, es. "/nuovoturno mario" → "mario". */
  args: string
  /** Dati del bottone premuto (callback_query.data), se presente. */
  callbackData?: string
  /** message_id del messaggio con la tastiera, per editMessageText. */
  messageId?: number
  callbackQueryId?: string
}

export interface ReplyOptions {
  reply_markup?: InlineKeyboardMarkup
  parse_mode?: 'Markdown'
  /** Se true e il contesto deriva da un callback, modifica il messaggio esistente invece di inviarne uno nuovo. */
  edit?: boolean
}

export async function reply(ctx: Ctx, text: string, options: ReplyOptions = {}) {
  const { edit, ...rest } = options
  if (edit && ctx.messageId) {
    return editMessageText(ctx.chatId, ctx.messageId, text, rest)
  }
  return sendMessage(ctx.chatId, text, rest)
}
```

`src/lib/telegram/format.ts`

```

import { addDays, format } from 'date-fns'
import { formatInTimeZone } from 'date-fns-tz'
import { it } from 'date-fns/locale'
import type { InlineKeyboardMarkup } from './types'

export const TZ = 'Europe/Rome'

export function todayStr(): string {
  return formatInTimeZone(new Date(), TZ, 'yyyy-MM-dd')
}

export function dateAfter(days: number): string {
  return formatInTimeZone(addDays(new Date(), days), TZ, 'yyyy-MM-dd')
}

export function formatDateLabel(dateStr: string): string {
  return formatInTimeZone(`${dateStr}T12:00:00Z`, TZ, 'EEEE d MMMM', { locale: it })
}

export function formatDateShort(dateStr: string): string {
  return formatInTimeZone(`${dateStr}T12:00:00Z`, TZ, 'dd/MM', { locale: it })
}

const TIME_RE = /^[01]\d|2[0-3])([0-5]\d)$/

export function isValidTime(text: string): boolean {
  return TIME_RE.test(text.trim())
}

export function normalizeTime(text: string): string {
  return `${text.trim():00}`
}

// Caratteri speciali Markdown (legacy "Markdown" mode di Telegram).
export function escapeMd(text: string): string {
  return text.replace(/([_]*[\])/g, '\\$1')
}

// — Inline keyboard builders —————

export function buildDateKeyboard(prefix: string, days = 7): InlineKeyboardMarkup {
  const today = new Date()
  const labels = ['Oggi', 'Domani']
  const rows: InlineKeyboardMarkup['inline_keyboard'] = []
  let row: InlineKeyboardMarkup['inline_keyboard'][number] = []

  for (let i = 0; i < days; i++) {
    const d = addDays(today, i)
    const dateStr = format(d, 'yyyy-MM-dd')
    const label = labels[i] ?? formatInTimeZone(`${dateStr}T12:00:00Z`, TZ, 'EEE d/MM', { locale: it })
    row.push({ text: label, callback_data: `${prefix}:${dateStr}` })
    if (row.length === 2) {
      rows.push(row)
      row = []
    }
  }
  if (row.length) rows.push(row)
  rows.push([ { text: ' Annulla', callback_data: 'cancel' } ])
  return { inline_keyboard: rows }
}

export function buildEmployeeKeyboard(
  employees: { id: string; full_name: string }[],
  prefix: string,
): InlineKeyboardMarkup {
  const rows = employees.map(e => [ { text: e.full_name, callback_data: `${prefix}:${e.id}` } ])
  rows.push([ { text: ' Annulla', callback_data: 'cancel' } ])
  return { inline_keyboard: rows }
}

export function buildYesNoKeyboard(prefix: string): InlineKeyboardMarkup {
  return {
    inline_keyboard: [

```

```
[
  { text: '  Si', callback_data: `${prefix}:yes` },
  { text: '  No', callback_data: `${prefix}:no` },
],
],
}
}

export function buildCancelKeyboard(): InlineKeyboardMarkup {
  return { inline_keyboard: [[{ text: '  Annulla', callback_data: 'cancel' }]] }
}
```

`src/lib/telegram/router.ts`

```

import { answerCallbackQuery, sendMessage } from './api'
import { getTelegramUser } from './auth'
import { createAdminClient } from '@lib/supabase/admin'
import { getSession, clearSession } from './session'
import { canManageTurni, canManageOds, canManagePresenze } from './scope'
import type { Ctx } from './context'
import { reply } from './context'
import type { TgUpdate } from './types'

import { runAiAssistant } from './ai'
import { cmdHelp, handleStart } from './commands/help'
import { cmdTurni, cmdNuovoTurno, cmdRiposo, cmdEliminaTurno, handleTurniCallback, handleTurniText } from './commands/turni'
import { cmdOds, cmdNuovoOds, cmdCompletaOds, handleOdsCallback, handleOdsText } from './commands/ods'
import { cmdPresenze, cmdModificaPresenza, cmdNuovaPresenza, handlePresenzeCallback, handlePresenzeText } from './commands/presenze'

export async function handleUpdate(update: TgUpdate): Promise<void> {
  const cb = update.callback_query
  const msg = update.message

  const telegramId = cb?.from.id ?? msg?.from?.id
  const chatId = cb?.message?.chat.id ?? msg?.chat.id
  if (!telegramId || !chatId) return

  if (cb) {
    // Risponde subito per chiudere lo spinner del bottone su Telegram.
    await answerCallbackQuery(cb.id).catch(() => {})
  }

  const text = msg?.text ?? ''
  const callbackData = cb?.data
  const messageId = cb?.message?.message_id

  // — /start — gestito prima della risoluzione del profilo —————
  if (text.startsWith('/start')) {
    const args = text.replace(/^\/start(@\w+)?\s*/, '')
    const existing = await getTelegramUser(telegramId)
    return handleStart(telegramId, chatId, args, {
      username: msg?.from?.username,
      first_name: msg?.from?.first_name,
    }, existing)
  }

  const profile = await getTelegramUser(telegramId)
  if (!profile) {
    await sendMessage(
      chatId,
      ' Il tuo account Telegram non è collegato a Turni.\n\nGenera un codice dall\'app (Dashboard → "Collega Telegram") e invialo con /start <codice>.',
    )
    return
  }

  const ctx: Ctx = {
    telegramId,
    chatId,
    profile,
    admin: createAdminClient(),
    text,
    args: text.replace(/^\/\w+(@\w+)?\s*/, ''),
    callbackData,
    messageId,
    callbackQueryId: cb?.id,
  }

  // — Annulla globale —————
  if (callbackData === 'cancel' || text === '/annulla') {
    await clearSession(telegramId)
    await reply(ctx, ' Operazione annullata.', { edit: !!callbackData })
    return
  }
}

```



```

// — Sessione attiva (wizard multi-step) —————
const session = await getSession(telegramId)
if (session) {
  const handled = callbackData
    ? await dispatchCallback(ctx, session.state, session.data)
    : await dispatchText(ctx, session.state, session.data)
  if (handled) return
  // Nessun handler ha gestito l'input: ignora silenziosamente i bottoni
  // obsoleti, o avvisa per il testo libero inatteso.
  if (!callbackData) {
    await reply(ctx, ' Non ho capito. Usa /annulla per interrompere l\'operazione in corso.')
  }
  return
}

// — Comandi —————
if (callbackData) return // bottone orfano senza sessione: ignorato

const match = text.match(/^\/(\w+)/)
const command = match?.[1]?.toLowerCase()
if (!command) {
  // — Messaggio libero senza comando: passa all'assistente AI —————
  if (text.trim()) {
    const aiReply = await runAiAssistant(ctx, text.trim())
    try {
      await reply(ctx, aiReply, { parse_mode: 'Markdown' })
    } catch {
      await reply(ctx, aiReply)
    }
  }
  return
}

switch (command) {
  case 'help':
    await cmdHelp(ctx)
    return

  case 'turni':
    if (!canManageTurni(profile)) return unauthorized(ctx)
    await cmdTurni(ctx)
    return

  case 'nuovoturno':
    if (!canManageTurni(profile)) return unauthorized(ctx)
    await cmdNuovoTurno(ctx)
    return

  case 'riposo':
    if (!canManageTurni(profile)) return unauthorized(ctx)
    await cmdRiposo(ctx)
    return

  case 'eliminaturno':
    if (!canManageTurni(profile)) return unauthorized(ctx)
    await cmdEliminaTurno(ctx)
    return

  case 'ods':
    if (!canManageOds(profile)) return unauthorized(ctx)
    await cmdOds(ctx)
    return

  case 'nuovoods':
    if (!canManageOds(profile)) return unauthorized(ctx)
    await cmdNuovoOds(ctx)
    return

  case 'completaods':
    if (!canManageOds(profile)) return unauthorized(ctx)
    await cmdCompletaOds(ctx)
    return

  case 'presenze':
    if (!canManagePresenze(profile)) return unauthorized(ctx)
    await cmdPresenze(ctx)
    return

  case 'modificapresenza':
    if (!canManagePresenze(profile)) return unauthorized(ctx)

```

```

        await cmdModificaPresenza(ctx)
        return
    case 'nuovapresenza':
        if (!canManagePresenze(profile)) return unauthorized(ctx)
        await cmdNuovaPresenza(ctx)
        return

    default:
        await reply(ctx, "    Comando non riconosciuto. Usa /help per l'elenco dei comandi.")
    }
}

async function unauthorized(ctx: Ctx): Promise<void> {
    await reply(ctx, '    Non sei autorizzato a usare questo comando.')
}

async function dispatchCallback(ctx: Ctx, state: string, data: Record<string, unknown>): Promise<boolean> {
    if (state.startsWith('nt_') || state.startsWith('r_') || state.startsWith('dt_')) {
        return handleTurniCallback(ctx, state, data)
    }
    if (state.startsWith('no_') || state.startsWith('co_')) {
        return handleOdsCallback(ctx, state, data)
    }
    if (state.startsWith('mp_') || state.startsWith('np_')) {
        return handlePresenzeCallback(ctx, state, data)
    }
    return false
}

async function dispatchText(ctx: Ctx, state: string, data: Record<string, unknown>): Promise<boolean> {
    if (state.startsWith('nt_')) return handleTurniText(ctx, state, data)
    if (state.startsWith('no_')) return handleOdsText(ctx, state, data)
    if (state.startsWith('mp_') || state.startsWith('np_')) return handlePresenzeText(ctx, state, data)
    return false
}

```

```

import { scopeStaffQuery, scopeTurnsQuery, type ScopeProfile } from '@lib/turniScope'
import type { TelegramProfile } from './auth'

// — RBAC helpers per i comandi Telegram —————
// Stessa gerarchia dell'app web (turniScope.ts), riapplicata qui perché
// il bot opera con il client admin (service role) e bypassa la RLS:
// tutta l'autorizzazione deve essere fatta in questo livello applicativo.
//
// - manager                → GLOBALE (tutti i ristoranti/reparti)
// - capo_servizio + is_direttore → tutti i reparti del proprio ristorante
// - capo_servizio (non direttore) → solo proprio ristorante + reparto

export function toScopeProfile(profile: TelegramProfile): ScopeProfile {
  return {
    role: profile.role,
    restaurant_id: profile.restaurant_id,
    department: profile.department,
    is_direttore: profile.is_direttore,
  }
}

export function isManager(profile: TelegramProfile): boolean {
  return profile.role === 'manager'
}

export function isDirettore(profile: TelegramProfile): boolean {
  return profile.role === 'capo_servizio' && profile.is_direttore === true
}

export function isCapoServizio(profile: TelegramProfile): boolean {
  return profile.role === 'capo_servizio'
}

// Presenze: solo manager e direttori possono modificarle/crearle da Telegram.
export function canManagePresenze(profile: TelegramProfile): boolean {
  return isManager(profile) || isDirettore(profile)
}

// Turni e ODS: manager, direttori e capi servizio (nel proprio scope).
export function canManageTurni(profile: TelegramProfile): boolean {
  return isManager(profile) || isCapoServizio(profile)
}

export function canManageOds(profile: TelegramProfile): boolean {
  return isManager(profile) || isCapoServizio(profile)
}

export { scopeStaffQuery, scopeTurnsQuery }

// Verifica che un dipendente target sia nello scope del chiamante
// (stesso ristorante, e se non direttore anche stesso reparto).
export function assigneeInScope(profile: TelegramProfile, assignee: { restaurant_id: string | null; department: string | null }): boolean {
  if (isManager(profile)) return true
  if (profile.restaurant_id !== assignee.restaurant_id) return false
  if (isDirettore(profile)) return true
  return profile.department === assignee.department
}

```

```
import { createAdminClient } from '@lib/supabase/admin'

// — Telegram conversational sessions (wizard state) —————
// Vercel functions are stateless: multi-step commands persist their
// progress here, keyed by telegram_id, between webhook invocations.

export interface TelegramSession<T = Record<string, unknown>> {
  state: string
  data: T
}

export async function getSession<T = Record<string, unknown>>(telegramId: number): Promise<TelegramSession<T> | null> {
  const admin = createAdminClient()
  const { data } = await admin
    .from('telegram_sessions')
    .select('state, data')
    .eq('telegram_id', telegramId)
    .maybeSingle()

  if (!data) return null
  return { state: data.state, data: data.data as T }
}

export async function setSession(telegramId: number, state: string, data: Record<string, unknown> = {}): Promise<void> {
  const admin = createAdminClient()
  await admin
    .from('telegram_sessions')
    .upsert({ telegram_id: telegramId, state, data, updated_at: new Date().toISOString() })
}

export async function clearSession(telegramId: number): Promise<void> {
  const admin = createAdminClient()
  await admin.from('telegram_sessions').delete().eq('telegram_id', telegramId)
}
```

```
// — Telegram Bot API — minimal type subset —————
```

```
export interface TgUser {  
  id: number  
  is_bot: boolean  
  first_name: string  
  last_name?: string  
  username?: string  
}
```

```
export interface TgChat {  
  id: number  
  type: string  
}
```

```
export interface TgMessage {  
  message_id: number  
  from?: TgUser  
  chat: TgChat  
  date: number  
  text?: string  
}
```

```
export interface TgCallbackQuery {  
  id: string  
  from: TgUser  
  message?: TgMessage  
  data?: string  
}
```

```
export interface TgUpdate {  
  update_id: number  
  message?: TgMessage  
  callback_query?: TgCallbackQuery  
}
```

```
export interface InlineKeyboardButton {  
  text: string  
  callback_data?: string  
  url?: string  
}
```

```
export interface InlineKeyboardMarkup {  
  inline_keyboard: InlineKeyboardButton[][]  
}
```

src/lib/telegram/wizard.ts

```
import type { createAdminClient } from '@lib/supabase/admin'
import type { TelegramProfile } from './auth'
import { isManager, isDirettore } from './scope'
import type { InlineKeyboardMarkup } from './types'

type Admin = ReturnType<typeof createAdminClient>

// — Selezione ristorante (solo manager, che opera a livello globale) —

export async function listRestaurants(admin: Admin) {
  const { data } = await admin.from('restaurants').select('id, name').order('name')
  return data ?? []
}

export function buildRestaurantKeyboard(restaurants: { id: string; name: string }[], prefix: string):
InlineKeyboardMarkup {
  const rows = restaurants.map(r => [{ text: r.name, callback_data: `${prefix}:${r.id}` }])
  rows.push([{ text: ' Annulla', callback_data: 'cancel' }])
  return { inline_keyboard: rows }
}

// — Selezione dipendente, nello scope del chiamante —
// manager      → tutti i dipendenti/capi servizio del ristorante scelto
// direttore     → tutti i dipendenti/capi servizio del proprio ristorante
// capo_servizio → solo dipendenti del proprio ristorante + reparto

export async function listAssignableStaff(
  admin: Admin,
  profile: TelegramProfile,
  restaurantId?: string,
) {
  let query = admin
    .from('profiles')
    .select('id, full_name, department, restaurant_id')
    .in('role', ['dipendente', 'capo_servizio'])
    .order('full_name')

  if (isManager(profile)) {
    if (restaurantId) query = query.eq('restaurant_id', restaurantId)
  } else if (isDirettore(profile)) {
    query = query.eq('restaurant_id', profile.restaurant_id)
  } else {
    query = query.eq('restaurant_id', profile.restaurant_id).eq('department', profile.department)
  }

  const { data } = await query
  return data ?? []
}
```

src/lib/turnColors.ts

```
export const EXTRAORDINARY_BADGE = 'bg-amber-50 text-amber-700 border-amber-200 dark:bg-amber-950/30 dark:text-amber-400 dark:border-amber-800'
export const STANDARD_BADGE = 'bg-blue-100 text-blue-800 border-blue-300 dark:bg-blue-950/50 dark:text-blue-300 dark:border-blue-700'
export const RIPOSO_BADGE = 'bg-red-50 text-red-700 border-red-200 dark:bg-red-950/30 dark:text-red-400 dark:border-red-800'
```

```
// — Gestione Turni: Query Scoping (RBAC) —————
// Regole ferree, applicate sia in lettura che in scrittura:
// - manager → GLOBALE (nessun vincolo)
// - capo_servizio + is_direttore → LOCALE (solo restaurant_id proprio)
// - capo_servizio (non direttore) → DIPARTIM. (restaurant_id + department propri)
// - dipendente → PERSONALE (solo user_id proprio, sola lettura)

export interface ScopeProfile {
  role: string
  restaurant_id: string | null
  department: string | null
  is_direttore: boolean
}

type ScopableQuery<T> = {
  eq: (column: string, value: unknown) => T
}

export function scopeTurnsQuery<T extends ScopableQuery<T>>(
  query: T,
  profile: ScopeProfile,
  userId: string,
): T {
  if (profile.role === 'manager') {
    // Manager: visibilità e modifica GLOBALE su tutti i ristoranti/reparti
    return query
  } else if (profile.role === 'capo_servizio' && profile.is_direttore) {
    // Direttore: visibilità e modifica LOCALE – tutti i reparti del proprio ristorante
    return query.eq('restaurant_id', profile.restaurant_id)
  } else if (profile.role === 'capo_servizio') {
    // Capo Servizio: visibilità e modifica DIPARTIMENTALE – solo proprio ristorante + reparto
    return query.eq('restaurant_id', profile.restaurant_id).eq('department', profile.department)
  } else {
    // Dipendente: visibilità PERSONALE, sola lettura – solo i propri turni
    return query.eq('user_id', userId)
  }
}

export function scopeStaffQuery<T extends ScopableQuery<T>>(
  query: T,
  profile: ScopeProfile,
): T {
  if (profile.role === 'manager') {
    return query
  } else if (profile.role === 'capo_servizio' && profile.is_direttore) {
    return query.eq('restaurant_id', profile.restaurant_id)
  } else if (profile.role === 'capo_servizio') {
    return query.eq('restaurant_id', profile.restaurant_id).eq('department', profile.department)
  }
  return query
}
```

src/lib/utils.ts

```
import { type ClassValue, clsx } from "clsx"
import { twMerge } from "tailwind-merge"
import { format, parseISO } from "date-fns"

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}

/**
 * Single source of truth for user-facing date strings.
 *
 * Accepts an ISO date-only string ("2026-05-25"), ISO datetime ("2026-05-25T18:00:00Z"),
 * or a Date object, and returns the Italian numeric format "dd-MM-yyyy".
 *
 * DO NOT use this for values sent to the database, for <input type="date"> values,
 * or for groupBy/filter keys – those must stay in ISO 8601 ("yyyy-MM-dd").
 */
export function formatDate(input: string | Date): string {
  const d = typeof input === "string" ? parseISO(input) : input
  return format(d, "dd-MM-yyyy")
}
```

src/middleware.ts

```
import { createServerClient } from '@supabase/ssr'
import { NextResponse, type NextRequest } from 'next/server'

export async function middleware(request: NextRequest) {
  let supabaseResponse = NextResponse.next({ request })

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() { return request.cookies.getAll() },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) => request.cookies.set(name, value))
          supabaseResponse = NextResponse.next({ request })
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options)
          )
        },
      },
    },
  )

  const { data: { user } } = await supabase.auth.getUser()
  const path = request.nextUrl.pathname

  if (!user && !path.startsWith('/login') && !path.startsWith('/register') && !path.startsWith('/api')) {
    return NextResponse.redirect(new URL('/login', request.url))
  }

  if (user && (path === '/login' || path === '/register')) {
    return NextResponse.redirect(new URL('/', request.url))
  }

  return supabaseResponse
}

export const config = {
  matcher: ['/(?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*/'],
}
```

```

export type Role = 'manager' | 'capo_servizio' | 'dipendente' | 'consulente_lavoro'
export type AbsenceType = 'ferie' | 'malattia' | 'riposo' | 'assenza_ingiustificata'
export type AbsenceStatus = 'pending' | 'approved' | 'rejected'
export type BulletinTarget = 'all' | 'restaurant' | 'role' | 'users' | 'department'

export type AccountStatus = 'pending' | 'active'

export interface Restaurant {
  id: string
  name: string
  address: string | null
  latitude: number | null
  longitude: number | null
  qr_secret: string
  closing_days: number[] // 0=Dom..6=Sab
  is_demo: boolean
  owner_id: string | null
  created_at: string
}

export type Department = 'Sala' | 'Pizzeria' | 'Bar' | 'Cucina'

export const DEPARTMENTS: Department[] = ['Sala', 'Pizzeria', 'Bar', 'Cucina']

// 0=Dom..6=Sab (date-fns getDay convention)
export const WEEK_DAYS_SHORT = ['Dom', 'Lun', 'Mar', 'Mer', 'Gio', 'Ven', 'Sab'] as const
export const WEEK_DAYS_FULL = ['Domenica', 'Lunedì', 'Martedì', 'Mercoledì', 'Giovedì', 'Venerdì', 'Sabato'] as const

export interface SecondaryDepartment {
  slot_id: string // ID della fascia oraria (shift_slot)
  priority: number // 1=alta..3=bassa
}

export interface Profile {
  id: string
  full_name: string
  username: string | null
  role: Role
  department: Department | null
  restaurant_id: string | null
  can_post_bulletin: boolean
  is_direttore: boolean
  consultant_restaurant_ids: string[]
  can_view_hours: boolean
  last_active_at: string | null
  // — AI scheduling fields —————
  weekly_rest_days: number // default 1
  preferred_rest_day: number | null // 0=Dom..6=Sab, opzionale
  primary_slot_ids: string[] // fasce principali del dipendente
  secondary_departments: SecondaryDepartment[] // fasce jolly in altri reparti (solo Manager edita)
  weekly_hours_target: number | null // null=full-time; es. 20=part-time
  can_substitute_capo_servizio: boolean // può stare da solo / fare da senior
  // — SaaS fields —————
  account_status: AccountStatus // pending = demo; active = pieno accesso
  managed_restaurant_ids: string[] | null // null = platform owner (vede tutto)
  created_at: string
  updated_at: string
  restaurant?: Restaurant
}

export interface Attendance {
  id: string
  user_id: string
  restaurant_id: string | null
  check_in: string
  check_out: string | null
  is_split_shift: boolean
  notes: string | null
  fallback_photo_path: string | null
  needs_manager_approval: boolean
  created_at: string
  updated_at: string
}

```

```

    profile?: Profile
    restaurant?: Restaurant
}

export interface Absence {
  id: string
  user_id: string
  restaurant_id: string | null
  type: AbsenceType
  start_date: string
  end_date: string
  certificate_code: string | null
  notes: string | null
  status: AbsenceStatus
  created_by: string | null
  created_at: string
  updated_at: string
  profile?: Profile
  restaurant?: Restaurant
}

export interface Bulletin {
  id: string
  title: string
  body: string
  target: BulletinTarget
  target_roles: string[]
  target_user_ids: string[]
  target_department: string | null
  restaurant_id: string | null
  created_by: string
  created_at: string
  restaurant?: Restaurant
  author?: Profile
}

export interface BulletinRead {
  id: string
  bulletin_id: string
  user_id: string
  read_at: string
  profile?: { id: string; full_name: string }
}

export interface PushSubscription {
  id: string
  user_id: string
  endpoint: string
  p256dh: string
  auth_key: string
  created_at: string
}

export const ABSENCE_LABELS: Record<AbsenceType, string> = {
  ferie: 'Ferie',
  malattia: 'Malattia',
  riposo: 'Riposo',
  assenza_ingiustificata: 'Assenza Ingiustificata',
}

export const ABSENCE_CODES: Record<AbsenceType, string> = {
  ferie: 'F',
  malattia: 'M',
  riposo: 'R',
  assenza_ingiustificata: 'AI',
}

export const ROLE_LABELS: Record<Role, string> = {
  manager: 'Manager',
  capo_servizio: 'Capo Servizio',
  dipendente: 'Dipendente',
  consulente_lavoro: 'Consulente del Lavoro',
}

```

```

// — Consultant Messaging —————

export interface ConsultantMessage {
  id: string
  manager_id: string
  consultant_id: string
  title: string
  body: string
  attachments: Array<{ name: string; path: string }>
  sent_by_manager: boolean
  created_at: string
  read_at: string | null
  downloaded_at: string | null
  reply_to_id: string | null
}

// — ODS (Ordine di Servizio) —————

export type OdsTaskType = 'quotidiana' | 'settimanale' | 'bisettimanale' | 'straordinaria'

export const ODS_TYPE_LABELS: Record<OdsTaskType, string> = {
  quotidiana: 'Quotidiana',
  settimanale: 'Settimanale',
  bisettimanale: 'Bisettimanale',
  straordinaria: 'Straordinaria',
}

export const ODS_DAYS_IT = [
  'lunedì', 'martedì', 'mercoledì', 'giovedì', 'venerdì', 'sabato', 'domenica',
] as const

export interface OdsTask {
  id: string
  title: string
  department: string
  restaurant_id: string
  creator_id: string | null
  assigned_to: string | null
  type: OdsTaskType
  recurrence_days: string[]
  created_at: string
  updated_at: string
  assignee?: { id: string; full_name: string } | null
}

export interface OdsCompletion {
  id: string
  task_id: string
  user_id: string
  completed_at: string
}

// — In-app Notifications —————

export interface AppNotification {
  id: string
  user_id: string
  title: string
  message: string
  link: string | null
  read_at: string | null
  created_at: string
}

// — AI Scheduling —————

export interface ShiftSlot {
  id: string
  restaurant_id: string
  department: Department
  name: string // es. "Pranzo", "Cena", "Apertura"
  start_time: string
  end_time: string
  required_count: number
}

```

```

    days_of_week: number[] // 0=Dom..6=Sab; se vuoto = tutti i giorni
    created_at: string
    updated_at: string
  }

export type AiDraftStatus = 'draft' | 'confirmed' | 'cancelled'
export type AiDraftTurnStatus = 'pending' | 'modified' | 'rejected'
export type ExistingTurnsMode = 'integrate' | 'replace'

export interface AiScheduleWarning {
  day: string // yyyy-MM-dd
  department: Department
  slot_name: string
  message: string
  missing_count?: number
}

export interface ExtraordinaryClosure {
  date: string // yyyy-MM-dd
  department?: Department // null = intero ristorante
}

export interface AiScheduleDraft {
  id: string
  restaurant_id: string
  week_start: string // yyyy-MM-dd (sempre lunedì)
  status: AiDraftStatus
  department_scope: Department[] | null // null = tutti
  generated_by: string | null
  generation_params: Record<string, unknown>
  extraordinary_closures: ExtraordinaryClosure[]
  existing_turns_mode: ExistingTurnsMode
  warnings: AiScheduleWarning[]
  created_at: string
  updated_at: string
}

export interface AiScheduleDraftTurn {
  id: string
  draft_id: string
  user_id: string
  department: Department | null
  date: string
  start_time: string
  end_time: string
  is_rest_day: boolean
  is_extraordinary: boolean
  is_cross_dept: boolean
  original_department: Department | null // reparto di appartenenza se jolly
  warning: string | null
  status: AiDraftTurnStatus
  created_at: string
  profile?: { id: string; full_name: string }
}

// — Gestione Turni (Shift Management) —————

export interface Turn {
  id: string
  user_id: string
  restaurant_id: string
  department: Department | null
  date: string
  start_time: string
  end_time: string
  is_extraordinary: boolean
  is_rest_day: boolean
  notes: string | null
  created_by: string | null
  created_at: string
  updated_at: string
  profile?: { id: string; full_name: string } | null
  restaurant?: { id: string; name: string } | null
}

```

```
export interface StandardShift {  
  id: string  
  user_id: string  
  restaurant_id: string  
  department: Department | null  
  day_of_week: number // 0=Dom .. 6=Sab  
  start_time: string  
  end_time: string  
  created_by: string | null  
  created_at: string  
  updated_at: string  
  profile?: { id: string; full_name: string } | null  
}
```

supabase/migrations/20260626_saas_multitenancy.sql


```

-- -----
-- SaaS Multi-tenancy Migration
-- Esegui questo script nel SQL Editor di Supabase
-- -----

-- 1. Colonne nuove su profiles
ALTER TABLE profiles
  ADD COLUMN IF NOT EXISTS account_status text DEFAULT 'active'
  CHECK (account_status IN ('pending', 'active')),
  ADD COLUMN IF NOT EXISTS managed_restaurant_ids uuid[] DEFAULT NULL;
-- NULL = platform owner (vede tutto); '{}' = nessun ristorante; '{uuid,...}' = ristoranti assegnati

-- 2. Colonne nuove su restaurants
ALTER TABLE restaurants
  ADD COLUMN IF NOT EXISTS is_demo boolean DEFAULT false,
  ADD COLUMN IF NOT EXISTS owner_id uuid REFERENCES auth.users(id) ON DELETE SET NULL;

-- 3. I manager esistenti diventano platform owner (accesso totale mantenuto)
UPDATE profiles
SET managed_restaurant_ids = NULL
WHERE role = 'manager';

-- -----
-- 4. Helper function per RLS – restituisce true se il manager corrente
-- può accedere al ristorante indicato
-- -----

CREATE OR REPLACE FUNCTION can_manage_restaurant(rid uuid)
RETURNS boolean
LANGUAGE sql SECURITY DEFINER STABLE AS $$
  SELECT EXISTS (
    SELECT 1 FROM profiles
    WHERE id = auth.uid()
    AND role = 'manager'
    AND (
      managed_restaurant_ids IS NULL          -- platform owner: accesso totale
      OR rid = ANY(managed_restaurant_ids)    -- accesso esplicito
    )
  )
  $$;

-- -----
-- 5. RLS: restaurants
-- -----

DROP POLICY IF EXISTS "managers_can_manage_restaurants" ON restaurants;
DROP POLICY IF EXISTS "manager_select_restaurants" ON restaurants;
DROP POLICY IF EXISTS "manager_insert_restaurants" ON restaurants;
DROP POLICY IF EXISTS "manager_update_restaurants" ON restaurants;
DROP POLICY IF EXISTS "manager_delete_restaurants" ON restaurants;

CREATE POLICY "manager_select_restaurants" ON restaurants
  FOR SELECT USING (can_manage_restaurant(id));

CREATE POLICY "manager_insert_restaurants" ON restaurants
  FOR INSERT WITH CHECK (
    EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'manager')
  );

CREATE POLICY "manager_update_restaurants" ON restaurants
  FOR UPDATE USING (can_manage_restaurant(id));

CREATE POLICY "manager_delete_restaurants" ON restaurants
  FOR DELETE USING (can_manage_restaurant(id));

-- -----
-- 6. RLS: profiles – i manager vedono solo dipendenti dei propri ristoranti
-- -----

DROP POLICY IF EXISTS "manager_select_profiles" ON profiles;
DROP POLICY IF EXISTS "manager_update_profiles" ON profiles;
DROP POLICY IF EXISTS "manager_delete_profiles" ON profiles;

CREATE POLICY "manager_select_profiles" ON profiles
  FOR SELECT USING (
    id = auth.uid()                                -- ognuno vede se stesso
  )

```

```

OR EXISTS (
  SELECT 1 FROM profiles p
  WHERE p.id = auth.uid()
  AND p.role = 'manager'
  AND (
    p.managed_restaurant_ids IS NULL
    OR profiles.restaurant_id = ANY(p.managed_restaurant_ids)
    OR profiles.role = 'manager' -- i manager vedono gli altri manager
  )
)
)
OR EXISTS (
  SELECT 1 FROM profiles p
  WHERE p.id = auth.uid()
  AND p.role IN ('capo_servizio', 'consulente_lavoro')
)
);

CREATE POLICY "manager_update_profiles" ON profiles
FOR UPDATE USING (
  id = auth.uid()
  OR EXISTS (
    SELECT 1 FROM profiles p
    WHERE p.id = auth.uid()
    AND p.role = 'manager'
    AND (
      p.managed_restaurant_ids IS NULL
      OR profiles.restaurant_id = ANY(p.managed_restaurant_ids)
    )
  )
);

CREATE POLICY "manager_delete_profiles" ON profiles
FOR DELETE USING (
  EXISTS (
    SELECT 1 FROM profiles p
    WHERE p.id = auth.uid()
    AND p.role = 'manager'
    AND (
      p.managed_restaurant_ids IS NULL
      OR profiles.restaurant_id = ANY(p.managed_restaurant_ids)
    )
  )
);

-- -----
-- 7. RLS: turns, shift_slots, absences, attendances, standard_shifts
-- Pattern comune: manager accede se può gestire il restaurant_id della riga
-- -----

-- turns
DROP POLICY IF EXISTS "manager_select_turns" ON turns;
DROP POLICY IF EXISTS "manager_insert_turns" ON turns;
DROP POLICY IF EXISTS "manager_update_turns" ON turns;
DROP POLICY IF EXISTS "manager_delete_turns" ON turns;

CREATE POLICY "manager_select_turns" ON turns
FOR SELECT USING (
  EXISTS (SELECT 1 FROM profiles p WHERE p.id = auth.uid()
    AND (p.role IN ('capo_servizio', 'dipendente') OR can_manage_restaurant(turns.restaurant_id)))
  OR user_id = auth.uid()
);
CREATE POLICY "manager_insert_turns" ON turns
FOR INSERT WITH CHECK (can_manage_restaurant(restaurant_id));
CREATE POLICY "manager_update_turns" ON turns
FOR UPDATE USING (can_manage_restaurant(restaurant_id));
CREATE POLICY "manager_delete_turns" ON turns
FOR DELETE USING (can_manage_restaurant(restaurant_id));

-- shift_slots
DROP POLICY IF EXISTS "manager_select_shift_slots" ON shift_slots;
DROP POLICY IF EXISTS "manager_insert_shift_slots" ON shift_slots;
DROP POLICY IF EXISTS "manager_update_shift_slots" ON shift_slots;
DROP POLICY IF EXISTS "manager_delete_shift_slots" ON shift_slots;

```

```

CREATE POLICY "manager_select_shift_slots" ON shift_slots
  FOR SELECT USING (
    can_manage_restaurant(restaurant_id)
    OR EXISTS (SELECT 1 FROM profiles p WHERE p.id = auth.uid()
      AND p.restaurant_id = shift_slots.restaurant_id)
  );
CREATE POLICY "manager_insert_shift_slots" ON shift_slots
  FOR INSERT WITH CHECK (can_manage_restaurant(restaurant_id));
CREATE POLICY "manager_update_shift_slots" ON shift_slots
  FOR UPDATE USING (can_manage_restaurant(restaurant_id));
CREATE POLICY "manager_delete_shift_slots" ON shift_slots
  FOR DELETE USING (can_manage_restaurant(restaurant_id));

-- ai_schedule_drafts
DROP POLICY IF EXISTS "manager_select_ai_drafts" ON ai_schedule_drafts;
DROP POLICY IF EXISTS "manager_insert_ai_drafts" ON ai_schedule_drafts;
DROP POLICY IF EXISTS "manager_update_ai_drafts" ON ai_schedule_drafts;

CREATE POLICY "manager_select_ai_drafts" ON ai_schedule_drafts
  FOR SELECT USING (can_manage_restaurant(restaurant_id));
CREATE POLICY "manager_insert_ai_drafts" ON ai_schedule_drafts
  FOR INSERT WITH CHECK (can_manage_restaurant(restaurant_id));
CREATE POLICY "manager_update_ai_drafts" ON ai_schedule_drafts
  FOR UPDATE USING (can_manage_restaurant(restaurant_id));

-- ai_schedule_draft_turns (read via draft's restaurant)
DROP POLICY IF EXISTS "manager_select_ai_draft_turns" ON ai_schedule_draft_turns;
DROP POLICY IF EXISTS "manager_insert_ai_draft_turns" ON ai_schedule_draft_turns;
DROP POLICY IF EXISTS "manager_update_ai_draft_turns" ON ai_schedule_draft_turns;

CREATE POLICY "manager_select_ai_draft_turns" ON ai_schedule_draft_turns
  FOR SELECT USING (
    EXISTS (SELECT 1 FROM ai_schedule_drafts d
      WHERE d.id = ai_schedule_draft_turns.draft_id
      AND can_manage_restaurant(d.restaurant_id))
    OR user_id = auth.uid()
  );
CREATE POLICY "manager_insert_ai_draft_turns" ON ai_schedule_draft_turns
  FOR INSERT WITH CHECK (
    EXISTS (SELECT 1 FROM ai_schedule_drafts d
      WHERE d.id = draft_id AND can_manage_restaurant(d.restaurant_id))
  );
CREATE POLICY "manager_update_ai_draft_turns" ON ai_schedule_draft_turns
  FOR UPDATE USING (
    EXISTS (SELECT 1 FROM ai_schedule_drafts d
      WHERE d.id = ai_schedule_draft_turns.draft_id
      AND can_manage_restaurant(d.restaurant_id))
  );

```

[next.config.ts](#)

```

import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  devIndicators: false,
  env: {
    NEXT_PUBLIC_SUPABASE_URL: 'https://myvflezdghrypytvdfgm.supabase.co',
    NEXT_PUBLIC_SUPABASE_ANON_KEY:
      'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6Im15dmZsZXpkZ2hyeXB5dHkZkdmdtIiwicm9sZSI6ImFuY24iLCJpYXQiOiJlE3Nzk1MTg4NjQsImV4cCI6ImJA5NTA5NDg2NH0.97kxoGVMM4Ptey6erKyXbpqgYYP0jTlI8XkvIlrxxB4',
    NEXT_PUBLIC_VAPID_PUBLIC_KEY: 'BF0F7j9TKRhnnN4r2m0Q6SPAyctMW2Du03yJ1XUIb2q7ACpr3h-tSA0skmbrCCw5BQMMjr_vROWIozkZu98MyN0o',
  },
};

export default nextConfig;

```

package.json

```
{
  "name": "inturno",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "@ai-sdk/google": "^3.0.81",
    "@radix-ui/react-avatar": "^1.1.11",
    "@radix-ui/react-checkbox": "^1.3.3",
    "@radix-ui/react-dialog": "^1.1.15",
    "@radix-ui/react-dropdown-menu": "^2.1.16",
    "@radix-ui/react-label": "^2.1.8",
    "@radix-ui/react-popover": "^1.1.15",
    "@radix-ui/react-scroll-area": "^1.2.10",
    "@radix-ui/react-select": "^2.2.6",
    "@radix-ui/react-separator": "^1.1.8",
    "@radix-ui/react-slot": "^1.2.4",
    "@radix-ui/react-switch": "^1.2.6",
    "@radix-ui/react-tabs": "^1.1.13",
    "@radix-ui/react-toast": "^1.2.15",
    "@supabase/ssr": "^0.10.3",
    "@supabase/supabase-js": "^2.106.1",
    "ai": "^6.0.202",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "date-fns": "^4.3.0",
    "date-fns-tz": "^3.2.0",
    "exceljs": "^4.4.0",
    "framer-motion": "^12.40.0",
    "html5-qrcode": "^2.3.8",
    "lucide-react": "^1.16.0",
    "next": "16.2.6",
    "next-themes": "^0.4.6",
    "pdf-lib": "^1.17.1",
    "qrcode": "^1.5.4",
    "react": "19.2.4",
    "react-dom": "19.2.4",
    "resend": "^6.16.0",
    "tailwind-merge": "^3.6.0",
    "web-push": "^3.6.7",
    "zod": "^4.4.3"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/node": "^20",
    "@types/qrcode": "^1.5.6",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "@types/web-push": "^3.6.4",
    "eslint": "^9",
    "eslint-config-next": "16.2.6",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

vercel.json

```
{}
```